

실전 시스템 설계

AI 시대의 아키텍처, 운영, 신뢰성, 그리고 확장성

KLIC



PRACTICAL SYSTEM DESIGN 2026

실전 시스템 설계 2026

AI 시대의 아키텍처, 운영, 신뢰성, 그리고 확장성

원작 **Donne Martin** · 한국어 개정 **KLIC** 지음

38장 전체 1차 초고 · 기술/편집/시각자료 검수 전

PDF 편집본 안내

PDF

이 문서는 제공된 한국어 Markdown 원고(7부 38장·부록)를 하나의 책으로 편집한 PDF 편집본이다. 원문의 장·절 구조와 본문 내용을 유지했다.

표지, 제목 페이지, 목차, 장 표제지, 머리말·꼬리말, 링크와 인용문 스타일은 읽기와 탐색을 돕도록 재구성했다. 원고의 사실 관계와 출처 표기는 원문을 따른다.

원고의 일부 구조와 주제는 Donne Martin의 The System Design Primer(CC BY 4.0)를 자료원으로 삼아 번역·재구성·수정했다. 원저자는 이 개정 초안을 보증하거나 후원하지 않는다.

이 책의 일부 구조와 학습 주제는 Donne Martin의 The System Design Primer(CC BY 4.0)를 자료원으로 번역·재구성·수정했다. 원저자는 이 개정 초안을 보증하거나 후원하지 않는다. 자체 작성 본문·명세는 CC BY 4.0 배포를 전제로 하며, 최종 출판 조건과 제3자 인용 범위는 출판 전 법률 검토가 필요하다. 본 판본은 38장 1차 초고로 기술·편집·시각자료 최종 검수 전이다.

이 판본은 38장 1차 초고로, 기술·편집·시각자료에 대한 최종 검수를 앞두고 있다.

PDF 편집일: 2026-08-09

목차

장과 절을 한 줄 흐름으로 읽을 수 있도록 1단 목차로 구성했습니다.

	머리말	24
--	-----------	----

	이 책을 사용하는 방법	25
--	--------------------	----

1	Part 1. 설계 문제를 푸는 방법	26
	포함 장	27
	읽는 방법	27

01	01. 요구사항에서 시스템 경계까지	28
	이 장에서 해결할 문제	30
	먼저 결론	30
	요구사항과 실패 모델	30
	핵심 개념	31
	기준 아키텍처	31
	요청-데이터 흐름	32
	대안과 트레이드오프	32
	장애 시나리오	33
	확장 전략	34
	보안과 개인정보	34
	관측 가능성	34
	비용과 운영 복잡도	34
	흔한 오해와 안티패턴	35
	설계 리뷰	35
	연습문제	35
	핵심 요약	35
	출처	35

02	02. 트래픽·저장공간·대역폭 계산	37
	이 장에서 해결할 문제	39
	먼저 결론	39
	요구사항과 실패 모델	39
	핵심 개념	40
	기준 아키텍처	41
	요청-데이터 흐름	42
	대안과 트레이드오프	43
	장애 시나리오	43
	확장 전략	44
	보안과 개인정보	44
	관측 가능성	45
	비용과 운영 복잡도	45
	흔한 오해와 안티패턴	45
	설계 리뷰	45
	연습문제	46
	핵심 요약	46
	출처	46
03	03. 트레이드오프와 Architecture Decision Record	47
	이 장에서 해결할 문제	49
	먼저 결론	49
	요구사항과 실패 모델	49
	핵심 개념	50
	기준 아키텍처	50
	요청-데이터 흐름	51
	대안과 트레이드오프	52
	장애 시나리오	52
	확장 전략	53
	보안과 개인정보	53
	관측 가능성	54
	비용과 운영 복잡도	54
	흔한 오해와 안티패턴	54
	설계 리뷰	54
	연습문제	55
	핵심 요약	55
	출처	55

04	04. SLI·SLO·SLA와 Error Budget	56
	이 장에서 해결할 문제	58
	먼저 결론	58
	요구사항과 실패 모델	58
	핵심 개념	59
	기준 아키텍처	60
	요청-데이터 흐름	61
	대안과 트레이드오프	61
	장애 시나리오	62
	확장 전략	63
	보안과 개인정보	63
	관측 가능성	63
	비용과 운영 복잡도	63
	흔한 오해와 안티패턴	64
	설계 리뷰	64
	연습문제	64
	핵심 요약	64
	출처	64
2	Part 2. 분산 시스템의 기본 원리	66
	포함 장	67
	읽는 방법	67
05	05. 지연시간·처리량·동시성과 Tail Latency	68
	이 장에서 해결할 문제	70
	먼저 결론	70
	요구사항과 실패 모델	70
	핵심 개념	71
	기준 아키텍처	72
	요청-데이터 흐름	73
	대안과 트레이드오프	73
	장애 시나리오	74
	확장 전략	75
	보안과 개인정보	75
	관측 가능성	75
	비용과 운영 복잡도	75
	흔한 오해와 안티패턴	76
	설계 리뷰	76
	연습문제	76
	핵심 요약	76
	출처	77

06	06. 가용성·신뢰성·내구성과 장애 도메인	78
	이 장에서 해결할 문제	80
	먼저 결론	80
	요구사항과 실패 모델	80
	핵심 개념	81
	기준 아키텍처	82
	요청-데이터 흐름	83
	대안과 트레이드오프	83
	장애 시나리오	84
	확장 전략	85
	보안과 개인정보	85
	관측 가능성	85
	비용과 운영 복잡도	85
	흔한 오해와 안티패턴	86
	설계 리뷰	86
	연습문제	86
	핵심 요약	86
	출처	87
07	07. CAP를 넘어선 일관성 모델	88
	이 장에서 해결할 문제	90
	먼저 결론	90
	요구사항과 실패 모델	90
	핵심 개념	91
	기준 아키텍처	92
	요청-데이터 흐름	92
	대안과 트레이드오프	93
	장애 시나리오	93
	확장 전략	94
	보안과 개인정보	94
	관측 가능성	95
	비용과 운영 복잡도	95
	흔한 오해와 안티패턴	95
	설계 리뷰	95
	연습문제	96
	핵심 요약	96
	출처	96

08	08. 트랜잭션·격리 수준·MVCC	97
	이 장에서 해결할 문제	99
	먼저 결론	99
	요구사항과 실패 모델	99
	핵심 개념	100
	기준 아키텍처	100
	요청-데이터 흐름	101
	대안과 트레이드오프	102
	장애 시나리오	102
	확장 전략	103
	보안과 개인정보	103
	관측 가능성	104
	비용과 운영 복잡도	104
	흔한 오해와 안티패턴	104
	설계 리뷰	104
	연습문제	105
	핵심 요약	105
	출처	105
09	09. 시간·순서·논리 시계·분산 ID	106
	이 장에서 해결할 문제	108
	먼저 결론	108
	요구사항과 실패 모델	108
	핵심 개념	109
	기준 아키텍처	109
	요청-데이터 흐름	110
	대안과 트레이드오프	111
	장애 시나리오	111
	확장 전략	112
	보안과 개인정보	112
	관측 가능성	113
	비용과 운영 복잡도	113
	흔한 오해와 안티패턴	113
	설계 리뷰	113
	연습문제	114
	핵심 요약	114
	출처	114

10	10. 복제·Quorum·Failover	115
	이 장에서 해결할 문제	117
	먼저 결론	117
	요구사항과 실패 모델	117
	핵심 개념	118
	기준 아키텍처	119
	요청·데이터 흐름	120
	대안과 트레이드오프	120
	장애 시나리오	121
	확장 전략	122
	보안과 개인정보	122
	관측 가능성	123
	비용과 운영 복잡도	123
	흔한 오해와 안티패턴	123
	설계 리뷰	123
	연습문제	124
	핵심 요약	124
	출처	124
11	11. 파티셔닝·Sharding·Consistent Hashing	125
	이 장에서 해결할 문제	127
	먼저 결론	127
	요구사항과 실패 모델	127
	핵심 개념	128
	기준 아키텍처	129
	요청·데이터 흐름	130
	대안과 트레이드오프	130
	장애 시나리오	131
	확장 전략	132
	보안과 개인정보	132
	관측 가능성	132
	비용과 운영 복잡도	132
	흔한 오해와 안티패턴	133
	설계 리뷰	133
	연습문제	133
	핵심 요약	133
	출처	134

12	12. Consensus · Leader Election · Fencing	135
	이 장에서 해결할 문제	137
	먼저 결론	137
	요구사항과 실패 모델	137
	핵심 개념	138
	기준 아키텍처	139
	요청-데이터 흐름	139
	대안과 트레이드오프	140
	장애 시나리오	140
	확장 전략	141
	보안과 개인정보	141
	관측 가능성	142
	비용과 운영 복잡도	142
	흔한 오해와 안티패턴	142
	설계 리뷰	142
	연습문제	143
	핵심 요약	143
	출처	143
3	Part 3. 네트워크와 서비스 실행 구조	144
	포함 장	145
	읽는 방법	145
13	13. DNS · CDN · Edge와 전역 트래픽	146
	이 장에서 해결할 문제	148
	먼저 결론	148
	요구사항과 실패 모델	148
	핵심 개념	149
	기준 아키텍처	150
	요청-데이터 흐름	150
	대안과 트레이드오프	151
	장애 시나리오	151
	확장 전략	152
	보안과 개인정보	152
	관측 가능성	153
	비용과 운영 복잡도	153
	흔한 오해와 안티패턴	153
	설계 리뷰	153
	연습문제	154
	핵심 요약	154
	출처	154

14	14. L4/L7 Load Balancing · Proxy · Gateway	155
	이 장에서 해결할 문제	157
	먼저 결론	157
	요구사항과 실패 모델	157
	핵심 개념	158
	기준 아키텍처	159
	요청-데이터 흐름	159
	대안과 트레이드오프	160
	장애 시나리오	160
	확장 전략	161
	보안과 개인정보	161
	관측 가능성	162
	비용과 운영 복잡도	162
	흔한 오해와 안티패턴	162
	설계 리뷰	162
	연습문제	163
	핵심 요약	163
	출처	163
15	15. HTTP/1.1 · HTTP/2 · HTTP/3와 QUIC	164
	이 장에서 해결할 문제	166
	먼저 결론	166
	요구사항과 실패 모델	166
	핵심 개념	167
	기준 아키텍처	168
	요청-데이터 흐름	169
	대안과 트레이드오프	169
	장애 시나리오	169
	확장 전략	170
	보안과 개인정보	170
	관측 가능성	171
	비용과 운영 복잡도	171
	흔한 오해와 안티패턴	171
	설계 리뷰	171
	연습문제	172
	핵심 요약	172
	출처	172

16	16. REST·gRPC·GraphQL·WebSocket·SSE	173
	이 장에서 해결할 문제	175
	먼저 결론	175
	요구사항과 실패 모델	175
	핵심 개념	176
	기준 아키텍처	177
	요청·데이터 흐름	177
	대안과 트레이드오프	178
	장애 시나리오	178
	확장 전략	179
	보안과 개인정보	179
	관측 가능성	180
	비용과 운영 복잡도	180
	흔한 오해와 안티패턴	180
	설계 리뷰	180
	연습문제	181
	핵심 요약	181
	출처	181
17	17. 모듈러 모놀리스·마이크로서비스·Service Mesh	182
	이 장에서 해결할 문제	184
	먼저 결론	184
	요구사항과 실패 모델	184
	핵심 개념	185
	기준 아키텍처	186
	요청·데이터 흐름	186
	대안과 트레이드오프	187
	장애 시나리오	187
	확장 전략	188
	보안과 개인정보	188
	관측 가능성	189
	비용과 운영 복잡도	189
	흔한 오해와 안티패턴	189
	설계 리뷰	189
	연습문제	190
	핵심 요약	190
	출처	190
4	Part 4. 데이터·캐시·이벤트	191
	포함 장	192
	읽는 방법	192

18	18. 워크로드에서 저장소 선택하기	193
	이 장에서 해결할 문제	195
	먼저 결론	195
	요구사항과 실패 모델	195
	핵심 개념	196
	기준 아키텍처	196
	요청-데이터 흐름	197
	대안과 트레이드오프	198
	장애 시나리오	198
	확장 전략	199
	보안과 개인정보	199
	관측 가능성	200
	비용과 운영 복잡도	200
	흔한 오해와 안티패턴	200
	설계 리뷰	200
	연습문제	201
	핵심 요약	201
	출처	201

19	19. 관계형 DB·분산 SQL·인덱스	202
	이 장에서 해결할 문제	204
	먼저 결론	204
	요구사항과 실패 모델	204
	핵심 개념	205
	기준 아키텍처	206
	요청-데이터 흐름	206
	대안과 트레이드오프	207
	장애 시나리오	207
	확장 전략	208
	보안과 개인정보	208
	관측 가능성	209
	비용과 운영 복잡도	209
	흔한 오해와 안티패턴	209
	설계 리뷰	209
	연습문제	210
	핵심 요약	210
	출처	210

20	20. Key-Value · Document · Wide-column · Graph	211
	이 장에서 해결할 문제	213
	먼저 결론	213
	요구사항과 실패 모델	213
	핵심 개념	214
	기준 아키텍처	215
	요청-데이터 흐름	215
	대안과 트레이드오프	216
	장애 시나리오	216
	확장 전략	217
	보안과 개인정보	217
	관측 가능성	218
	비용과 운영 복잡도	218
	흔한 오해와 안티패턴	218
	설계 리뷰	218
	연습문제	219
	핵심 요약	219
	출처	219

21	21. Object Storage · Search · Vector Store	220
	이 장에서 해결할 문제	222
	먼저 결론	222
	요구사항과 실패 모델	222
	핵심 개념	223
	기준 아키텍처	224
	요청-데이터 흐름	224
	대안과 트레이드오프	225
	장애 시나리오	225
	확장 전략	226
	보안과 개인정보	226
	관측 가능성	227
	비용과 운영 복잡도	227
	흔한 오해와 안티패턴	227
	설계 리뷰	227
	연습문제	228
	핵심 요약	228
	출처	228

22	22. 캐시·무효화·Stampede·Hot Key	229
	이 장에서 해결할 문제	231
	먼저 결론	231
	요구사항과 실패 모델	231
	핵심 개념	232
	기준 아키텍처	233
	요청·데이터 흐름	234
	대안과 트레이드오프	234
	장애 시나리오	235
	확장 전략	236
	보안과 개인정보	236
	관측 가능성	236
	비용과 운영 복잡도	236
	흔한 오해와 안티패턴	237
	설계 리뷰	237
	연습문제	237
	핵심 요약	237
	출처	238

23	23. Queue·Durable Log·Delivery Semantics	239
	이 장에서 해결할 문제	241
	먼저 결론	241
	요구사항과 실패 모델	241
	핵심 개념	242
	기준 아키텍처	243
	요청·데이터 흐름	243
	대안과 트레이드오프	244
	장애 시나리오	244
	확장 전략	245
	보안과 개인정보	245
	관측 가능성	246
	비용과 운영 복잡도	246
	흔한 오해와 안티패턴	246
	설계 리뷰	246
	연습문제	247
	핵심 요약	247
	출처	247

24	24. Event Streaming · CDC · Outbox · Saga	248
	이 장에서 해결할 문제	250
	먼저 결론	250
	요구사항과 실패 모델	250
	핵심 개념	251
	기준 아키텍처	252
	요청-데이터 흐름	253
	대안과 트레이드오프	253
	장애 시나리오	253
	확장 전략	254
	보안과 개인정보	254
	관측 가능성	255
	비용과 운영 복잡도	255
	흔한 오해와 안티패턴	255
	설계 리뷰	255
	연습문제	256
	핵심 요약	256
	출처	256
5	Part 5. 프로덕션 시스템	257
	포함 장	258
	읽는 방법	258
25	25. Timeout · Deadline · Retry · Backoff · Jitter	259
	이 장에서 해결할 문제	261
	먼저 결론	261
	요구사항과 실패 모델	261
	핵심 개념	262
	기준 아키텍처	263
	요청-데이터 흐름	264
	대안과 트레이드오프	264
	장애 시나리오	265
	확장 전략	266
	보안과 개인정보	266
	관측 가능성	266
	비용과 운영 복잡도	266
	흔한 오해와 안티패턴	267
	설계 리뷰	267
	연습문제	267
	핵심 요약	267
	출처	268

26	26. Circuit Breaker·Bulkhead·Backpressure·Load Shedding	269
	이 장에서 해결할 문제	271
	먼저 결론	271
	요구사항과 실패 모델	271
	핵심 개념	272
	기준 아키텍처	273
	요청·데이터 흐름	273
	대안과 트레이드오프	274
	장애 시나리오	274
	확장 전략	275
	보안과 개인정보	275
	관측 가능성	276
	비용과 운영 복잡도	276
	흔한 오해와 안티패턴	276
	설계 리뷰	276
	연습문제	277
	핵심 요약	277
	출처	277

27	27. Metrics·Logs·Traces와 OpenTelemetry	278
	이 장에서 해결할 문제	280
	먼저 결론	280
	요구사항과 실패 모델	280
	핵심 개념	281
	기준 아키텍처	282
	요청·데이터 흐름	283
	대안과 트레이드오프	283
	장애 시나리오	284
	확장 전략	285
	보안과 개인정보	285
	관측 가능성	285
	비용과 운영 복잡도	285
	흔한 오해와 안티패턴	286
	설계 리뷰	286
	연습문제	286
	핵심 요약	286
	출처	287

28	28. 인증·인가·Zero Trust·Secrets·공급망 보안	288
	이 장에서 해결할 문제	290
	먼저 결론	290
	요구사항과 실패 모델	290
	핵심 개념	291
	기준 아키텍처	292
	요청·데이터 흐름	293
	대안과 트레이드오프	293
	장애 시나리오	293
	확장 전략	294
	보안과 개인정보	295
	관측 가능성	295
	비용과 운영 복잡도	295
	흔한 오해와 안티패턴	295
	설계 리뷰	296
	연습문제	296
	핵심 요약	296
	출처	296
29	29. Multi-region·Backup·재해 복구	298
	이 장에서 해결할 문제	300
	먼저 결론	300
	요구사항과 실패 모델	300
	핵심 개념	301
	기준 아키텍처	302
	요청·데이터 흐름	303
	대안과 트레이드오프	303
	장애 시나리오	304
	확장 전략	305
	보안과 개인정보	305
	관측 가능성	305
	비용과 운영 복잡도	305
	흔한 오해와 안티패턴	306
	설계 리뷰	306
	연습문제	306
	핵심 요약	306
	출처	307

30	30. Container · Kubernetes · Serverless · IaC · GitOps · FinOps	308
	이 장에서 해결할 문제	310
	먼저 결론	310
	요구사항과 실패 모델	310
	핵심 개념	311
	기준 아키텍처	312
	요청-데이터 흐름	313
	대안과 트레이드오프	313
	장애 시나리오	314
	확장 전략	315
	보안과 개인정보	315
	관측 가능성	316
	비용과 운영 복잡도	316
	흔한 오해와 안티패턴	316
	설계 리뷰	316
	연습문제	317
	핵심 요약	317
	출처	317
6	Part 6. AI 네이티브 시스템	318
	포함 장	319
	읽는 방법	319
31	31. RAG 데이터 파이프라인과 Retrieval 품질	320
	이 장에서 해결할 문제	322
	먼저 결론	322
	요구사항과 실패 모델	322
	핵심 개념	323
	기준 아키텍처	324
	요청-데이터 흐름	325
	대안과 트레이드오프	326
	장애 시나리오	326
	확장 전략	327
	보안과 개인정보	327
	관측 가능성	328
	비용과 운영 복잡도	328
	흔한 오해와 안티패턴	328
	설계 리뷰	328
	연습문제	329
	핵심 요약	329
	출처	329

32	32. LLM Inference · Batching · KV Cache · Model Routing	330
	이 장에서 해결할 문제	332
	먼저 결론	332
	요구사항과 실패 모델	332
	핵심 개념	333
	기준 아키텍처	334
	요청-데이터 흐름	335
	대안과 트레이드오프	336
	장애 시나리오	336
	확장 전략	337
	보안과 개인정보	337
	관측 가능성	338
	비용과 운영 복잡도	338
	흔한 오해와 안티패턴	338
	설계 리뷰	338
	연습문제	339
	핵심 요약	339
	출처	339
33	33. Agent 상태·메모리·도구 실행·승인 경계	340
	이 장에서 해결할 문제	342
	먼저 결론	342
	요구사항과 실패 모델	342
	핵심 개념	343
	기준 아키텍처	344
	요청-데이터 흐름	345
	대안과 트레이드오프	346
	장애 시나리오	346
	확장 전략	347
	보안과 개인정보	347
	관측 가능성	348
	비용과 운영 복잡도	348
	흔한 오해와 안티패턴	348
	설계 리뷰	348
	연습문제	349
	핵심 요약	349
	출처	349

34	34. AI 평가·관측 가능성·보안·비용	350
	이 장에서 해결할 문제	352
	먼저 결론	352
	요구사항과 실패 모델	352
	핵심 개념	353
	기준 아키텍처	354
	요청·데이터 흐름	355
	대안과 트레이드오프	356
	장애 시나리오	356
	확장 전략	357
	보안과 개인정보	357
	관측 가능성	358
	비용과 운영 복잡도	358
	흔한 오해와 안티패턴	358
	설계 리뷰	358
	연습문제	359
	핵심 요약	359
	출처	359
7	Part 7. 단계별 종합 설계	360
	포함 장	361
	읽는 방법	361
35	35. URL 단축 서비스: 단일 노드에서 전역 서비스까지	362
	이 장에서 해결할 문제	364
	먼저 결론	364
	요구사항과 실패 모델	364
	핵심 개념	365
	기준 아키텍처	366
	요청·데이터 흐름	367
	대안과 트레이드오프	367
	장애 시나리오	368
	종합 설계 보조 도표	369
	확장 전략	370
	보안과 개인정보	371
	관측 가능성	371
	비용과 운영 복잡도	371
	흔한 오해와 안티패턴	372
	설계 리뷰	372
	연습문제	372
	핵심 요약	372
	출처	373

36	36. 실시간 채팅과 알림 플랫폼	374
	이 장에서 해결할 문제	376
	먼저 결론	376
	요구사항과 실패 모델	376
	핵심 개념	377
	기준 아키텍처	378
	요청-데이터 흐름	379
	대안과 트레이드오프	379
	장애 시나리오	380
	종합 설계 보조 도표	381
	확장 전략	382
	보안과 개인정보	383
	관측 가능성	383
	비용과 운영 복잡도	383
	흔한 오해와 안티패턴	384
	설계 리뷰	384
	연습문제	384
	핵심 요약	384
	출처	385

37	37. 주문·재고·결제 원장 시스템	386
	이 장에서 해결할 문제	388
	먼저 결론	388
	요구사항과 실패 모델	388
	핵심 개념	389
	기준 아키텍처	390
	요청-데이터 흐름	391
	대안과 트레이드오프	392
	장애 시나리오	392
	종합 설계 보조 도표	393
	확장 전략	394
	보안과 개인정보	395
	관측 가능성	395
	비용과 운영 복잡도	395
	흔한 오해와 안티패턴	396
	설계 리뷰	396
	연습문제	396
	핵심 요약	396
	출처	397

38	38. 멀티테넌트 RAG·AI 고객지원 플랫폼	398
	이 장에서 해결할 문제	400
	먼저 결론	400
	요구사항과 실패 모델	400
	핵심 개념	401
	기준 아키텍처	402
	요청-데이터 흐름	403
	대안과 트레이드오프	404
	장애 시나리오	404
	종합 설계 보조 도표	405
	확장 전략	407
	보안과 개인정보	407
	관측 가능성	408
	비용과 운영 복잡도	408
	흔한 오해와 안티패턴	408
	설계 리뷰	409
	연습문제	409
	핵심 요약	409
	출처	409
A	부록 A. 시스템 설계 리뷰 체크리스트	411
	문제와 경계	412
	수치와 성능	412
	데이터와 일관성	412
	장애와 운영	412
	보안과 비용	413
B	부록 B. 용량 계산 공식	414
C	부록 C. 이미지 제작과 검수 흐름	416

머리말

시스템 설계 자료는 흔하지만, 많은 자료가 두 극단 중 하나에 머문다. 하나는 제품 이름과 구성도만 나열하고 왜 그런 경계를 선택했는지 설명하지 않는다. 다른 하나는 면접에서 외울 수 있는 정답 모양을 제공하지만 실제 장애, 운영, 데이터 정확성, 보안, 비용을 뒤로 미룬다.

이 책은 요구사항을 **검증 가능한 설계 입력**으로 바꾸는 데서 시작한다. 사용자가 끝내야 할 일, 깨지면 안 되는 불변조건, 피크 규모, 허용 실패, 데이터 소유권을 먼저 정의한다. 그 다음에 일관성, transaction, replication, partitioning, protocol, cache, event, cloud runtime, AI pipeline을 선택한다. 제품은 이 선택을 구현하는 수단이며, 선택의 근거 자체는 아니다.

원본 The System Design Primer가 제공한 학습 지도와 일부 핵심 개념은 `upstream-map.yaml` 에서 계보를 추적한다. 오래된 용어·단순화·고정 수치는 그대로 재사용하지 않았다. `master/slave` 대신 역할에 맞춰 `primary/replica`, `leader/follower`, `single-writer/multi-writer` 를 사용한다. “세 가지 중 두 가지” 같은 CAP 요약은 분할 중의 실제 선택과 사용자에게 필요한 일관성 모델로 다시 설명한다. 고정 latency 숫자는 보편 법칙처럼 심지 않고 계산식·측정 환경·분포를 요구한다.

현재 파일은 **출판 가능한 최종 원고가 아니라 실제 내용이 들어 있는 1차 초고**다. 모든 장애 장애 시나리오, 대안, 운영 지표, 보안, 비용, 연습문제와 출처를 넣었다. 그러나 `current` 와 `volatile` 장은 release 직전 공식 원문을 다시 확인해야 하고, Image2.0·SVG·차트는 상세 명세까지만 포함돼 있다. 이 경계를 숨기지 않는 것이 좋은 기술 문서의 출발점이다.

이 책을 사용하는 방법

각 장을 처음부터 끝까지 읽는 것보다 실제 시스템 하나를 정해 반복 대입하는 방식이 효과적이다.

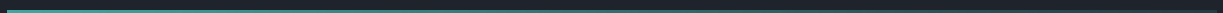
1. **요구사항과 실패 모델**에서 자신의 사용자 여정과 불변조건을 쓴다.
2. **기준 아키텍처**의 제품 이름을 복사하지 말고 책임과 데이터 소유자를 매핑한다.
3. **대안과 트레이드오프**에서 현재 선택이 어떤 비용을 감수하는지 ADR로 기록한다.
4. **장애 시나리오**를 부하 테스트, chaos/game day, 복구 훈련 항목으로 바꾼다.
5. **관측 가능성** 항목을 dashboard·SLO·alert·trace schema에 연결한다.
6. **설계 리뷰** 체크박스가 모두 체크되더라도 남은 가정과 risk owner를 기록한다.

본문의 `figure-spec` 주석은 사람이 읽는 이미지 지시이자 도구가 파싱할 수 있는 계약이다. 실제 그림이 없는 현재 단계에서는 깨진 이미지 링크 대신 제작 위치·출력 경로·대체 텍스트를 보인다. `assets.yaml`의 상태가 `approved`가 된 뒤에만 최종 빌드가 이미지를 포함해야 한다.

PART

1

설계 문제를 푸는 방법



시스템 설계의 출발점을 제품 요구사항, 수치, 실패 모델, 운영 목표로 고정한다.

이 Part에서는 특정 제품의 설정법보다 반복해서 적용할 수 있는 설계 질문과 실패 모델을 먼저 익힌다. 각 장의 체크리스트를 실제 시스템의 ADR·런북·대시보드와 연결해 읽는 것이 목표다.

포함 장

ID	장 제목	최신성	원본 관계
ch01	요구사항에서 시스템 경계까지	durable	REWRITE
ch02	트래픽·저장공간·대역폭 계산	durable	REWRITE
ch03	트레이드오프와 Architecture Decision Record	durable	ADD
ch04	SLI·SLO·SLA와 Error Budget	current	ADD

읽는 방법

1. 먼저 각 장의 **요구사항과 실패 모델**을 자신의 시스템에 대입한다.
2. **대안과 트레이드오프**에서 현재 선택이 어떤 비용을 감수하는지 확인한다.
3. **장애 시나리오**를 게임데이 또는 설계 리뷰 질문으로 바꾼다.
4. 시각자료는 설명을 대신하지 않고 책임·흐름·복구 경계를 검증하는 용도로 사용한다.

01

요구사항에서 시스템 경계까지

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

좋은 구성도는 박스를 많이 그린 그림이 아니라, 무엇을 책임지고 무엇을 책임지지 않는지 합의한 결과다. 첫 단계에서 경계와 실패 모델을 틀리면 이후의 데이터베이스·캐시·클라우드 선택은 정교해 보여도 문제를 해결하지 못한다.

이 절의 기준 출처: [iso-42010; @upstream-primer].

학습 목표

- 기능 요구와 품질 요구를 분리한다.
- 시스템 경계와 신뢰 경계를 명시한다.
- 모호한 요구를 검증 가능한 설계 입력으로 바꾼다.

먼저 결론

- 사용자 여정과 사업 불변조건부터 적는다.
- 평균 트래픽보다 피크·증가율·실패 시 행동을 먼저 확인한다.
- 시스템 경계, 데이터 소유권, 외부 의존성을 한 장에 표시한다.
- 미확정 사항은 숨기지 말고 가정·검증 방법·결정 시한으로 관리한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
기능	사용자가 반드시 끝낼 수 있어야 하는 일은 무엇인가?	핵심 여정과 보조 기능을 분리한다.
품질	얼마나 빠르고, 가용하며, 안전해야 하는가?	측정 가능한 지표와 관측 창을 정한다.
규모	현재·피크·성장 후 부하는 얼마인가?	평균값과 최대값을 따로 계산한다.
경계	우리가 소유하는 데이터와 외부 계약은 무엇인가?	책임·신뢰·장애 경계를 분리해 그린다.
계약	법규, 일정, 인력, 기존 시스템 제약은 무엇인가?	바꿀 수 있는 것과 바꿀 수 없는 것을 구분한다.

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

시스템 경계

책임지는 기능·데이터·운영 범위를 둘러싼 선이다. 조직도나 네트워크 구간과 항상 같지는 않다.

불변조건

어떤 장애나 재시도에서도 깨지면 안 되는 규칙이다. 예: 주문 총액은 승인된 항목 합과 일치해야 한다.

실패 모델

프로세스 중단, 네트워크 지연, 중복 메시지, 부분 성공처럼 설계가 견뎌야 할 실패의 집합이다.

외부 계약

API, 이벤트, 파일, 사용자 인터페이스처럼 경계를 넘는 약속이다. 버전·소유자·오류 의미가 필요하다.

비목표

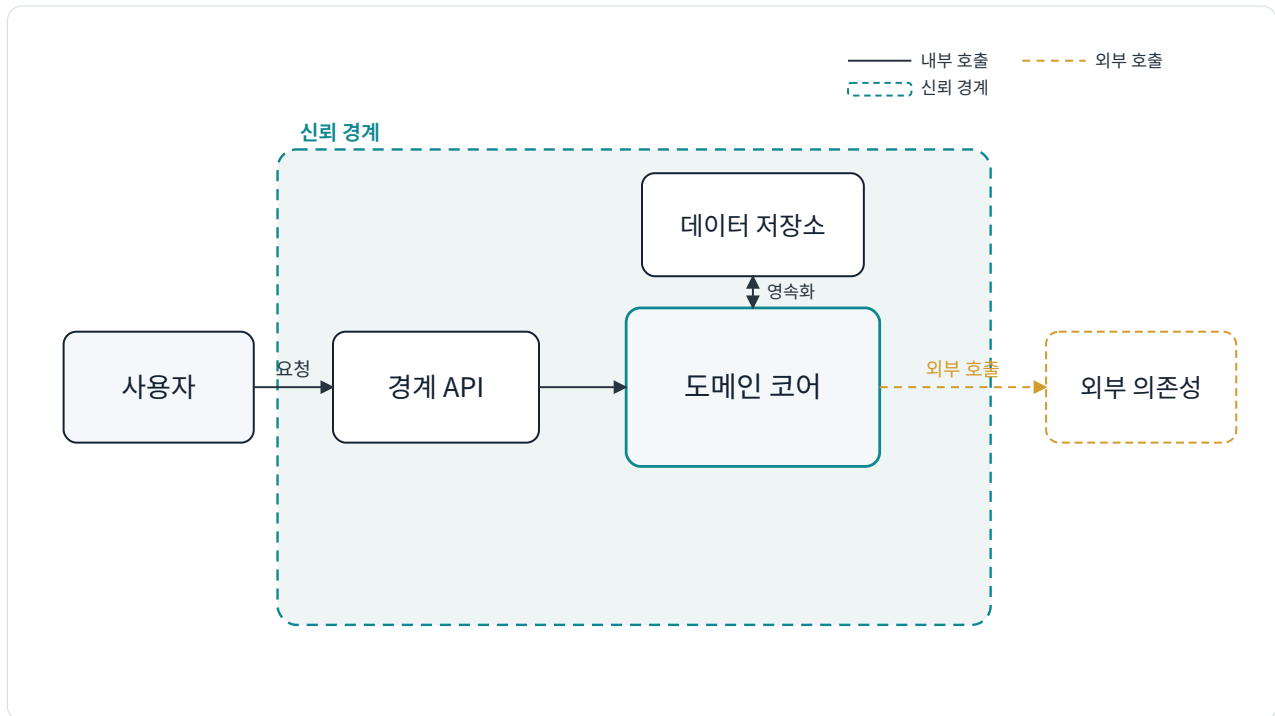
이번 시스템이 해결하지 않는 범위를 명시해 설계가 무한 확장되는 것을 막는다.

핵심 개념의 정의와 범위는 [iso-42010; upstream-primer]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
사용자/호출자	목표와 입력을 제공하고 결과의 의미를 판단한다.
경계 API	요청을 검증하고 내부 모델로 변환한다.
도메인 코어	불변조건과 상태 전이를 소유한다.
데이터 저장소	도메인 상태와 감사 증거를 보존한다.
외부 의존성	결제·인증·알림처럼 별도 장애 도메인에 속한다.
운영 제어면	설정, 배포, 관측, 접근 통제를 제공한다.



사용자·도메인 코어·외부 의존성과 신뢰 경계를 한눈에 보여준다

요청·데이터 흐름

1. 핵심 사용자 여정을 한 문장으로 정의한다.
2. 입력·출력·상태 변화를 식별한다.
3. 각 상태의 소유자와 원장을 지정한다.
4. 경계를 넘는 호출과 이벤트에 실패 의미를 붙인다.
5. 성공 경로뿐 아니라 부분 성공·취소·재시도를 그린다.
6. 가정 목록을 검증 가능한 질문으로 바꾼다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

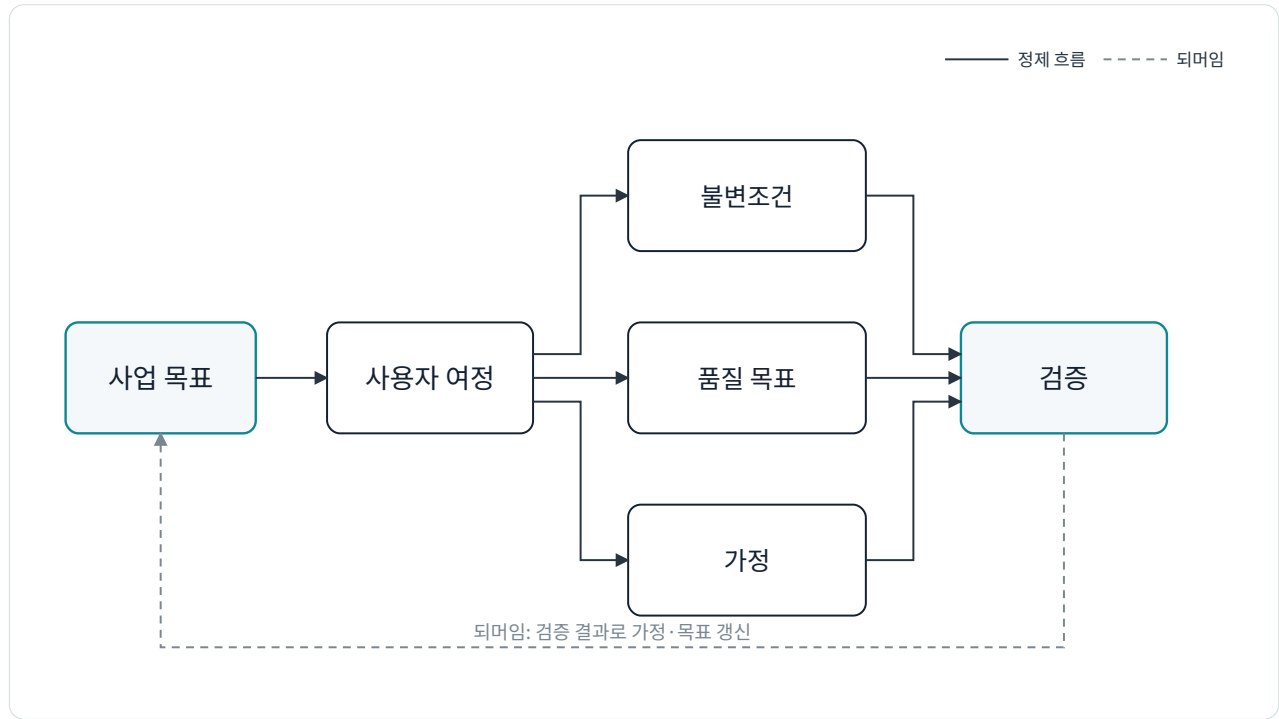
대안	장점	비용·위험	적합한 조건
도메인 중심 경계	업무 불변조건과 데이터 소유권이 선명하다.	초기 도메인 분석이 필요하다.	업무 규칙이 복잡한 시스템
기술 계층 중심 경계	조직이 익숙하고 시작이 빠르다.	데이터 소유권과 책임이 여러 계층에 흩어질 수 있다.	단순 CRUD 또는 기존 구조 유지

대안	장점	비용·위험	적합한 조건
외부 서비스 조합	구현 범위를 줄일 수 있다.	가용성·비용·데이터 정책을 외 부 계약에 의존한다.	차별화되지 않는 범용 기능

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [iso-42010; @upstream-primer]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
요구 누락	정상 경로만 구현되어 취소·중복·부분 실패 가 운영 사고가 된다.	사전 조건·후속 보상·운영 수동 절차를 요 구사항에 포함한다.
경계 중복	여러 서비스가 같은 상태를 수정해 진실의 원천이 모호해진다.	쓰기 소유자를 하나로 정하고 복제는 읽기 모델로 제한한다.
외부 계약 변화	공급자 API 변경이나 제한이 내부 장애로 전파된다.	어댑터, 계약 테스트, 버전 정책, 격리된 재 시도를 둔다.
가정의 사실화	확인되지 않은 트래픽·보존 기간이 용량 설 계를 왜곡한다.	가정마다 소유자·근거·검증일을 기록한다.



모호한 사업 요구가 기능·품질·규모·실패 모델·검증 항목으로 좁혀지는 과정을 보여준다

확장 전략

- 경계를 기능 목록이 아니라 데이터 소유권과 실패 격리 단위로 재검토한다.
- 핵심 경로와 비핵심 경로를 분리해 비핵심 장애가 전체 성공률을 떨어뜨리지 않게 한다.
- 성장 시 분리할 가능성이 높은 책임은 인터페이스를 선명하게 하되 처음부터 서비스로 쪼개지 않는다.
- 외부 시스템마다 timeout, 재시도 가능성, 중복 처리 계약을 문서화한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 개인정보가 경계를 넘는 지점을 데이터 흐름에 표시한다.
- 외부 입력은 신뢰하지 않고 스키마·권한·크기를 검증한다.
- 운영자 기능은 사용자 경로와 별도의 권한·감사 정책을 둔다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 핵심 여정 성공률과 단계별 실패율
- 외부 의존성별 지연·오류·timeout
- 불변조건 위반 탐지 건수
- 수동 복구·보상 작업 건수

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 경계가 많아질수록 네트워크·배포·관측 비용이 증가한다.
- 외부 서비스는 개발 비용을 줄이지만 호출량·데이터 반출·탈퇴 비용을 만든다.
- 모호한 책임은 평상시보다 장애 시 인력 비용을 크게 만든다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- “마이크로서비스여야 확장된다”는 결론부터 시작한다.
- 사용자 수만 묻고 피크 패턴과 쓰기 비율을 묻지 않는다.
- 구성도에 제품 이름만 있고 데이터 소유자와 실패 의미가 없다.
- 비목표와 수동 운영 절차를 문서에서 제외한다.

설계 리뷰

- ☐ 핵심 불변조건이 문장으로 적혀 있는가?
- ☐ 각 데이터의 단일 쓰기 소유자가 정해졌는가?
- ☐ 외부 의존성 실패가 사용자에게 어떻게 보이는가?
- ☐ 검증되지 않은 가정에 소유자와 날짜가 있는가?
- ☐ 시스템 밖에서 수행할 수동 복구 절차가 정의됐는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 온라인 강의 결제 기능의 시스템 경계를 그리고 결제·수강권·영수증의 쓰기 소유자를 정하라.
2. “월 사용자 100만 명”이라는 요구를 설계 가능한 질문 열 개로 바꾸라.
3. 외부 SMS 공급자가 30분 중단될 때 핵심 사용자 여정을 유지하는 방식을 설계하라.

핵심 요약

- 설계는 제품 선택보다 문제 경계와 불변조건 정의에서 시작한다.
- 기능·품질·규모·실패·제약을 분리해 적는다.
- 데이터 소유권과 외부 계약이 시스템 경계를 결정한다.
- 확인되지 않은 내용은 가정으로 표시하고 검증한다.
- 정상 경로와 복구 경로를 같은 수준으로 설계한다.

출처

- [iso-42010] ISO/IEC/IEEE. **ISO/IEC/IEEE 42010:2022 — Architecture description** (2022). <https://www.iso.org/standard/74393.html>

-
- [@upstream-primer] Donne Martin. **The System Design Primer** (2026). <https://github.com/donnemartin/system-design-primer>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

02

트래픽·저장공간·대역폭 계산

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

02

CHAPTER MAP

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

용량 계산의 목적은 정답 숫자를 맞추는 것이 아니라 병목 후보와 민감한 가정을 드러내는 것이다. 모든 숫자는 단위, 관측 창, 복제·압축·보존 조건을 함께 가져야 한다.

이 절의 기준 출처: [upstream-primer; dean-tail-at-scale].

학습 목표

- 요청률·동시성·저장량을 단위와 계산식으로 산출한다.
- 평균과 피크를 분리하고 불확실성 범위를 제시한다.
- 계산 결과를 아키텍처 결정과 검증 계획으로 연결한다.

먼저 결론

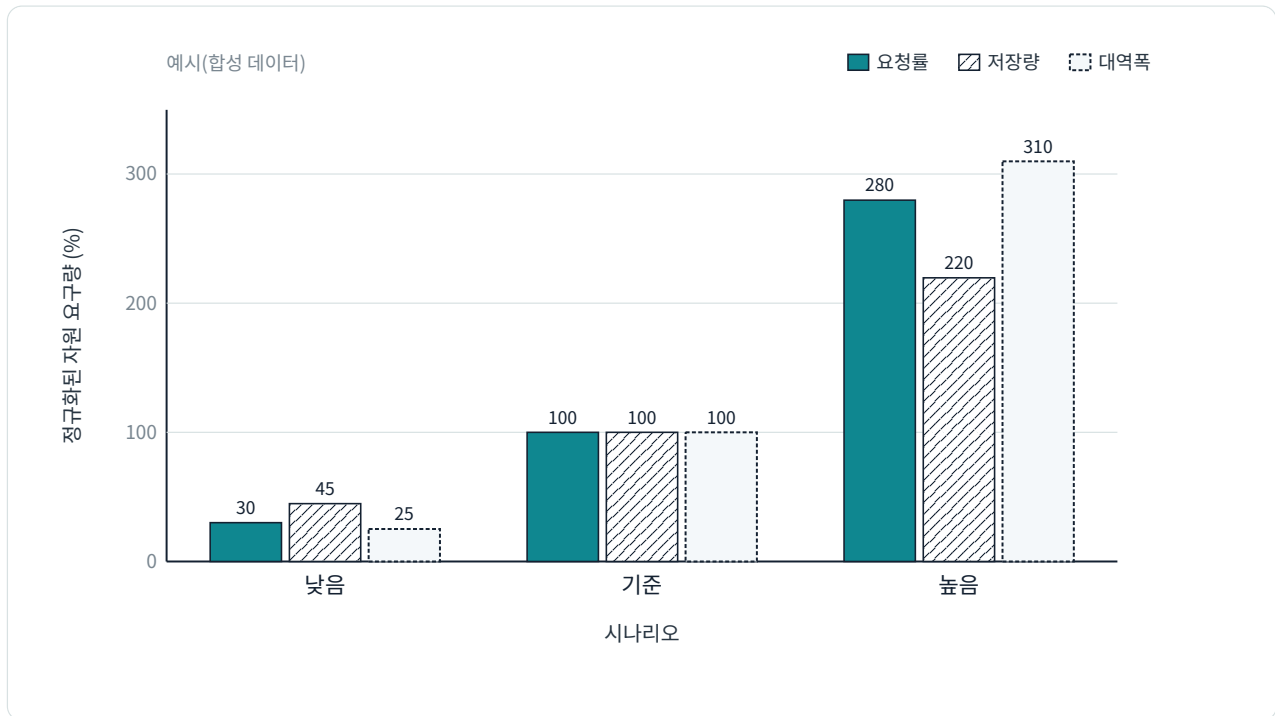
- 일 단위 총량을 초당 평균으로 나누되 피크 계수를 별도로 둔다.
- 동시성은 요청률 $\times$ 체류 시간으로 추정하고 긴 연결은 별도로 계산한다.
- 논리 데이터와 물리 저장량을 복제·색인·로그·여유 공간까지 포함해 구분한다.
- 단일 값 대신 낮음·기준·높음 시나리오를 제시한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
수요	일·시간·초 단위 요청과 이벤트가 얼마나 발생하는가?	평균·피크·burst를 분리해 계산한다.
체류 시간	요청·연결·작업이 자원을 얼마나 오래 점유하는가?	평균과 p95·p99를 이용해 동시성 범위를 계산한다.
데이터 크기	요청·응답·이벤트·객체의 실제 크기는 얼마인가?	표본 측정값과 프로토콜 오버헤드를 함께 기록한다.
보존·복제	데이터를 얼마나 오래, 몇 개 사본과 색인으로 보존하는가?	논리 저장량과 물리 저장량을 별도로 산출한다.

차원	확인 질문	설계 판단
증설 여유	예측 오차와 증설 lead time을 얼마나 흡수해야 하는가?	낮음·기준·높음 시나리오와 안전 여유를 둔다.

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



낮음·기준·높음 용량 시나리오

핵심 개념

요청률

초당 요청 수(RPS) 또는 이벤트 수(EPS)다. 읽기·쓰기·배치·스트리밍을 분리해야 한다.

동시성

동시에 자원을 점유하는 작업 수다. 대략 요청률 × 평균 체류 시간으로 추정할 수 있지만 꼬리 지연과 긴 연결을 따로 본다.

대역폭

초당 전송 바이트다. 요청·응답·헤더·복제·리전 간 전송을 분리한다.

논리 저장량

사용자가 의미 있게 생성한 데이터 크기다.

물리 저장량

복제본, 색인, WAL·로그, 임시 공간, 스냅샷, 여유 용량을 포함한다.

민감도

어떤 입력 가정이 결과를 가장 크게 흔드는지 나타낸다.

핵심 개념의 정의와 범위는 [upstream-primer; @dean-tail-at-scale]를 기준으로 재검토해야 한다.

계산 예시: 이벤트 저장량

가정이 다음과 같다고 하자.

- 평균 이벤트: 25,000 events/s
- 평균 이벤트 크기: 900 bytes/event
- 보존 기간: 30 days
- 복제 계수: 3

논리 원본량은 다음과 같다.

```
25,000 × 900 × 86,400 × 30
= 58,320,000,000,000 bytes
≈ 58.32 TB (10진 기준)
```

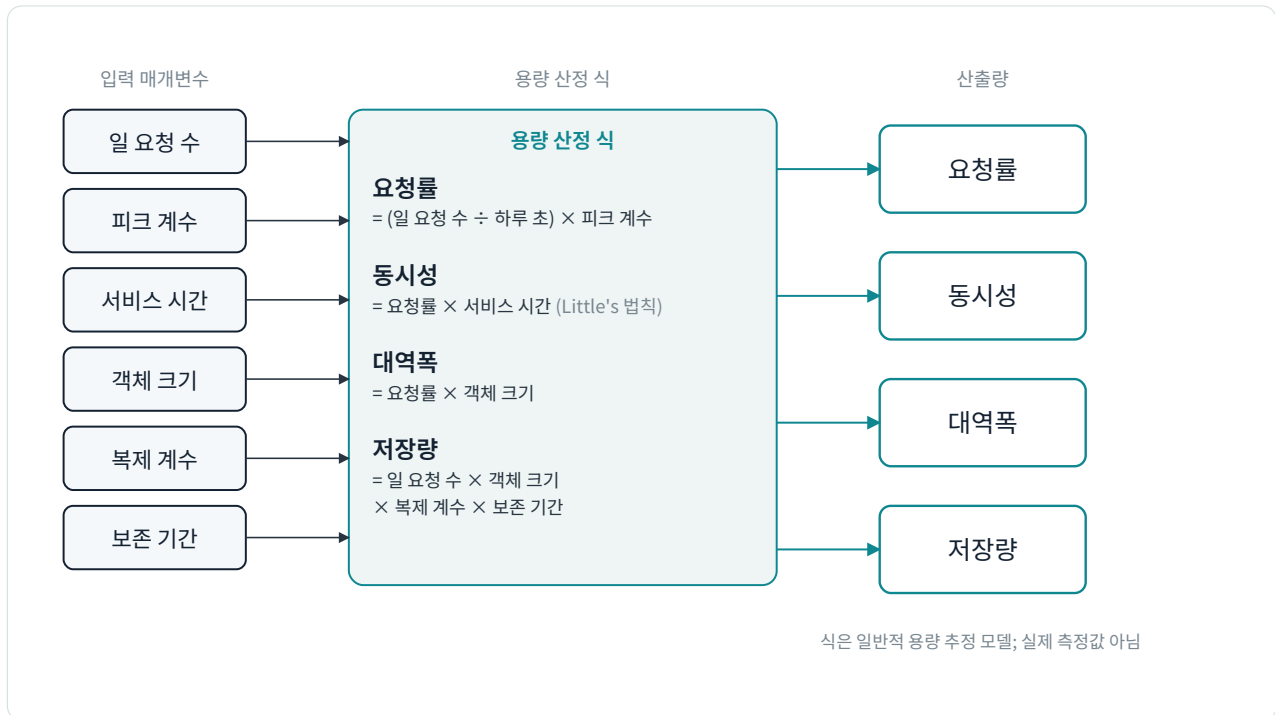
3중 복제만 적용한 물리량은 $58.32 \text{ TB} \times 3 = 174.96 \text{ TB}$ 다. 실제 계획에는 색인, 로그, 스냅샷, 압축률, compaction 임시 공간, 안전 여유를 별도 항목으로 더해야 한다. 이 수치는 예시 입력을 사용한 계산이며 실측값이 아니다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
입력 가정표	사용자 수, 행동 빈도, 객체 크기, 보존 기간을 근거와 함께 저장한다.
계산 시트	단위가 보이는 식으로 RPS·동시성·저장량·대역폭을 계산한다.

구성 요소	책임
시나리오 모델	낮음·기준·높음 값을 같은 식으로 비교한다.
병목 지도	CPU, 메모리, I/O, 연결 수, 네트워크 중 먼저 닿는 제한을 표시한다.
검증 계획	부하 테스트와 실제 관측으로 어떤 가정을 갱신할지 정한다.



요청률·동시성·대역폭·저장량의 입력과 식을 연결한다

요청·데이터 흐름

1. 사용자 행동을 읽기·쓰기·업로드·실시간 연결로 나눈다.
2. 각 행동의 일 총량과 피크 시간대를 구한다.
3. 평균 RPS를 계산하고 피크 계수와 burst를 적용한다.
4. 객체 크기와 프로토콜 오버헤드로 네트워크를 계산한다.
5. 보존 기간·복제·색인 계수로 물리 저장량을 계산한다.
6. 가장 민감한 가정을 부하 테스트 입력으로 바꾼다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

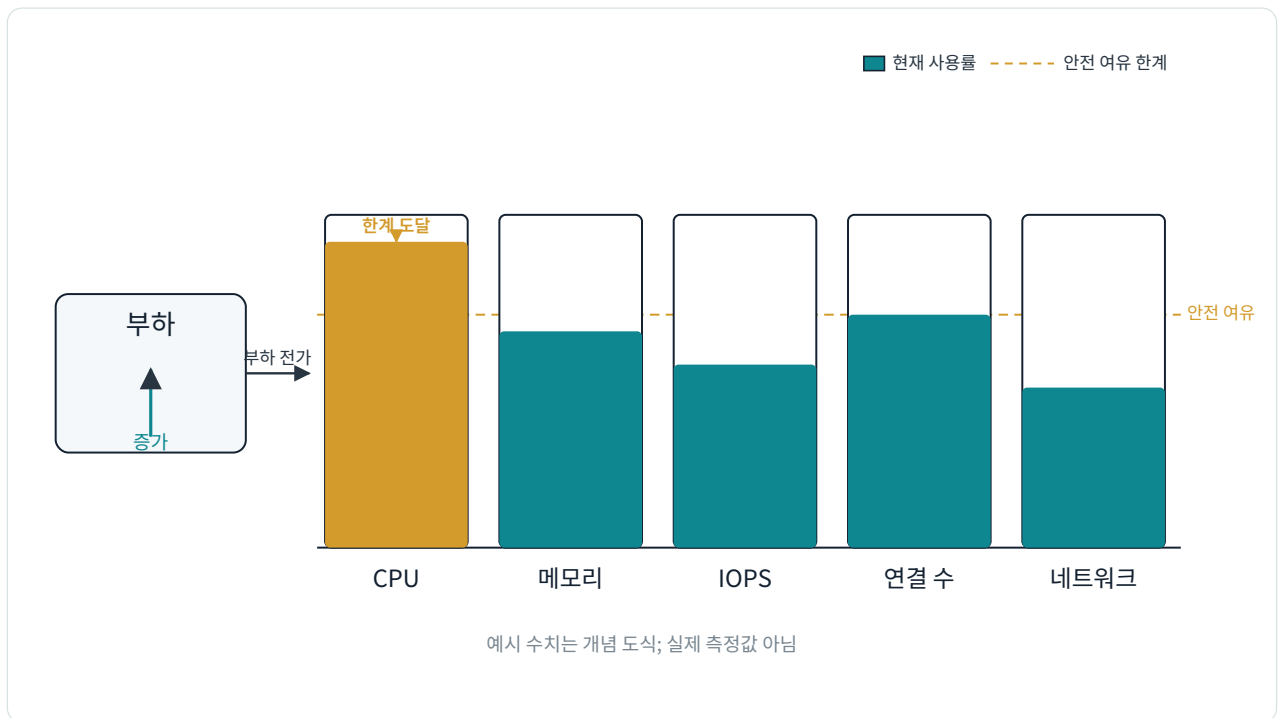
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
상향식 계산	행동과 객체 크기에서 자원량을 도출해 근거가 명확하다.	행동 데이터가 없으면 가정이 많아진다.	신규 기능 설계
관측 기반 외삽	현재 트래픽을 성장률로 확장해 현실적이다.	새로운 사용 패턴과 구조 변화는 반영하지 못한다.	기존 서비스 증설
공급자 한도 역산	DB IOPS·API quota 같은 한도에서 안전 처리량을 역산한다.	한도 하나에 과도하게 맞춘 구조가 될 수 있다.	명확한 외부 제한이 있는 시스템

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [upstream-primer; @dean-tail-at-scale]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
평균값 함정	평균 RPS는 낮지만 짧은 이벤트 시간에 burst가 집중된다.	분 단위·초 단위 피크와 burst 지속시간을 따로 모델링한다.
단위 혼동	GB와 GiB, bit와 byte, 일과 초가 섞여 8배 이상 오차가 난다.	식마다 단위를 적고 자동 변환 테스트를 둔다.
숨은 복제 비용	논리 데이터만 계산해 디스크와 리전 전송이 부족해진다.	복제·색인·WAL·백업·여유 계수를 별도 항목으로 둔다.
꼬리 지연 누락	평균 체류 시간만으로 연결 수를 계산해 큐가 급증한다.	p95·p99 서비스 시간과 timeout 시나리오를 함께 계산한다.



트래픽 증가에 따라 CPU·메모리·IOPS·연결·네트워크 중 어느 한계에 먼저 닿는지 보여준다

확장 전략

- 읽기와 쓰기를 독립적으로 확장할 수 있는지 확인한다.
- 고비용 객체나 hot tenant를 평균에서 분리한다.
- 용량 임계치 전에 증설할 운영 lead time을 반영한다.
- 수직 확장 한도와 수평 확장 시 재분배 비용을 같이 계산한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 용량 로그에 실제 사용자 식별 정보가 들어가지 않게 집계한다.
- 지역 간 데이터 전송량 계산에 데이터 주권·암호화 비용을 포함한다.
- 부하 테스트가 실제 외부 서비스에 피해를 주지 않도록 격리한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- RPS/EPS와 피크 대 평균 비율
- p50·p95·p99 서비스 시간과 동시성
- CPU·메모리·IOPS·네트워크 포화도
- 논리/물리 저장량 증가율과 여유 일수

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 과잉 증설과 장애 위험 사이의 비용을 error budget과 함께 평가한다.
- 리전 간 전송, 검색 색인, 로그 보존은 원본 저장량보다 빠르게 비용이 커질 수 있다.
- 수평 확장은 인스턴스 비용뿐 아니라 재분배·운영 자동화 비용을 만든다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 근거 없는 “피크는 평균의 10배”를 모든 시스템에 적용한다.
- 압축률과 캐시 적중률을 보장값으로 취급한다.
- 스토리지 용량만 보고 IOPS와 재구축 시간을 무시한다.
- 계산 결과를 부하 테스트나 실제 관측으로 갱신하지 않는다.

설계 리뷰

- ☐ 모든 숫자에 단위와 기준 시간이 있는가?
- ☐ 피크·burst·성장률이 평균과 분리됐는가?
- ☐ 복제·색인·로그·백업을 물리량에 포함했는가?
- ☐ 가장 민감한 가정이 무엇인지 설명할 수 있는가?
- ☐ 계산을 검증할 부하 테스트와 관측 항목이 정해졌는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 하루 2천만 건, 평균 1.2KB 이벤트를 90일 보존하고 3중 복제할 때 원본·물리 저장량을 계산하라.
2. 피크 8,000 RPS, 평균 서비스 시간 120ms, p99 900ms인 API의 평균·꼬리 동시성을 비교하라.
3. 캐시 적중률이 95%에서 85%로 떨어질 때 원본 DB 읽기 부하가 몇 배가 되는지 식으로 설명하라.

핵심 요약

- 용량 계산은 가정과 단위를 드러내는 설계 도구다.
- 평균, 피크, burst, 성장률을 분리한다.
- 논리량과 물리량을 구분한다.
- 단일 예측값보다 시나리오와 민감도를 제시한다.
- 계산은 부하 테스트와 운영 데이터로 반복 갱신한다.

출처

- [@upstream-primer] Donne Martin. **The System Design Primer** (2026). <https://github.com/donnemartin/system-design-primer>
- [@dean-tail-at-scale] Jeffrey Dean and Luiz André Barroso. **The Tail at Scale** (2013). <https://research.google/pubs/the-tail-at-scale/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

03

트레이드오프와 Architecture Decision Record

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

아키텍처 결정은 “최고의 기술”을 고르는 일이 아니라 특정 제약에서 어떤 비용을 감수할지 명시하는 일이다. ADR은 결과보다 당시의 맥락과 버린 대안을 보존해 이후 변경을 가능하게 한다.

이 절의 기준 출처: [iso-42010; adr-nygard].

학습 목표

- 설계 선택을 비교 가능한 기준으로 평가한다.
- ADR에 맥락·결정·대안·결과를 기록한다.
- 되돌릴 수 있는 결정과 비가역적 결정을 다르게 관리한다.

먼저 결론

- 평가 기준을 선택지보다 먼저 합의한다.
- 근거가 약한 항목은 점수로 위장하지 말고 불확실성으로 표시한다.
- 되돌리기 어려운 결정은 작은 실험과 탈출 전략을 요구한다.
- 결정 이후 실제 결과를 기록해 ADR을 학습 자산으로 만든다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	트레이드오프와 Architecture Decision Record에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	평가 기준을 선택지보다 먼저 합의한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	자주 바뀌는 구현은 인터페이스 뒤에 두고 데이터 계약은 더 엄격히 관리한다.
실패·복구	“결정 근거 소실” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	ADR에 맥락·대안·반대 의견·재검토 조건을 보존한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	보안·개인정보 결정은 위험 수용자와 만료일을 명시한다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	ADR별 재검토 예정일과 만료 상태

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

트레이드오프

한 속성을 얻기 위해 다른 속성이나 비용을 포기하는 관계다.

결정 드라이버

사업 목표, SLO, 규제, 팀 역량처럼 선택을 실제로 구속하는 기준이다.

ADR

하나의 중요한 결정을 맥락·상태·대안·결과와 함께 남기는 짧은 기록이다.

가역성

낮은 비용으로 되돌리거나 교체할 수 있는 정도다.

옵션 가치

지금 모든 것을 확정하지 않고 미래 선택 가능성을 보존해 얻는 가치다.

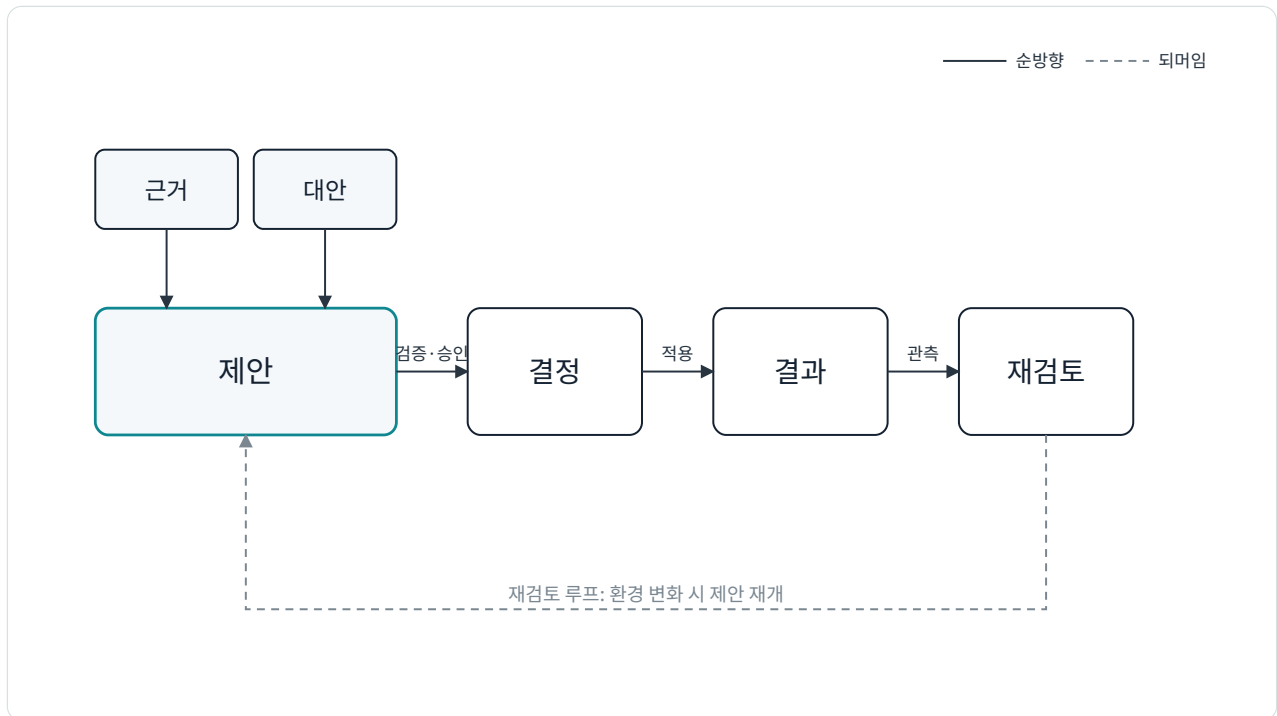
핵심 개념의 정의와 범위는 [iso-42010; adr-nygard]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
결정 제안	문제와 후보 대안을 정의한다.
근거 패킷	측정값, 실험, 표준, 비용 모델을 모은다.

구성 요소	책임
리뷰 그룹	영향받는 개발·운영·보안·제품 관점을 검토한다.
ADR 저장소	결정 번호, 상태, 대안, 결과를 버전 관리한다.
후속 검증	결정이 만든 실제 지표와 재검토 조건을 추적한다.



제안·검증·승인·적용·관측·재검토로 이어지는 ADR 생명주기를 보여준다

요청·데이터 흐름

1. 결정이 필요한 이유와 시한을 적는다.
2. 평가 기준과 가중치 대신 최소 통과 조건을 먼저 정한다.
3. 2~4개 현실적인 대안을 같은 수준으로 조사한다.
4. 실험 가능한 불확실성을 작은 spike로 줄인다.
5. 결정과 부정적 결과까지 ADR에 기록한다.
6. 재검토 트리거와 탈출 경로를 정한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

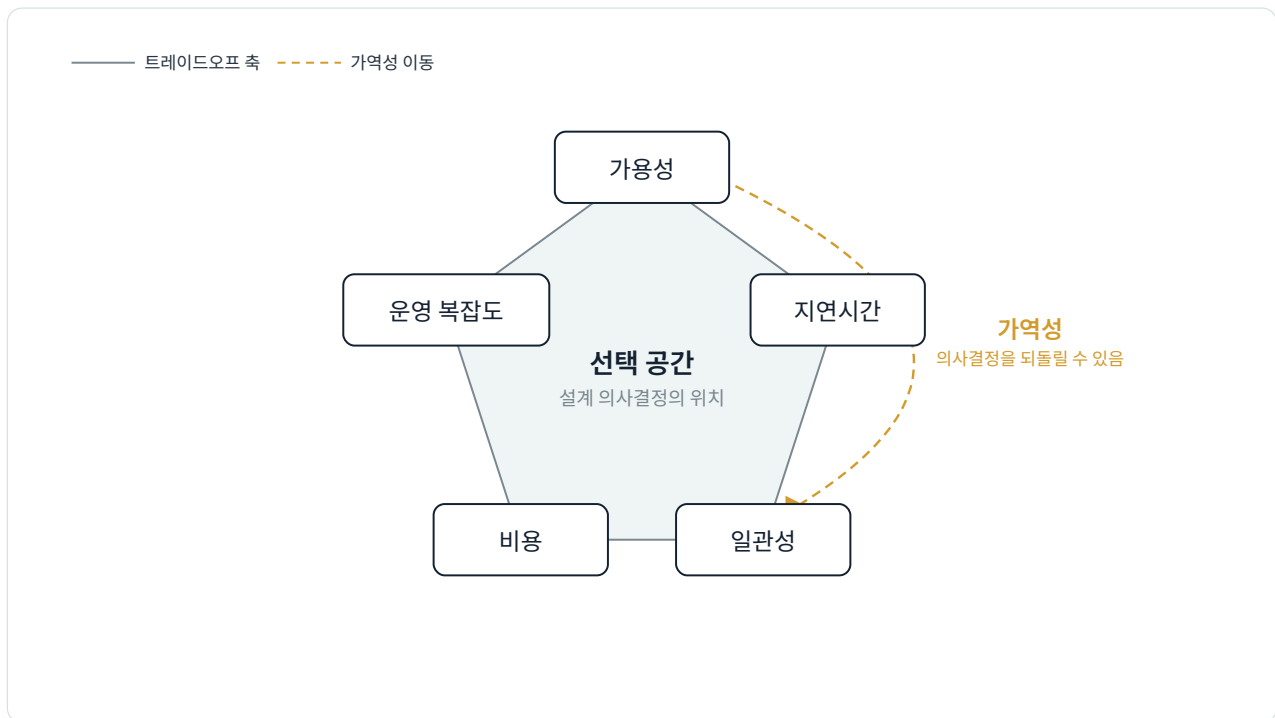
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
정량 점수표	많은 대안을 한눈에 비교할 수 있다.	근거 없는 점수가 정밀함을 가질 수 있다.	측정 가능한 기준이 충분할 때
원칙·계약 기반 선택	핵심 제약 위반 여부가 명확하다.	미묘한 차이를 놓칠 수 있다.	규제·SLO·호환성 같은 강한 제약
시간 제한 실험	실제 위험을 빠르게 확인한다.	실험 환경이 운영 현실을 대표하지 않을 수 있다.	성능·운영성·통합 불확실성

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [iso-42010; adr-nygard]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
결정 근거 소실	몇 달 뒤 제품명만 남고 왜 선택했는지 알 수 없다.	ADR에 맥락·대안·반대 의견·재검토 조건을 보존한다.
점수 조작	원하는 결론에 맞춰 가중치와 점수를 바꾼다.	필수 조건과 측정 근거를 먼저 고정하고 민감도 분석을 한다.
영구 임시방편	시간 압박으로 선택한 대안이 종료 조건 없이 고착된다.	만료일·부채 소유자·제거 신호를 결정에 포함한다.
결정 과잉	작은 구현 세부까지 ADR을 만들어 기록 비용이 커진다.	장기간 영향을 주거나 여러 팀 계약을 바꾸는 결정에 집중한다.



가용성·지연·일관성·비용·운영 복잡도 사이의 선택 공간을 보여준다

확장 전략

- 자주 바뀌는 구현은 인터페이스 뒤에 두고 데이터 계약은 더 엄격히 관리한다.
- 공급자 종속이 큰 선택은 export·migration·dual-run 경로를 검증한다.
- 운영 복잡도를 팀의 on-call 역량과 함께 평가한다.
- 새 대안이 생겨도 재검토 트리거가 없으면 자동으로 바꾸지 않는다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 보안·개인정보 결정은 위험 수용자와 만료일을 명시한다.
- 비밀, 내부 취약점, 개인정보를 ADR 본문에 직접 넣지 않고 보호된 증거 위치를 참조한다.
- 공급망과 데이터 위치가 대안 평가 기준에 포함됐는지 확인한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- ADR별 재검토 예정일과 만료 상태
- 결정 후 기대 지표와 실제 지표 차이
- 예외·우회 구현 수와 기술 부채
- 마이그레이션·롤백 실험 성공률

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 결정 기록 비용은 작지만, 근거가 없는 재논의와 잘못된 마이그레이션 비용을 줄인다.
- 이중 운영과 탈출 경로는 단기 비용이지만 비가역성 위험을 낮춘다.
- 기술 선택보다 팀 교육·운영 자동화가 총비용에 더 큰 영향을 줄 수 있다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- “업계 표준”이라는 말로 맥락 없는 선택을 정당화한다.
- 모든 속성을 1~5점으로 매겨 숫자를 사실처럼 취급한다.
- 결정 이후 실제 결과와 부작용을 ADR에 갱신하지 않는다.
- 도구 선택과 아키텍처 원칙을 같은 수준의 결정으로 취급한다.

설계 리뷰

- ☐ 결정 드라이버가 선택지보다 먼저 정의됐는가?
- ☐ 버린 대안과 버린 이유가 공정하게 기록됐는가?
- ☐ 불확실성을 줄일 실험이 있었는가?
- ☐ 롤백·마이그레이션·재검토 조건이 있는가?
- ☐ 보안·운영·비용 담당자가 결과를 검토했는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 관계형 DB와 분산 KV 저장소 선택을 위한 ADR을 작성하라.
2. 관리형 메시징과 자체 운영 Kafka의 탈출 비용을 비교하라.
3. 이미 채택한 기술 결정 하나를 골라 당시 가정과 현재 결과의 차이를 기록하라.

핵심 요약

- 결정은 맥락과 제약에 상대적이다.
- 평가 기준은 대안보다 먼저 정한다.
- ADR은 결정의 이유와 결과를 보존한다.
- 비가역적 결정은 실험과 탈출 전략이 필요하다.
- 재검토 조건이 없는 기록은 살아 있는 의사결정 체계가 아니다.

출처

- [iso-42010] ISO/IEC/IEEE. **ISO/IEC/IEEE 42010:2022 — Architecture description** (2022). <https://www.iso.org/standard/74393.html>
- [adr-nygard] Michael Nygard. **Documenting Architecture Decisions** (2011). <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>

“

검증 기준일: 2026-08-06. `current` 와 `volatile` 내용은 `release` 전에 공식 원문을 다시 확인한다.

04

SLI·SLO·SLA와 Error Budget

이 장의 구성

- 01 이 장에서 해결할 문제
- 02 먼저 결론
- 03 요구사항과 실패 모델
- 04 핵심 개념
- 05 기준 아키텍처
- 06 요청·데이터 흐름
- 07 대안과 트레이드오프
- 08 장애 시나리오
- 09 확장 전략
- 10 보안과 개인정보
- 11 관측 가능성
- 12 비용과 운영 복잡도
- 13 흔한 오해와 안티패턴
- 14 설계 리뷰
- 15 연습문제
- 16 핵심 요약
- 17 출처

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

“항상 안정적”이라는 요구는 설계 입력이 아니다. 사용자 관점의 신호를 SLI로 측정하고, 관측 창과 목표 비율을 SLO로 정하며, 허용 실패량인 error budget을 변경 속도와 신뢰성 투자에 사용해야 한다.

이 절의 기준 출처: [@google-sre-slo; @google-sre-error-budget].

학습 목표

- 사용자 경험을 나타내는 SLI를 정의한다.
- SLO와 error budget을 계산한다.
- 신뢰성 목표를 배포·투자·장애 대응 결정에 연결한다.

먼저 결론

- SLI는 시스템 내부 CPU보다 사용자가 완료한 결과를 우선한다.
- 분모와 좋은 이벤트의 조건을 명시한다.
- 모든 기능에 같은 SLO를 주지 말고 사용자 여정과 중요도에 따라 분리한다.
- SLA는 외부 약속이고 SLO는 내부 운영 목표이므로 동일시하지 않는다.

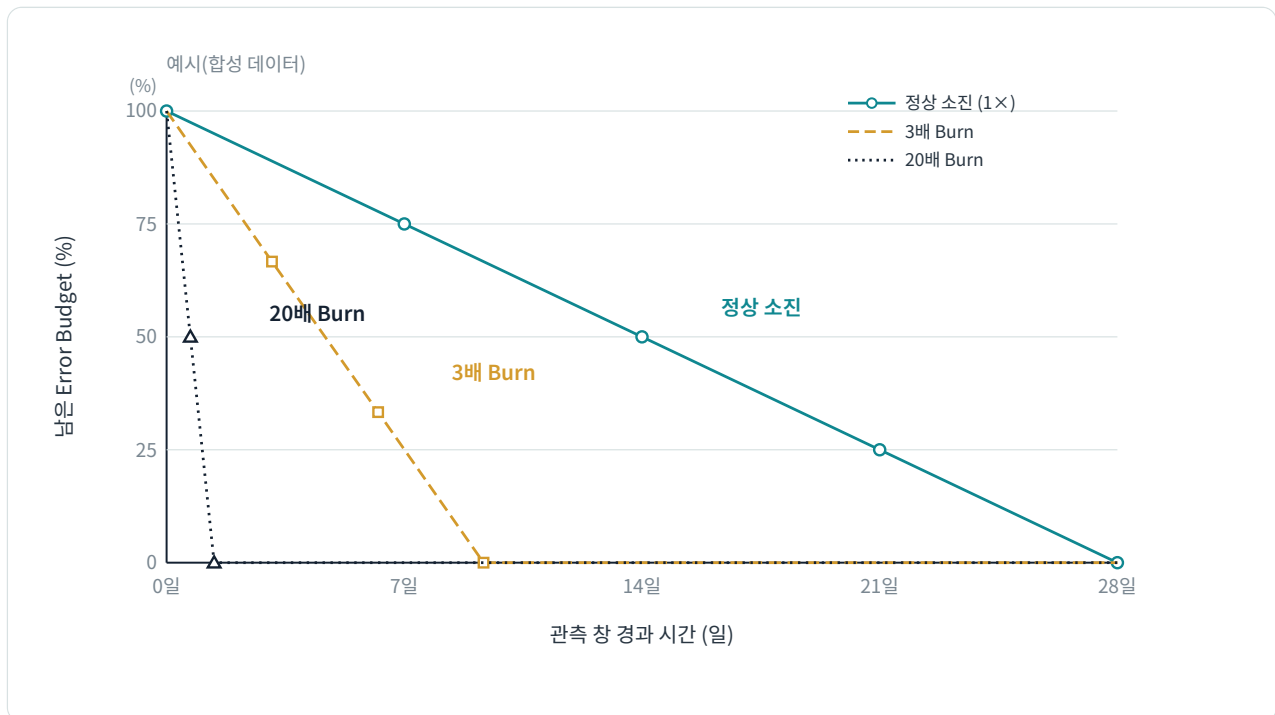
::: current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06 이며, 출판 직전 공식 문서를 다시 확인한다. :::

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	SLI·SLO·SLA와 Error Budget에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	SLI는 시스템 내부 CPU보다 사용자가 완료한 결과를 우선한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	서비스 계층을 나눌 때 의존성 SLO가 사용자 여정 SLO에 어떻게 합성되는지 계산한다.
실패·복구	“계측 공백” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	분모 소스와 독립적인 합성 검사, missing-data 정책을 둔다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	보안 차단과 정상 오류를 SLI에서 구분하되 공격 트래픽을 무조건 삭제하지 않는다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	좋은 이벤트/전체 유효 이벤트 비율

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



Error budget과 burn rate

핵심 개념

SLI

좋은 이벤트의 비율이나 지연 분포처럼 서비스 수준을 측정하는 지표다.

SLO

정해진 관측 창에서 SLI가 달성해야 하는 목표다.

SLA

서비스 제공자와 고객 사이의 약속이며 위반 시 조치가 포함될 수 있다.

Error budget

1 - SLO에 해당하는 허용 실패량이다.

Burn rate

error budget이 기준 속도보다 얼마나 빠르게 소진되는지 나타낸다.

관측 창

최근 28일 같은 rolling window 또는 달력 월처럼 목표를 평가하는 기간이다.

핵심 개념의 정의와 범위는 [@google-sre-slo; @google-sre-error-budget; @google-sre-book]를 기준으로 재검토해야 한다.

Error budget 계산

28일 관측 창에서 가용성 SLO가 99.9% 라면 허용 실패 비율은 $0.1\% = 0.001$ 이다.

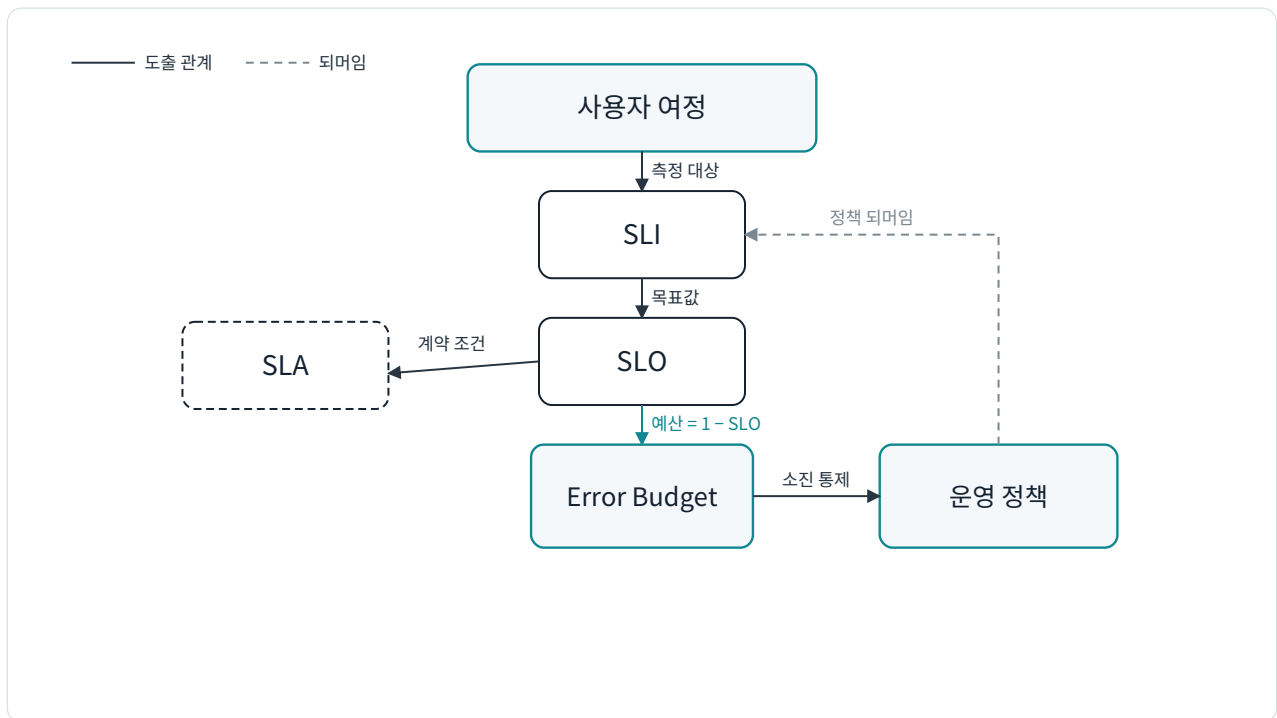
```
28 days × 24 h/day × 60 min/h × 0.001
= 40.32 minutes
```

이 값은 “40분 19.2초 동안 마음대로 장애가 나도 된다”는 뜻이 아니다. 요청 비율 SLI라면 허용 실패 요청 수로 계산해야 하고, 짧은 시간에 예산을 집중 소진하면 사용자의 실제 피해가 더 클 수 있다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
이벤트 계측	요청·작업·사용자 여정의 성공, 지연, 실패도를 기록한다.
SLI 집계	좋은 이벤트와 전체 유효 이벤트를 같은 정의로 집계한다.
SLO 평가	관측 창, 목표, 제외 규칙을 적용한다.
Burn-rate 경보	빠른 소진과 느린 소진을 서로 다른 창으로 감지한다.
정책 연결	배포 중단, 안정화 작업, 용량 투자, 사후 분석을 결정한다.



사용자 여정에서 SLI·SLO·SLA·error budget 정책으로 내려가는 관계를 보여준다

요청·데이터 흐름

1. 사용자가 가치 있는 결과를 얻는 여정을 고른다.
2. 좋은 이벤트와 유효한 전체 이벤트를 정의한다.
3. 계측 누락·봇·의도적 차단 같은 제외 규칙을 명시한다.
4. 달성 가능한 기준선과 사업 기대를 비교해 SLO를 정한다.
5. 빠른/느린 burn-rate 경보를 구성한다.
6. error budget 정책을 배포와 우선순위에 연결한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

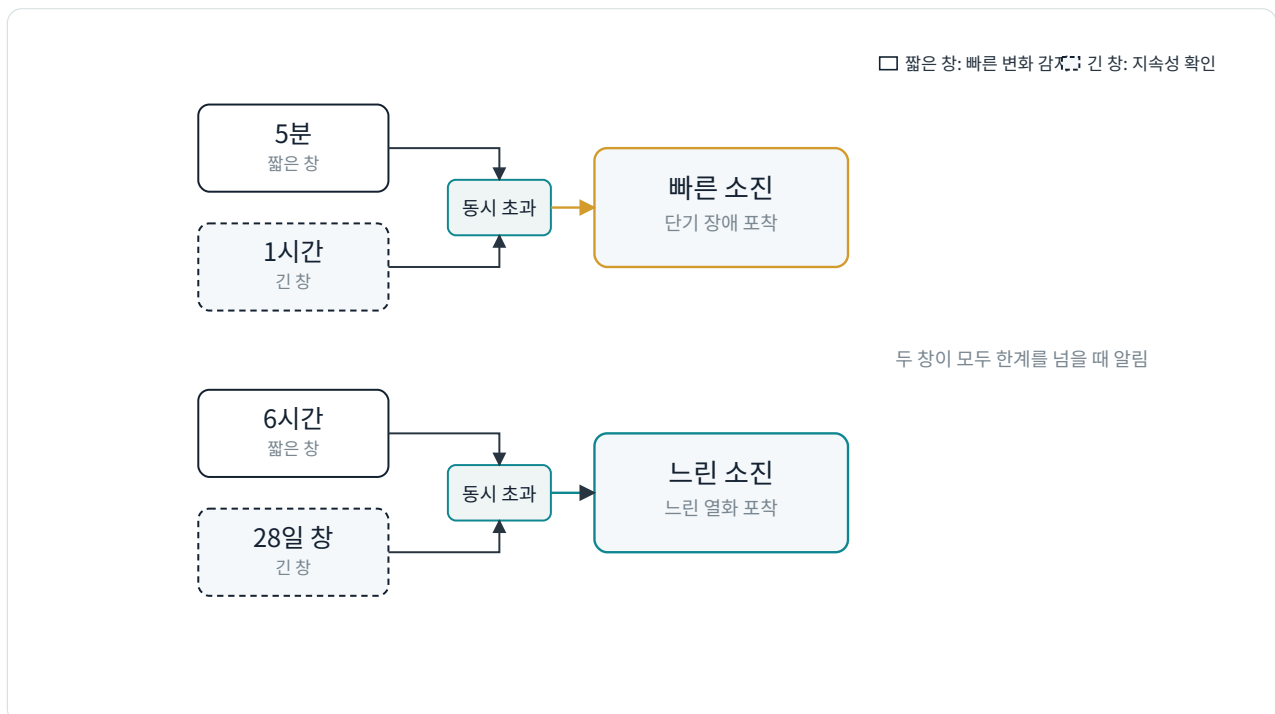
대안	장점	비용·위험	적합한 조건
요청 기반 SLI	계산과 설명이 단순하다.	다단계 비동기 여정의 완료율 놓칠 수 있다.	동기 API
사용자 여정 SLI	실제 가치 전달을 잘 반영한다.	상관관계와 지연된 완료 추적 어렵다.	주문·결제·업로드

대안	장점	비용·위험	적합한 조건
창 기반 가용성	긴 장애 시간을 직관적으로 보여준다.	짧은 고빈도 오류의 사용자 영향을 왜곡할 수 있다.	전통적 인프라 SLA

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [@google-sre-slo; @google-sre-error-budget; @google-sre-book]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
계측 공백	장애 중 telemetry도 사라져 성공처럼 보인다.	분모 소스와 독립적인 합성 검사, missing-data 정책을 둔다.
잘못된 분모	클라이언트 취소나 차단 요청을 임의로 제외해 목표가 부풀려진다.	제외 규칙을 코드·문서·리뷰로 관리한다.
SLO 과다	세부 엔드포인트마다 SLO를 만들어 운영자가 신호를 해석하지 못한다.	핵심 사용자 여정과 대표 서비스 지표로 제한한다.
예산 무시	error budget을 초과해도 배포 속도가 바뀌지 않는다.	사전에 합의한 정책으로 안정화와 출시 결정을 연결한다.



짧은 창과 긴 창의 burn rate가 빠른 장애와 느린 열화를 포착하는 방식을 보여준다

확장 전략

- 서비스 계층을 나눌 때 의존성 SLO가 사용자 여정 SLO에 어떻게 합성되는지 계산한다.
- 고객군·지역·기능별로 분리하되 전체 집계가 심각한 하위 집단을 숨기지 않게 한다.
- 트래픽이 적은 서비스는 이벤트 비율만으로 경보하지 않고 합성 검사와 시간 기반 조건을 함께 쓴다.
- 새 기능은 초기 기준선 관측 후 SLO를 조정하되 목표를 사후에 낮춰 실패를 숨기지 않는다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 보안 차단과 정상 오류를 SLI에서 구분하되 공격 트래픽을 무조건 삭제하지 않는다.
- 고객별 SLI를 만들 때 개인 식별 가능성을 줄이고 보존 기간을 제한한다.
- SLA 문구와 실제 계측 정의가 모순되지 않도록 법무·제품·운영이 공동 검토한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 좋은 이벤트/전체 유효 이벤트 비율
- 지연 임계값별 성공률
- 28일·1시간·5분 burn rate
- 계측 누락률과 합성 검사 성공률

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- SLO가 높아질수록 중복 인프라·운영 인력·변경 통제 비용이 비선형적으로 증가할 수 있다.
- 낮은 중요도의 기능을 핵심 경로와 분리하면 전체 신뢰성 비용을 줄일 수 있다.
- error budget은 장애 비용과 출시 지연 비용을 같은 언어로 논의하게 한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 모든 서비스에 99.99%를 복사한다.
- 서버 가동률을 사용자 성공률로 간주한다.
- 평균 지연만 SLI로 사용한다.
- SLO 위반을 평가와 처벌 도구로 사용해 계속 왜곡을 유도한다.

설계 리뷰

- ☐ SLI가 사용자 가치와 직접 연결되는가?
- ☐ 좋은 이벤트와 분모·제외 규칙이 명확한가?
- ☐ 관측 창과 목표 선택 근거가 있는가?
- ☐ 빠른 장애와 느린 품질 저하를 모두 감지하는가?
- ☐ error budget 초과 시 실제 행동 정책이 있는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 99.9% SLO의 30일 error budget을 분과 초로 계산하라.
2. 주문 생성은 성공했지만 배송 지시가 10분 지연되는 시스템의 여정 SLI를 정의하라.
3. 1시간 동안 20배 burn rate와 6시간 동안 3배 burn rate를 어떻게 다르게 대응할지 정책을 작성하라.

핵심 요약

- SLI는 사용자 관점의 측정값이다.
- SLO는 목표와 관측 창을 함께 정의한다.
- SLA는 외부 약속이며 SLO와 목적이 다르다.
- error budget은 허용 실패를 변경 정책에 연결한다.
- 계측 공백과 잘못된 분모도 신뢰성 위험이다.

출처

- [@google-sre-slo] Google. **SRE Workbook — Implementing SLOs** (2018). <https://sre.google/workbook/implementing-slos/>

-
- [@google-sre-error-budget] Google. **SRE Workbook — Error Budget Policy** (2018). <https://sre.google/workbook/error-budget-policy/>
 - [@google-sre-book] Google. **Site Reliability Engineering** (2016). <https://sre.google/sre-book/table-of-contents/>

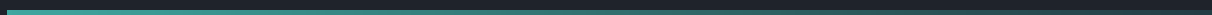
“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

PART

2

분산 시스템의 기본 원리



시간, 실패, 복제, 일관성, 합의처럼 분산 시스템을 어렵게 만드는 조건을 다룬다.

이 Part에서는 특정 제품의 설정정보보다 반복해서 적용할 수 있는 설계 질문과 실패 모델을 먼저 익힌다. 각 장의 체크리스트를 실제 시스템의 ADR·런북·대시보드와 연결해 읽는 것이 목표다.

포함 장

ID	장 제목	최신성	원본 관계
ch05	지연시간·처리량·동시성과 Tail Latency	durable	REWRITE
ch06	가용성·신뢰성·내구성과 장애 도메인	durable	REWRITE
ch07	CAP를 넘어선 일관성 모델	durable	REWRITE
ch08	트랜잭션·격리 수준·MVCC	durable	ADD
ch09	시간·순서·논리 시계·분산 ID	durable	ADD
ch10	복제·Quorum·Failover	durable	REPLACE
ch11	파티셔닝 ·Sharding·Consistent Hashing	durable	REWRITE
ch12	Consensus·Leader Election·Fencing	durable	ADD

읽는 방법

1. 먼저 각 장의 **요구사항과 실패 모델**을 자신의 시스템에 대입한다.
2. **대안과 트레이드오프**에서 현재 선택이 어떤 비용을 감수하는지 확인한다.
3. **장애 시나리오**를 게임데이 또는 설계 리뷰 질문으로 바꾼다.
4. 시각자료는 설명을 대신하지 않고 책임·흐름·복구 경계를 검증하는 용도로 사용한다.

05

지연시간·처리량·동시성과 Tail Latency

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

평균 지연시간은 사용자의 나쁜 경험을 숨길 수 있다. 여러 하위 호출을 병렬로 수행하는 서비스에서는 각 호출의 작은 꼬리가 전체 요청의 꼬리로 증폭되므로 p95·p99와 큐잉 상태를 설계 입력으로 사용해야 한다.

이 절의 기준 출처: [@dean-tail-at-scale; @google-sre-overload].

학습 목표

- 지연시간 분포와 처리량을 함께 해석한다.
- 큐잉과 fan-out이 tail latency를 증폭하는 이유를 설명한다.
- timeout·용량·복제 전략을 백분위 지표로 설계한다.

먼저 결론

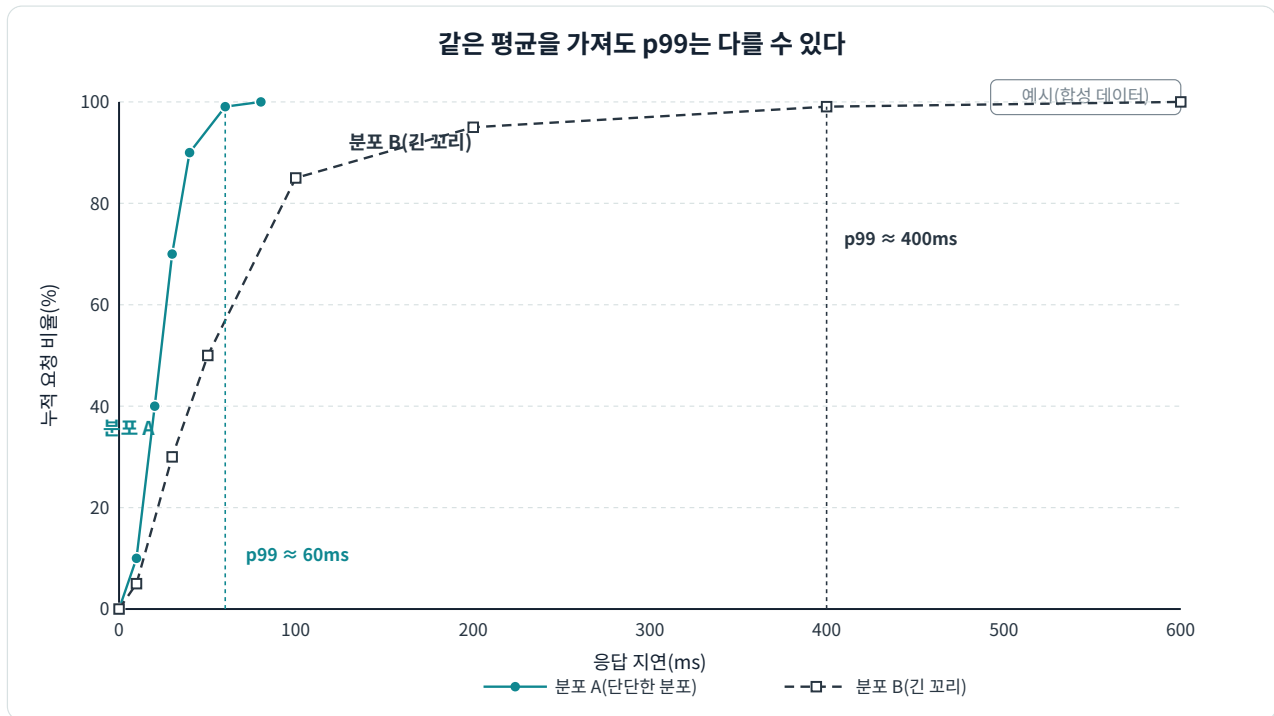
- 지연시간과 처리량은 같은 숫자가 아니며 부하가 포화점에 가까워지면 함께 악화될 수 있다.
- 평균뿐 아니라 p50·p95·p99와 timeout 비율을 본다.
- fan-out 수가 커질수록 하나 이상의 느린 하위 호출을 만날 확률이 증가한다.
- 여유 용량, 요청 축소, hedging은 비용과 중복 부작용을 함께 평가한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	지연시간·처리량·동시성과 Tail Latency에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	지연시간과 처리량은 같은 숫자가 아니며 부하가 포화점에 가까워지면 함께 악화될 수 있다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	포화점보다 낮은 목표 사용률을 유지해 burst와 장애 전환 여유를 둔다.
실패·복구	“큐 폭증” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	유한 큐, admission control, backpressure를 적용한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	고객 우선순위가 권한 우위로 악용되지 않게 서버가 정책을 결정한다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	단계별 p50·p95·p99·p99.9 지연

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



핵심 개념

지연시간

작업 하나가 시작부터 완료까지 걸린 시간이다.

처리량

단위 시간에 완료한 작업 수다.

동시성

같은 시점에 진행 중인 작업 수다.

Tail latency

분포의 상위 백분위에서 나타나는 느린 응답이다.

큐잉 지연

자원이 바빠 실제 처리를 시작하기 전 기다리는 시간이다.

Fan-out 증폭

하나의 요청이 여러 하위 요청 중 가장 느린 결과를 기다리며 꼬리가 커지는 현상이다.

핵심 개념의 정의와 범위는 [dean-tail-at-scale; @google-sre-overload; @google-sre-book]를 기준으로 재검토해야 한다.

간단한 fan-out 계산

하위 호출 하나가 목표 지연 안에 끝날 확률이 0.99 이고, 서로 독립이라고 단순 가정하자. 20개 호출이 모두 목표 안에 끝날 확률은 다음과 같다.

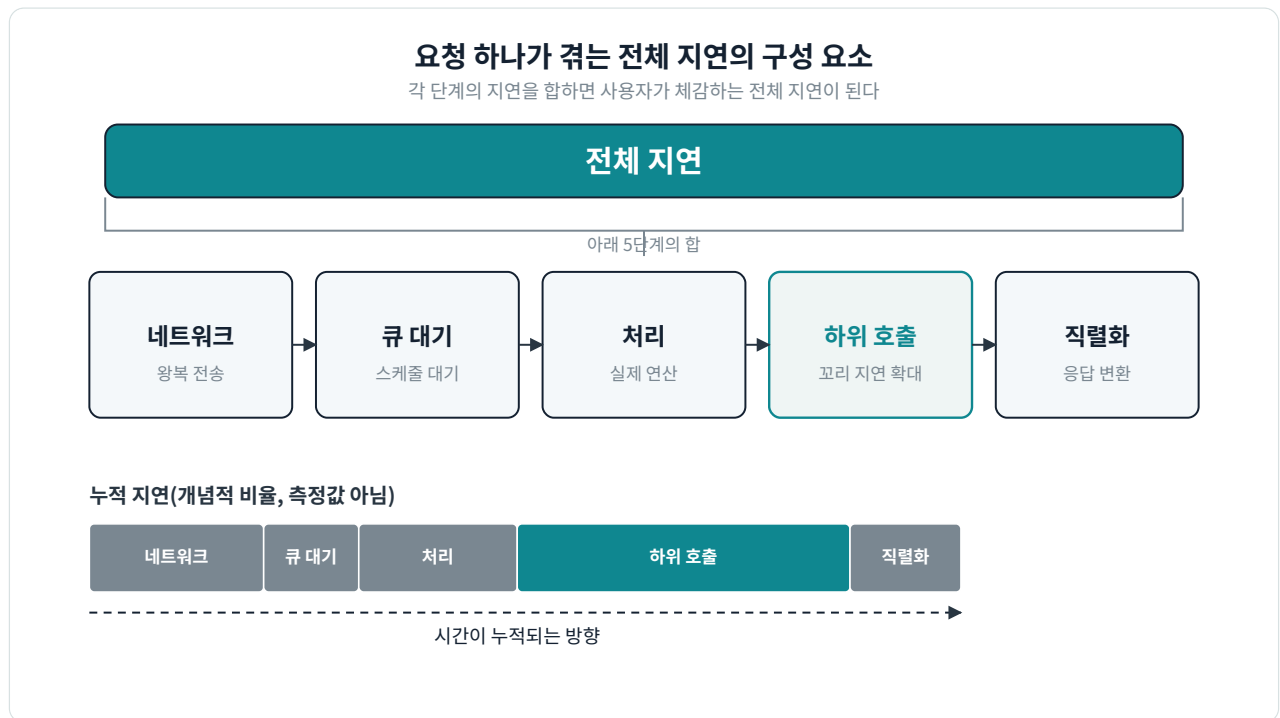
$$0.99^{20} \approx 0.8179$$

즉 약 18.2%의 상위 요청은 적어도 하나의 느린 하위 호출을 만난다. 실제 호출은 독립이 아닐 수 있고 공유 자원 때문에 함께 느려질 수 있으므로 이 계산은 하한·상한이 아니라 현상을 이해하기 위한 단순 모델이다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Ingress	요청을 받아 admission control과 deadline을 적용한다.
작업 큐	대기 작업과 우선순위를 관리한다.
Worker pool	제한된 동시성으로 실제 처리를 수행한다.
하위 의존성	DB·캐시·외부 API 호출을 제공한다.
분포 집계기	구간별 latency histogram과 timeout을 기록한다.
과부하 제어	큐 제한, load shedding, degradation을 수행한다.



전체 지연을 네트워크·큐 대기·처리·하위 호출·직렬화로 분해한다

요청·데이터 흐름

1. 클라이언트가 전체 deadline을 포함해 요청한다.
2. 입구에서 요청 크기·우선순위·현재 부하를 검사한다.
3. 큐에서 기다린 시간을 별도 기록한다.
4. 남은 deadline을 하위 호출에 분배한다.
5. 병렬 호출 중 필수·선택 결과를 구분한다.
6. 응답 후 전체 및 단계별 지연 분포를 기록한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
대기열 확장	짧은 burst를 흡수하고 손실을 줄인다.	오래된 요청이 쌓여 지연과 메모리가 폭증한다.	burst가 짧고 작업 가치가 유지될 때
동시성 확대	처리량을 늘릴 수 있다.	DB 연결·CPU 경쟁으로 오히려 tail이 악화될 수 있다.	하위 자원에 여유가 있을 때

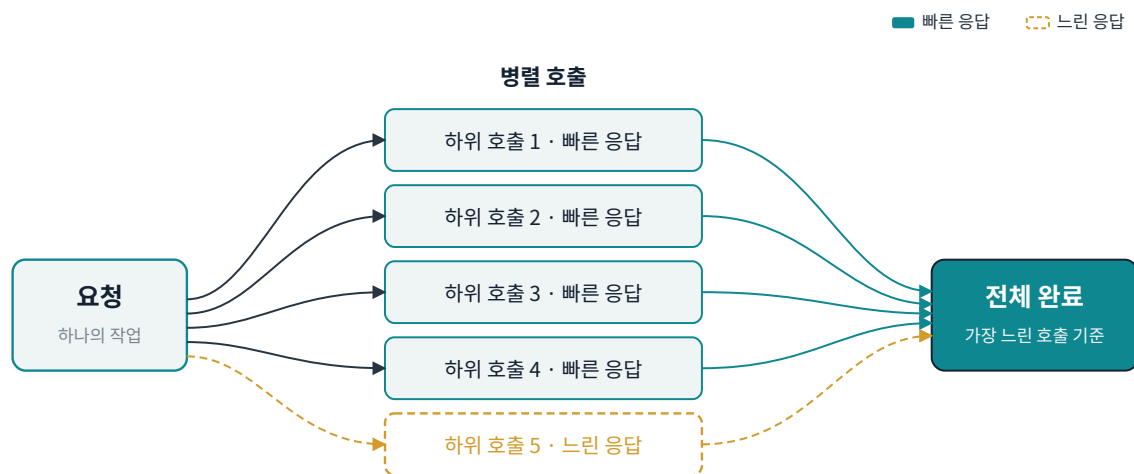
대안	장점	비용·위험	적합한 조건
요청 축소/거부	핵심 요청의 지연을 보호한다.	일부 기능 품질이나 성공률을 포기한다.	포화 상태와 우선순위가 명확할 때

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [dean-tail-at-scale; google-sre-overload; google-sre-book]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
큐 폭증	도착률이 처리율을 넘으며 대기 시간이 deadline을 초과한다.	유한 큐, admission control, backpressure를 적용한다.
느린 하위 의존성	한 DB shard나 외부 API가 p99를 지배한다.	단계별 deadline, 격리 pool, fallback을 둔다.
재시도 폭풍	timeout이 재시도를 만들고 추가 부하가 더 많은 timeout을 만든다.	재시도 예산, jitter, 멍등성, 서버 힌트를 사용한다.
GC·스케줄링 정지	짧은 정지가 일부 요청에 긴 꼬리로 나타난다.	런타임 pause와 CPU throttling을 요청 trace와 상관 분석한다.
Coordinated omission	부하 생성기가 느린 동안 새 요청을 보내지 않아 지연을 낮게 측정한다.	고정 도착률을 보존하는 부하 모델과 원시 분포를 사용한다.

fan-out이 많아질수록 느린 호출을 한 번이라도 만날 확률이 커진다



fan-out ↑ → 느린 호출 만날 확률 ↑

단일 호출이 정상일 확률이 높아도, 여러 개를 동시에 부르면 하나는 느리다.

fan-out 개수가 증가할수록 하나 이상의 느린 하위 호출을 만날 확률이 커지는 모습을 보여준다

확장 전략

- 포화점보다 낮은 목표 사용률을 유지해 burst와 장애 전환 여유를 둔다.
- hot key·large request·slow tenant를 별도 차원으로 분해한다.
- fan-out 단계에서 부분 결과와 quorum 완료 조건을 사용한다.
- hedged request는 취소·중복 비용과 함께 제한적으로 적용한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 고객 우선순위가 권한 우회로 악용되지 않게 서버가 정책을 결정한다.
- 지연 로그의 URL·쿼리·trace attribute에서 개인정보를 제거한다.
- load shedding이 특정 사용자군을 지속적으로 차별하지 않는지 분석한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 단계별 p50·p95·p99·p99.9 지연
- 큐 대기 시간과 queue depth
- 동시성·CPU throttling·GC pause
- timeout·취소·재시도·shed 비율
- fan-out 개수별 전체 요청 지연

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- tail을 낮추기 위한 여유 용량은 직접 비용을 만든다.
- hedging과 복제 읽기는 지연을 낮추지만 하위 시스템 부하를 증가시킨다.
- 더 긴 timeout은 실패율을 낮춰 보이지만 자원 점유와 사용자 대기 비용을 키운다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 평균 응답 시간이 좋으니 성능 문제가 없다고 결론낸다.
- 큐 크기를 늘리면 처리량도 늘어난다고 생각한다.
- 클라이언트 timeout보다 긴 서버 작업을 계속 수행한다.
- p99를 샘플 몇 개의 최대값처럼 해석한다.

설계 리뷰

- ☐ 사용자 SLO에 대응하는 백분위가 선택됐는가?
- ☐ 큐 대기과 실제 처리 시간이 분리됐는가?
- ☐ fan-out과 재시도가 하위 부하에 미치는 영향이 계산됐는가?
- ☐ 과부하 시 버릴 기능과 보호할 기능이 정해졌는가?
- ☐ 부하 테스트가 coordinated omission을 피하는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 하위 호출 20개 각각이 99% 확률로 빠를 때 전체 요청이 모두 빠를 확률을 계산하라.
2. 큐가 비어 있을 때와 1초치 작업이 쌓였을 때 같은 timeout 정책의 차이를 설명하라.
3. 검색 결과의 정확도 일부를 포기해 p99를 보호하는 degradation 단계를 설계하라.

핵심 요약

- 평균은 tail latency를 설명하지 못한다.
- 포화점 근처에서는 큐잉이 지연을 급격히 키운다.
- fan-out은 작은 하위 꼬리를 전체 꼬리로 증폭한다.
- deadline·유한 큐·부하 제어를 함께 설계한다.
- 성능 측정 자체의 편향도 검증해야 한다.

출처

- [@dean-tail-at-scale] Jeffrey Dean and Luiz André Barroso. **The Tail at Scale** (2013). <https://research.google/pubs/the-tail-at-scale/>
- [@google-sre-overload] Google. **Site Reliability Engineering — Handling Overload** (2016). <https://sre.google/sre-book/handling-overload/>
- [@google-sre-book] Google. **Site Reliability Engineering** (2016). <https://sre.google/sre-book/table-of-contents/>

“

검증 기준일: 2026-08-06. `current` 와 `volatile` 내용은 `release` 전에 공식 원문을 다시 확인한다.

06

가용성·신뢰성·내구성과 장애 도메인

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

서버를 두 대 배치했다고고가용성이 되는 것은 아니다. 두 복제본이 같은 전원, 제어면, 자격증명, 배포 파이프라인, 데이터 손상 경로를 공유하면 하나의 장애가 동시에 둘을 무너뜨릴 수 있다.

이 절의 기준 출처: [@google-sre-book; @nist-contingency].

학습 목표

- 가용성·신뢰성·내구성을 구분한다.
- 장애 도메인과 공통 원인 실패를 식별한다.
- 중복 구성의 실제 독립성과 복구 능력을 검증한다.

먼저 결론

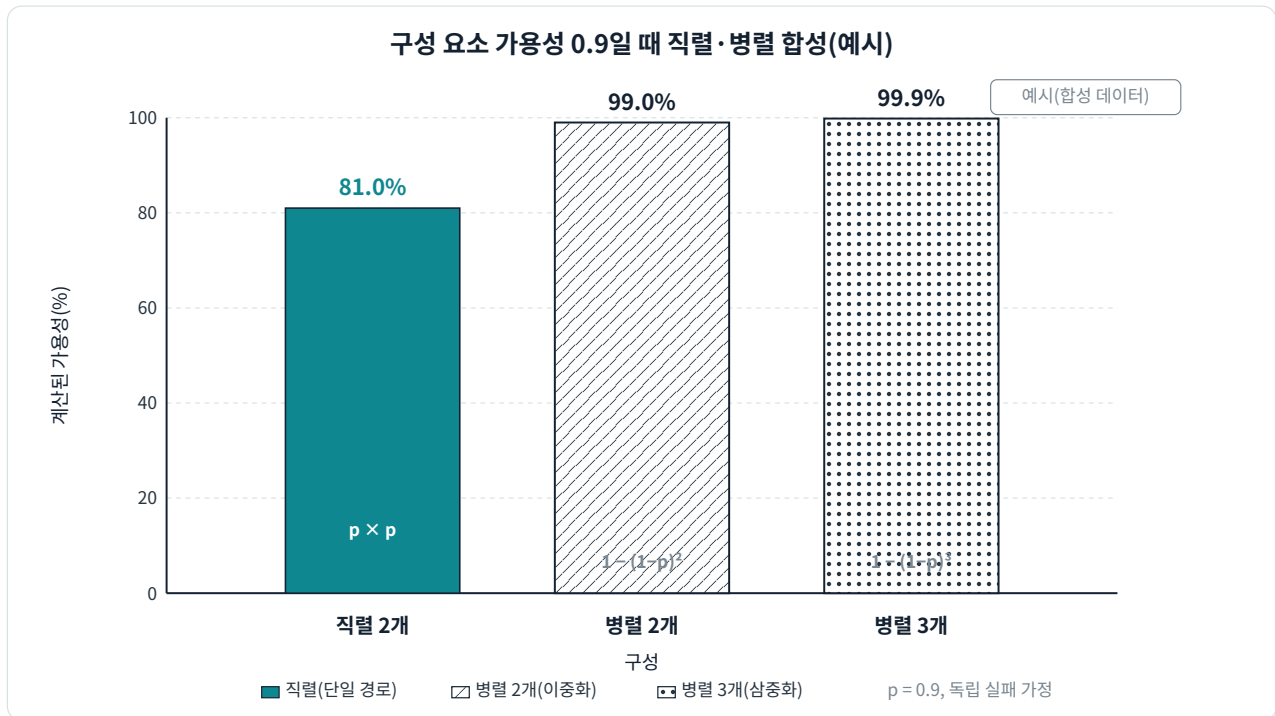
- 가용성은 요청 시 서비스가 유용한 결과를 제공하는 비율이다.
- 신뢰성은 일정 기간 올바르게 동작할 가능성과 실패 특성을 포함한다.
- 내구성은 이미 승인된 데이터가 장기간 보존될 가능성이다.
- 중복은 독립된 장애 도메인과 검증된 failover가 있을 때만 효과가 있다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	가용성·신뢰성·내구성과 장애 도메인에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	가용성은 요청 시 서비스가 유용한 결과를 제공하는 비율이다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	장애 도메인별로 최소 필요 복제본과 quorum을 계산한다.
실패·복구	“영역 장애” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	복제본과 quorum을 독립 zone에 배치한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	비상 계정과 복구 키는 평상시 계정과 독립적으로 보호하고 사용을 감사한다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	사용자 여정 가용성과 오류 예산

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



직렬·병렬 구성의 단순 가용성

핵심 개념

가용성

필요한 시점에 서비스를 사용할 수 있는 정도다.

신뢰성

요구된 조건에서 올바르게 동작하는 성질이다.

내구성

기록된 데이터가 손실되지 않고 보존되는 성질이다.

장애 도메인

한 원인으로 함께 실패할 수 있는 자원의 묶음이다.

공통 원인 실패

중복된 구성 요소가 공유 의존성이나 같은 결함으로 동시에 실패하는 현상이다.

MTTR

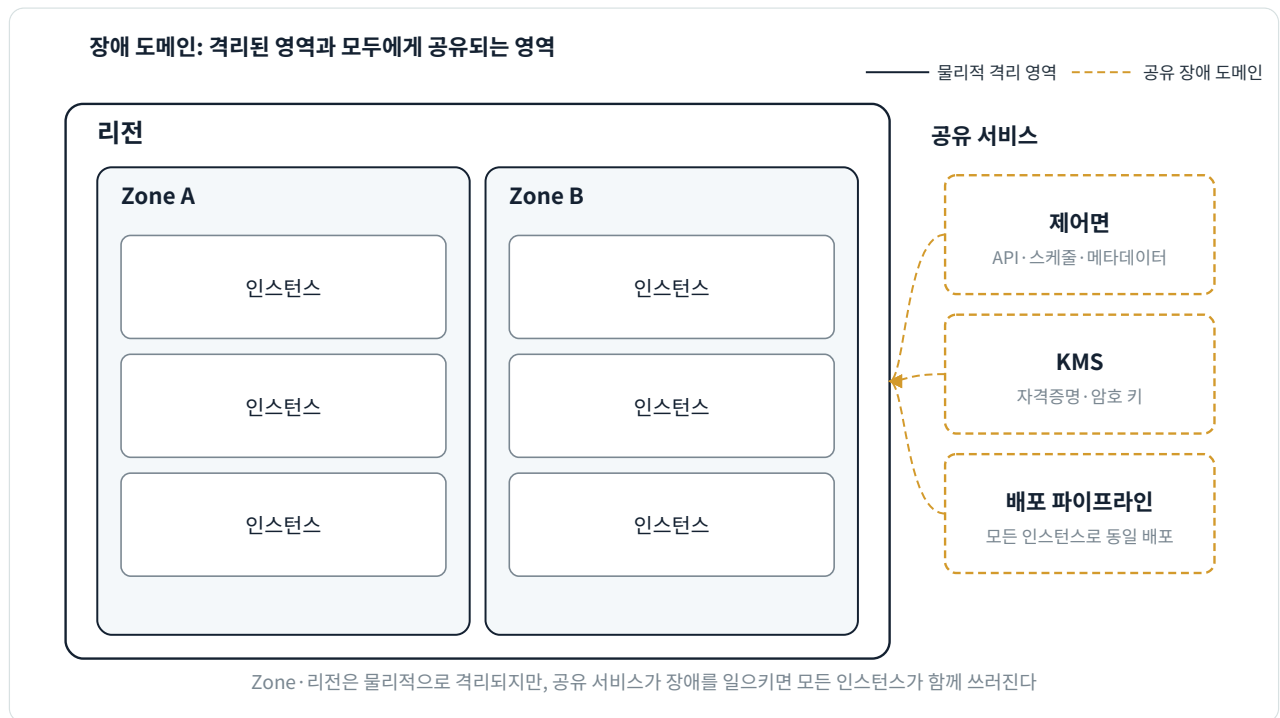
장애 후 정상 서비스로 복구하는 데 걸리는 평균 시간이며 분포와 단계별 시간도 함께 봐야 한다.

핵심 개념의 정의와 범위는 [a@google-sre-book; @nist-contingency]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
서비스 복제본	요청 처리를 여러 인스턴스로 분산한다.
상태 저장 계층	데이터 복제와 복구 가능한 원장을 제공한다.
트래픽 전환기	건강 상태와 정책에 따라 우회한다.
제어면	배포·구성·인증·DNS를 관리한다.
백업 저장소	운영 복제와 독립된 복구 지점을 보존한다.
복구 오케스트레이션	failover, restore, 검증, failback 절차를 자동화한다.



인스턴스·zone·리전·제어면·자격증명·배포 경로의 공유 장애 도메인을 보여준다

요청·데이터 흐름

1. 각 사용자 여정의 의존성 그래프를 그린다.
2. 노드마다 공유 장애 도메인을 태깅한다.
3. 단일 장애가 여정에 미치는 영향을 평가한다.
4. failover가 필요한 상태와 데이터 손실 범위를 정한다.
5. 백업 복구로만 해결되는 손상 시나리오를 분리한다.
6. 게임데이에서 탐지·의사결정·복구 시간을 측정한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

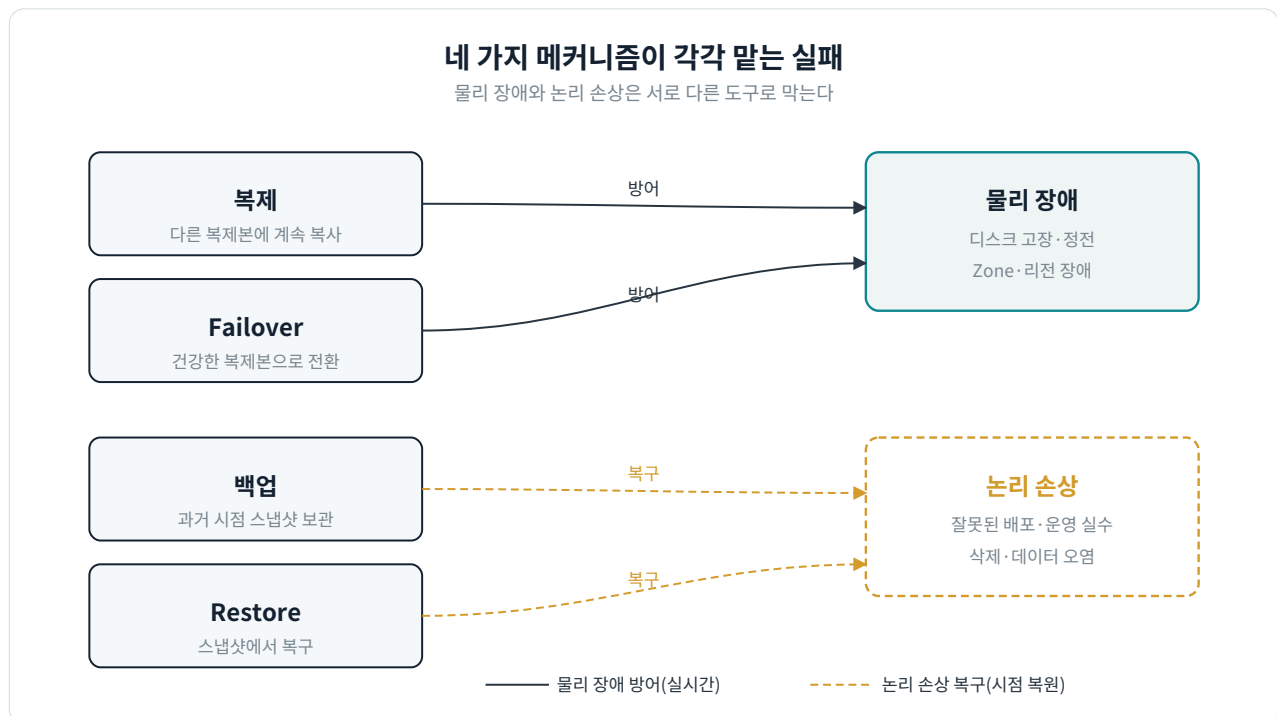
대안	장점	비용·위험	적합한 조건
Active/Standby	상태와 쓰기 경로가 단순하고 충돌이 적다.	대기 자원 비용과 승격 시간이 필요하다.	강한 쓰기 소유권이 필요한 시스템
Active/Active	지역별 지연과 일부 장애 격리가 좋다.	충돌·순서·데이터 정책이 복잡하다.	독립 분할 가능한 워크로드

대안	장점	비용·위험	적합한 조건
복구 중심 단일 운영	평상시 비용이 낮다.	RTO 동안 서비스를 제공하지 못한다.	낮은 중요도 또는 긴 RTO 허용

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [a@google-sre-book; @nist-contingency]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
영역 장애	같은 zone의 앱·DB·캐시가 동시에 중단된다.	복제본과 quorum을 독립 zone에 배치한다.
잘못된 배포	모든 복제본에 같은 결함이 동시에 배포된다.	점진 배포, 이전 버전 유지, 자동 rollback을 사용한다.
자격증명 만료	여러 리전에 복제했지만 공통 인증서·KMS가 실패한다.	만료 감시, 다중 경로, 비상 접근 절차를 검증한다.
논리적 데이터 손상	복제가 삭제·오염을 빠르게 전파한다.	불변 백업, point-in-time recovery, 복구 리허설을 둔다.
제어면 장애	데이터면은 살아 있지만 설정 변경이나 신규 배포가 불가능하다.	데이터면의 마지막 정상 설정 유지와 수동 절차를 설계한다.



복제·failover·백업·restore가 서로 다른 실패를 담당하는 관계를 보여준다

확장 전략

- 장애 도메인별로 최소 필요 복제본과 quorum을 계산한다.
- 읽기 전용·기능 축소 모드를 통해 전체 중단 대신 제한된 서비스를 제공한다.
- 복구 작업도 정상 트래픽과 자원을 경쟁하므로 복구 처리량을 용량 계획에 포함한다.
- failover 후 원래 위치로 돌아가는 failback과 데이터 재동기화까지 설계한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 비상 계정과 복구 키는 평상시 계정과 독립적으로 보호하고 사용을 감사한다.
- 백업에는 운영 데이터와 같은 암호화·보존·삭제 정책을 적용한다.
- 장애 대응 중 보안 통제를 무조건 해제하지 않고 제한된 break-glass 절차를 사용한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 사용자 여정 가용성과 오류 예산
- 장애 도메인별 건강·용량·복제 지연
- 탐지·승인·전환·복구·검증 단계 시간
- 백업 성공률보다 실제 restore 성공률
- 공통 의존성 오류와 제어면 가용성

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 독립 리전·zone·계정은 비용을 늘리지만 공통 원인 위험을 낮춘다.
- 대기 자원은 보험 비용이며 RTO·SLO와 연결해 정한다.
- 복구 자동화와 리허설 비용을 빼면 중복 인프라가 있어도 복구 가능성을 증명할 수 없다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 복제본 수를 가용성과 동일시한다.
- 같은 배포·DNS·KMS를 공유하면서 다중 리전이라고 안심한다.
- 백업 성공 로그만 보고 restore를 시험하지 않는다.
- failover만 설계하고 failback과 재동기화를 생략한다.

설계 리뷰

- ☐ 각 중복 구성의 실제 장애 도메인이 다른가?
- ☐ 제어면과 데이터면 실패가 분리돼 있는가?
- ☐ 논리 손상과 물리 손상의 복구 경로가 다른가?
- ☐ RTO·RPO가 기술 구성과 리허설 결과로 입증되는가?
- ☐ 복구 중 보안·감사 통제가 유지되는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 앱 3대와 DB 3대가 모두 같은 zone에 있을 때 어떤 실패를 견디지 못하는지 나열하라.
2. 99.9% 구성요소 두 개가 직렬·병렬로 연결될 때 단순 독립 가정의 가용성을 계산하라.
3. 잘못된 스키마 마이그레이션이 두 리전에 전파된 상황의 복구 절차를 설계하라.

핵심 요약

- 가용성·신뢰성·내구성은 서로 다른 목표다.
- 중복보다 장애 도메인의 독립성이 중요하다.
- 복제는 논리 손상으로부터 보호하지 못한다.
- failover·restore·failback을 모두 검증한다.
- 복구 능력은 리허설 증거로 판단한다.

출처

- [@google-sre-book] Google. **Site Reliability Engineering** (2016). <https://sre.google/sre-book/table-of-contents/>
- [@nist-contingency] NIST. **NIST SP 800-34 Rev. 1 — Contingency Planning Guide for Federal Information Systems** (2010). <https://csrc.nist.gov/pubs/sp/800/34/r1/final>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

07

CAP를 넘어서는 일관성 모델

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

CAP는 “세 가지 중 두 개를 고르는 제품 분류표”가 아니다. 네트워크 분할 중에도 모든 요청에 응답할지, 하나의 원자적 최신 값처럼 보이게 할지의 충돌을 설명한다. 실제 설계에서는 정상 상태 지연, 세션 보장, 충돌 해결, 격리 수준까지 별도로 선택해야 한다.

이 절의 기준 출처: [@gilbert-lynch-cap; @vogels-eventual].

학습 목표

- CAP가 다루는 조건과 다루지 않는 조건을 설명한다.
- 선형화 가능성·인과·최종 일관성을 사용자 경험과 연결한다.
- 읽기·쓰기 경로별 일관성 요구를 선택한다.

먼저 결론

- 분할 허용성은 선택 옵션이 아니라 분산 네트워크가 실패할 수 있다는 조건이다.
- 강한 일관성은 모든 데이터와 모든 연산에 동일하게 적용할 필요가 없다.
- 사용자에게 필요한 read-your-writes와 monotonic reads를 먼저 명시한다.
- 충돌을 허용하면 병합 규칙과 의미적 불변조건을 설계해야 한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	CAP를 넘어서는 일관성 모델에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	분할 허용성은 선택 옵션이 아니라 분산 네트워크가 실패할 수 있다는 조건이다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	키·tenant·지역별로 일관성 수준을 다르게 적용하되 API 의미를 명확히 한다.
실패·복구	“네트워크 분할” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	쓰기 소유권을 제한하거나 충돌을 의미적으로 병합한다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	권한·정책 변경이 오래된 복제본에서 허용되지 않도록 강한 읽기 또는 짧은 만료를 사용한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	복제 지연 분포와 최대 staleness

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

CAP의 C

모든 성공한 연산이 하나의 원자적 순서로 보이는 선형화 가능성에 해당하는 강한 조건이다.

Availability

분할 상황에서도 비실패 노드가 받은 모든 요청에 응답하는 성질을 의미한다.

Partition

노드 간 메시지가 손실되거나 임의로 지연되는 조건이다.

선형화 가능성

각 연산이 호출과 응답 사이 한 시점에 즉시 일어난 것처럼 보이는 모델이다.

인과 일관성

원인과 결과 관계가 있는 연산 순서는 모두가 동일하게 관찰하도록 보장한다.

세션 보장

read-your-writes, monotonic reads/writes처럼 한 사용자 세션에서 필요한 보장이다.

최종 일관성

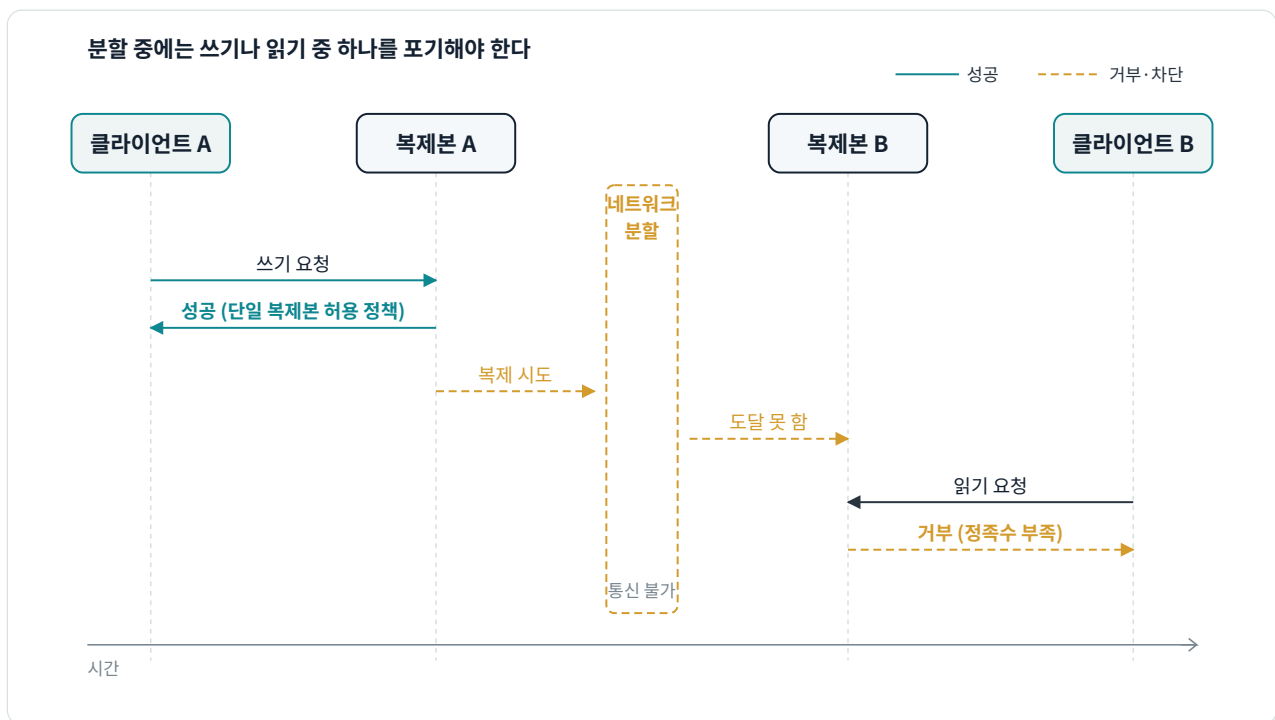
새 업데이트가 없으면 복제본이 언젠가 같은 값으로 수렴하는 성질이며 수렴 시간과 충돌 의미를 추가로 정의해야 한다.

핵심 개념의 정의와 범위는 [@gilbert-lynch-cap; @vogels-eventual; @dynamo-paper]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
쓰기 조정자	쓰기 순서·버전·quorum을 결정한다.
복제본 집합	독립 장애 도메인에 상태를 저장한다.
읽기 라우터	필요한 일관성 수준에 따라 leader·quorum·로컬 복제본을 선택한다.
버전 메타데이터	논리 시계·버전 벡터·타임스탬프로 충돌을 감지한다.
충돌 해석기	업무 규칙에 따라 병합·거부·사용자 확인을 수행한다.
세션 토큰	클라이언트가 본 최소 버전이나 region affinity를 전달한다.



두 복제본 사이 분할 중 쓰기·읽기 선택과 응답 결과를 시간축으로 보여준다

요청·데이터 흐름

1. 연산별 불변조건과 사용자 기대를 분류한다.
2. 분할·리더 상실 시 허용할 응답을 결정한다.
3. 쓰기 acknowledgement 조건과 읽기 소스를 정한다.
4. 세션 토큰 또는 버전 조건을 전달한다.

5. 동시 쓰기 충돌을 감지한다.

6. 자동 병합 불가능한 충돌은 명시적 워크플로로 보낸다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

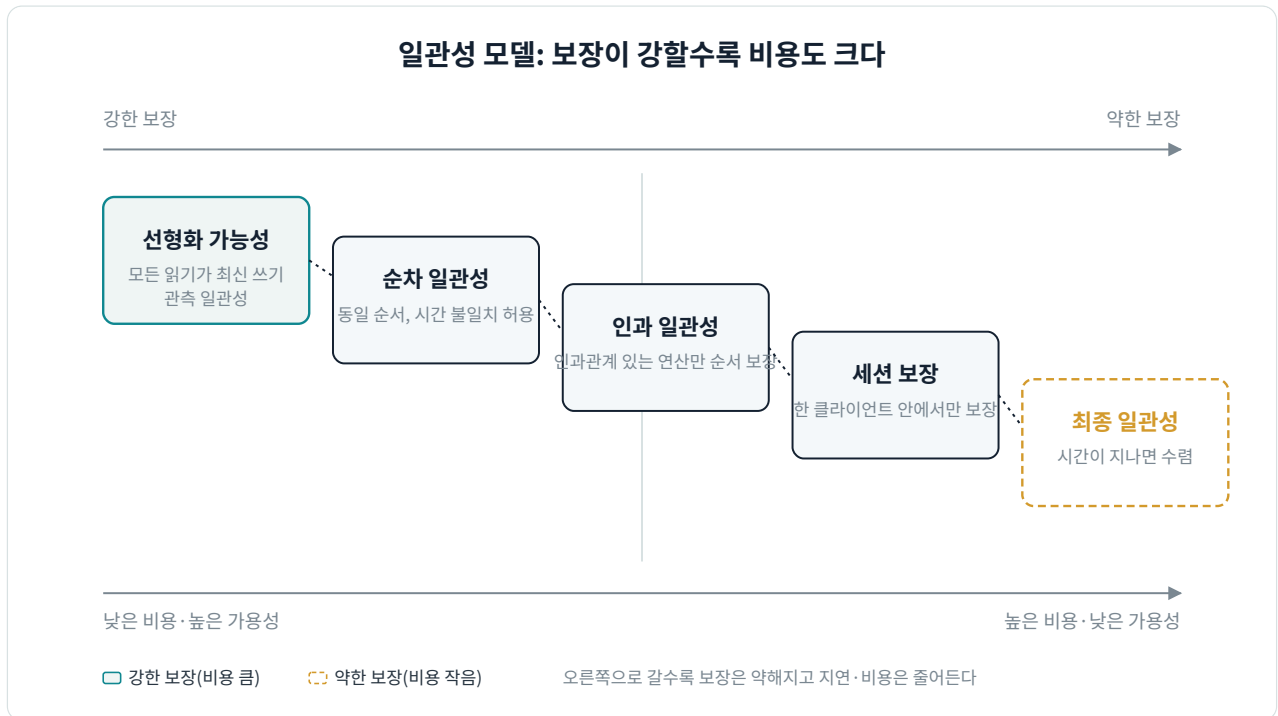
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
Leader 기반 강한 읽기	순서와 불변조건을 이해하기 쉽다.	원격 leader 지연과 분할 중 쓰기 중단이 발생한다.	결제 잔액·유일성·권한 변경
Quorum 읽기/쓰기	일부 노드 실패를 견디며 조정 가능하다.	지연·repair·sloppy quorum 의미가 복잡하다.	복제 KV·메타데이터
로컬 eventual 읽기	지역 지연과 가용성이 좋다.	오래된 값·역전·충돌을 사용자 흐름에서 처리해야 한다.	피드·통계·검색 색인

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [gilbert-lynch-cap; vogels-eventual; dynamo-paper]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
네트워크 분할	각 지역이 독립 쓰기를 받아 동일 키가 갈라진다.	쓰기 소유권을 제한하거나 충돌을 의미적으로 병합한다.
복제 지연	사용자가 방금 쓴 값을 다른 복제본에서 읽지 못한다.	세션 affinity, version token, leader read를 적용한다.
시계 오차	last-write-wins가 실제 인과관계를 뒤집는다.	물리 시간만으로 충돌을 해결하지 않고 논리 버전을 사용한다.
read repair 폭증	오래된 복제본 수리가 사용자 읽기 경로를 느리게 한다.	background repair와 repair budget을 둔다.
유령 성공	client timeout 후 쓰기는 성공했지만 재시도로 중복 상태가 생긴다.	idempotency key와 결과 조회 계약을 둔다.



확장 전략

- 키·tenant·지역별로 일관성 수준을 다르게 적용하되 API 의미를 명확히 한다.
- 강한 경로는 범위를 좁혀 quorum 지연과 coordinator 부하를 줄인다.
- eventual 경로는 anti-entropy와 최대 허용 staleness를 운영 지표로 둔다.
- 다중 writer는 충돌률과 병합 실패율이 낮을 때만 이점을 갖는다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 권한·정책 변경이 오래된 복제본에서 허용되지 않도록 강한 읽기 또는 짧은 만료를 사용한다.
- 삭제 요청이 모든 복제본·색인·백업에 전파되는 시간을 추적한다.
- 충돌 로그에 민감한 본문 대신 버전·해시·식별자를 최소한으로 기록한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 복제 지연 분포와 최대 staleness
- quorum 실패·leader unavailable 비율
- 충돌 감지·자동 병합·수동 해결 건수
- read-your-writes 위반 탐지
- repair backlog와 오래된 replica 수

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 강한 일관성은 조정과 원격 왕복 비용을 만든다.
- 다중 writer는 평상시 지연을 줄여도 충돌 처리와 운영 복잡도를 늘린다.
- 세션 보장은 전체 선형화보다 저렴하게 사용자 기대를 만족시킬 수 있다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- CAP를 데이터베이스 제품에 CP/AP 라벨 하나로 붙인다.
- “eventual”을 언제든지 틀린 값을 반환해도 된다는 뜻으로 사용한다.
- 격리 수준과 복제 일관성을 같은 개념으로 혼동한다.
- last-write-wins를 업무 의미와 무관한 안전한 기본값으로 둔다.

설계 리뷰

- ☐ 분할 중 각 연산이 성공·실패·대기 중 무엇을 하는가?
- ☐ 사용자에게 필요한 세션 보장이 정의됐는가?
- ☐ 충돌 감지와 병합이 도메인 불변조건을 보존하는가?
- ☐ 최대 staleness와 복제 지연을 관측하는가?
- ☐ 권한·삭제 같은 보안 상태에 더 강한 정책이 적용되는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 장바구니 수량, 은행 잔액, 좋아요 수 각각에 적절한 일관성 모델을 선택하고 이유를 쓰라.
2. 다중 리전에서 read-your-writes를 제공하는 세 가지 방법을 비교하라.
3. last-write-wins가 할인 쿠폰 사용 횟수 불변조건을 깨뜨리는 시나리오를 구성하라.

핵심 요약

- CAP는 분할 중 선형화 가능성과 모든 요청 응답의 충돌을 설명한다.
- 정상 상태의 지연·격리·세션 보장은 별도 설계 문제다.
- 일관성은 데이터가 아니라 연산과 불변조건 단위로 선택한다.
- eventual 시스템도 수렴·staleness·충돌 규칙이 필요하다.
- 사용자 경험에 필요한 최소 보장을 명시하는 것이 출발점이다.

출처

- [@gilbert-lynch-cap] Seth Gilbert and Nancy Lynch. **Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services** (2002). <https://doi.org/10.1145/564585.564601>
- [@vogels-eventual] Werner Vogels. **Eventually Consistent** (2009). <https://dl.acm.org/doi/10.1145/1435417.1435432>
- [@dynamo-paper] Giuseppe DeCandia et al.. **Dynamo: Amazon's Highly Available Key-value Store** (2007). <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

08

트랜잭션·격리 수준·MVCC

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

트랜잭션은 여러 SQL 문을 한 묶음으로 만드는 문법이 아니라 실패 중에도 불변조건을 보존하는 계약이다. 격리 수준 이름만 믿지 말고 애플리케이션이 막아야 할 write skew, lost update, phantom을 구체적인 테스트로 확인해야 한다.

이 절의 기준 출처: [@postgres-transaction-iso; @postgres-mvcc].

학습 목표

- 원자성·격리·내구성을 실제 실패와 연결한다.
- 격리 수준별 이상 현상을 설명한다.
- MVCC가 읽기와 쓰기 충돌을 관리하는 방식을 이해한다.

먼저 결론

- 원자성은 중간 상태 노출을 막지만 외부 부작용까지 자동으로 되돌리지 않는다.
- 격리 수준은 동시 실행이 어떤 순서로 보이는지 결정한다.
- MVCC는 읽기 스냅샷을 제공하지만 오래 열린 트랜잭션과 vacuum 지연 비용을 만든다.
- 유일성·잔액·재고 같은 불변조건은 제약·잠금·직렬화 재시도로 지킨다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	트랜잭션·격리 수준·MVCC에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	원자성은 중간 상태 노출을 막지만 외부 부작용까지 자동으로 되돌리지 않는다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	트랜잭션 범위를 최소 불변조건 단위로 좁힌다.
실패·복구	“Lost update” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	조건부 update, 버전 열, 행 잠금을 사용한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	행 수준 권한과 애플리케이션 권한이 같은 트랜잭션 스냅샷에서 평가되는지 확인한다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	트랜잭션 시간과 active transaction 수

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

원자성

트랜잭션의 변경이 모두 반영되거나 모두 반영되지 않는 성질이다.

격리

동시 트랜잭션이 서로의 중간 상태를 어떻게 관찰하는지에 대한 성질이다.

내구성

커밋이 성공했다고 응답한 상태가 약속된 실패 범위에서 보존되는 성질이다.

MVCC

여러 버전의 행을 유지해 읽기 스냅샷과 동시 쓰기를 조정하는 방식이다.

Write skew

서로 다른 행을 수정하지만 함께 보는 조건이 깨지는 이상 현상이다.

직렬화 가능성

동시 실행 결과가 어떤 직렬 실행 순서와 동등하도록 보장하는 격리 수준이다.

핵심 개념의 정의와 범위는 [[@postgres-transaction-iso](#); [@postgres-mvcc](#)]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
트랜잭션 경계	하나의 불변조건을 지켜야 하는 작업 범위를 정의한다.
제약 조건	UNIQUE, CHECK, FK 등 DB가 직접 검증하는 규칙이다.
버전 저장소	스냅샷별로 보이는 행 버전을 관리한다.
잠금 관리자	행·범위·predicate 충돌을 조정한다.
WAL/로그	커밋 복구와 복제를 위한 순서를 기록한다.
재시도 계층	serialization failure·deadlock을 안전하게 다시 실행한다.

격리 수준이 강해질수록 막을 수 있는 이상 현상이 늘어난다

PostgreSQL 기준 · 동시 실행에서 나타날 수 있는 현상

격리 수준	Dirty Read	Nonrepeatable Read	Phantom	Lost Update	Write Skew
Read Committed	방지	가능	가능	가능	가능
Repeatable Read	방지	방지	방지	방지	가능
Serializable	방지	방지	방지	방지	방지

범례: **방지** = 격리 수준이 막아주는 현상 **가능** = 동시 실행에서 여전히 발생할 수 있는 현상
아래로 내려갈수록 보장이 강해지지만 동시성은 줄고 비용은 커진다. 색을 빼도 '방지/가능' 글자로 구분된다.

격리 수준별 dirty read · nonrepeatable read · phantom · write skew 가능성을 비교한다

요청·데이터 흐름

- 요청을 idempotency key와 함께 받는다.
- 현재 상태를 적절한 스냅샷 또는 잠금으로 읽는다.
- 불변조건을 DB 제약과 애플리케이션 검증으로 확인한다.
- 변경을 수행하고 커밋한다.
- 충돌·deadlock·serialization failure는 전체 단위를 재시도한다.
- 외부 이벤트는 outbox 등 커밋 후 전달 가능한 기록으로 분리한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

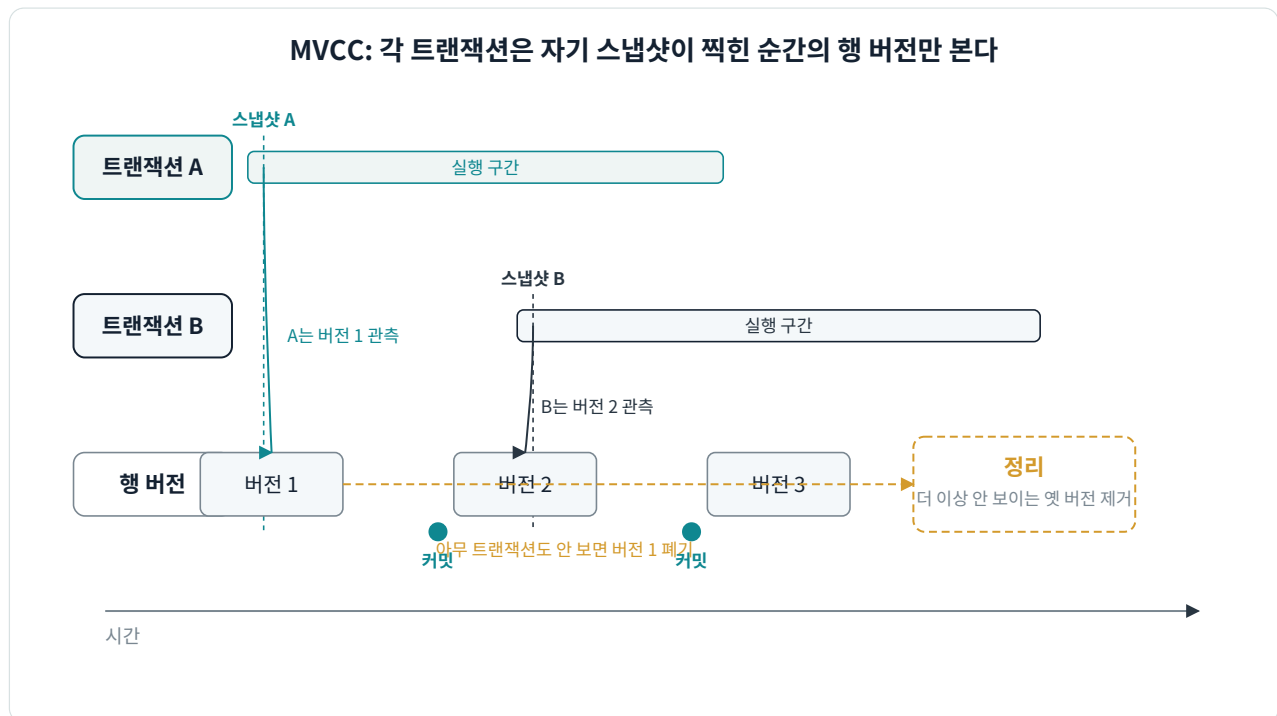
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
Read Committed	기본 비용이 낮고 각 문장은 커밋된 값만 본다.	같은 트랜잭션 안에서도 값이 바뀌고 복합 불변조건이 깨질 수 있다.	짧은 단일 행 CRUD
Repeatable Read/Snapshot	일관된 스냅샷으로 읽기가 안정적이다.	구현에 따라 write skew가 가능하다.	리포트·복수 읽기
Serializable	가장 강한 격리로 복잡한 불변조건을 단순화한다.	충돌 시 abort·재시도가 높고 긴 트랜잭션에 불리하다.	금융·재고·정책 변경

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [[@postgres-transaction-iso](#); [@postgres-mvcc](#)]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Lost update	두 요청이 같은 값을 읽고 각자 계산한 뒤 하나를 덮어쓴다.	조건부 update, 버전 열, 행 잠금을 사용한다.
Write skew	두 의사가 서로가 근무 중임을 보고 각각 퇴근해 최소 인원 규칙이 깨진다.	serializable, predicate lock, 별도 집계 행을 사용한다.
긴 스냅샷	오래 열린 트랜잭션이 오래된 버전 정리를 막아 저장량과 I/O가 증가한다.	트랜잭션 시간을 제한하고 배치 읽기를 폐이지화한다.
Deadlock	서로 다른 순서로 잠금을 획득한 작업이 대기한다.	잠금 순서를 통일하고 전체 트랜잭션을 재시도한다.
외부 부작용 불일치	DB rollback은 이미 보낸 이메일·결제 호출을 취소하지 못한다.	outbox, idempotency, 보상 워크플로를 사용한다.



두 트랜잭션의 스냅샷이 여러 행 버전을 다르게 보는 과정을 시간축으로 보여준다

확장 전략

- 트랜잭션 범위를 최소 불변조건 단위로 좁힌다.
- hot row를 분할하거나 append-only 원장과 비동기 집계를 사용한다.
- 읽기 전용 분석은 replica·snapshot export로 OLTP 경로와 격리한다.
- 재시도율과 lock wait가 임계치를 넘으면 데이터 모델과 경합 지점을 재설계한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 행 수준 권한과 애플리케이션 권한이 같은 트랜잭션 스냅샷에서 평가되는지 확인한다.
- 감사 로그는 원본 변경과 인과 관계를 유지하고 임의 수정이 어렵게 한다.
- 오류 로그에 SQL 파라미터와 개인정보를 그대로 남기지 않는다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 트랜잭션 시간과 active transaction 수
- lock wait·deadlock·serialization failure 비율
- 오래된 snapshot age와 vacuum/compaction backlog
- 불변조건 제약 위반과 재시도 성공률
- WAL 생성량과 commit latency

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 강한 격리와 큰 트랜잭션은 경합과 로그 비용을 높인다.
- 애플리케이션에서 불변조건을 재현하면 코드·테스트·복구 비용이 늘어난다.
- OLTP와 분석을 분리하면 인프라 비용은 늘지만 장애 격리와 예측 가능성이 좋아진다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- “ACID DB면 모든 동시성 문제가 해결된다”고 생각한다.
- 트랜잭션 안에서 외부 API를 오래 기다린다.
- serialization failure를 500 오류로 그대로 노출한다.
- read-modify-write를 조건 없는 UPDATE로 구현한다.

설계 리뷰

- ☐ 불변조건이 DB가 검증할 수 있는 형태인가?
- ☐ 선택한 격리 수준에서 가능한 이상 현상을 테스트했는가?
- ☐ 재시도 단위가 멍등적이며 전체 트랜잭션을 포함하는가?
- ☐ 외부 부작용과 DB 커밋의 불일치를 처리하는가?
- ☐ 오래 열린 트랜잭션과 lock wait를 관측하는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 좌석 1개를 두 사용자가 동시에 예약하는 시나리오를 세 가지 방식으로 구현하라.
2. write skew가 발생하는 온콜 근무표 예제를 구성하고 serializable로 해결하라.
3. DB 업데이트와 메시지 발행 사이 장애를 outbox로 처리하는 흐름을 그려라.

핵심 요약

- 트랜잭션은 불변조건을 실패 중에도 지키는 계약이다.
- 격리 수준은 가능한 동시성 이상 현상으로 이해한다.
- MVCC는 읽기 동시성을 높이지만 버전 정리 비용이 있다.
- 강한 격리도 충돌 재시도 설계가 필요하다.
- 외부 부작용은 DB 트랜잭션과 별도 조정해야 한다.

출처

- [@postgres-transaction-iso] PostgreSQL Global Development Group. **PostgreSQL Documentation — Transaction Isolation** (2026). <https://www.postgresql.org/docs/current/transaction-iso.html>
- [@postgres-mvcc] PostgreSQL Global Development Group. **PostgreSQL Documentation — Concurrency Control** (2026). <https://www.postgresql.org/docs/current/mvcc.html>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

09

시간·순서·논리 시계·분산 ID

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

분산 시스템에는 모든 노드가 공유하는 완벽한 현재 시각이 없다. 벽시계는 만료·사용자 표시·운영 분석에 필요하지만, 사건의 인과 순서와 단일 승자를 정하는 근거로 사용하려면 오차·점프·동률을 다뤄야 한다.

이 절의 기준 출처: [lamport-time; @rfc9562].

학습 목표

- 물리 시계와 논리적 순서를 구분한다.
- 인과 관계를 표현하는 시계와 버전 메타데이터를 선택한다.
- 분산 ID의 정렬성·고유성·정보 노출을 비교한다.

먼저 결론

- 벽시계 타임스탬프와 논리적 버전을 분리한다.
- “먼저”가 업무적으로 필요한 곳만 순서를 강제한다.
- 시간 기반 ID는 정렬과 locality를 얻지만 생성 시간·트래픽 패턴을 노출할 수 있다.
- 시계 오차 한계와 동기화 실패를 관측하고 안전 여유에 포함한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	시간·순서·논리 시계·분산 ID에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	벽시계 타임스탬프와 논리적 버전을 분리한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	전역 순서 대신 aggregate·partition 단위 순서를 사용해 조정을 줄인다.
실패·복구	“시계 역행” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	마지막 발급 값 보존, 논리 카운터, 역행 시 차단을 사용한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	ID에서 tenant·시간·region 정보를 불필요하게 노출하지 않는다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	노드별 clock offset·동기화 상태

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

벽시계

달력상의 시각을 나타내며 NTP 조정, leap second 처리, VM 일시정지 등으로 단조 증가하지 않을 수 있다.

단조 시계

한 프로세스 안에서 경과 시간을 측정하는 데 적합하며 절대 시각과는 다르다.

Lamport clock

인과 관계가 있으면 논리 시계 값도 증가하도록 만드는 단순한 순서 메타데이터다.

Vector clock

여러 노드의 진행 상태를 벡터로 기록해 인과와 동시성을 구분한다.

Hybrid logical clock

물리 시간에 가까운 정렬성과 논리적 단조성을 결합한다.

분산 ID

중앙 병목 없이 고유 식별자를 만들기 위한 UUID, 시간 정렬 ID, 구간 할당 등이다.

핵심 개념의 정의와 범위는 [lamport-time; @rfc9562; @dynamo-paper]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
시간 동기화 계층	노드 시계 오차와 동기 상태를 관리한다.
ID 생성기	고유성·정렬성·가용성 정책에 따라 ID를 발급한다.
버전 메타데이터	업데이트의 인과 관계와 동시성을 표현한다.
순서 결정기	필요한 도메인 범위에서 단일 순서를 부여한다.
저장·색인 계층	ID와 버전의 정렬 특성을 활용하거나 hot partition을 방지한다.
감사 계층	사용자 표시 시간과 처리 순서를 구분해 기록한다.

다섯 가지 시계는 각각 다른 질문에 답한다



물리 시계는 ‘언제’를, 논리 시계는 ‘어떤 순서로’를 답한다. 목적이 다르므로 함께 쓴다.

벽시계·단조 시계·Lamport·vector·hybrid logical clock의 목적을 비교한다

요청·데이터 흐름

1. 요구되는 순서의 범위를 전역·tenant·aggregate·partition으로 정한다.
2. 표시 시간, timeout 경과, 인과 버전에 서로 다른 시계를 선택한다.
3. ID에 필요한 고유성·정렬성·예측 불가능성을 정한다.
4. 노드 재시작·시계 역행·동일 밀리초 burst를 처리한다.
5. 수신 측에서 버전 비교와 중복 검사를 수행한다.
6. 감사 로그에는 발생 시각·수신 시각·처리 순서를 구분해 저장한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

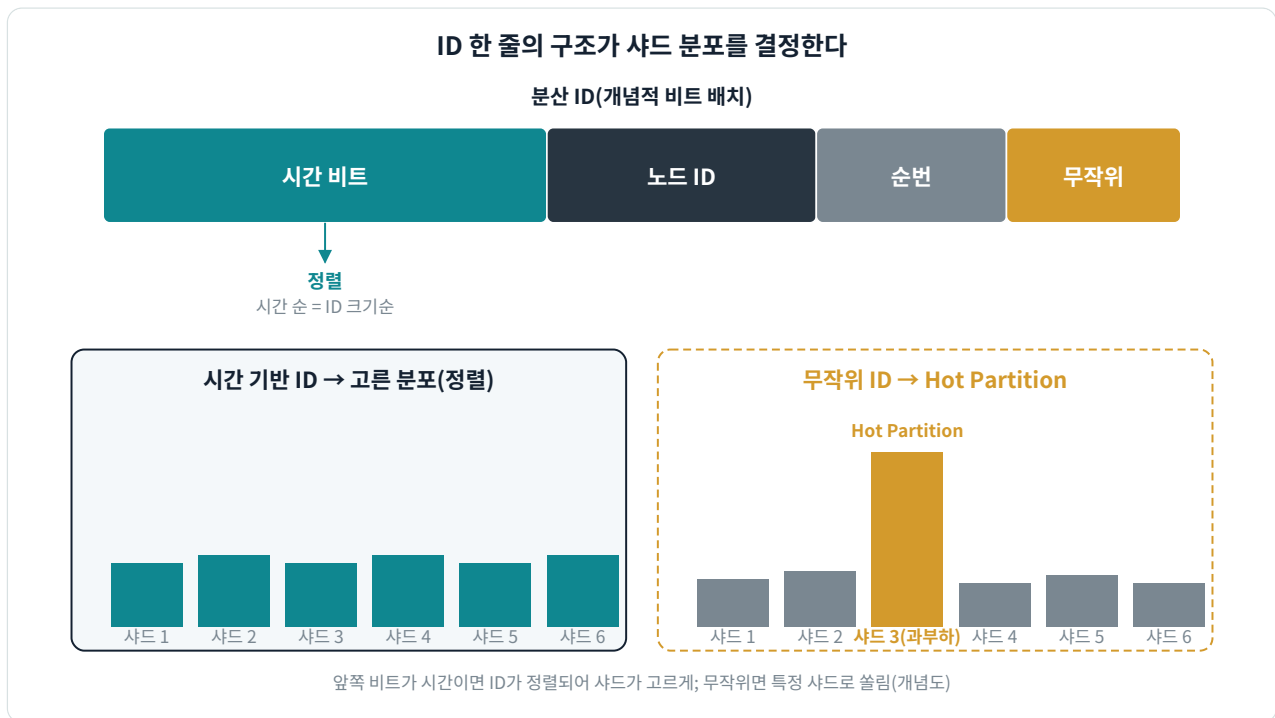
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
UUID v4	중앙 조정 없이 예측하기 어렵고 단순하다.	무작위 인덱스 locality가 낮고 시간 정렬이 없다.	공개 식별자·일반 객체
시간 정렬 UUID/ID	B-tree locality와 시간순 조치가 좋다.	생성 시간 노출·동일 시각 충돌·노드 식별 관리가 필요하다.	고속 쓰기·로그·이벤트
중앙 sequence	작고 완전한 증가 순서를 제공한다.	중앙 의존성과 다중 리전 지연이 생긴다.	단일 DB 범위의 업무 번호

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [lamport-time; @rfc9562; @dynamo-paper]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
시계 역행	VM 이동이나 동기화로 시간이 뒤로 가며 ID 중복·만료 오류가 난다.	마지막 발급 값 보존, 논리 카운터, 역행 시 차단을 사용한다.
노드 ID 충돌	두 생성기가 같은 worker ID로 같은 시각에 같은 값을 만든다.	lease 기반 할당과 fencing, 시작 전 충돌 검사를 둔다.
전역 순서 착각	시간 정렬 ID를 모든 사건의 정확한 발생 순서로 해석한다.	인과 관계와 수신·커밋 순서를 별도 기록한다.
hot partition	시간 접두사가 같은 새 ID가 한 shard에 집중된다.	hash prefix, bucket, range split 전략을 적용한다.
정보 노출	공개 ID로 생성 시각이나 대략적 거래량을 추정할 수 있다.	외부 식별자와 내부 정렬 키를 분리한다.



시간·노드·카운터·무작위 비트로 구성된 분산 ID와 인덱스 분포를 보여준다

확장 전략

- 전역 순서 대신 aggregate·partition 단위 순서를 사용해 조정을 줄인다.
- ID 발급 서비스가 필요하다면 구간 선할당과 지역별 namespace로 병목을 줄인다.
- 벡터 메타데이터 크기는 참여 노드 수와 함께 커지므로 안정된 replica set 또는 압축 전략을 사용한다.
- 감사·분석 파이프라인은 late event와 out-of-order event를 정상 조건으로 처리한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- ID에서 tenant·시간·region 정보를 불필요하게 노출하지 않는다.
- 인증 토큰 만료 판단은 서버의 신뢰 가능한 시간과 허용 오차를 사용한다.
- 감사 로그 시간 조작을 탐지하고 원본 순서 증거를 보호한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 노드별 clock offset·동기화 상태
- ID 충돌·재발급·시계 역행 차단 건수
- out-of-order·late event 분포
- 버전 충돌과 동시 업데이트 비율
- 순서 결정 서비스의 지연·lease 상태

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 전역 순서는 조정·가용성·리전 지연 비용을 만든다.
- 긴 ID는 저장·색인·네트워크 비용을 조금 늘리지만 운영 단순성을 줄 수 있다.
- 시간 정렬 키는 쓰기 효율을 높일 수 있으나 shard hotspot 비용을 만든다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- created_at을 인과 순서와 동일시한다.
- 밀리초 timestamp만으로 고유 ID를 만든다.
- DB auto-increment를 전 세계 서비스의 전역 순서로 확장한다.
- 시간 기반 공개 ID의 정보 노출을 무시한다.

설계 리뷰

- ☐ 업무적으로 필요한 순서 범위가 최소화됐는가?
- ☐ 표시 시간·경과 시간·인과 버전에 올바른 시계를 쓰는가?
- ☐ 시계 역행과 worker ID 충돌을 시험했는가?
- ☐ ID가 shard 분포와 개인정보에 미치는 영향을 평가했는가?
- ☐ late/out-of-order 사건을 운영 지표로 보고 있는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 채팅 메시지의 전역 순서가 꼭 필요한지 방·사용자·서버 범위로 나누어 논증하라.
2. UUID v4, UUID v7, 중앙 sequence를 주문 ID 요구에 맞춰 비교하라.
3. 서버 시계가 90초 앞으로 갔다가 복구되는 토큰 만료 시나리오를 설계하라.

핵심 요약

- 분산 시스템에서 절대 시간과 인과 순서는 다르다.
- 순서 보장은 필요한 범위로 제한한다.
- 논리 시계는 인과 관계와 동시성을 표현한다.
- 분산 ID는 고유성·정렬성·노출·분산 특성을 함께 선택한다.
- 시계와 ID 생성기도 장애·관측 대상이다.

출처

- [lamport-time] Leslie Lamport. **Time, Clocks, and the Ordering of Events in a Distributed System** (1978). <https://lamport.azurewebsites.net/pubs/time-clocks.pdf>
- [rfc9562] IETF. **RFC 9562 — Universally Unique IDentifiers (UUIDs)** (2024). <https://www.rfc-editor.org/rfc/rfc9562.html>
- [dynamo-paper] Giuseppe DeCandia et al.. **Dynamo: Amazon's Highly Available Key-value Store** (2007). <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>

“

검증 기준일: 2026-08-06. `current` 와 `volatile` 내용은 `release` 전에 공식 원문을 다시 확인한다.

10

복제·Quorum·Failover

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

복제는 읽기 확장, 장애 대응, 지역 지연, 내구성을 위해 사용되지만 이 목적들은 같은 구성을 요구하지 않는다. 쓰기 성공 시점, replica 적용 시점, leader 승격 조건을 명시하지 않으면 장애 중 데이터 손실과 split brain을 설명할 수 없다.

이 절의 기준 출처: [raft-paper; dynamo-paper].

학습 목표

- 복제의 목적과 acknowledgement 조건을 구분한다.
- read/write quorum과 복제 지연을 계산한다.
- failover에서 데이터 손실·중복 leader·재동기화를 다룬다.

먼저 결론

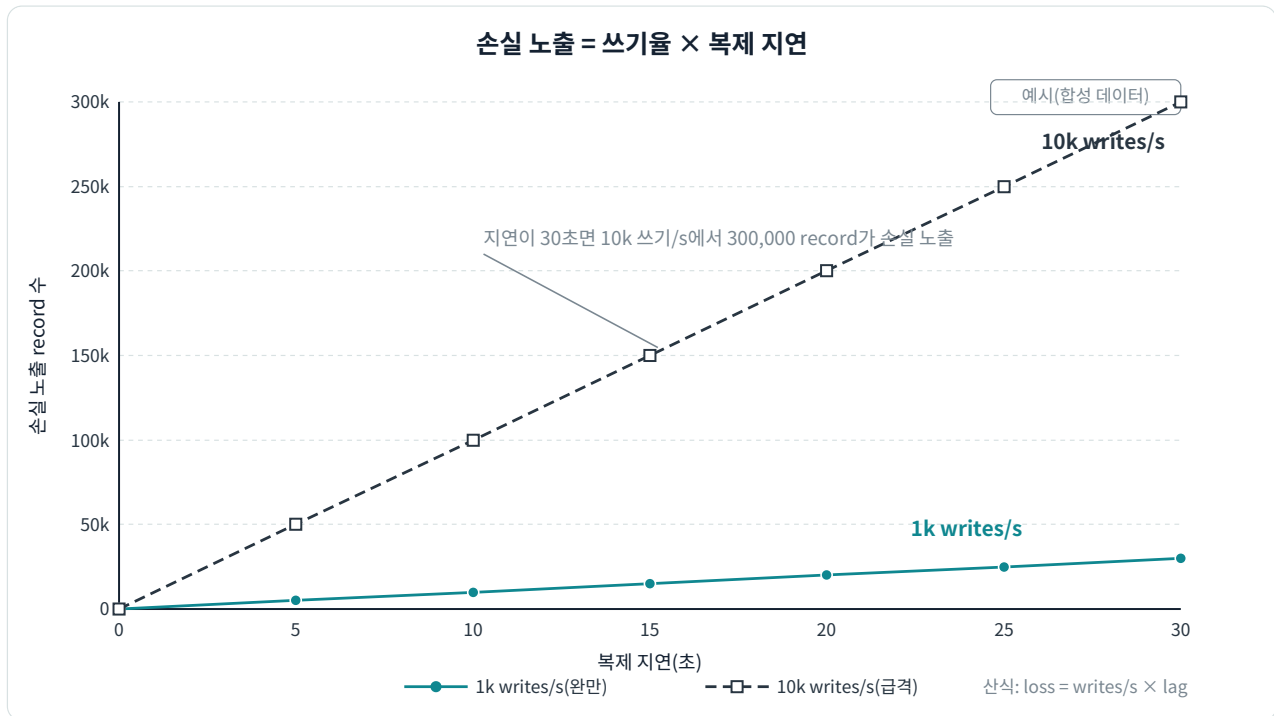
- Primary/Replica 또는 Leader/Follower 용어로 역할과 쓰기 소유권을 명확히 한다.
- 동기 복제는 acknowledgement 지연을 늘리고 비동기 복제는 데이터 손실 창을 만든다.
- $R + W > N$ 같은 식은 replica 집합과 실패 모델이 동일할 때만 의미가 있다.
- failover는 선출뿐 아니라 fencing·client 재연결·재동기화·failback을 포함한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	복제·Quorum·Failover에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	Primary/Replica 또는 Leader/Follower 용어로 역할과 쓰기 소유권을 명확히 한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	읽기 replica를 늘리기 전에 복제 로그·leader I/O·connection fan-out 한계를 확인한다.
실패·복구	“미복제 커밋 손실” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	동기 acknowledgement 또는 손실 허용 RPO를 명시한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	승격 권한과 membership 변경 권한을 최소화하고 감사한다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	replica apply/flush lag와 WAL backlog

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



복제 지연과 데이터 손실 노출 창

핵심 개념

복제 계수 N

하나의 데이터 항목을 보유하는 replica 수다.

쓰기 quorum W

성공 응답 전에 확인받아야 하는 replica 수다.

읽기 quorum R

읽기에서 조회하거나 비교하는 replica 수다.

복제 지연

leader 커밋과 follower 적용 사이 시간이다.

Commit index

합의된 로그에서 안전하게 적용할 수 있는 위치다.

Fencing

이전 leader가 뒤늦게 쓰기를 계속하지 못하도록 epoch·term·token으로 차단하는 방법이다.

Failover

새 writer 선택, 트래픽 전환, 데이터 검증, 이전 writer 격리를 포함한 복구 과정이다.

핵심 개념의 정의와 범위는 [@raft-paper; @dynamo-paper; @spanner-paper]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

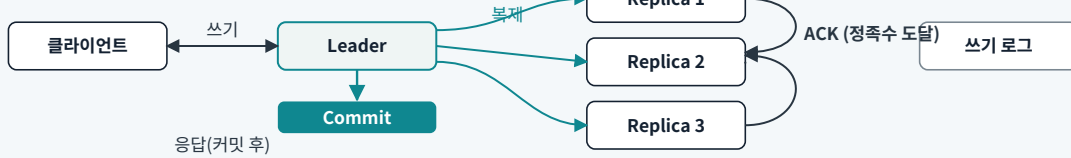
구성 요소	책임
쓰기 leader	순서를 부여하고 로그를 append한다.
복제 로그	변경을 내구성 있게 전달한다.
Follower 집합	로그를 저장하고 적용해 읽기·승격 후보가 된다.
Membership/term 저장소	현재 구성과 leader epoch를 관리한다.
라우터	leader 또는 허용된 replica로 요청을 보낸다.
Repair/재동기화	누락 로그나 snapshot으로 replica를 복구한다.
Failover 제어	건강 판단, 승격, fencing, 트래픽 변경을 수행한다.

동기 복제는 응답 전 ACK를 기다리고, 비동기 복제는 먼저 커밋한다

— 정상(동기) ——— 지연(비동기)

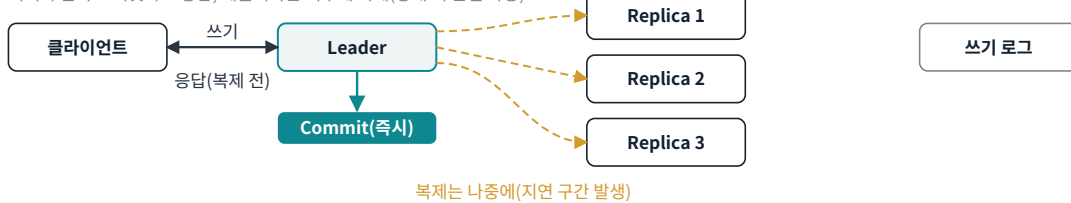
동기 복제

정족수 ACK를 받은 뒤에야 클라이언트에게 응답



비동기 복제

리더가 곧바로 커밋하고 응답; 레플리카는 이후에 복제(장애 시 손실 가능)



leader와 세 replica 사이 쓰기 로그·ack·commit 경로를 동기/비동기로 비교한다

요청·데이터 흐름

1. 클라이언트가 idempotency key와 쓰기 요청을 보낸다.
2. leader가 현재 term을 확인하고 로그 위치를 할당한다.
3. 설정된 W 또는 다수 replica가 로그를 내구성 있게 확인한다.
4. commit 조건을 만족하면 적용하고 성공을 응답한다.
5. 읽기는 요구 일관성에 따라 leader·R replica·stale replica를 선택한다.
6. leader 장애 시 새 term을 획득한 후보만 승격한다.
7. 복구된 이전 leader는 follower로 재동기화한 뒤 트래픽을 받는다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

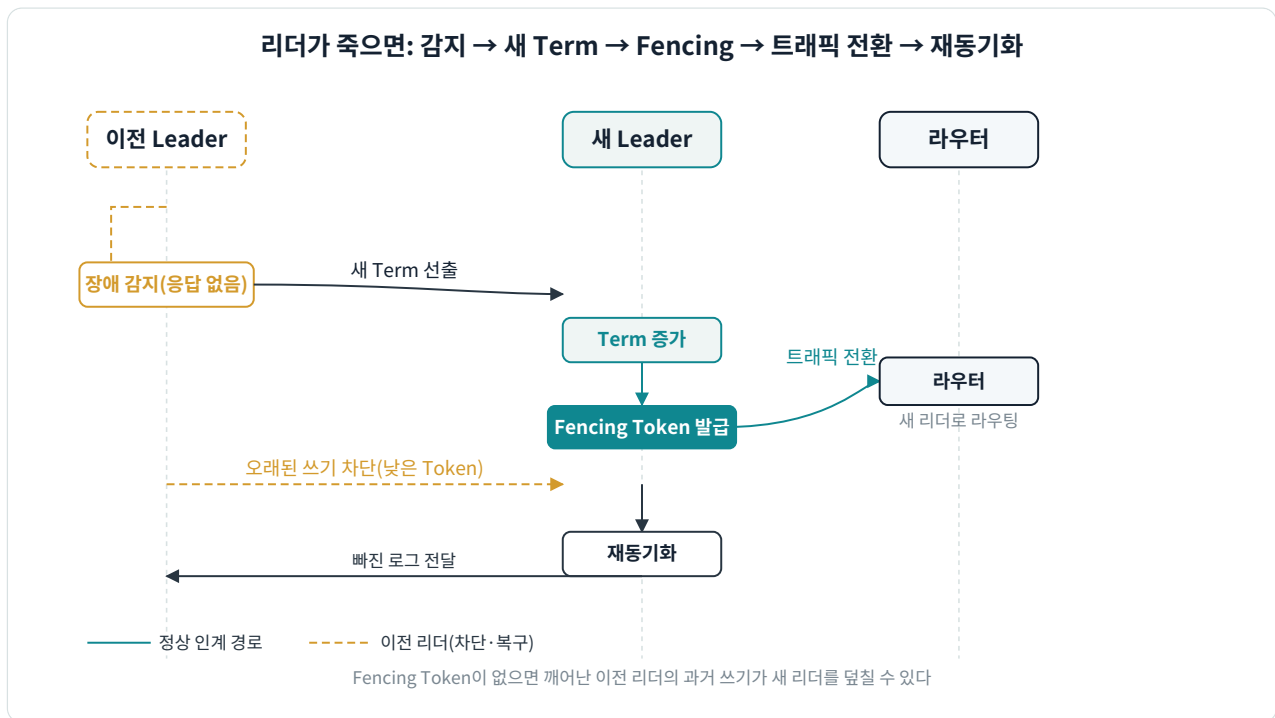
대안	장점	비용·위험	적합한 조건
비동기 Primary/Replica	쓰기 지연이 낮고 읽기 확장이 쉽다.	승격 시 미복제 쓰기 손실과 stale read가 가능하다.	일반 OLTP 읽기 replica

대안	장점	비용·위험	적합한 조건
동기 다수 복제	승인된 쓰기의 손실 범위를 줄인다.	느린 replica·zone 장애가 쓰기 지연과 가용성에 영향을 준다.	원장·메타데이터
Leaderless quorum	단일 leader 병목과 일부 장애를 피한다.	충돌·repair·sloppy quorum 의미가 복잡하다.	분산 KV·가용성 중심 쓰기

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [raft-paper; dynamo-paper; spanner-paper]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
미복제 커밋 손실	leader가 성공 응답 후 follower 전송 전 실패한다.	동기 acknowledgement 또는 손실 허용 RPO를 명시한다.
Split brain	네트워크 분할 양쪽이 writer로 동작한다.	quorum lease·term·fencing으로 단일 writer를 강제한다.
복제 지연 폭증	대량 쓰거나 느린 replica가 적용 backlog를 만든다.	lag 기반 읽기 차단, WAL 보존, replica 재구축을 준비한다.
Failover 반복	불안정한 감지로 leader가 계속 바뀌고 처리량이 붕괴한다.	히스테리시스, 최소 안정 시간, 수동 승인 단계를 둔다.
재동기화 포화	복구 replica의 snapshot 전송이 정상 트래픽 I/O를 압박한다.	속도 제한, 별도 네트워크, 점진 재가입을 사용한다.



leader 장애 감지부터 새 term 획득·fencing·트래픽 전환·재동기화까지 보여준다

확장 전략

- 읽기 replica를 늘리기 전에 복제 로그·leader I/O·connection fan-out 한계를 확인한다.
- geo replica는 사용자 지연을 줄이지만 데이터 전송과 staleness를 늘린다.
- 재구축 시간이 RTO보다 길어지지 않도록 snapshot 주기와 데이터 크기를 관리한다.
- hot shard는 replica 수보다 파티셔닝·쓰기 분산이 먼저 필요할 수 있다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 승격 권한과 membership 변경 권한을 최소화하고 감사한다.
- 복제 채널을 상호 인증·암호화한다.
- 삭제·권한 변경이 stale replica에서 되살아나지 않도록 tombstone과 버전 정책을 둔다.
- backup용 replica가 운영 접근 통제를 우회하지 않게 한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- replica apply/flush lag와 WAL backlog
- quorum 성공·timeout·unavailable 비율
- leader term 변경·failover·fencing 거부 건수
- 읽기 staleness와 read repair
- snapshot 전송량·재구축 예상 시간

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 동기 replica를 먼 region에 두면 지연과 전송 비용이 커진다.
- 읽기 replica는 쿼리 부하를 분산하지만 저장·백업·patch 비용을 늘린다.
- 빠른 failover를 위해 상시 대기 용량과 자동화에 투자해야 한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- replication factor 3이면 데이터 손실이 불가능하다고 말한다.
- health check 실패만으로 즉시 다른 writer를 승격한다.
- replica lag를 보지 않고 모든 읽기를 follower로 보낸다.
- failover 성공만 확인하고 이전 leader fencing과 failback을 생략한다.

설계 리뷰

- ☐ 쓰기 성공 응답이 의미하는 내구성 범위가 명확한가?
- ☐ R·W·N과 failure domain 배치가 일치하는가?
- ☐ 이전 leader가 쓰지 못하도록 fencing되는가?
- ☐ 복제 지연과 WAL 보존이 재구축 시간을 감당하는가?
- ☐ failover·재동기화·failback을 실제로 연습했는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. $N=3$ 에서 $W=2$, $R=2$ 인 경우와 $W=1$, $R=1$ 인 경우의 보장 차이를 설명하라.
2. 비동기 replica 승격 시 최대 데이터 손실량을 복제 지연과 쓰기율로 계산하라.
3. 두 데이터센터 간 네트워크 분할에서 단일 writer를 유지하는 fencing 절차를 설계하라.

핵심 요약

- 복제 목적에 따라 acknowledgement·읽기·failover 정책이 달라진다.
- 동기와 비동기는 지연과 데이터 손실 창을 교환한다.
- quorum 식은 실제 replica 집합과 장애 모델을 함께 봐야 한다.
- failover에는 fencing과 재동기화가 필수다.
- replica lag와 재구축 시간은 핵심 운영 지표다.

출처

- [raft-paper] Diego Ongaro and John Ousterhout. **In Search of an Understandable Consensus Algorithm** (2014). <https://raft.github.io/raft.pdf>
- [dynamo-paper] Giuseppe DeCandia et al.. **Dynamo: Amazon's Highly Available Key-value Store** (2007). <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>
- [spanner-paper] James C. Corbett et al.. **Spanner: Google's Globally-Distributed Database** (2012). <https://research.google/pubs/spanner-googles-globally-distributed-database/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

11

파티셔닝·Sharding·Consistent Hashing

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

샤딩은 저장공간을 여러 서버에 나누는 기술이 아니라 데이터·부하·실패를 어떤 키로 분리할지 결정하는 모델이다. 잘못된 파티션 키는 노드를 추가해도 hot key와 cross-shard transaction을 해결하지 못한다.

이 절의 기준 출처: [@consistent-hashing; @dynamo-paper].

학습 목표

- 파티션 키를 접근 패턴과 불변조건으로 선택한다.
- range·hash·directory 기반 분할을 비교한다.
- 재분배·hot partition·cross-shard 작업을 설계한다.

먼저 결론

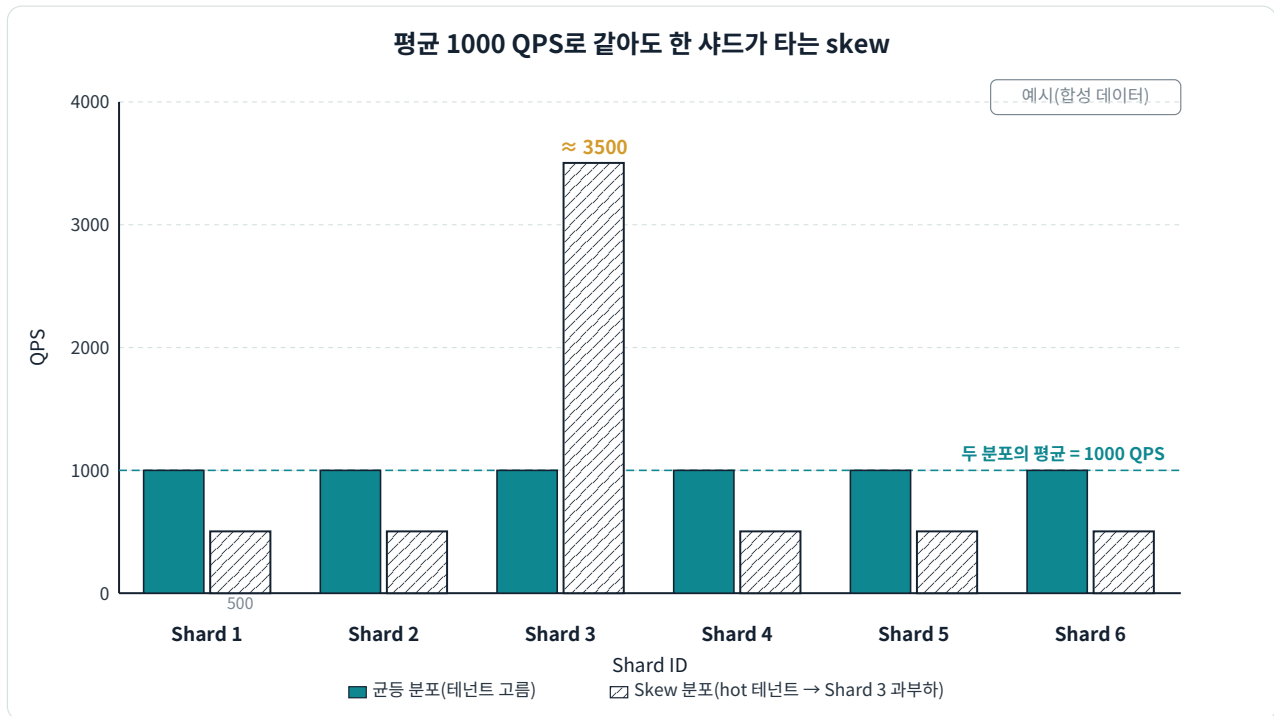
- 파티션 키는 분포뿐 아니라 함께 읽고 쓰는 데이터와 트랜잭션 경계를 결정한다.
- 균등 hash는 hotspot을 줄이지만 범위 조회와 지역성을 희생한다.
- range는 순차 조회에 좋지만 최신 구간·대형 tenant가 뜨거워질 수 있다.
- 재샤딩은 정상 트래픽과 경쟁하므로 온라인 이동·검증·rollback을 설계한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	파티셔닝·Sharding·Consistent Hashing에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	파티션 키는 분포뿐 아니라 함께 읽고 쓰는 데이터와 트랜잭션 경계를 결정한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	통계 기반 자동 split은 최대 파티션 크기와 요청률을 함께 고려한다.
실패·복구	“Hot key” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	key salting, 읽기 복제, tenant 전용 shard를 사용한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	tenant ID가 파티션 키일 때 라우팅 메타데이터가 고객 목록을 노출하지 않게 보호한다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	shard별 데이터 크기·QPS·p99·queue depth

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



샤드 부하 불균형

핵심 개념

파티션

데이터와 요청을 독립적으로 배치·복제·이동할 수 있는 단위다.

샤드 키

요청을 어떤 파티션으로 라우팅할지 결정하는 값이다.

Range partition

키 구간을 연속 범위로 나눈다.

Hash partition

키 hash 공간을 분할해 분포를 균등하게 만든다.

Consistent hashing

노드 변화 시 이동하는 키 범위를 줄이기 위해 hash ring 또는 유사한 토큰 공간을 사용한다.

Virtual node

물리 노드 하나가 여러 작은 token 범위를 소유해 균형과 이동 단위를 개선한다.

Directory routing

별도 메타데이터가 키·tenant의 위치를 직접 가리킨다.

핵심 개념의 정의와 범위는 [@consistent-hashing; @dynamo-paper; @bigtable-paper]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
파티션 맵	키 범위·token·tenant와 소유 노드를 관리한다.
라우터	요청 키를 읽고 올바른 shard로 보낸다.
Shard replica set	각 파티션의 저장·복제·leader를 제공한다.
Rebalancer	부하·용량·장애 도메인에 따라 파티션을 이동한다.
Global index	파티션 키가 아닌 조건의 검색을 지원한다.
Cross-shard coordinator	불가피한 다중 shard 작업의 순서·보상·결과를 관리한다.

세 가지 파티셔닝: 키를 어떻게 샤드에 배치하느냐



Global Index

파티션 키가 아닌 다른 필드로 검색할 때 모든 샤드에 걸친 인덱스 사용(비용 큼)

같은 라우터·샤드라도 키→샤드 배치 규칙이 다르면 분포와 질의 성격이 달라진다

range·hash·directory 파티셔닝의 라우팅과 데이터 배치를 비교한다

요청·데이터 흐름

1. 접근 패턴과 함께 변경되는 aggregate를 식별한다.
2. 후보 키의 cardinality·skew·성장·tenant 크기를 분석한다.
3. range/hash/directory 방식과 secondary index 비용을 비교한다.
4. 라우터가 파티션 맵 버전과 이동 상태를 확인한다.
5. 이동 중 dual-read/forwarding 또는 ownership epoch를 적용한다.
6. 검증 후 이전 소유권을 해제하고 stale writer를 fencing한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

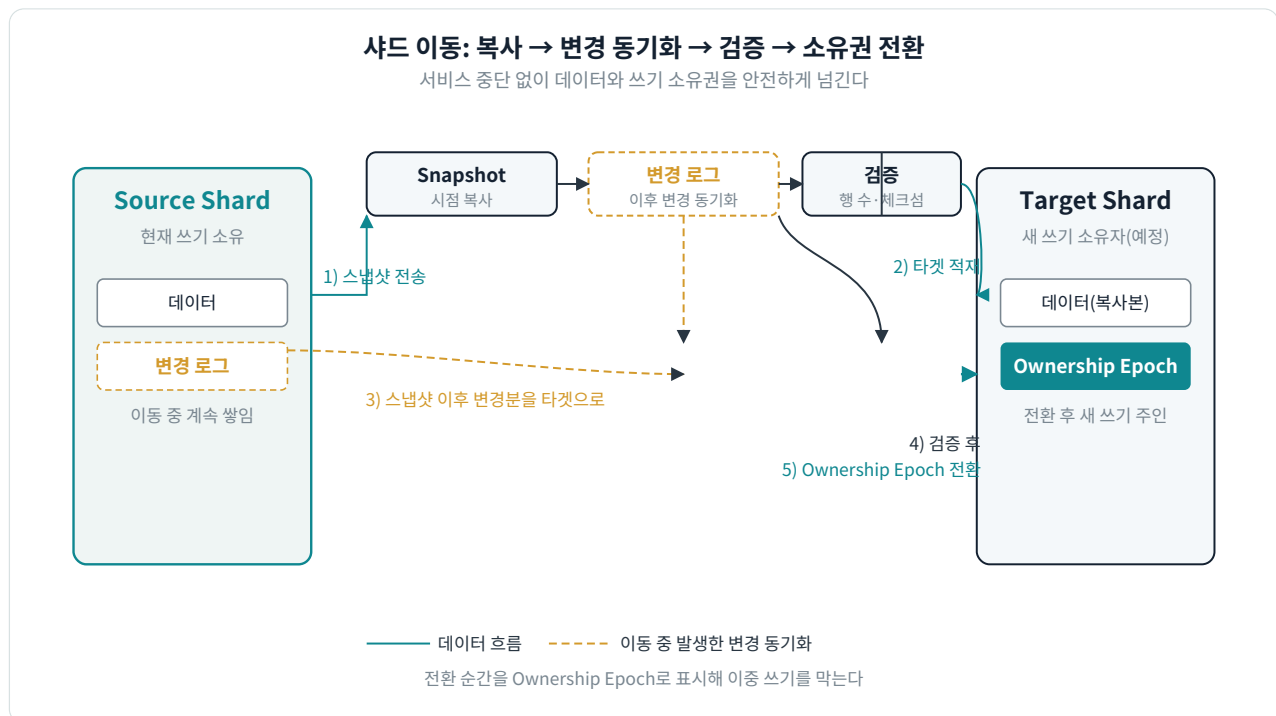
대안	장점	비용·위험	적합한 조건
Range	범위 조회와 순차 스캔이 효율적이다.	최신 키·특정 구간에 쓰기가 집중될 수 있다.	시간 구간·정렬 조회
Hash	분포가 균등하고 단일 키 lookup이 단순하다.	범위 조회와 같은 tenant 데이터 모으기가 어렵다.	대규모 KV

대안	장점	비용·위험	적합한 조건
Directory/Tenant	대형 tenant를 독립 배치하고 이동하기 쉽다.	메타데이터 가용성과 라우팅 cache 일관성이 필요하다.	멀티테넌트 SaaS

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [consistent-hashing; dynamo-paper; bigtable-paper]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Hot key	유명 콘텐츠·대형 tenant 한 키가 한 shard 처리량을 초과한다.	key salting, 읽기 복제, tenant 전용 shard를 사용한다.
재분배 폭풍	노드 추가 후 너무 많은 데이터가 동시에 이동한다.	이동 budget, 우선순위, 작은 단위 migration을 적용한다.
Stale routing	클라이언트가 이전 shard로 쓰기를 보낸다.	ownership epoch와 redirect, idempotency를 사용한다.
Cross-shard 불변조건	유일성·잔액·재고가 여러 shard에 걸쳐 깨진다.	파티션 경계를 바꾸거나 조정된 원장·saga를 둔다.
Global index 불일치	원본 이동·삭제와 색인 업데이트가 어긋난다.	버전·CDC·reconciliation으로 수렴을 검증한다.



source shard에서 target shard로 복사·변경 동기화·검증·소유권 전환하는 단계를 보여준다

확장 전략

- 통계 기반 자동 split은 최대 파티션 크기와 요청률을 함께 고려한다.
- 대형 tenant는 shared pool에서 전용 shard로 승격 가능한 경로를 둔다.
- scatter-gather 쿼리는 fan-out 한도·timeout·partial result 정책을 갖는다.
- 파티션 수를 노드 수와 동일시하지 말고 이동 가능한 작은 단위로 유지한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- tenant ID가 파티션 키일 때 라우팅 메타데이터가 고객 목록을 노출하지 않게 보호한다.
- 파티션 이동 중 암호화 키와 데이터 지역 정책을 보존한다.
- 삭제가 global index·cache·이전 replica에 남는 시간을 추적한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- shard별 데이터 크기·QPS·p99·queue depth
- 키/tenant skew와 상위 hot key
- rebalance backlog·전송량·예상 완료 시간
- cross-shard 요청 비율과 fan-out 폭
- routing redirect·epoch mismatch

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 샤드 수 증가는 인스턴스 비용 외에 연결·백업·메타데이터·운영 자동화 비용을 늘린다.
- 균등 분포를 위해 over-partitioning하면 작은 파티션 관리 비용이 생긴다.
- global secondary index는 읽기 편의 대신 쓰기 증폭과 저장 비용을 만든다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 사용자 ID를 습관적으로 샤드 키로 선택한다.
- 노드 수만큼 샤드를 만들어 이동 단위를 지나치게 크게 만든다.
- consistent hashing이 hot key를 해결한다고 생각한다.
- 재샤딩을 offline maintenance로만 가정한다.

설계 리뷰

- ☐ 파티션 키가 핵심 aggregate와 불변조건을 함께 보존하는가?
- ☐ 키 분포의 p99 tenant와 hot key를 분석했는가?
- ☐ 재분배 중 stale routing과 이중 쓰기를 처리하는가?
- ☐ scatter-gather와 global index 비용이 제한되는가?
- ☐ 데이터 지역·삭제 정책이 이동 중 유지되는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 시간 기반 이벤트를 range partition할 때 최신 파티션 hotspot을 완화하는 방법을 설계하라.
2. tenant 크기가 1MB에서 10TB까지 분포하는 SaaS의 shard 정책을 작성하라.
3. 4개 노드에서 5개 노드로 consistent hash ring을 확장할 때 이동 단위를 설명하라.

핵심 요약

- 샤딩의 핵심은 파티션 키와 데이터 경계다.
- range·hash·directory는 서로 다른 조회와 운영 비용을 가진다.
- hot key는 균등 hash만으로 해결되지 않는다.
- 온라인 재분배와 ownership fencing이 필요하다.
- cross-shard 작업은 데이터 모델 문제로 되돌아가 검토한다.

출처

- [consistent-hashing] David Karger et al.. **Consistent Hashing and Random Trees** (1997). <https://doi.org/10.1145/258533.258660>
- [dynamo-paper] Giuseppe DeCandia et al.. **Dynamo: Amazon's Highly Available Key-value Store** (2007). <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>
- [bigtable-paper] Fay Chang et al.. **Bigtable: A Distributed Storage System for Structured Data** (2006). <https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/>

“

검증 기준일: 2026-08-06. `current` 와 `volatile` 내용은 release 전에 공식 원문을 다시 확인한다.

12

Consensus · Leader Election · Fencing

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

합의는 모든 데이터를 전역 정렬하는 만능 도구가 아니다. 소수의 중요한 메타데이터, leader, membership, lock ownership처럼 단일 결정을 공유해야 할 때 사용하며, 장애 중 안전성과 가용성의 경계를 명확히 한다.

이 절의 기준 출처: [raft-paper; @paxos-made-simple].

학습 목표

- 합의가 필요한 문제와 불필요한 문제를 구분한다.
- term · log replication · majority의 역할을 설명한다.
- lease와 fencing으로 stale leader를 차단한다.

먼저 결론

- majority가 없으면 안전한 새 leader를 선출할 수 없으므로 쓰기를 중단하는 편이 낫은 leader 두 개보다 낫다.
- leader election만으로 stale writer가 사라지지 않으므로 저장소가 term · fencing token을 검증해야 한다.
- 합의 그룹 크기와 지리적 배치는 지연·장애 허용·운영 비용을 함께 결정한다.
- 대용량 사용자 데이터보다 작은 제어 메타데이터에 합의를 집중한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	Consensus · Leader Election · Fencing에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	majority가 없으면 안전한 새 leader를 선출할 수 없으므로 쓰기를 중단하는 편이 낫은 leader 두 개보다 낫다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	합의 그룹을 지나치게 크게 만들지 않고 여러 독립 그룹으로 분할한다.
실패·복구	“Minority island” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	quorum 상실 시 쓰기를 중단하고 fencing을 검증한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	membership · leader 강제 이전 · snapshot 접근 권한을 분리한다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	term 변경·election 시간·leader churn

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

합의

비동기 통신과 일부 실패가 있는 노드들이 하나의 값 또는 로그 순서에 동의하는 문제다.

Term/Epoch

leader 세대를 증가시키는 논리 번호다.

Majority quorum

구성원 절반 초과가 참여한 겹치는 집합으로 두 개의 독립 leader 결정을 막는다.

Log replication

leader가 명령 순서를 복제하고 commit된 prefix를 모든 노드가 동일하게 적용하도록 한다.

Lease

일정 시간 동안 권한이 유효하다는 계약이며 시계 오차와 지연 상한을 고려해야 한다.

Fencing token

새 소유자가 더 큰 번호를 받아 하위 저장소가 오래된 소유자의 쓰기를 거부하게 하는 값이다.

Membership change

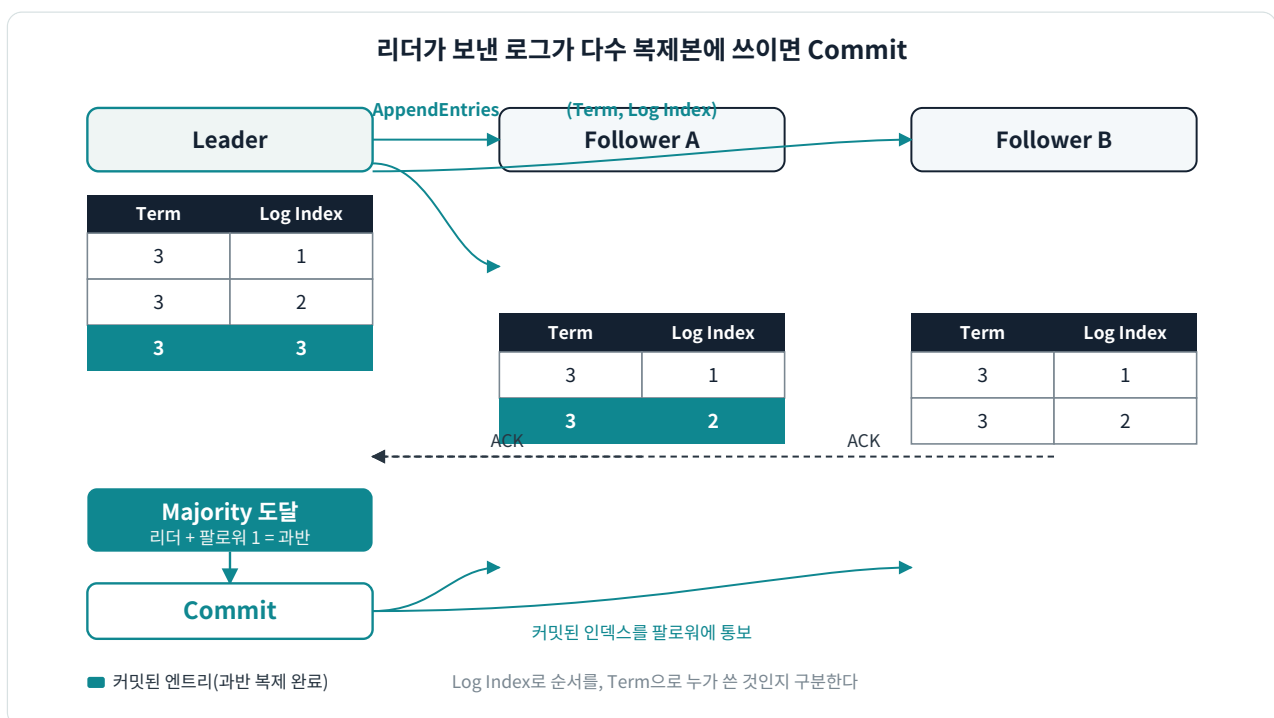
합의 그룹 구성원을 안전하게 추가·제거하는 절차다.

핵심 개념의 정의와 범위는 [raft-paper; @paxos-made-simple; @chubby-paper]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Consensus members	term, vote, replicated log를 보존한다.
Leader	클라이언트 명령을 log에 순서대로 제안한다.
Follower	log를 복제하고 leader 건강을 관찰한다.
Client proxy	현재 leader를 찾고 redirect·retry를 처리한다.
State machine	commit된 명령을 결정적으로 적용한다.
Fenced resource	DB·파일·작업 실행기가 token을 확인해 stale owner를 막는다.
Snapshot/compaction	오래된 log를 압축하고 신규 노드를 빠르게 합류시킨다.



leader가 term·index를 가진 log를 복제하고 majority commit하는 과정을 보여준다

요청·데이터 흐름

1. 클라이언트가 요청 ID와 명령을 leader에 보낸다.
2. leader가 현재 term과 log index를 붙여 replica에 전송한다.
3. majority가 내구성 있게 저장하면 commit index를 전진시킨다.

4. 모든 노드는 같은 순서로 상태 기계에 적용한다.
5. leader 응답이 유실되면 클라이언트는 요청 ID로 결과를 재조회한다.
6. leader 상실 시 더 최신 log를 가진 후보가 새 term에서 선출된다.
7. 외부 자원은 새 fencing token보다 작은 쓰기를 거부한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

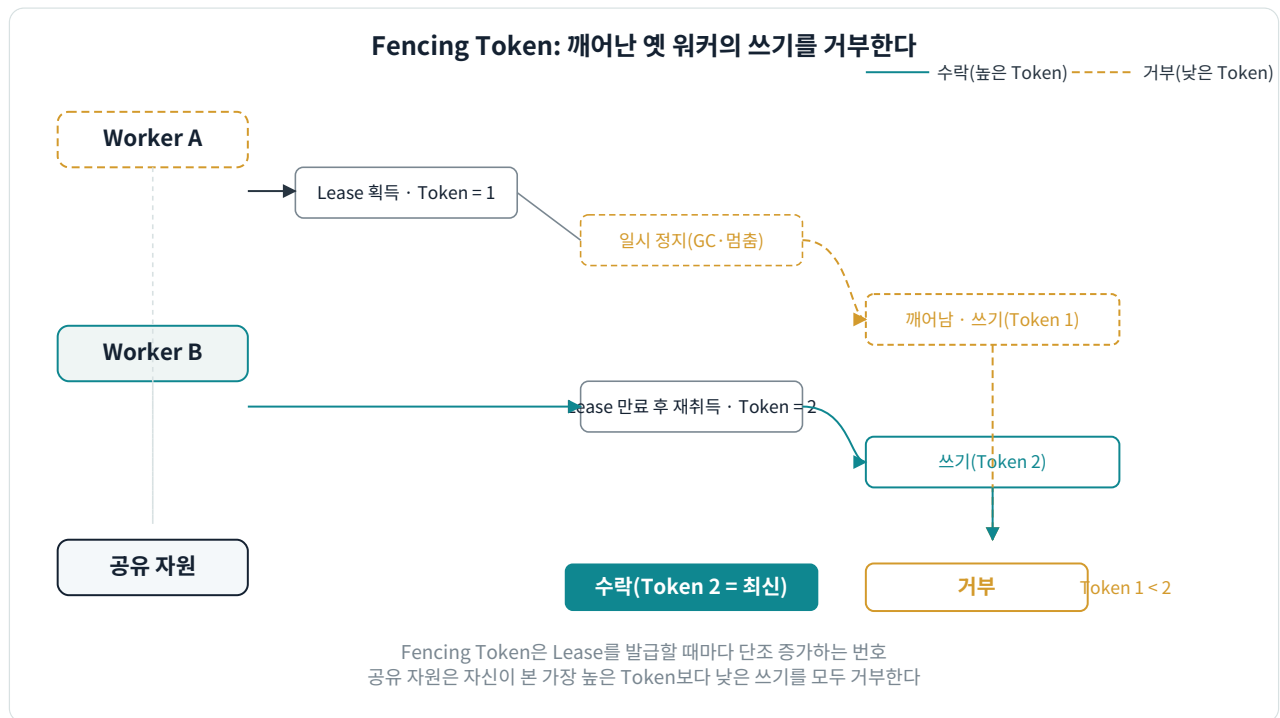
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
Raft형 replicated log	역할과 log 규칙을 설명하기 쉽고 구현이 널리 검증됐다.	membership·snapshot·운영 구현은 여전히 복잡하다.	제어 메타데이터·구성 저장소
외부 합의 서비스 사용	애플리케이션이 직접 알고리즘을 구현하지 않아도 된다.	외부 서비스의 SLO·세션·watch 의미를 이해해야 한다.	leader lease·service discovery
DB row lock/lease	작은 범위에서 단순하고 기존 트랜잭션을 활용한다.	시계·연결 끊김·긴 작업에 stale owner 위험이 있다.	단일 DB 범위 작업 잠금

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [raft-paper; @paxos-made-simple; @chubby-paper]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Minority island	분할된 소수 노드가 이전 leader를 계속 신뢰한다.	quorum 상실 시 쓰기를 중단하고 fencing을 검증한다.
장기 GC pause	leader가 멈춘 동안 새 leader가 선출되고, 이전 leader가 깨어나 다시 쓴다.	term/token을 모든 외부 쓰기에 전달한다.
Disk full	일부 노드가 log를 저장하지 못해 quorum과 snapshot이 정체된다.	디스크 여유·compaction·read-only 보호 모드를 둔다.
잘못된 membership 변경	동시에 여러 구성을 바꿔 겹치지 않는 quorum이 생긴다.	joint consensus 또는 검증된 순차 절차를 사용한다.
결정적 적용 위반	노드별 시간·무작위·외부 호출로 상태 기계 결과가 달라진다.	명령에 필요한 결과를 포함하고 적용 함수를 결정적으로 만든다.



오래된 worker와 새 worker가 같은 자원에 접근할 때 fencing token이 stale write를 막는 모습을 보여준다

확장 전략

- 합의 그룹을 지나치게 크게 만들지 않고 여러 독립 그룹으로 분할한다.
- read-only 요청은 linearizable read 필요 여부에 따라 lease/read-index/stale replica를 선택한다.
- snapshot 생성·전송이 정상 log 복제와 경쟁하지 않도록 제한한다.
- 장거리 다중 리전 quorum은 쓰기 지연을 직접 증가시키므로 배치와 쓰기 소유권을 재검토한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- membership · leader 강제 이전 · snapshot 접근 권한을 분리한다.
- 합의 로그에 비밀 원문을 넣지 않고 암호화 또는 참조를 사용한다.
- fencing token을 클라이언트 주장만 믿지 않고 신뢰된 저장소가 검증한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- term 변경·election 시간·leader churn
- commit latency와 quorum unavailable
- replication match index · snapshot backlog
- stale token 거부 건수
- membership 변경 상태와 disk fsync 오류

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 합의 노드는 소수라도 상시 다중 장애 도메인 비용이 든다.
- 원격 quorum은 매 쓰기에 네트워크 왕복을 추가한다.
- 자체 구현보다 검증된 시스템 운영이 대개 저렴하지만 그 시스템의 장애 의미를 학습해야 한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 분산 lock API를 호출했으니 오래 걸리는 작업이 안전하다고 믿는다.
- heartbeats만으로 lease 안전성을 보장한다.
- 합의 그룹에 대용량 blob과 모든 이벤트를 넣는다.
- quorum 수만 맞추고 failure domain과 membership 변경을 무시한다.

설계 리뷰

- ☐ 합의가 필요한 단일 결정이 정확히 무엇인가?
- ☐ quorum 상실 시 안전한 동작이 정의됐는가?
- ☐ stale leader의 외부 쓰기를 누가 거부하는가?
- ☐ membership·snapshot·disk full을 운영에서 시험했는가?
- ☐ 합의 데이터 범위가 최소화됐는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 작업 스케줄러 leader가 GC pause 후 돌아오는 상황을 fencing token으로 해결하라.
2. 5노드 합의 그룹이 두 zone에 3:2로 배치될 때 zone 장애별 가용성을 분석하라.
3. 합의 로그에 비결정적 명령을 넣었을 때 상태가 갈라지는 예를 만들라.

핵심 요약

- 합의는 중요한 단일 결정과 로그 순서를 공유하는 도구다.
- majority quorum은 서로 겹쳐 두 leader 결정을 막는다.
- leader election과 stale writer 차단은 별도 문제다.
- fencing token은 하위 자원이 검증해야 한다.
- membership · snapshot · disk도 합의 시스템의 핵심 운영 영역이다.

출처

- [raft-paper] Diego Ongaro and John Ousterhout. **In Search of an Understandable Consensus Algorithm** (2014). <https://raft.github.io/raft.pdf>
- [paxos-made-simple] Leslie Lamport. **Paxos Made Simple** (2001). <https://lamport.azurewebsites.net/pubs/paxos-simple.pdf>
- [chubby-paper] Mike Burrows. **The Chubby Lock Service for Loosely-Coupled Distributed Systems** (2006). <https://research.google/pubs/the-chubby-lock-service-for-loosely-coupled-distributed-systems/>

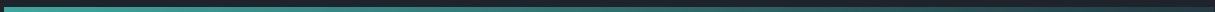
“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

PART

3

네트워크와 서비스 실행 구조



사용자 요청이 전역 네트워크와 서비스 계층을 지나 응답으로 돌아오는 전체 경로를 설계한다.

이 Part에서는 특정 제품의 설정법보다 반복해서 적용할 수 있는 설계 질문과 실패 모델을 먼저 익힌다. 각 장의 체크리스트를 실제 시스템의 ADR·런북·대시보드와 연결해 읽는 것이 목표다.

포함 장

ID	장 제목	최신성	원본 관계
ch13	DNS·CDN·Edge와 전역 트래픽	current	REWRITE
ch14	L4/L7 Load Balancing·Proxy·Gateway	current	REWRITE
ch15	HTTP/1.1·HTTP/2·HTTP/3와 QUIC	current	REPLACE
ch16	REST·gRPC·GraphQL·Web Socket·SSE	current	REPLACE
ch17	모듈러 모놀리스·마이크로서비스·Service Mesh	current	REPLACE

읽는 방법

1. 먼저 각 장의 **요구사항과 실패 모델**을 자신의 시스템에 대입한다.
2. **대안과 트레이드오프**에서 현재 선택이 어떤 비용을 감수하는지 확인한다.
3. **장애 시나리오**를 게임데이 또는 설계 리뷰 질문으로 바꾼다.
4. 시각자료는 설명을 대신하지 않고 책임·흐름·복구 경계를 검증하는 용도로 사용한다.

13

DNS·CDN·Edge와 전역 트래픽

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

전역 트래픽은 “가장 가까운 리전”으로 보내는 문제만이 아니다. DNS TTL, resolver cache, CDN cache key, 원본 건강, 데이터 주권, failover 수렴 시간을 함께 설계해야 사용자가 실제로 안전한 리전에 도달한다.

이 절의 기준 출처: [rfc1034; rfc1035].

학습 목표

- DNS와 CDN이 요청 경로와 장애 복구에 미치는 영향을 설명한다.
- 전역 라우팅 정책을 지연·건강·데이터 위치 요구로 선택한다.
- 캐시·원본·purge 실패를 포함한 edge 운영을 설계한다.

먼저 결론

- DNS 변경은 즉시 전파되지 않으므로 TTL과 resolver 행동을 복구 시간에 포함한다.
- CDN cache key와 개인화 경계를 잘못 잡으면 정보가 섞이거나 적중률이 붕괴한다.
- edge는 정적 자산뿐 아니라 인증 전 검증·rate limit·간단한 계산을 수행할 수 있지만 원본과 정책 일관성을 관리해야 한다.
- 전역 failover는 트래픽 전환 후 데이터 쓰기 권한과 용량까지 검증해야 한다.

∴ current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06이며, 출판 직전 공식 문서를 다시 확인한다. ∴

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	DNS·CDN·Edge와 전역 트래픽에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	DNS 변경은 즉시 전파되지 않으므로 TTL과 resolver 행동을 복구 시간에 포함한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	정적·공개·개인화·쓰기 요청을 서로 다른 edge 정책으로 분리한다.

차원	확인 질문	설계 판단
실패·복구	“DNS stale cache” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	짧은 TTL만 믿지 말고 endpoint 자체의 redirect·proxy fallback을 둔다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	TLS 개인키와 인증서 배포 범위를 최소화하고 갱신 실패를 감시한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	DNS 응답·TTL·resolver별 stale 비율

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Authoritative DNS

도메인에 대한 권한 있는 레코드를 제공한다.

Recursive resolver

클라이언트를 대신해 DNS 계층을 조회하고 TTL 동안 캐시한다.

TTL

레코드를 재조회하기 전 캐시할 수 있는 시간이다.

Anycast

여러 위치가 같은 IP prefix를 광고해 네트워크 경로상 가까운 곳으로 라우팅한다.

CDN cache key

어떤 요청을 같은 객체로 취급할지 정하는 키다.

Origin shield

edge miss를 한 단계에서 모아 원본 fan-in과 burst를 줄인다.

Edge compute

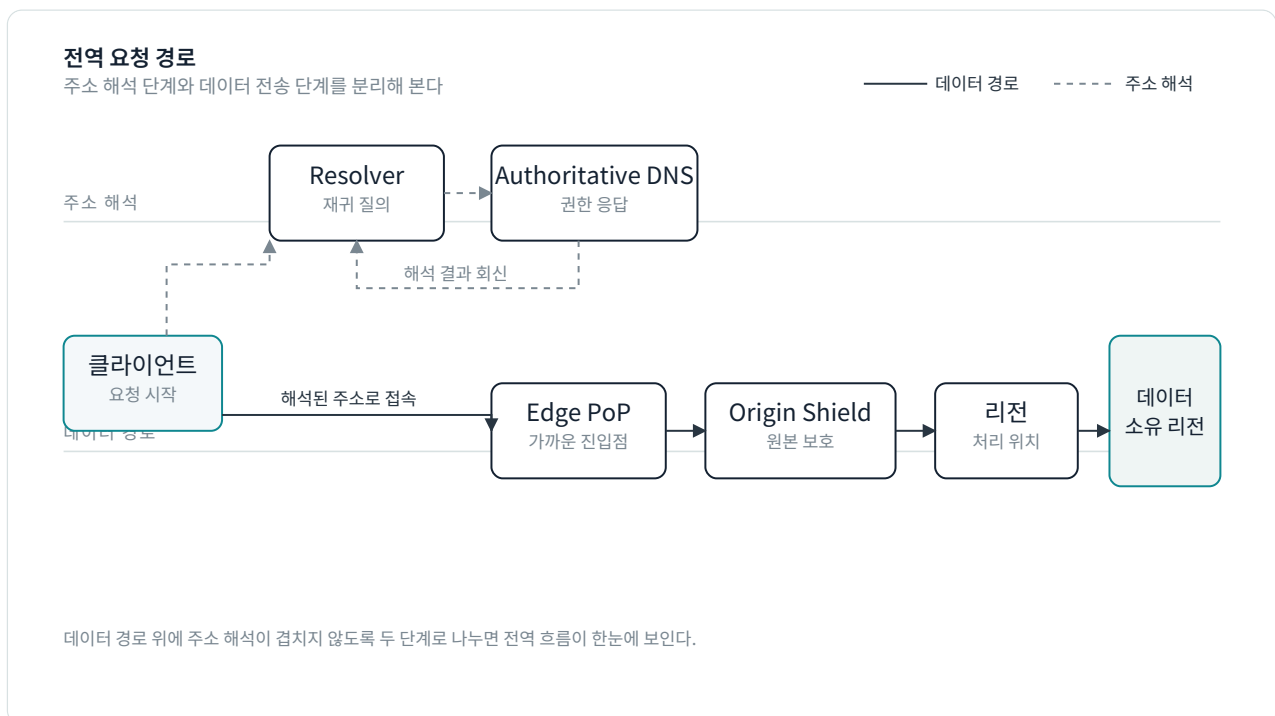
사용자 가까이에서 제한된 요청 처리·정책·변환을 수행한다.

핵심 개념의 정의와 범위는 [rfc1034; rfc1035; rfc9111]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
권한 DNS	지역·건강·정책에 따라 endpoint를 반환한다.
CDN/Edge PoP	TLS 종료, cache, WAF, 요청 정규화를 수행한다.
Origin shield	원본 요청을 집계하고 재검증한다.
지역 ingress	해당 리전의 gateway로 요청을 받는다.
데이터 소유 리전	쓰기 권한과 일관성 정책을 보유한다.
전역 제어면	설정, 인증서, purge, 라우팅 정책을 배포한다.
합성 검사	외부 관점의 DNS·TLS·콘텐츠 건강을 확인한다.



resolver·authoritative DNS·edge·shield·region·data owner를 지나는 전역 요청 경로를 보여준다

요청·데이터 흐름

1. 클라이언트가 recursive resolver를 통해 도메인을 조회한다.
2. resolver가 TTL과 정책에 따라 endpoint를 캐시한다.
3. 요청이 edge PoP에 도착해 TLS·보안·cache key를 평가한다.

4. hit이면 edge에서 응답하고 miss면 shield 또는 origin으로 전달한다.
5. 지역 ingress가 쓰기 소유권과 데이터 위치를 확인한다.
6. 응답 cacheability와 vary 조건을 명시한다.
7. 건강 저하 시 DNS·anycast·CDN 정책이 단계적으로 우회한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
DNS 지리 라우팅	단순하고 다양한 endpoint로 유도할 수 있다.	캐시된 응답 때문에 전환이 느리고 세밀한 요청 판단이 어렵다.	리전 단위 전환
Anycast/Global proxy	빠른 네트워크 우회와 단일 IP 경험을 제공한다.	공급자 제어면과 경로 정책에 의존한다.	글로벌 HTTP ingress
애플리케이션 리다이렉트	사용자·tenant·데이터 위치를 정밀하게 반영한다.	첫 요청 왕복과 redirect loop 위험이 있다.	로그인 후 home region 고정

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [RFC1034; RFC1035; RFC9111]를 참조한다.

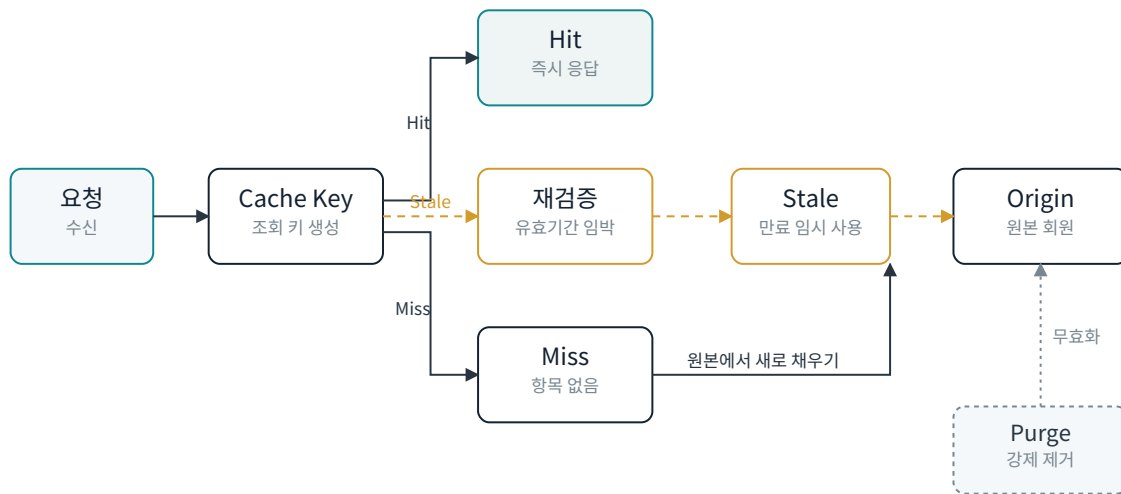
장애 시나리오

시나리오	영향	대응 원칙
DNS stale cache	장애 리전 주소가 TTL 동안 계속 사용된다.	짧은 TTL만 믿지 말고 endpoint 자체의 redirect·proxy fallback을 둔다.
Cache poisoning/키 오류	인증 헤더·쿠키가 cache key에서 빠져 사용자 응답이 섞인다.	기본 비공개, 명시적 cache key, 보안 테스트를 적용한다.
Thundering miss	인기 객체 만료와 함께 모든 PoP가 원본을 호출한다.	request coalescing, stale-while-revalidate, shield를 사용한다.
Purge 지연	잘못된 콘텐츠나 보안 패치가 일부 PoP에 남는다.	versioned URL, purge 상태 관측, 짧은 비상 TTL을 둔다.
전환 후 원본 포화	장애 리전 트래픽이 남은 리전 용량을 초과한다.	failover capacity와 admission policy를 사전 검증한다.

캐시 수명 주기

Cache Key 한 개가 Hit·Miss·재검증·Stale로 갈라지는 흐름

—— 정상 경로 - - - - 검증·갱신 Purge



cache key 생성·hit/miss·revalidation·stale fallback·purge 흐름을 보여준다

확장 전략

- 정적·공개·개인화·쓰기 요청을 서로 다른 edge 정책으로 분리한다.
- cache key cardinality와 객체 크기 분포를 관리한다.
- 리전 전환은 트래픽 비율을 단계적으로 올리고 데이터 소유권을 함께 이동한다.
- edge 설정 배포도 canary와 rollback을 지원해야 한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- TLS 개인키와 인증서 배포 범위를 최소화하고 갱신 실패를 감시한다.
- 개인화 응답은 기본적으로 공유 cache에 저장하지 않는다.
- 지역별 데이터 주권과 로그 반출 정책을 edge까지 적용한다.
- Host, X-Forwarded-For 같은 전달 헤더의 신뢰 경계를 명확히 한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- DNS 응답·TTL·resolver별 stale 비율
- edge hit/miss/revalidation과 cache key cardinality
- PoP·region별 p95/p99와 origin fetch 지연
- purge 전파 시간과 stale object 수
- failover 후 트래픽·용량·오류 분포

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- CDN은 origin egress와 compute 비용을 줄일 수 있지만 request·purge·edge compute 비용을 만든다.
- 짧은 TTL은 DNS 조회량을 늘리고 반드시 빠른 전환을 보장하지 않는다.
- 다중 CDN은 공급자 장애를 줄일 수 있으나 설정·로그·인증서·계약 비용이 크게 늘어난다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- TTL을 0으로 하면 즉시 failover된다고 믿는다.
- 모든 GET 응답을 안전하게 공유 cache할 수 있다고 생각한다.
- edge 건강만 보고 데이터 쓰기 가능 여부를 확인하지 않는다.
- CDN purge를 배포 전략 대신 사용한다.

설계 리뷰

- ☐ DNS cache 수렴 시간이 RTO에 포함됐는가?
- ☐ cache key가 인증·언어·압축·query 의미를 정확히 반영하는가?
- ☐ origin shield와 stale 정책이 원본 burst를 줄이는가?
- ☐ 전환 대상 리전의 데이터 권한과 여유 용량이 검증됐는가?
- ☐ edge 설정과 인증서의 제어면 장애를 고려했는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 개인화된 뉴스 홈과 공개 기사 본문에 서로 다른 CDN cache key 정책을 설계하라.
2. TTL 300초인 레코드의 리전 failover가 사용자에게 보이는 최악 경로를 그려라.
3. 인기 파일 1개가 동시에 만료될 때 origin 요청을 제한하는 계층을 설계하라.

핵심 요약

- DNS와 CDN cache는 복구 시간을 지연시킬 수 있다.
- cache key는 성능뿐 아니라 데이터 격리 규칙이다.
- edge와 origin의 책임·설정·관측을 분리한다.
- 전역 failover는 트래픽과 데이터 쓰기 권한을 함께 전환한다.
- TTL·purge·origin capacity를 실제로 시험한다.

출처

- [rfc1034] IETF. **RFC 1034 — Domain Names: Concepts and Facilities** (1987). <https://www.rfc-editor.org/rfc/rfc1034.html>
- [rfc1035] IETF. **RFC 1035 — Domain Names: Implementation and Specification** (1987). <https://www.rfc-editor.org/rfc/rfc1035.html>
- [rfc9111] IETF. **RFC 9111 — HTTP Caching** (2022). <https://www.rfc-editor.org/rfc/rfc9111.html>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

14

L4/L7 Load Balancing · Proxy · Gateway

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

트래픽 분산 계층은 단순한 round-robin 장치가 아니다. 연결과 요청의 수명, 건강 상태, 정책, 재시도, TLS, 헤더 신뢰, 배포 전환을 다루기 때문에 계층을 늘릴 때마다 책임과 중복 기능을 명확히 해야 한다.

이 절의 기준 출처: [rfc9110; rfc9112].

학습 목표

- L4와 L7 분산의 관측·정책 차이를 설명한다.
- reverse proxy · API gateway · service proxy의 책임을 구분한다.
- 건강 검사·connection draining·재시도가 장애에 미치는 영향을 설계한다.

먼저 결론

- L4는 연결 단위, L7은 HTTP 요청 의미를 활용해 라우팅한다.
- health check 통과와 실제 사용자 요청 성공은 다를 수 있다.
- gateway가 무제한 재시도하면 하위 서비스 과부하를 확대한다.
- proxy가 추가한 identity · client IP 헤더는 신뢰 가능한 hop에서만 받아들인다.

∴ current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06 이며, 출판 직전 공식 문서를 다시 확인한다. ∴

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	L4/L7 Load Balancing · Proxy · Gateway에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	L4는 연결 단위, L7은 HTTP 요청 의미를 활용해 라우팅한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	연결 수와 요청 수를 모두 고려해 endpoint 부하를 계산한다.
실패·복구	“거짓 건강” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	readiness에 핵심 의존성을 제한적으로 반영하고 passive health를 사용한다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	TLS 종료 지점마다 평문 구간과 키 보유 범위를 기록한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	계층별 active connection · request rate · queue

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

L4 load balancing

IP · 포트 · 연결 정보를 중심으로 전달한다.

L7 load balancing

Host, path, method, header 등 애플리케이션 프로토콜 정보를 활용한다.

Reverse proxy

서버 앞에서 연결 종료 · 라우팅 · 캐시 · 정책을 수행한다.

API gateway

외부 API의 인증, quota, 버전, routing, 변환 같은 공통 경계를 제공한다.

Connection draining

배포 · 제거 중 신규 연결을 막고 기존 요청을 제한 시간 동안 마치는 절차다.

Passive health

실제 요청 오류를 건강 판단에 반영하는 방식이다.

Outlier detection

비정상 endpoint를 일시 격리하는 정책이다.

핵심 개념의 정의와 범위는 [rfc9110; rfc9112; google-sre-overload]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
전역 ingress	공용 endpoint와 DDoS·TLS 정책을 담당한다.
L7 gateway	API 인증·라우팅·request limit을 수행한다.
L4 balancer	연결을 지역 서비스 endpoint로 분산한다.
서비스 proxy	retry·timeout·mTLS·관측을 하위 서비스 호출에 적용한다.
Endpoint registry	건강한 인스턴스와 배포 버전을 제공한다.
Policy store	route·quota·header 신뢰 정책을 버전 관리한다.



전역 ingress·L7 gateway·L4 balancer·service proxy·endpoint의 책임을 계층별로 보여준다

요청·데이터 흐름

1. 연결이 L4 또는 L7 ingress에 도착한다.
2. TLS 종료 위치와 client identity 전달 방식을 결정한다.
3. L7 계층이 route·권한·크기·quota를 검증한다.
4. healthy endpoint 집합에서 locality·load·hash 정책으로 대상을 선택한다.

5. 남은 deadline과 재시도 예산을 전달한다.
6. 응답·reset·timeout을 passive health에 반영한다.
7. 배포 제거 시 endpoint를 먼저 제외하고 연결을 drain한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

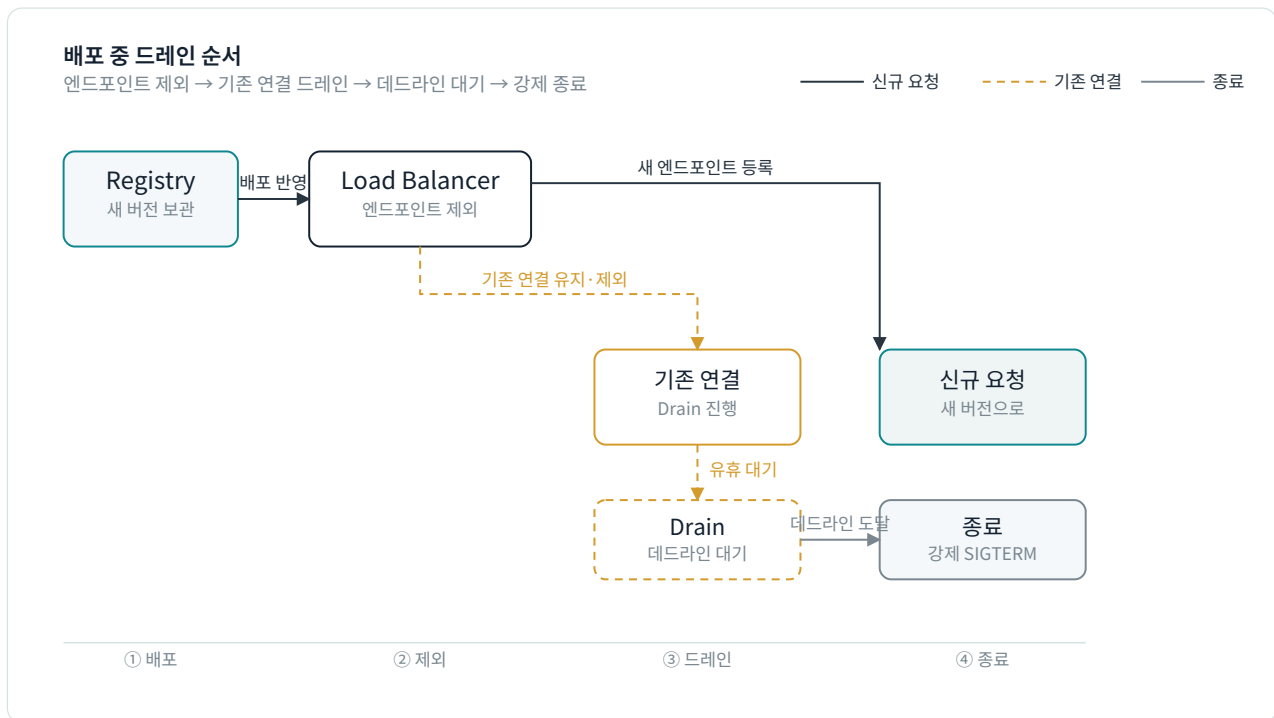
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
L4 중심	프로토콜 독립적이고 처리 오버헤드가 낮다.	요청 단위 정책·관측·세밀한 라우팅이 어렵다.	TCP/UDP 서비스·고성능 ingress
L7 중심	콘텐츠 기반 routing과 공통 보안 정책이 풍부하다.	CPU·메모리·설정 복잡도와 새로운 장애 지점이 생긴다.	HTTP API·점진 배포
Client-side balancing	중간 hop을 줄이고 서비스별 판단이 가능하다.	클라이언트 라이브러리·서비스 발견 일관성이 필요하다.	내부 RPC

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [RFC9110; RFC9112; @google-sre-overload]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
거짓 건강	얕은 /health 는 성공하지만 DB·thread pool이 포화됐다.	readiness에 핵심 의존성을 제한적으로 반영하고 passive health를 사용한다.
Retry amplification	gateway와 client와 service proxy가 각 각 재시도한다.	한 계층이 retry budget을 소유하고 시도 횟수를 전달한다.
Drain 실패	종료 신호 직후 기존 연결이 끊기고 긴 요청이 유실된다.	endpoint 제거→drain→종료 순서와 최대 요청 시간을 정한다.
Sticky overload	세션 affinity가 일부 endpoint에 부하를 고정한다.	bounded-load hashing과 세션 상태 외부화를 검토한다.
Header spoofing	외부 사용자가 내부 identity·IP 헤더를 직접 보낸다.	edge에서 제거 후 신뢰된 proxy가 재작성한다.



배포 중 endpoint 제외·기존 연결 drain·deadline 대기·강제 종료 순서를 보여준다

확장 전략

- 연결 수와 요청 수를 모두 고려해 endpoint 부하를 계산한다.
- long-lived connection은 신규 endpoint에 자동 재분배되지 않으므로 reconnect 정책을 둔다.
- route config가 커질수록 계층별 ownership과 validation을 자동화한다.
- gateway를 비즈니스 orchestration 계층으로 키우지 않고 공통 경계 책임에 제한한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- TLS 종료 지점마다 평문 구간과 키 보유 범위를 기록한다.
- gateway 인증 결과를 서명·mTLS·짧은 수명 토큰으로 하위에 전달한다.
- request smuggling을 막기 위해 hop 간 HTTP parsing과 header 정규화를 일치시킨다.
- 관리 API와 데이터면 endpoint를 분리하고 최소 권한을 적용한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 계층별 active connection · request rate · queue
- endpoint별 p95/p99 · reset · 5xx · ejection
- route match · no route · auth · quota 거부
- retry attempts와 amplification factor
- drain 중 강제 종료된 요청 수

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace · log · 변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- proxy hop은 컴퓨팅 · TLS · 네트워크 비용을 추가한다.
- 중복 gateway · mesh 기능은 라이선스보다 설정 · 디버깅 인력 비용이 더 클 수 있다.
- client-side balancing은 인프라 hop을 줄이지만 SDK 배포 · 호환 비용을 만든다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제 · 백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- L4와 L7을 “빠름/느림” 한 문장으로만 비교한다.
- 모든 공통 로직을 gateway에 넣는다.
- health check endpoint 하나로 실제 사용자 경로를 대표한다.
- 재시도와 timeout을 각 계층이 독립 설정한다.

설계 리뷰

- ☐ 각 proxy 계층의 단일 책임이 명확한가?
- ☐ TLS와 identity 신뢰 경계가 hop별로 문서화됐는가?
- ☐ 건강 · drain · retry 정책이 실제 요청 수명과 맞는가?
- ☐ long-lived connection과 sticky routing을 고려했는가?
- ☐ 설정 오류를 canary · 검증 · rollback할 수 있는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. WebSocket 서비스의 L4/L7 분산과 배포 drain 절차를 설계하라.
2. client · gateway · service가 모두 3회 재시도할 때 최대 시도 수를 계산하고 단일 retry budget으로 바꾸라.
3. 신뢰할 수 있는 client IP 전달 헤더 체인을 설계하라.

핵심 요약

- L4는 연결, L7은 요청 의미를 중심으로 분산한다.
- proxy 계층마다 책임과 재시도 소유자를 하나로 둔다.
- health check와 실제 사용자 건강을 분리해 본다.
- 배포에는 endpoint 제거와 connection draining이 필요하다.
- 전달 헤더와 TLS 종료는 보안 경계다.

출처

- [rfc9110] IETF. **RFC 9110 — HTTP Semantics** (2022). <https://www.rfc-editor.org/rfc/rfc9110.html>
- [rfc9112] IETF. **RFC 9112 — HTTP/1.1** (2022). <https://www.rfc-editor.org/rfc/rfc9112.html>
- [google-sre-overload] Google. **Site Reliability Engineering — Handling Overload** (2016). <https://sre.google/sre-book/handling-overload/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

15

HTTP/1.1 · HTTP/2 · HTTP/3와 QUIC

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

HTTP/3가 항상 더 빠른 것은 아니다. HTTP semantics는 유지되지만 전송은 QUIC 위에서 이루어지고, 독립 스트림·TLS 통합·connection migration 같은 특성을 얻는 대신 UDP 경로, 관측 도구, proxy 지원, QPACK 동작을 함께 검증해야 한다.

이 절의 기준 출처: [rfc9110; rfc9112].

학습 목표

- HTTP 세대별 연결·스트림·head-of-line 특성을 비교한다.
- QUIC의 연결 설정·암호화·경로 변경이 운영에 미치는 영향을 설명한다.
- 프로토콜 선택을 실제 client·network·proxy 지원 조건과 연결한다.

먼저 결론

- HTTP/1.1은 여러 연결과 순차 요청, HTTP/2는 한 TCP 연결의 다중 스트림, HTTP/3는 QUIC 연결의 다중 스트림을 사용한다.
- HTTP/2의 서로 다른 스트림도 TCP packet loss 때문에 전송 계층 head-of-line 영향을 함께 받을 수 있다.
- HTTP/3는 QUIC stream 간 손실 격리를 제공하지만 congestion과 네트워크 경로는 공유한다.
- 0-RTT는 재전송 가능성을 고려해 안전한 요청에만 사용한다.

∴ current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06 이며, 출판 직전 공식 문서를 다시 확인한다. ∴

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	HTTP/1.1·HTTP/2·HTTP/3와 QUIC에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	HTTP/1.1은 여러 연결과 순차 요청, HTTP/2는 한 TCP 연결의 다중 스트림, HTTP/3는 QUIC 연결의 다중 스트림을 사용한다.

차원	확인 질문	설계 판단
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	연결 수보다 stream 수 · congestion · CPU 암호화 비용을 함께 본다.
실패·복구	“UDP 차단/제한” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	빠른 fallback과 protocol별 성공률을 측정한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	QUIC은 기본적으로 암호화되지만 endpoint 인증과 애플리케이션 권한은 여전히 필요하다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	protocol별 연결 성공 · fallback · handshake 시간

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

HTTP semantics

method, status, field, representation 같은 의미는 HTTP 버전 간 공유된다.

Multiplexing

한 연결에서 여러 요청·응답 스트림을 동시에 진행하는 방식이다.

Head-of-line blocking

앞선 손실이나 작업 때문에 뒤의 독립 작업도 대기하는 현상이다.

QUIC connection

UDP 위에서 암호화·신뢰성·혼잡 제어·다중 스트림을 제공한다.

Connection ID

IP·port 변경과 독립적으로 연결을 식별해 경로 변경을 지원한다.

QPACK

HTTP/3에서 field compression을 수행하며 동적 table 의존과 blocking 한계를 관리한다.

0-RTT

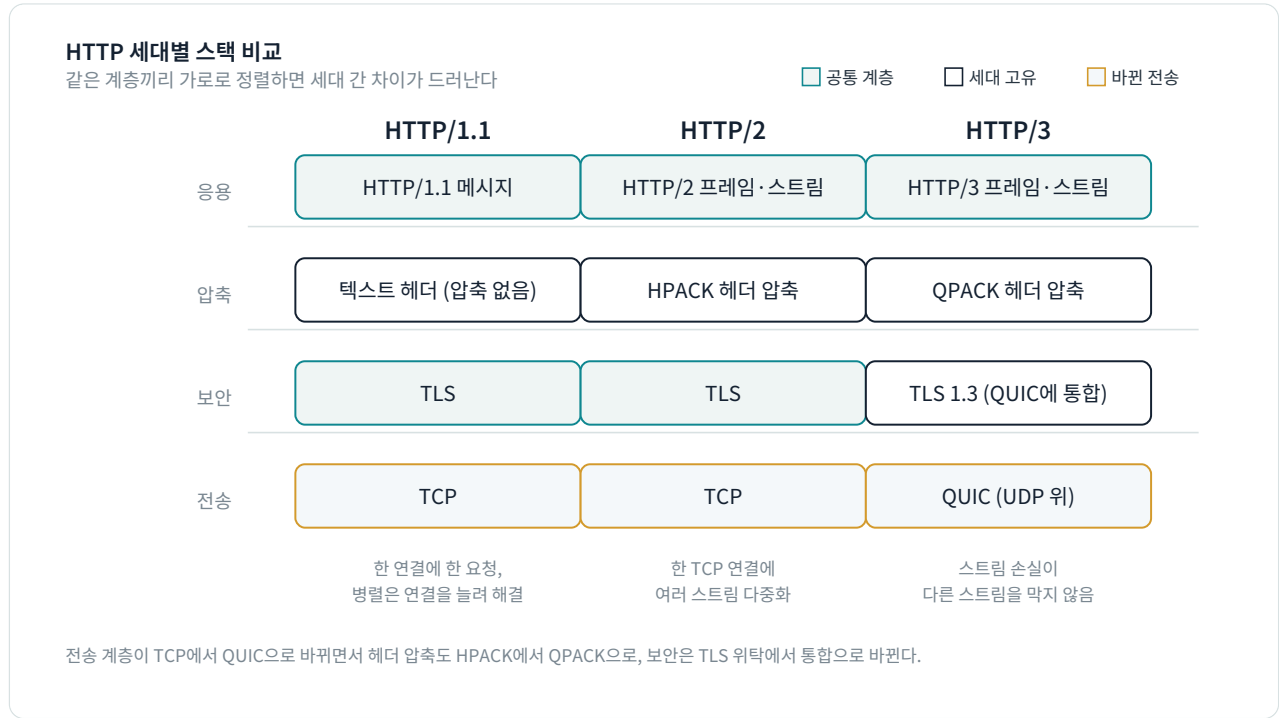
이전 연결 상태를 이용해 handshake 완료 전 application data를 보내는 방식으로 replay 위험이 있다.

핵심 개념의 정의와 범위는 [@rfc9110; @rfc9112; @rfc9113; @rfc9000; @rfc9114; @rfc9204]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
클라이언트	지원 protocol을 협상하고 연결·stream을 관리한다.
DNS/Alt-Svc 계층	HTTP/3 endpoint 사용 가능성을 알린다.
Edge/QUIC endpoint	UDP 수신, TLS 1.3, QUIC transport를 처리한다.
HTTP gateway	버전 간 변환과 request policy를 적용한다.
Origin service	HTTP semantics에 따라 요청을 처리한다.
Telemetry pipeline	protocol, handshake, loss, fallback을 기록한다.



HTTP/1.1·HTTP/2·HTTP/3의 application·compression·transport·security stack을 비교한다

요청·데이터 흐름

1. 클라이언트가 DNS와 이전 Alt-Svc 정보를 확인한다.
2. HTTP/3 가능 시 QUIC handshake와 TLS 인증을 수행한다.
3. 요청마다 독립 stream을 열고 HTTP field를 QPACK으로 표현한다.
4. packet loss는 해당 stream 데이터 재전송에 영향을 주되 다른 stream 전송은 계속될 수 있다.
5. gateway가 필요하면 origin의 HTTP/2 또는 HTTP/1.1로 변환한다.
6. 경로 변경 시 connection ID로 연결을 유지한다.
7. 실패 시 정책에 따라 다른 protocol로 fallback하고 이유를 기록한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
HTTP/1.1	도구·서버·중간장비 호환성이 넓고 단순하다.	병렬 처리를 위해 여러 연결이 필요하고 순차화 문제가 있다.	단순 origin·legacy path
HTTP/2	한 TCP 연결에서 다중 스트림과 header compression을 제공한다.	TCP loss가 연결 내 모든 stream 전송을 지연시킬 수 있다.	안정된 네트워크·내부 RPC
HTTP/3	stream 손실 격리, 빠른 handshake, 경로 변경을 제공한다.	UDP 차단·CPU·관측·proxy 지원을 검증해야 한다.	모바일·장거리·손실 네트워크

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [rfc9110; rfc9112; rfc9113]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
UDP 차단/제한	기업망·방화벽이 QUIC을 차단하거나 낮은 timeout을 적용한다.	빠른 fallback과 protocol별 성공률을 측정한다.
Handshake 실패	인증서·TLS·version negotiation 문제로 연결이 성립하지 않는다.	failure reason과 client/network 구간을 구분한다.
0-RTT replay	재전송된 초기 요청이 부작용을 두 번 만든다.	역등·안전한 요청만 허용하고 anti-replay 정책을 둔다.

- 암호화로 전통적 네트워크 관측이 어려워지므로 endpoint telemetry를 강화한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- protocol별 연결 성공·fallback·handshake 시간
- stream별 p95/p99와 packet loss·retransmission
- 0-RTT 시도·승인·거부·재시도
- QUIC CPU·메모리·connection migration
- edge-origin version 조합별 오류

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- HTTP/3는 지연을 줄일 수 있지만 edge·CPU·관측·운영 도구 비용을 늘릴 수 있다.
- 한 connection의 효율은 좋아져도 장거리 egress 비용 자체는 줄지 않는다.
- 두 protocol을 장기간 동시에 운영하면 테스트 행렬과 장애 분석 비용이 증가한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- HTTP/3를 UDP 기반이라 신뢰성이 없다고 설명한다.
- HTTP/2면 애플리케이션의 모든 head-of-line 문제가 사라진다고 생각한다.
- 0-RTT를 모든 POST 요청에 허용한다.
- 벤치마크 한 번으로 모든 네트워크에서 HTTP/3가 빠르다고 결론낸다.

설계 리뷰

- ☐ 지원 client·network·proxy 조합이 실제 트래픽으로 검증됐는가?
- ☐ fallback이 빠르고 이유를 관측할 수 있는가?

- 0-RTT 허용 요청이 replay-safe한가?
- protocol 변환 hop에서 deadline·reset 의미가 보존되는가?
- CPU·loss·mobile 경로에서 p99 효과를 측정했는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 같은 리전 내부 RPC와 모바일 글로벌 API에 서로 다른 HTTP 버전 전략을 선택하라.
2. 0-RTT로 중복 결제가 발생할 수 있는 요청 흐름을 그리고 방지책을 제시하라.
3. HTTP/2 TCP loss와 HTTP/3 stream loss 격리를 시간축으로 비교하라.

핵심 요약

- HTTP semantics는 버전 간 공유되지만 transport 특성이 다르다.
- HTTP/2는 TCP, HTTP/3는 QUIC 위에서 다중 stream을 제공한다.
- HTTP/3는 stream 손실 격리와 경로 변경을 제공한다.
- 0-RTT는 replay 위험 때문에 제한적으로 사용한다.
- 실제 이득은 client·network·proxy별 관측으로 판단한다.

출처

- [rfc9110] IETF. **RFC 9110 — HTTP Semantics** (2022). <https://www.rfc-editor.org/rfc/rfc9110.html>
- [rfc9112] IETF. **RFC 9112 — HTTP/1.1** (2022). <https://www.rfc-editor.org/rfc/rfc9112.html>
- [rfc9113] IETF. **RFC 9113 — HTTP/2** (2022). <https://www.rfc-editor.org/rfc/rfc9113.html>
- [rfc9000] IETF. **RFC 9000 — QUIC: A UDP-Based Multiplexed and Secure Transport** (2021). <https://www.rfc-editor.org/rfc/rfc9000.html>
- [rfc9114] IETF. **RFC 9114 — HTTP/3** (2022). <https://www.rfc-editor.org/rfc/rfc9114.html>
- [rfc9204] IETF. **RFC 9204 — QPACK: Field Compression for HTTP/3** (2022). <https://www.rfc-editor.org/rfc/rfc9204.html>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

16

REST · gRPC · GraphQL · WebSocket · SSE

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

REST, gRPC, GraphQL, WebSocket, SSE는 서로를 완전히 대체하는 경쟁 제품이 아니다. 자원 중심 공개 API, 타입이 강한 내부 RPC, 클라이언트 조합 조회, 양방향 실시간 연결, 서버 단방향 이벤트라는 서로 다른 상호작용을 해결한다.

이 절의 기준 출처: [rfc9110; @grpc-core].

학습 목표

- 요청·스트리밍·구독 패턴에 맞춰 통신 방식을 선택한다.
- deadline·취소·버전·오류 계약을 protocol보다 먼저 설계한다.
- 실시간 연결의 재연결·순서·backpressure를 다룬다.

먼저 결론

- protocol 이름보다 호출 방향, 메시지 빈도, 연결 수명, 브라우저 지원, 캐시, 실패 복구 요구를 먼저 적는다.
- deadline·취소·idempotency·오류 의미는 어떤 protocol에서도 필요하다.
- GraphQL은 over/under-fetch를 줄일 수 있지만 비용 제한과 field-level 권한이 필요하다.
- WebSocket과 SSE는 연결이 끊어지는 것을 정상 조건으로 보고 resume·중복·순서 계약을 둔다.

∴ current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06이며, 출판 직전 공식 문서를 다시 확인한다. ∴

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	REST·gRPC·GraphQL·WebSocket·SSE에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	protocol 이름보다 호출 방향, 메시지 빈도, 연결 수명, 브라우저 지원, 캐시, 실패 복구 요구를 먼저 적는다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	realtime connection state를 stateless gateway와 shared subscription index로 분리한다.
실패·복구	“Schema breaking change” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	additive evolution, deprecation, compatibility CI를 사용한다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	field·method 단위 권한을 schema와 함께 검증한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	method/operation별 p95·오류 code·deadline exceeded

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

REST 스타일

HTTP method와 resource representation을 활용하는 API 설계 방식이다.

gRPC

IDL 기반 service/method와 unary·streaming RPC를 제공한다.

GraphQL

클라이언트가 schema의 field를 선택해 query/mutation/subscription을 수행한다.

WebSocket

HTTP handshake 후 양방향 message channel을 제공한다.

SSE

HTTP response를 유지하며 서버가 text event stream을 단방향 전송한다.

Deadline propagation

상위 요청의 남은 시간을 하위 호출에 전달하는 계약이다.

Resume token

재연결 시 마지막으로 처리한 위치에서 이어받기 위한 cursor·event ID다.

핵심 개념의 정의와 범위는 [rfc9110; @grpc-core; @graphql-spec; @rfc6455; @html-sse]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
외부 API gateway	인증·quota·version·HTTP 정책을 적용한다.
REST/GraphQL facade	클라이언트 요구를 도메인 호출로 조합한다.
gRPC service	내부 타입 계약과 streaming을 제공한다.
Realtime gateway	WebSocket/SSE connection과 subscription을 관리한다.
Event backbone	실시간 fan-out 전에 내구성 있는 순서와 replay를 제공한다.
Schema registry	IDL·GraphQL schema·event contract 호환성을 검증한다.

RPC 패턴과 프로토콜 적합도

세로는 호출 형태, 가로는 프로토콜. 채운 정도가 적합도를 뜻한다

● 매우 적합 ● 부분 적합 ○ 부적합·과잉

	REST	gRPC	GraphQL	WebSocket	SSE
Unary 요청 1 · 응답 1	●	●	●	○	○
Server Stream 요청 1 · 응답 여러	●	●	●	●	●
Client Stream 요청 여러 · 응답 1	○	●	●	●	○
Bidirectional 양방향 지속	○	●	●	●	○

행을 읽으면 그 호출 형태에 어떤 프로토콜이 어울리는지가 보인다. 예: Server Stream엔 SSE와 gRPC, Bidirectional엔 WebSocket과 gRPC가 강하다.

unary · server streaming · client streaming · bidirectional과 REST/gRPC/GraphQL/WebSocket/SSE의 적합도를 비교한다

요청·데이터 흐름

- 클라이언트의 상호작용 패턴을 unary · server stream · client stream · bidirectional로 분류한다.
- 공개·내부·브라우저 경계에 맞는 protocol을 선택한다.
- 요청 ID, deadline, auth context, idempotency를 전달한다.
- 서버가 오류를 retryable · permanent · auth · quota로 구분한다.

5. stream은 sequence/cursor와 backpressure를 관리한다.
6. 재연결 시 resume token으로 누락·중복을 보정한다.
7. schema 변경을 compatibility test로 배포한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
REST/HTTP JSON	브라우저·도구·캐시 친화적이고 공개 계약이 쉽다.	타입·streaming·세밀한 조합에 추가 규칙이 필요하다.	공개 API·CRUD
gRPC	강한 schema와 효율적 streaming·codegen을 제공한다.	브라우저·proxy·디버깅 경로를 준비해야 한다.	내부 서비스·고빈도 RPC
GraphQL	클라이언트별 화면 조합과 schema 탐색성이 좋다.	query 비용·N+1·권한·cache가 복잡하다.	다양한 UI의 aggregation
WebSocket/SSE	낮은 지연의 지속 업데이트를 제공한다.	connection state·재연결·fan-out 운영이 필요하다.	채팅·알림·진행 상태

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [rfc9110; @grpc-core; @graphql-spec]를 참조한다.

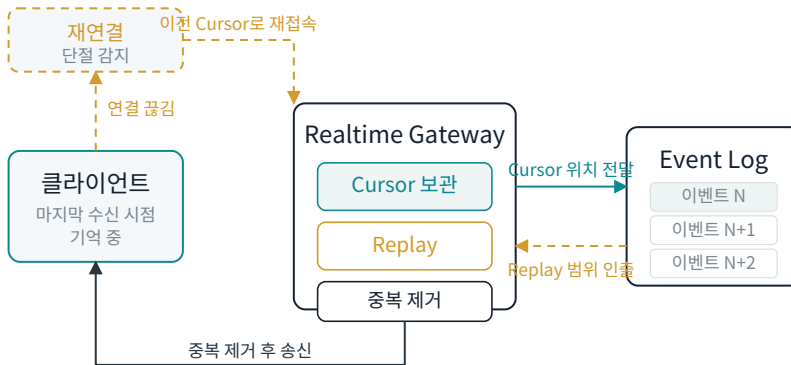
장애 시나리오

시나리오	영향	대응 원칙
Schema breaking change	필드 삭제·의미 변경이 오래된 client를 깨뜨린다.	additive evolution, deprecation, compatibility CI를 사용한다.
Unbounded query	GraphQL 깊이·fan-out이 DB와 downstream을 포화시킨다.	cost budget, depth/field limit, persisted query를 둔다.
Zombie stream	모바일 단절 후 서버가 connection을 오래 유지한다.	heartbeat, idle timeout, lease, disconnect cleanup을 적용한다.
Resume gap	재연결 동안 이벤트가 유실되거나 중복된다.	durable cursor, sequence, replay window, idempotent consumer를 사용한다.
Deadline loss	gateway가 timeout을 새로 시작해 하위 작업이 사용자 취소 후에도 계속된다.	absolute deadline 또는 남은 budget을 hop마다 전달한다.

실시간 연결 재개 흐름

단절 이후 Cursor·Replay·중복 제거로 같은 시점부터 이어받는다

—— 정상 스트림 - - - - - 재연결·Replay — Cursor



Cursor가 없으면 처음부터, 너무 오래되면 Replay 창 밖이어서 전체 동기화로 넘어간다. 중복 제거는 클라이언트가 이미 본 이벤트를 다시 보내지 않는다.

실시간 연결 단절 후 cursor·replay window·deduplication으로 이어받는 흐름을 보여준다

확장 전략

- realtime connection state를 stateless gateway와 shared subscription index로 분리한다.
- fan-out은 연결별 반복 DB query 대신 event backbone과 batch delivery를 사용한다.
- GraphQL resolver는 DataLoader/배치와 field cost를 적용한다.
- 큰 payload와 slow consumer를 별도 queue·drop 정책으로 격리한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- field·method 단위 권한을 schema와 함께 검증한다.
- WebSocket upgrade 이후에도 token 만료·권한 변경을 재평가한다.
- GraphQL introspection·error가 내부 schema·데이터를 과도하게 노출하지 않게 한다.
- message size·compression bomb·subscription 수를 제한한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- method/operation별 p95·오류 code·deadline exceeded
- GraphQL complexity·resolver fan-out·N+1
- active connection·reconnect·heartbeat timeout
- stream lag·resume gap·duplicate event
- schema version·deprecated field 사용률

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- IDL/codegen은 개발 효율을 높이지만 다언어 toolchain 유지 비용이 있다.
- 실시간 연결은 요청 수보다 동시 연결·메모리·egress 비용이 중요하다.
- GraphQL facade는 클라이언트 개발을 줄여도 backend aggregation과 관측 비용을 만든다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 모든 내부 호출을 REST로 해야 단순하다고 단정한다.
- GraphQL 하나로 서비스 경계를 대체한다.
- WebSocket이면 메시지가 자동으로 내구성 있고 순서 보장된다고 생각한다.
- streaming API에 deadline과 backpressure를 두지 않는다.

설계 리뷰

- ☐ 상호작용 방향과 연결 수명이 protocol 선택 근거인가?
- ☐ 오류·deadline·취소·idempotency가 계약에 포함됐는가?
- ☐ schema evolution과 오래된 client를 시험하는가?
- ☐ 재연결·resume·중복·slow consumer가 정의됐는가?
- ☐ query·message·subscription 비용이 제한되는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 배송 진행 상태를 SSE와 WebSocket으로 각각 설계하고 선택 근거를 쓰라.
2. GraphQL query 하나가 1만 개 DB 호출을 만드는 경로를 비용 모델로 차단하라.
3. gRPC deadline이 REST gateway를 거쳐 하위 서비스까지 전달되는 규칙을 정의하라.

핵심 요약

- 통신 방식은 상호작용 패턴에 맞춰 선택한다.
- deadline·오류·idempotency는 protocol 공통 계약이다.
- GraphQL에는 query 비용과 field 권한이 필요하다.
- 실시간 연결은 단절·resume·중복을 정상 조건으로 처리한다.
- schema compatibility는 배포 전 자동 검증한다.

출처

- [rfc9110] IETF. **RFC 9110 — HTTP Semantics** (2022). <https://www.rfc-editor.org/rfc/rfc9110.html>
- [grpc-core] gRPC Authors. **gRPC Core Concepts** (2026). <https://grpc.io/docs/what-is-grpc/core-concepts/>
- [graphql-spec] GraphQL Foundation. **GraphQL Specification** (2025). <https://spec.graphql.org/>
- [rfc6455] IETF. **RFC 6455 — The WebSocket Protocol** (2011). <https://www.rfc-editor.org/rfc/rfc6455.html>
- [html-sse] WHATWG. **HTML Living Standard — Server-sent events** (2026). <https://html.spec.whatwg.org/multipage/server-sent-events.html>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

17

모듈러 모놀리스·마이크로서비스 ·Service Mesh

이 장의 구성

- 01 이 장에서 해결할 문제
- 02 먼저 결론
- 03 요구사항과 실패 모델
- 04 핵심 개념
- 05 기존 아키텍처
- 06 요청·데이터 흐름
- 07 대안과 트레이드오프
- 08 장애 시나리오
- 09 확장 전략
- 10 보안과 개인정보
- 11 관측 가능성
- 12 비용과 운영 복잡도
- 13 흔한 오해와 안티패턴
- 14 설계 리뷰
- 15 연습문제
- 16 핵심 요약
- 17 출처

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

모놀리스와 마이크로서비스는 성숙도 순서가 아니다. 한 프로세스에서도 모듈·데이터 소유권을 엄격히 나눌 수 있고, 여러 서비스여도 같은 DB와 배포를 공유하면 독립성이 없다. 분리는 변경·확장·소유권의 실제 압력이 있을 때 수행한다.

이 절의 기준 출처: [kubernetes-concepts; @istio-architecture].

학습 목표

- 배포 단위와 모듈 경계를 구분한다.
- 서비스 분리의 비용과 조건을 평가한다.
- service mesh가 해결하는 통신 문제와 해결하지 않는 도메인 문제를 설명한다.

먼저 결론

- 먼저 모듈 경계와 의존 방향을 만들고 배포 분리는 나중에 선택한다.
- 서비스마다 데이터 쓰기 소유권과 운영 책임이 있어야 한다.
- 분산 호출은 부분 실패·지연·버전·관측 비용을 추가한다.
- service mesh는 mTLS·traffic policy·telemetry를 지원하지만 데이터 소유권과 업무 saga를 설계해주지 않는다.

::: current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06이며, 출판 직전 공식 문서를 다시 확인한다. :::

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	모듈러 모놀리스·마이크로서비스·Service Mesh에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	먼저 모듈 경계와 의존 방향을 만들고 배포 분리는 나중에 선택한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	scale profile이 다른 모듈만 독립적으로 분리한다.

차원	확인 질문	설계 판단
실패·복구	“Chatty calls” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	API coarse-graining, local composition, 비동기 이벤트를 사용한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	서비스 identity와 사용자 identity를 구분해 전달한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	서비스별 SLO·call graph·dependency latency

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

모듈러 모놀리스

하나의 배포 단위 안에서 명시적 모듈 경계와 의존 규칙을 유지하는 구조다.

마이크로서비스

독립 배포·소유·데이터 경계를 가진 작은 서비스 집합이다.

분산 모놀리스

서비스 수는 많지만 데이터·배포·변경이 강하게 결합된 구조다.

Bounded context

용어와 모델이 일관된 업무 경계다.

Service mesh

서비스 간 통신의 proxy와 control plane을 통해 보안·정책·관측을 제공하는 인프라 계층이다.

Strangler migration

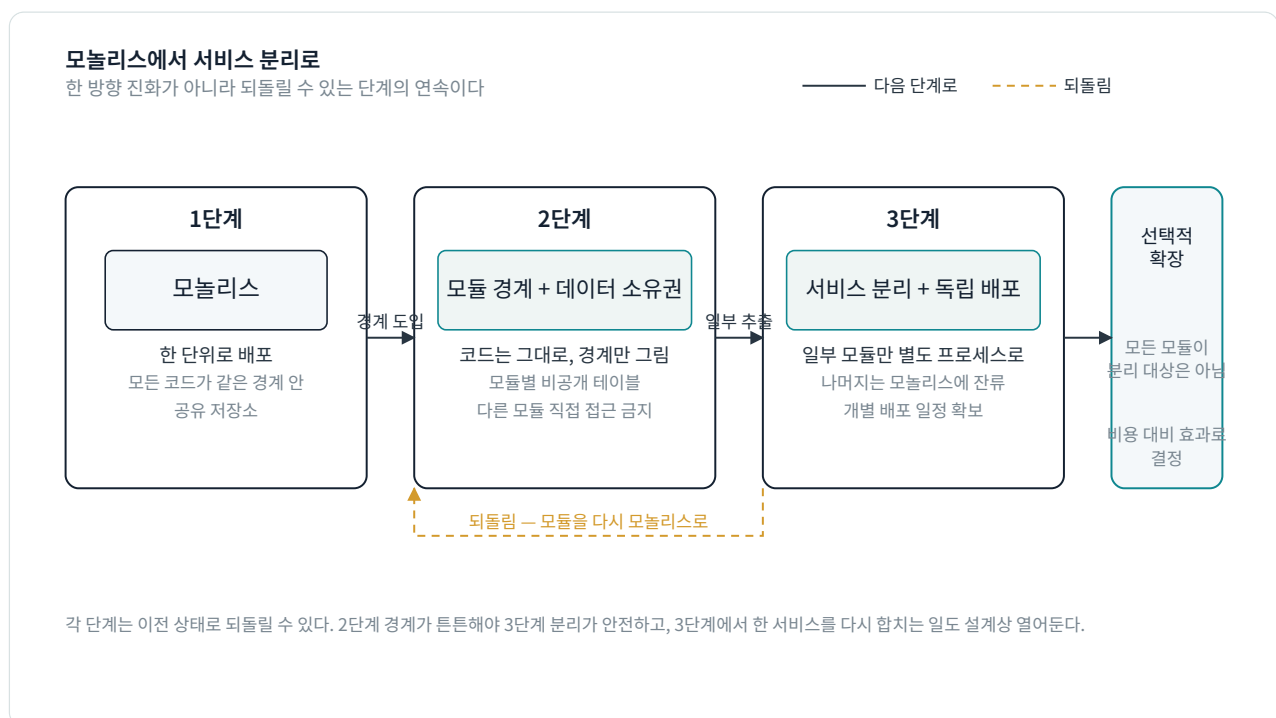
기존 시스템 주변에서 기능을 단계적으로 새 경계로 옮기는 방식이다.

핵심 개념의 정의와 범위는 [kubernetes-concepts; @istio-architecture; @iso-42010]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
도메인 모듈	업무 규칙과 자체 데이터 접근을 소유한다.
내부 인터페이스	모듈 간 허용된 호출과 이벤트를 정의한다.
서비스 API	배포 분리 후 네트워크 계약이 된다.
Event channel	비동기 통합과 느슨한 결합을 지원한다.
Sidecar/ambient data plane	서비스 간 mTLS·routing·telemetry를 수행한다.
Mesh control plane	identity·policy·route 설정을 배포한다.
Platform layer	배포·관측·secret·template을 표준화한다.



모놀리스→모듈러 모놀리스→선택적 서비스 분리의 단계와 되돌림 지점을 보여준다

요청·데이터 흐름

1. 변경 이유와 데이터 소유권으로 모듈을 정의한다.
2. 모듈 내부 DB table 접근을 외부에서 금지한다.
3. 호출 graph와 transaction boundary를 측정한다.

4. 독립 확장·배포·보안 요구가 큰 모듈을 분리 후보로 정한다.
5. API/event 계약과 데이터 migration을 단계적으로 적용한다.
6. mesh는 반복 통신 정책이 충분히 많을 때 도입한다.
7. 분리 후 지연·오류·운영 비용이 목표를 만족하는지 검증한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
모듈러 모놀리스	로컬 transaction·디버깅·배포가 단순하다.	강제 장치가 없으면 경계가 무너질 수 있다.	초기·중간 규모, 복잡한 도메인
마이크로서비스	독립 배포·확장·팀 소유가 가능하다.	네트워크·데이터·관측·플랫폼 비용이 크다.	독립 변화 압력이 검증된 경계
Service mesh	일관된 mTLS·traffic policy·telemetry를 제공한다.	control/data plane 복잡도와 리소스 비용이 있다.	많은 서비스의 공통 통신 정책

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [@kubernetes-concepts; @istio-architecture; @iso-42010]를 참조한다.

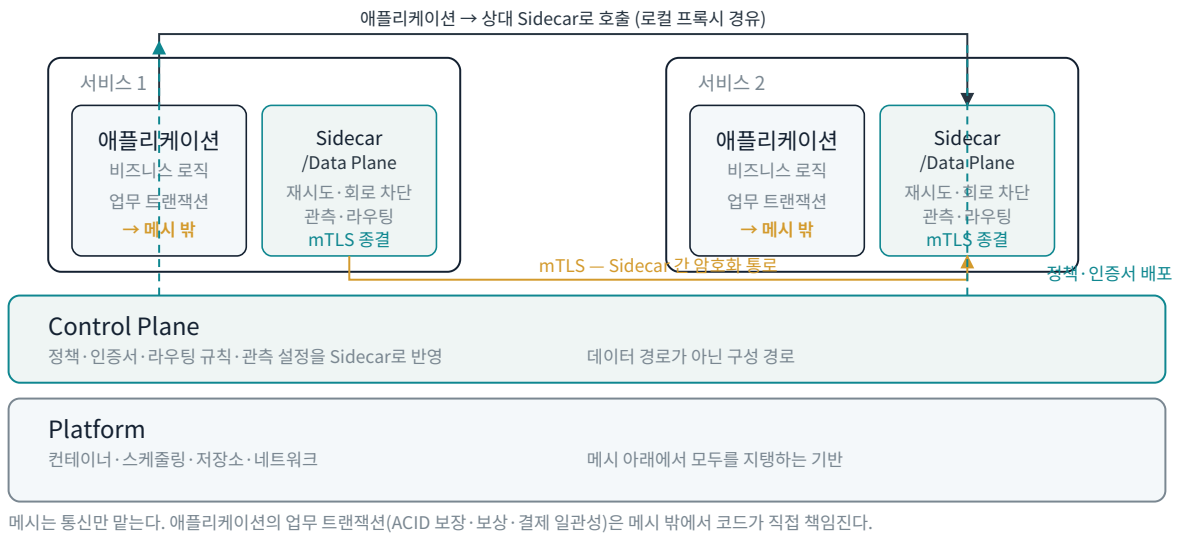
장애 시나리오

시나리오	영향	대응 원칙
Chatty calls	로컬 함수였던 호출이 수십 개 동기 RPC가 되어 tail이 악화된다.	API coarse-graining, local composition, 비동기 이벤트를 사용한다.
공유 DB	여러 서비스가 같은 table을 수정해 배포 독립성이 사라진다.	쓰기 소유자를 정하고 API/CDC로 읽기 모델을 제공한다.
Mesh outage	control plane 설정 오류나 인증서 문제로 광범위 통신 장애가 난다.	last-known config, canary, fail-safe 정책, 범위 축소를 둔다.
Version lockstep	서비스가 함께 배포돼야만 호환된다.	additive contract, consumer-driven test, expand-contract migration을 사용한다.
조직 경계 불일치	한 팀이 수십 서비스를 맡아 on-call과 변경 속도가 악화된다.	서비스 수를 팀 인지 부하와 운영 능력에 맞춘다.

서비스 메시의 책임 범위

통신은 메시가 맡고, 업무 트랜잭션은 여전히 애플리케이션이 줘다

—— 앱 호출 - - - - 제어 신호 — mTLS



애플리케이션·data plane·control plane·platform의 책임과 mesh가 다루지 않는 업무 트랜잭션을 보여준다

확장 전략

- scale profile이 다른 모듈만 독립적으로 분리한다.
- 동기 call depth와 fan-out budget을 제한한다.
- 플랫폼 자동화 없이는 서비스 수 증가를 멈추고 표준 golden path를 먼저 만든다.
- mesh policy는 namespace·tenant 단위로 점진 적용하고 config blast radius를 제한한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 서비스 identity와 사용자 identity를 구분해 전달한다.
- mTLS가 애플리케이션 권한을 대체하지 않는다는 점을 명시한다.
- 공유 DB를 분리하는 동안 최소 권한·감사·dual-write 위험을 관리한다.
- mesh admin과 workload deploy 권한을 분리한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 서비스별 SLO·call graph·dependency latency
- 배포 빈도·변경 실패율·MTTR
- 동기 call depth·fan-out·retry amplification
- mesh config reject·certificate expiry·proxy resource
- 공유 DB 접근·contract break 건수

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 서비스 하나마다 build·deploy·runtime·observability·on-call 비용이 생긴다.
- mesh proxy는 CPU·메모리·지연과 control plane 운영 비용을 추가한다.
- 모듈러 모놀리스는 인프라 비용이 낮지만 경계 테스트와 코드 ownership 투자가 필요하다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 서비스 수를 현대성 지표로 사용한다.
- 테이블을 서비스별로 나누면 도메인 경계가 생겼다고 생각한다.
- service mesh가 retry·transaction·보안을 자동으로 해결한다고 믿는다.
- 공통 라이브러리 업데이트를 위해 모든 서비스를 동시에 배포한다.

설계 리뷰

- ☐ 독립 배포가 실제로 필요한 변경 압력이 있는가?
- ☐ 서비스마다 데이터 쓰기 소유자와 on-call이 있는가?
- ☐ 동기 call graph가 사용자 SLO를 만족하는가?
- ☐ mesh 도입 전에 해결할 반복 문제와 성공 지표가 명확한가?
- ☐ 되돌리기 가능한 단계적 분리 계획이 있는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 공유 DB를 사용하는 5개 서비스가 왜 분산 모놀리스인지 장애 시나리오로 설명하라.
2. 모듈러 모놀리스의 주문 모듈을 서비스로 분리하는 expand-contract 단계를 설계하라.
3. mesh의 자동 재시도가 비역등 결제 호출을 중복시키는 경로를 막아라.

핵심 요약

- 배포 단위와 모듈 경계는 다른 개념이다.
- 마이크로서비스는 독립성의 이익과 분산 비용을 함께 가진다.
- 데이터 소유권이 없는 서비스 분리는 독립적이지 않다.
- service mesh는 통신 인프라 문제를 해결할 뿐 도메인 설계를 대신하지 않는다.
- 팀의 운영 능력이 서비스 수의 현실적 상한을 결정한다.

출처

- [@kubernetes-concepts] Kubernetes Authors. **Kubernetes Concepts** (2026). <https://kubernetes.io/docs/concepts/>
- [@istio-architecture] Istio Authors. **Istio Architecture** (2026). <https://istio.io/latest/docs/ops/deployment/architecture/>
- [@iso-42010] ISO/IEC/IEEE. **ISO/IEC/IEEE 42010:2022 — Architecture description** (2022). <https://www.iso.org/standard/74393.html>

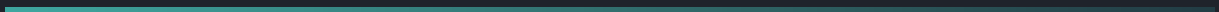
“

검증 기준일: 2026-08-06. `current` 와 `volatile` 내용은 `release` 전에 공식 원문을 다시 확인한다.

PART

4

데이터·캐시·이벤트



데이터의 형태와 접근 패턴에 따라 저장소, 캐시, 이벤트 경로를 선택한다.

이 Part에서는 특정 제품의 설정법보다 반복해서 적용할 수 있는 설계 질문과 실패 모델을 먼저 익힌다. 각 장의 체크리스트를 실제 시스템의 ADR·런북·대시보드와 연결해 읽는 것이 목표다.

포함 장

ID	장 제목	최신성	원본 관계
ch18	워크로드에서 저장소 선택하기	durable	REWRITE
ch19	관계형 DB·분산 SQL·인덱스	current	REWRITE
ch20	Key-Value·Document·Wide-column·Graph	current	REWRITE
ch21	Object Storage·Search·Vector Store	current	ADD
ch22	캐시·무효화·Stampede·Hot Key	durable	REWRITE
ch23	Queue·Durable Log·Delivery Semantics	durable	REWRITE
ch24	Event Streaming·CDC·Outbox·Saga	current	ADD

읽는 방법

1. 먼저 각 장의 **요구사항과 실패 모델**을 자신의 시스템에 대입한다.
2. **대안과 트레이드오프**에서 현재 선택이 어떤 비용을 감수하는지 확인한다.
3. **장애 시나리오**를 게임데이 또는 설계 리뷰 질문으로 바꾼다.
4. 시각자료는 설명을 대신하지 않고 책임·흐름·복구 경계를 검증하는 용도로 사용한다.

18

워크로드에서 저장소 선택하기

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

저장소 선택은 “SQL 대 NoSQL” 투표가 아니다. 쓰기 단위, 조회 형태, 일관성, 데이터 수명, 재구축 가능성, 운영 역량을 먼저 적고 그 요구를 가장 단순하게 만족하는 저장소를 선택해야 한다.

이 절의 기준 출처: [@postgres-transaction-iso; @dynamo-paper].

학습 목표

- 접근 패턴과 불변조건으로 저장소 요구를 도출한다.
- 하나의 저장소와 다중 저장소 전략의 비용을 비교한다.
- 벤치마크를 실제 데이터·쿼리·장애 조건으로 설계한다.

먼저 결론

- 데이터 모델보다 먼저 읽기·쓰기·삭제·분석·복구 패턴을 표로 만든다.
- 핵심 원장은 가장 강한 불변조건을 지키는 저장소에 둔다.
- 파생 색인·캐시는 재구축 경로와 허용 staleness를 명시한다.
- polyglot persistence는 기능 이점보다 데이터 동기화·백업·운영 비용을 함께 계산한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	워크로드에서 저장소 선택하기에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	데이터 모델보다 먼저 읽기·쓰기·삭제·분석·복구 패턴을 표로 만든다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	한계에 도달한 축이 저장량인지 QPS인지 쿼리 복잡도인지 먼저 측정한다.
실패·복구	“벤치마크 왜곡” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	운영 분포·payload·concurrency·장애를 재현한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	민감 데이터의 저장 위치·암호화 키·접근 감사를 후보 평가에 포함한다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	쿼리 패턴별 p95/p99·rows scanned·cache hit

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

System of record

업무상 진실의 원천과 승인된 상태를 소유하는 저장소다.

Access pattern

키 조회, 범위, join, graph traversal, full-text, vector search 같은 실제 읽기·쓰기 형태다.

Working set

짧은 시간에 반복 접근되는 데이터 집합이다.

Write amplification

하나의 논리 쓰기가 복제·색인·compaction으로 여러 물리 쓰기를 만드는 정도다.

Derived store

원본에서 다시 만들 수 있는 cache·search index·warehouse·feature store다.

Operational envelope

데이터 크기, latency, throughput, failure, 복구, 인력 범위에서 검증된 운영 영역이다.

핵심 개념의 정의와 범위는 [@postgres-transaction-iso; @dynamo-paper; @bigtable-paper]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

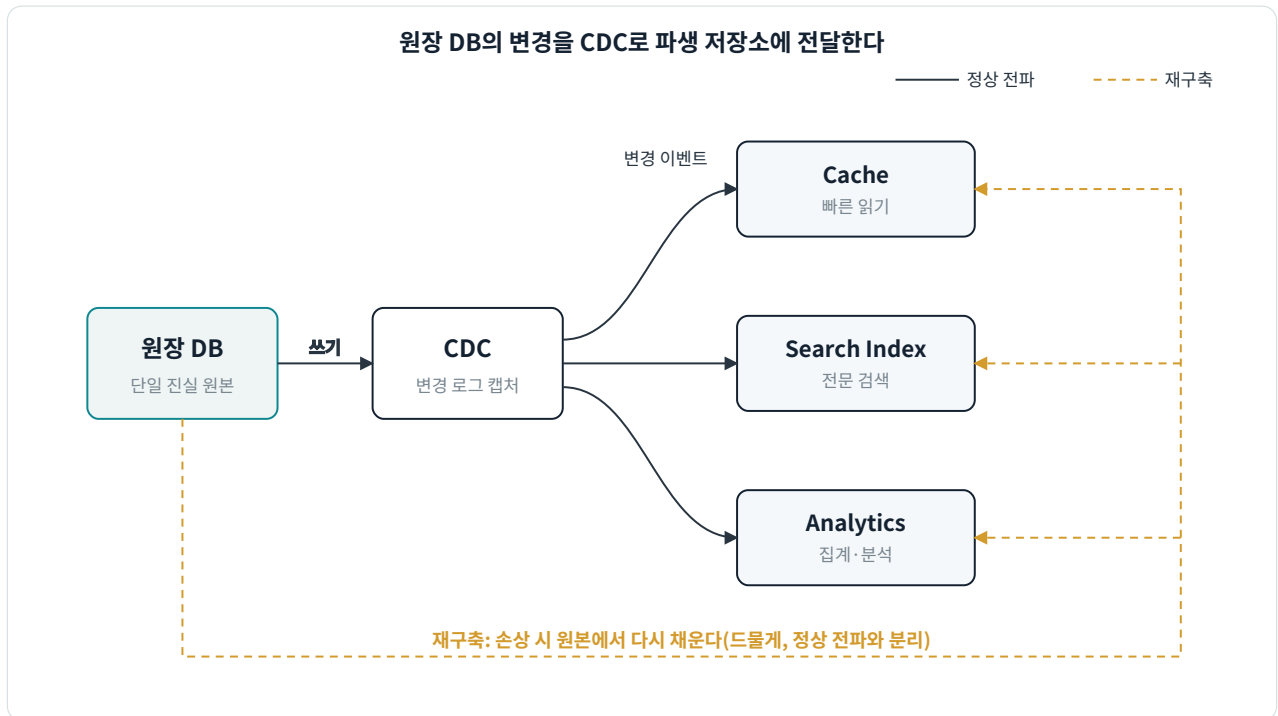
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
단일 범용 DB	transaction·backup·운영이 단순하다.	검색·graph·대규모 blob 같은 특수 패턴 효율이 낮을 수 있다.	대부분의 초기·중간 시스템
원장+파생 저장소	핵심 불변조건과 특수 조회를 각각 최적화한다.	동기화·staleness·rebuild 운영이 필요하다.	검색·분석·추천이 있는 시스템
다중 독립 원장	도메인별 독립 확장과 소유가 가능하다.	cross-domain transaction 과 데이터 거버넌스가 복잡하다.	명확한 bounded context와 운영 역량

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [postgres-transaction-iso; @dynamo-paper; @bigtable-paper]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
벤치마크 왜곡	균등 random key로만 시험해 실제 hot tenant와 범위 조회를 놓친다.	운영 분포·payload·concurrency·장애를 재현한다.
기능 체크리스트 선택	제품 기능 수는 많지만 핵심 쿼리와 복구가 불안정하다.	필수 여정과 운영 증거에 가중한다.
파생 저장소 원장화	검색 index가 직접 수정돼 원본과 수렴 경로가 사라진다.	쓰기 소유자를 원장으로 제한하고 재구축을 정기 시험한다.
Schema lock-in	데이터 변환과 export가 검증되지 않아 탈퇴가 어렵다.	정기 export·restore·dual-read proof를 수행한다.
운영 역량 부족	기술은 맞지만 on-call과 backup·upgrade가 감당되지 않는다.	관리형 서비스 또는 더 단순한 저장소를 선택한다.



원장 DB에서 CDC로 cache·search·analytics를 만드는 흐름과 rebuild 경계를 보여준다

확장 전략

- 한계에 도달한 축이 저장량인지 QPS인지 쿼리 복잡도인지 먼저 측정한다.
- 읽기 모델을 추가하기 전에 index·query·connection·batch를 최적화한다.
- 데이터를 수명·온도·tenant로 tiering한다.
- 재구축 가능한 파생 데이터는 원장과 다른 RPO/RTO를 적용한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 민감 데이터의 저장 위치·암호화 키·접근 감사를 후보 평가에 포함한다.
- 삭제·보존 정책이 replica·index·backup에 실제 적용되는지 시험한다.
- 관리형 서비스의 운영자 접근·지원 데이터 처리·export 권한을 검토한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 쿼리 패턴별 p95/p99·rows scanned·cache hit
- write amplification·compaction·replication lag
- storage growth·working set·index size
- backup/restore·rebuild 시간
- schema change와 failed migration

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 저장소 라이선스·인스턴스보다 인덱스·egress·backup·운영 인력 비용을 포함한다.
- 여러 저장소는 각 connector·schema·security·upgrade 비용을 곱한다.
- 과도한 미래 대비는 현재 학습과 장애 표면을 키운다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 유명 기업이 쓰는 저장소를 규모 근거 없이 채택한다.
- “NoSQL은 schema가 없다”고 생각한다.
- 벤치마크에서 평균 latency와 정상 상태만 측정한다.
- 파생 저장소의 rebuild 시간을 모른다.

설계 리뷰

- ☐ 핵심 불변조건과 원장이 명확한가?
- ☐ 실제 access pattern과 데이터 분포가 문서화됐는가?
- ☐ 후보의 실패·복구·migration을 시험했는가?
- ☐ 파생 데이터의 staleness와 rebuild 경로가 있는가?
- ☐ 팀이 운영할 수 있는 기술 수를 넘지 않는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 쇼핑물의 주문, 상품 검색, 이미지, 추천 embedding에 저장소 역할을 배정하라.
2. 후보 DB 벤치마크에 포함할 데이터 분포·쿼리·장애 항목을 작성하라.
3. 새 검색 저장소를 제거하고 원장으로 되돌아갈 탈출 계획을 설계하라.

핵심 요약

- 저장소는 access pattern과 불변조건에서 선택한다.
- 원장과 파생 저장소의 책임을 분리한다.
- 실제 분포와 실패를 포함해 벤치마크한다.
- polyglot은 동기화·보안·복구 비용을 만든다.
- 탈출과 재구축 가능성을 채택 전에 검증한다.

출처

- [@postgres-transaction-iso] PostgreSQL Global Development Group. **PostgreSQL Documentation — Transaction Isolation** (2026). <https://www.postgresql.org/docs/current/transaction-iso.html>
- [@dynamo-paper] Giuseppe DeCandia et al.. **Dynamo: Amazon's Highly Available Key-value Store** (2007). <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>
- [@bigtable-paper] Fay Chang et al.. **Bigtable: A Distributed Storage System for Structured Data** (2006). <https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/>

“

검증 기준일: 2026-08-06. `current` 와 `volatile` 내용은 release 전에 공식 원문을 다시 확인한다.

19

관계형 DB·분산 SQL·인덱스

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

관계형 데이터베이스의 강점은 단순히 SQL 문법이 아니라 제약, transaction, optimizer, 성숙한 복구 도구가 결합된 데 있다. 분산 SQL은 이 모델을 여러 노드로 확장하지만 원격 transaction과 데이터 배치 비용을 없애지는 않는다.

이 절의 기준 출처: [postgres-indexes; postgres-transaction-iso].

학습 목표

- 관계형 모델과 인덱스의 비용을 쿼리 계획으로 설명한다.
- 수직 확장·읽기 복제·샤딩·분산 SQL의 경계를 비교한다.
- 온라인 schema·index 변경을 안전하게 수행한다.

먼저 결론

- 정규화와 denormalization은 읽기·쓰기·불변조건 비용의 선택이다.
- 인덱스는 읽기를 줄이는 대신 쓰기·저장·vacuum 비용을 늘린다.
- query plan과 실제 cardinality가 성능 판단의 근거다.
- 분산 SQL에서도 locality와 transaction 범위를 데이터 모델에 반영한다.

::: current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06 이며, 출판 직전 공식 문서를 다시 확인한다. :::

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	관계형 DB·분산 SQL·인덱스에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	정규화와 denormalization은 읽기·쓰기·불변조건 비용의 선택이다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	connection pool을 DB 처리량과 transaction 길이에 맞추고 무제한 연결을 막는다.
실패·복구	“통계 부정확” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	analyze, extended statistics, plan regression 감시를 사용한다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	DB role을 애플리케이션 기능별 최소 권한으로 나눈다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	query fingerprint별 latency·rows·buffer I/O

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

관계형 제약

PK, FK, UNIQUE, CHECK로 데이터 규칙을 DB가 검증한다.

B-tree index

정렬된 키 구조로 equality·range·order query를 지원한다.

Covering index

쿼리에 필요한 열을 index에서 충족해 table lookup을 줄인다.

Query optimizer

통계와 비용 모델로 join 순서와 access path를 선택한다.

Read replica

원장 변경을 복제해 읽기 부하를 분산한다.

Distributed SQL

여러 노드에 partition·replication하면서 SQL transaction을 제공하는 계열이다.

Online schema change

오래 잠그지 않고 expand·backfill·switch·contract로 구조를 변경하는 방식이다.

핵심 개념의 정의와 범위는 [@postgres-indexes; @postgres-transaction-iso; @spanner-paper]를 기준으로 재검토해야 한다.

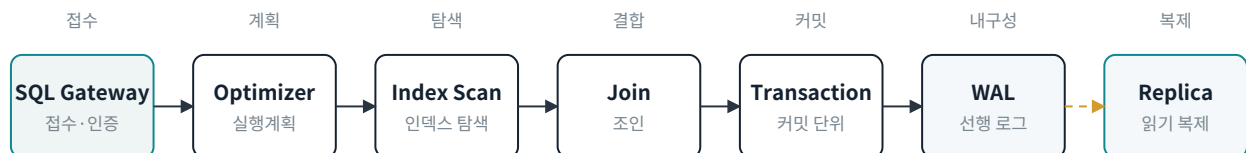
기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
SQL gateway	connection·parse·auth·route를 처리한다.
Transaction coordinator	분산된 read/write의 commit을 조정한다.
Range/Shard replica	키 범위를 저장하고 consensus로 복제한다.
Optimizer/statistics	분산 비용과 cardinality를 추정한다.
Index set	주요 access path와 제약을 지원한다.
Change pipeline	schema migration·backfill·validation을 수행한다.
Backup/PITR	WAL·snapshot으로 복구 지점을 제공한다.

SQL이 parse·optimize·탐색/조인·커밋·WAL을 거쳐 replica에 도달한다

—— 정상 경로 - - - - 비동기 복제



WAL이 디스크에 쓰히면 커밋이 완료된다. Replica는 WAL을 따라가며, 장애 시 약간의 손실 노출이 있다.
Optimizer가 Index Scan과 Join의 순서를 정하고, Transaction이 원자성 경계를 제공한다.

SQL이 parse·optimize·index/join·transaction·WAL로 처리되는 경로를 보여준다

요청·데이터 흐름

1. 요청이 transaction과 query를 시작한다.
2. optimizer가 통계로 local/remote plan을 선택한다.
3. route key가 있으면 필요한 shard로 직접 보낸다.

4. index scan·join·filter를 수행한다.
5. 다중 range 쓰기는 coordinator가 commit protocol을 수행한다.
6. WAL/log가 replica와 backup 경로로 전달된다.
7. slow query와 plan change를 관측해 통계를 갱신한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

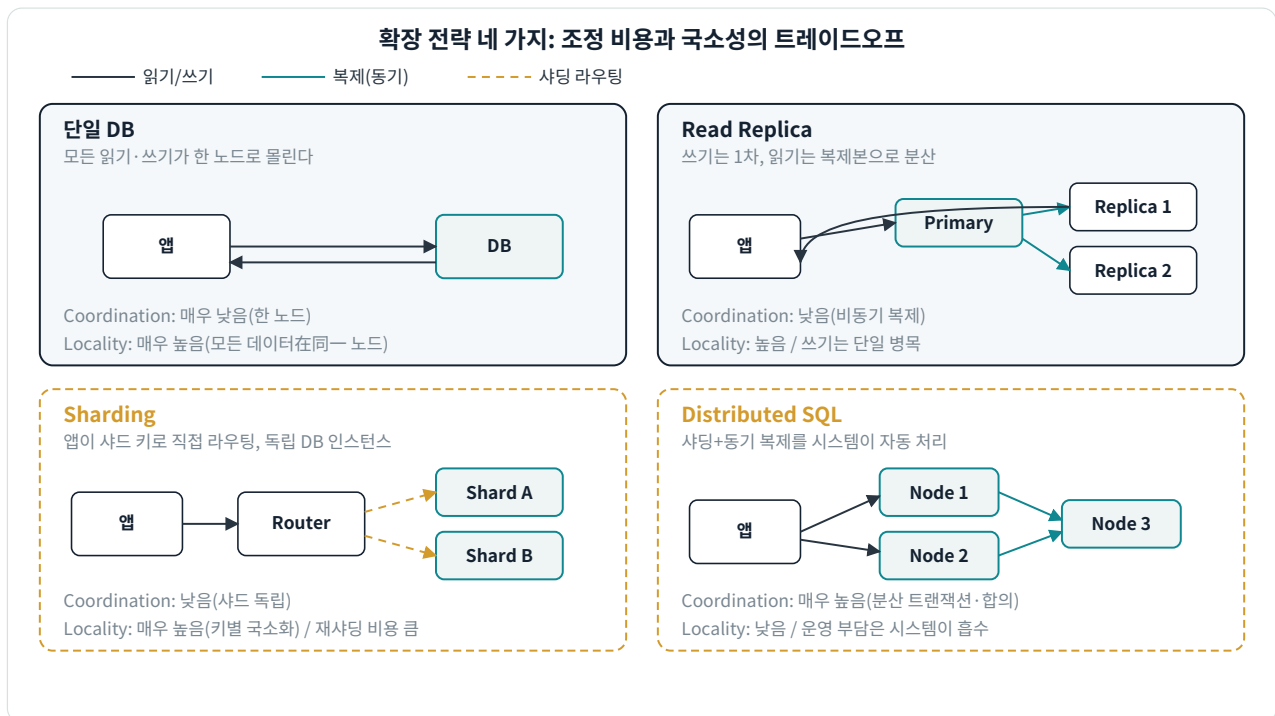
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
단일 관계형 DB	강한 transaction과 운영 단순성이 좋다.	한 노드 한계와 지역 지연이 있다.	대부분의 OLTP
Primary+read replica	읽기 확장과 분석 격리에 유리하다.	stale read·lag·승격 복잡도가 있다.	읽기 비중 높은 시스템
분산 SQL	수평 저장·고가용성과 SQL 모델을 결합한다.	원격 transaction·hot range·운영 비용이 있다.	큰 데이터·다중 zone 강한 transaction

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [@postgres-indexes; @postgres-transaction-iso; @spanner-paper]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
통계 부정확	optimizer가 작은 table로 예상한 결과가 커져 잘못된 join을 선택한다.	analyze, extended statistics, plan regression 감시를 사용한다.
인덱스 폭증	모든 쿼리마다 index를 추가해 쓰기와 vacuum이 느려진다.	사용률·중복·쓰기 비용을 정기 감사한다.
긴 migration lock	DDL이 table을 잠가 요청이 쌓인다.	expand-contract, online build, lock timeout을 사용한다.
분산 hot range	순차 key가 한 range leader에 쓰기를 집중시킨다.	hash prefix·range split·키 설계를 조정한다.
Replica stale read	방금 쓴 데이터를 follower에서 읽어 사용자 흐름이 깨진다.	session routing·LSN token·leader read를 사용한다.



단일 DB·read replica·application sharding·distributed SQL의 경계와 비용을 비교한다

확장 전략

- connection pool을 DB 처리량과 transaction 길이에 맞추고 무제한 연결을 막는다.
- partition pruning과 route key로 scatter query를 줄인다.
- index-only scan·batch write·prepared statement를 실제 plan으로 검증한다.
- 분산 전환 전에 vertical scale·query·schema·archive로 한계를 늦춘다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- DB role을 애플리케이션 기능별 최소 권한으로 나눈다.
- row-level security를 사용해도 애플리케이션 tenant 검증과 테스트를 유지한다.
- backup·replica·query log에 동일한 민감 데이터 정책을 적용한다.
- migration 계정과 runtime 계정을 분리한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- query fingerprint별 latency·rows·buffer I/O
- index hit·size·write amplification·unused index
- lock wait·deadlock·transaction age
- replication lag·WAL generation·checkpoint
- range hotspot·remote transaction 비율

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- index와 replica는 저장·I/O·backup 비용을 지속적으로 만든다.
- 분산 SQL은 노드 수 외에 cross-region traffic와 operational expertise 비용이 있다.
- 쿼리 최적화와 archive가 더 싼 해결책인지 먼저 비교한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- ORM이 생성한 SQL을 보지 않는다.
- index가 많을수록 항상 빠르다고 생각한다.
- read replica를 강한 read처럼 사용한다.
- 분산 SQL이 data locality 문제를 자동 제거한다고 믿는다.

설계 리뷰

- ☐ 핵심 쿼리 plan과 cardinality가 측정됐는가?
- ☐ 제약이 애플리케이션 불변조건을 직접 보호하는가?
- ☐ index의 읽기 이득과 쓰기 비용을 평가했는가?
- ☐ schema 변경이 online·rollback 가능하게 설계됐는가?
- ☐ 분산 transaction과 hot range 비율이 알려져 있는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 주문 목록 쿼리의 복합 index 열 순서를 access pattern으로 설계하라.
2. 순차 timestamp PK가 분산 SQL hot range를 만드는 이유와 대안을 설명하라.
3. NOT NULL 열 추가를 expand-backfill-validate-contract로 배포하라.

핵심 요약

- 관계형 DB는 제약·transaction·optimizer·복구의 결합이다.
- 인덱스는 읽기와 쓰기 비용을 교환한다.
- query plan과 실제 통계로 판단한다.
- 분산 SQL에도 locality와 coordination 비용이 있다.
- schema 변경은 단계적이고 되돌릴 수 있어야 한다.

출처

- [@postgres-indexes] PostgreSQL Global Development Group. **PostgreSQL Documentation — Indexes** (2026). <https://www.postgresql.org/docs/current/indexes.html>
- [@postgres-transaction-iso] PostgreSQL Global Development Group. **PostgreSQL Documentation — Transaction Isolation** (2026). <https://www.postgresql.org/docs/current/transaction-iso.html>
- [@spanner-paper] James C. Corbett et al.. **Spanner: Google's Globally-Distributed Database** (2012). <https://research.google/pubs/spanner-googles-globally-distributed-database/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

20

Key-Value · Document · Wide-column · Graph

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

NoSQL은 하나의 일관된 기술 범주가 아니다. Key-value, document, wide-column, graph는 서로 다른 조회와 분할 문제를 해결한다. “join이 없다”는 단순함은 쓰기 중복, 비동기 index, 애플리케이션 병합 비용으로 이동할 수 있다.

이 절의 기준 출처: [dynamo-paper; @bigtable-paper].

학습 목표

- 비관계형 데이터 모델을 access pattern과 aggregate 경계로 선택한다.
- denormalization · secondary index · 일관성 비용을 설명한다.
- 모델별 hotspot과 schema evolution을 설계한다.

먼저 결론

- key-value는 key 기반 직접 조회와 단순 partition에 적합하다.
- document는 함께 변경되는 aggregate를 한 단위로 저장하지만 무제한 중첩과 큰 문서는 피한다.
- wide-column은 partition key와 clustering key로 미리 아는 쿼리를 최적화한다.
- graph는 다단계 관계 탐색에 유리하지만 큰 supernode와 분산 traversal을 관리해야 한다.

::: current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06 이며, 출판 직전 공식 문서를 다시 확인한다. :::

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	Key-Value · Document · Wide-column · Graph에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	key-value는 key 기반 직접 조회와 단순 partition에 적합하다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	query별 materialized view를 추가하되 view 수와 update fan-out을 제한한다.
실패·복구	“큰 document” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	크기 상한과 별도 child collection/blob을 둔다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	document의 자유로운 field에 민감 정보가 무단 추가되지 않게 schema validation과 분류를 둔다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	partition/document 크기 분포와 hot key

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Key-value model

key로 opaque value를 저장·조회하며 partition과 cache에 적합하다.

Document model

중첩 field를 가진 문서를 aggregate 단위로 저장한다.

Wide-column model

partition 안의 정렬된 clustering row를 큰 sparse table처럼 저장한다.

Property graph

node·edge와 속성으로 관계를 표현하고 traversal을 수행한다.

Denormalization

읽기 경로를 단순화하기 위해 데이터를 중복 저장하는 설계다.

Materialized view

원본 변경에서 파생해 특정 query를 위한 형태로 유지하는 데이터다.

Supernode

edge가 매우 많은 graph node로 traversal과 lock hotspot을 만든다.

핵심 개념의 정의와 범위는 [@dynamo-paper; @bigtable-paper; @mongodb-data-model; @neo4j-graph-modeling]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Request router	key · partition · graph 영역에 요청을 보낸다.
Primary data model	aggregate를 선택한 형태로 저장한다.
Secondary index/view	비주요 access pattern을 비동기 또는 동기로 지원한다.
Change stream	중복 데이터와 파생 view를 갱신한다.
Reconciliation	누락·중복·순서 오류를 주기적으로 찾아 수리한다.
Schema/version adapter	오래된 record를 읽고 새 형식으로 변환한다.

네 가지 NoSQL 데이터 모델: 형태와 대표 질의

—— 주요 접근 키

..... 연결(참조)

Key-Value

키 하나로 값 전체를 조회

Partition Key	Value(불투명 blob)
user:42	{ ... }
user:43	{ ... }

질의: get(partitionKey) → 값 전체

Document

중첩 필드 구조, 부분 갱신 · 필터 질의

```
{ "id": "order-7",
  "고객": { "이름": "홍", "등급": "A" },
  "항목": [ { "sku": "X", "수량": 2, ... },
  "배송": { "주소": "...", "상태": "준비" } }
```

질의: 필터(고객.등급=A) → 문서 전체 · 일부

Wide-Column

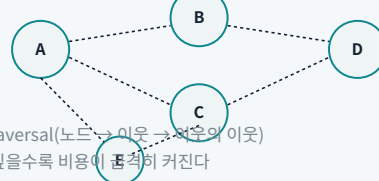
행키+컬럼 패밀리, 최소 행 저장

Partition Key	패밀리 A	패밀리 B
user:42	a1	—
user:43	—	b1

질의: rowKey 범위 + 컬럼 패밀리 스캔
최소 행이 많아도 저장 비용이 작다

Graph

노드 · 간선으로 구성, 이웃 확장 질의



질의: Traversal(노드 → 이웃 → 이웃의 이웃)
관계가 깊을수록 비용이 급격히 커진다

key-value · document · wide-column · graph의 데이터 형태와 대표 query를 비교한다

요청·데이터 흐름

1. 업무 query를 key · range · aggregate · relationship traversal로 분류한다.
2. 함께 원자적으로 변경할 범위를 정한다.
3. partition key와 문서/row 크기 상한을 정한다.
4. 중복 field와 secondary view의 source of truth를 지정한다.

5. 변경 이벤트로 파생 모델을 갱신한다.
6. staleness와 repair를 관측한다.
7. schema version을 읽기·쓰기 양쪽에서 점진 전환한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

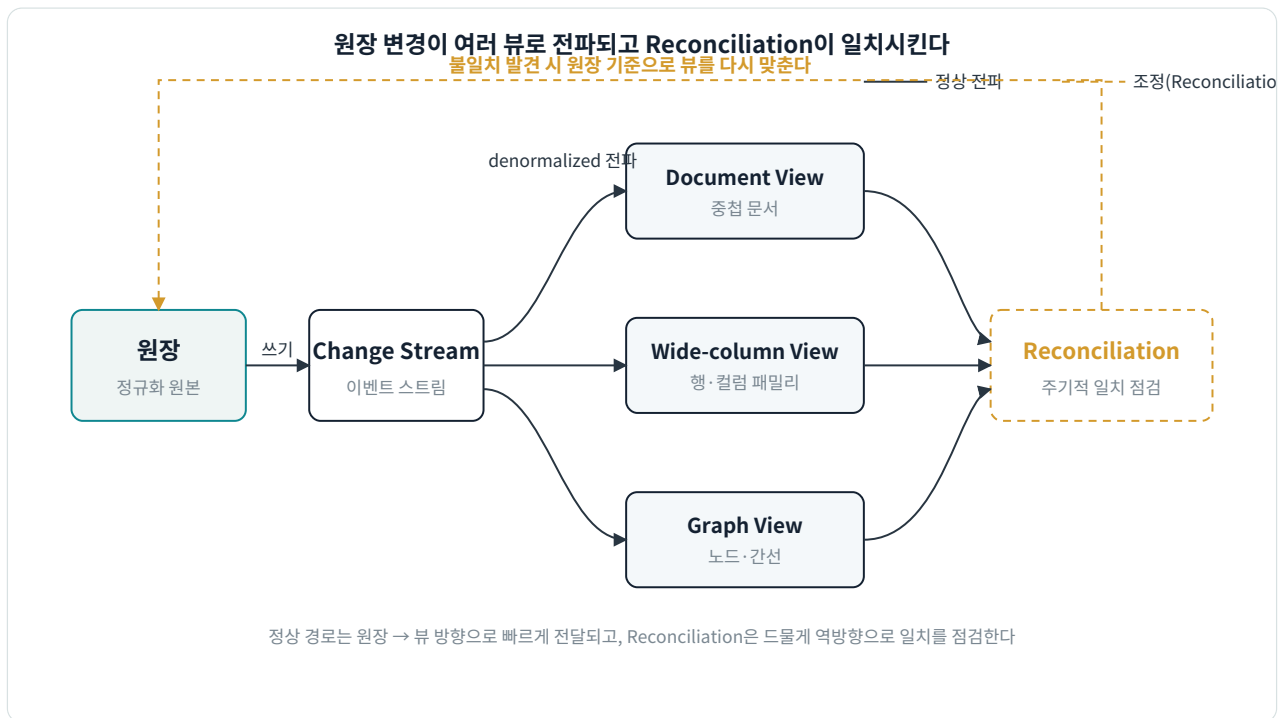
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
Key-value	단일 key 조회·분할·확장이 단순하다.	다양한 query와 관계 검증을 애플리케이션이 맡는다.	세션 · profile · cache · metadata
Document	aggregate 읽기와 schema evolution이 유연하다.	큰 문서·중복·다문서 transaction 비용이 있다.	catalog · content · 설정
Wide-column	높은 write throughput과 시간/범위 query에 적합하다.	query-first schema와 partition size 관리가 필요하다.	이벤트·시계열·대규모 로그
Graph	다중 hop 관계와 경로 query가 자연스럽다.	분산 traversal · supernode · 운영 비용이 있다.	권한 관계·사기 탐지·지식 graph

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [dynamo-paper; @bigtable-paper; @mongodb-data-model]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
큰 document	한 aggregate가 계속 커져 update · 복제 · 전송 비용이 폭증한다.	크기 상한과 별도 child collection/blob을 둔다.
Wide partition	한 partition key에 수년 데이터가 모여 compaction과 hotspot이 생긴다.	time bucket · hash suffix로 경계를 나눈다.
중복 불일치	여러 document의 복사 field가 일부만 갱신된다.	원장 지정 · change stream · reconciliation을 사용한다.
Secondary index lag	색인에서 새 데이터가 누락돼 사용자에게 모순이 보인다.	staleness SLO와 fallback read를 둔다.
Graph supernode	유명 사용자·공통 권한 node에 traversal이 집중된다.	edge type/partition · precomputed view · limit를 적용한다.



원장 변경이 여러 denormalized view와 secondary index로 전파되고 reconciliation되는 흐름을 보여준다

확장 전략

- query별 materialized view를 추가하되 view 수와 update fan-out을 제한한다.
- tenant·time bucket으로 partition을 나누고 skew를 모니터링한다.
- large value는 object storage로 분리하고 metadata만 모델에 둔다.
- graph traversal depth·result·CPU budget을 명시한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- document의 자유로운 field에 민감 정보가 무단 추가되지 않게 schema validation과 분류를 둔다.
- graph 관계가 권한 정보를 노출할 수 있으므로 traversal 결과에 정책을 적용한다.
- tenant key와 partition key를 일치시키거나 모든 query에서 격리 조건을 강제한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- partition/document 크기 분포와 hot key
- secondary index lag · view update failure
- read/write amplification과 compaction
- schema version 분포와 lazy migration 실패
- graph traversal depth · visited node · supernode

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace · log · 변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- denormalization은 저장량과 write fan-out을 늘린다.
- 특수 DB는 query 개발을 줄여도 별도 backup · upgrade · 운영 인력을 요구한다.
- graph · secondary index의 무제한 query를 허용하면 비용 예측이 어렵다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- NoSQL은 transaction이 없다고 일반화한다.
- document를 크기 제한 없는 객체 dump로 사용한다.
- wide-column에서 ad-hoc query를 나중에 해결하려 한다.
- graph DB가 모든 join을 더 빠르게 한다고 생각한다.

설계 리뷰

- ☐ 모델이 핵심 query와 aggregate 경계를 직접 반영하는가?
- ☐ partition/document/supernode 크기 상한이 있는가?
- ☐ 중복 데이터의 원장과 repair 경로가 명확한가?
- ☐ schema evolution이 오래된 record를 안전하게 처리하는가?
- ☐ query 비용과 tenant 격리가 제한되는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 사용자 profile을 document로 설계하고 무한히 커지는 활동 기록을 분리하라.
2. IoT 이벤트를 wide-column에 저장할 partition/clustering key를 설계하라.
3. 권한 graph에서 supernode가 되는 조직 전체 그룹을 다루는 방법을 제안하라.

핵심 요약

- 비관계형 모델은 서로 다른 access pattern을 해결한다.
- aggregate와 partition 경계가 모델의 핵심이다.
- denormalization은 읽기 이득과 동기화 비용을 교환한다.
- secondary view에는 staleness와 repair가 필요하다.
- 크기·fan-out·query 비용 상한을 명시한다.

출처

- [@dynamo-paper] Giuseppe DeCandia et al.. **Dynamo: Amazon's Highly Available Key-value Store** (2007). <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>
- [@bigtable-paper] Fay Chang et al.. **Bigtable: A Distributed Storage System for Structured Data** (2006). <https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/>
- [@mongodb-data-model] MongoDB. **MongoDB Data Modeling Introduction** (2026). <https://www.mongodb.com/docs/manual/data-modeling/>
- [@neo4j-graph-modeling] Neo4j. **Graph Data Modeling Guidelines** (2026). <https://neo4j.com/docs/getting-started/data-modeling/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

21

Object Storage · Search · Vector Store

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

객체 저장소, 검색 엔진, vector store는 범용 원장의 대체물이 아니라 큰 불변 blob과 파생 검색 구조를 제공하는 계층이다. 원본 문서와 metadata를 보존하고 검색 index는 언제든지 재구축할 수 있어야 한다.

이 절의 기준 출처: [@s3-consistency; @lucene-docs].

학습 목표

- blob · full-text · vector 검색의 서로 다른 저장·조회 모델을 설명한다.
- 원장 metadata와 파생 index의 경계를 설계한다.
- ingestion · version · 삭제·재색인을 운영한다.

먼저 결론

- object key와 metadata DB의 일관성 경계를 명시한다.
- 검색 index는 tokenization · mapping · ranking 버전에 따라 결과가 바뀐다.
- vector 검색은 embedding model · distance metric · filter · index parameter를 함께 버전 관리한다.
- 삭제는 object, text index, vector, cache, backup에 비동기로 전파되므로 완료 상태를 추적한다.

::: current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06 이며, 출판 직전 공식 문서를 다시 확인한다. :::

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	Object Storage · Search · Vector Store에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	object key와 metadata DB의 일관성 경계를 명시한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	object는 content-addressed key · multipart · lifecycle tiering으로 운영한다.
실패·복구	“Orphan object” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	pending state · idempotent finalize · garbage collector를 둔다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	signed URL을 짧은 만료·정확한 method/object로 제한한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	object upload/finalize/orphan · checksum failure

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Object storage

큰 immutable 또는 versioned blob을 key로 저장하는 계층이다.

Inverted index

term에서 포함 document 목록으로 연결해 full-text 검색을 지원한다.

Analyzer

텍스트를 token으로 분해·정규화하는 규칙이다.

Embedding

문서나 query를 수치 vector로 표현한 값이다.

ANN index

정확한 전체 비교 대신 근사 최근접 탐색으로 latency와 recall을 교환한다.

Metadata filter

tenant·권한·날짜 같은 구조적 조건으로 검색 후보를 제한한다.

Reindex

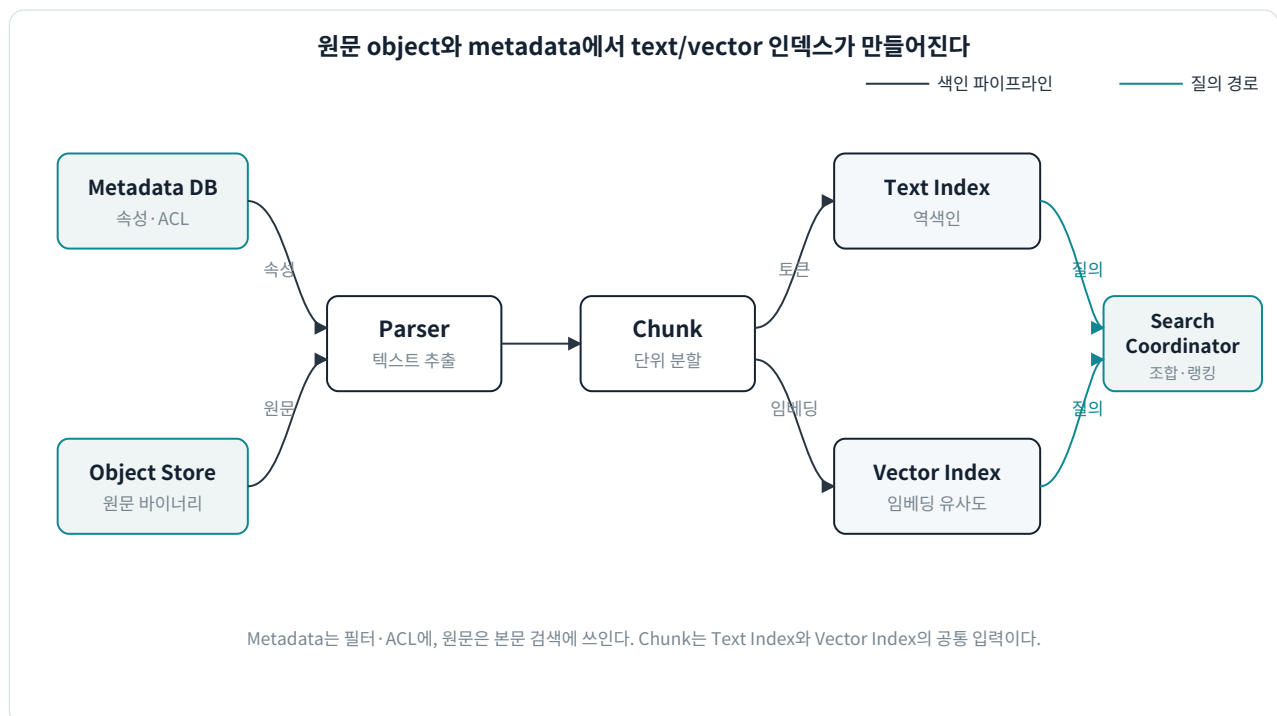
새 mapping·analyzer·model로 파생 index를 다시 만드는 작업이다.

핵심 개념의 정의와 범위는 [@s3-consistency; @lucene-docs; @hnsb-paper]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Metadata DB	object ownership · version · 상태·권한을 원장으로 저장한다.
Object store	원문·이미지·chunk 원본을 보존한다.
Ingestion worker	scan · parse · normalize · chunk · hash를 수행한다.
Text index	lexical search와 filter를 제공한다.
Vector index	embedding ANN search를 제공한다.
Search coordinator	query rewrite · hybrid retrieval · reranking을 조합한다.
Reindex controller	새 index를 병렬 구축·검증·alias 전환한다.



원문 object와 metadata에서 parser · chunk · text/vector index가 생성되는 흐름을 보여준다

요청·데이터 흐름

1. 업로드 요청이 metadata에 pending record를 만든다.
2. object를 저장하고 checksum · version을 확정한다.
3. worker가 안전하게 문서를 parse하고 chunk를 만든다.

- 4. text analyzer와 embedding model 버전을 붙여 index한다.
- 5. query는 tenant·권한 filter 후 lexical/vector 후보를 얻는다.
- 6. reranker가 상위 결과를 정렬하고 원문 version을 확인한다.
- 7. 삭제·변경은 tombstone과 job 상태로 모든 파생 index에 전파한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

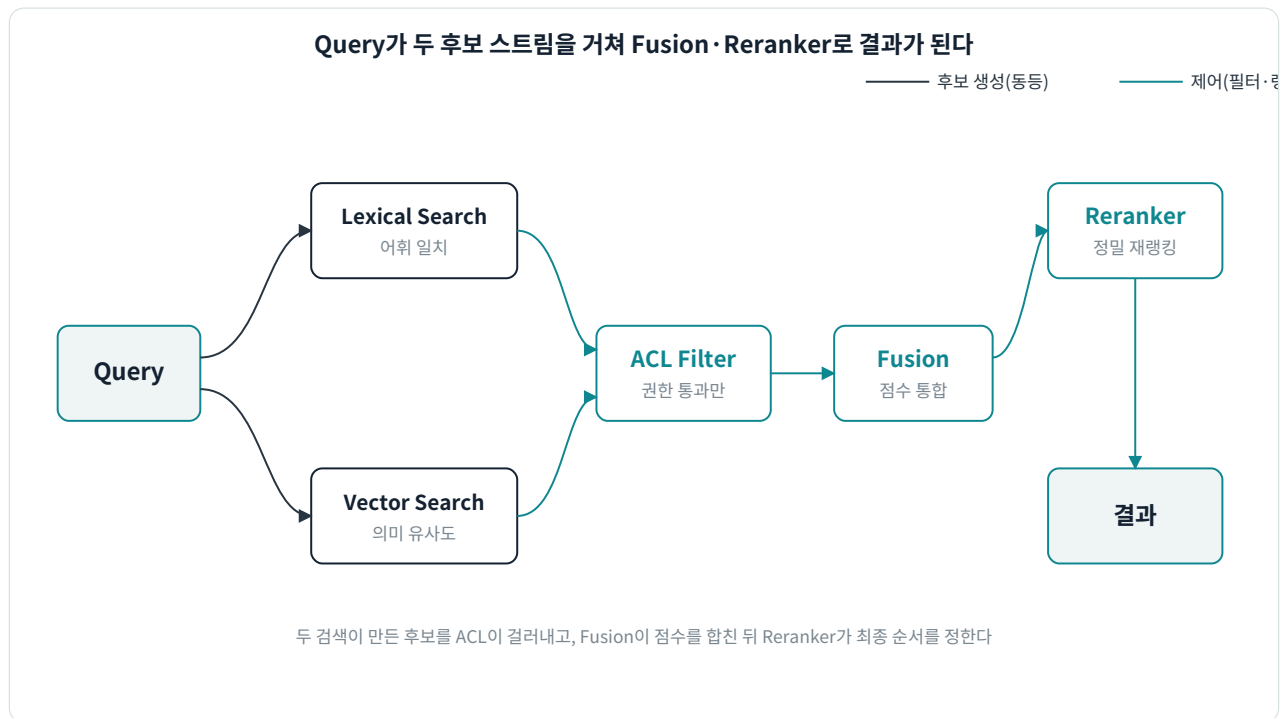
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
Object+DB	큰 blob과 transaction metadata를 분리해 비용·내 구성이 좋다.	두 저장소 사이 orphan·pending 상태를 처리해야 한다.	파일·미디어·문서 원장
Full-text search	정확한 term·filter·phrase 검색과 설명 가능성이 좋다.	동의어·의미 변형에 약하고 analyzer 운영이 필요하다.	검색·로그·catalog
Vector ANN	의미 유사 검색에 유리하다.	모델 drift·근사 recall·filter 비용·reindex가 필요하다.	RAG·추천·유사도
Hybrid	lexical과 semantic 신호를 결합한다.	점수 정규화·reranking·운영 경로가 복잡하다.	정확어와 의미를 함께 요구하는 검색

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [s3-consistency; @lucene-docs; @hnsf-paper]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Orphan object	object 업로드는 성공했지만 metadata commit이 실패한다.	pending state·idempotent finalize·garbage collector를 둔다.
Mapping explosion	동적 field가 무제한 index되어 memory 와 cluster state가 커진다.	schema allowlist와 field cardinality limit을 둔다.
Model drift	embedding model 변경 후 query와 document vector 공간이 달라진다.	model version을 키에 포함하고 dual index 전환을 한다.
권한 누출	vector 후보를 얻은 뒤 filter해 다른 tenant 존재가 노출된다.	가능하면 pre-filter하고 결과 단계에서 다시 검증한다.
삭제 지연	원문 삭제 후 search/vector에 결과가 남는다.	deletion ledger와 end-to-end completion SLO를 둔다.



query가 lexical · vector 후보와 metadata filter · reranker를 거쳐 결과가 되는 흐름을 보여준다

확장 전략

- object는 content-addressed key · multipart · lifecycle tiering으로 운영한다.
- text/vector index를 tenant · time · size 기준으로 shard하고 hot shard를 감시한다.
- reindex는 정상 query와 resource를 경쟁하므로 rate limit · canary · shadow query를 사용한다.
- hybrid search는 후보 수와 reranker budget을 명시한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- signed URL을 짧은 만료 · 정확한 method/object로 제한한다.
- 문서 parser를 격리하고 압축 폭탄 · 악성 파일 · macro를 차단한다.
- search index와 vector에 민감 원문을 불필요하게 중복하지 않는다.
- tenant · ACL filter를 retrieval 전후 두 번 검증한다.

보안 요구는 별도 부록이 아니라 요청 · 데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존 · 삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- object upload/finalize/orphan · checksum failure
- indexing lag · failed document · reindex progress
- query latency · candidate count · cache hit
- ANN recall proxy · filter selectivity · model version
- deletion propagation 완료 시간

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- object storage는 저렴한 용량 대신 request·egress·small object 비용을 만든다.
- search replica와 vector memory는 원문 크기보다 크게 비용이 늘 수 있다.
- model 변경마다 전체 embedding 재생성 비용이 발생한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- object store listing을 transaction database처럼 사용한다.
- 검색 index를 유일한 원장으로 수정한다.
- embedding dimension이 높을수록 무조건 품질이 좋다고 생각한다.
- ACL filter를 검색 후 애플리케이션에서만 적용한다.

설계 리뷰

- ☐ 원본·metadata·text index·vector의 쓰기 소유자가 명확한가?
- ☐ orphan과 partial indexing 상태를 복구하는가?
- ☐ model/analyzer/mapping 버전과 reindex 절차가 있는가?
- ☐ 권한과 삭제가 모든 파생 저장소에 전파되는가?
- ☐ 검색 품질·latency·비용을 함께 평가하는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 문서 업로드 중 DB commit이 실패하는 상태 기계를 설계하라.
2. lexical+vector hybrid 검색에서 candidate와 rerank budget을 정하라.
3. embedding model 교체를 dual-index · shadow query · alias switch로 배포하라.

핵심 요약

- object storage는 blob, search는 lexical index, vector store는 semantic 후보를 담당한다.
- metadata 원장과 파생 index를 분리한다.
- ingestion과 삭제는 상태가 있는 비동기 workflow다.
- model · analyzer · mapping을 versioning한다.
- 권한 filter는 retrieval의 일부다.

출처

- [@s3-consistency] Amazon Web Services. **Amazon S3 Data Consistency Model** (2026). <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html#ConsistencyModel>
- [@lucene-docs] Apache Software Foundation. **Apache Lucene Documentation** (2026). <https://lucene.apache.org/core/>
- [@hnsb-paper] Yu. A. Malkov and D. A. Yashunin. **Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs** (2018). <https://arxiv.org/abs/1603.09320>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

22

캐시·무효화·Stampede·Hot Key

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

캐시는 느린 원본을 가리는 마법이 아니라 복제된 파생 상태다. 어떤 데이터를 얼마 동안 오래되게 보여도 되는지, miss가 몰릴 때 원본을 어떻게 보호할지, 삭제·권한 변경을 얼마나 빨리 반영할지를 먼저 정해야 한다.

이 절의 기준 출처: [rfc9111; @redis-cache].

학습 목표

- cache 역할과 일관성 경계를 정의한다.
- cache-aside·write-through·refresh 전략을 비교한다.
- stampede·hot key·negative cache 실패를 완화한다.

먼저 결론

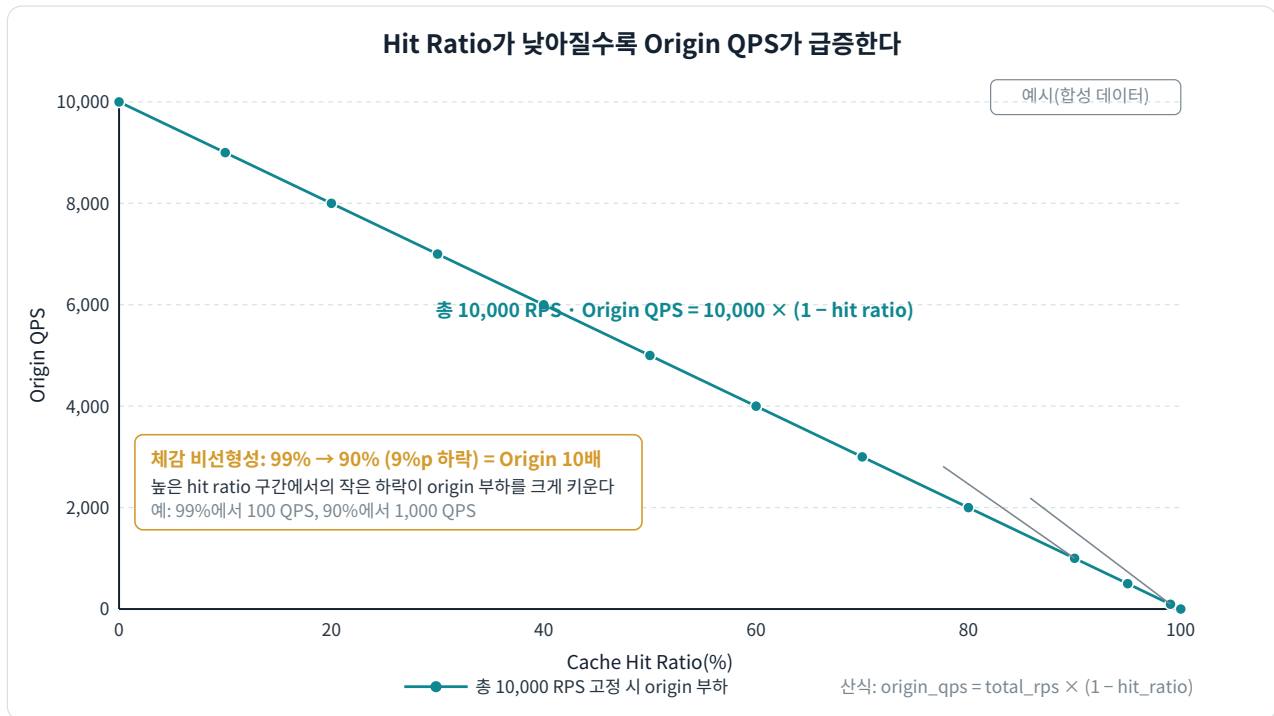
- cache hit ratio만이 아니라 miss cost와 원본 보호 효과를 본다.
- TTL은 무효화 정책의 대체물이 아니라 최대 staleness·정리 수단이다.
- hot key와 동시에 만료되는 key가 전체 원본을 무너뜨릴 수 있다.
- 권한·잔액·재고 같은 상태는 stale 허용 범위와 실패 시 행동을 별도로 정한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	캐시·무효화·Stampede·Hot Key에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	cache hit ratio만이 아니라 miss cost와 원본 보호 효과를 본다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	cache key cardinality와 value size를 함께 관리한다.
실패·복구	“Stampede” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	single-flight, TTL jitter, stale fallback을 사용한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	tenant·locale·권한 scope를 cache key에 포함한다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	hit/miss/stale/negative 비율과 key cardinality

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



캐시 적중률과 원본 부하

핵심 개념

Cache-aside

애플리케이션이 cache를 먼저 읽고 miss 시 원본에서 가져와 채운다.

Write-through

쓰기 경로가 cache와 원본을 함께 갱신한다.

Write-behind

cache가 먼저 변경을 받아 원본에 나중에 반영하며 데이터 손실·순서 위험이 있다.

TTL

entry가 자동 만료되기까지의 시간이다.

Stampede

인기 key가 만료되자 다수 요청이 동시에 원본을 조회하는 현상이다.

Hot key

일부 key에 요청이 지나치게 집중되는 현상이다.

Negative cache

존재하지 않음·오류 같은 결과를 제한 시간 저장해 반복 miss를 줄이는 방식이다.

Stale-while-revalidate

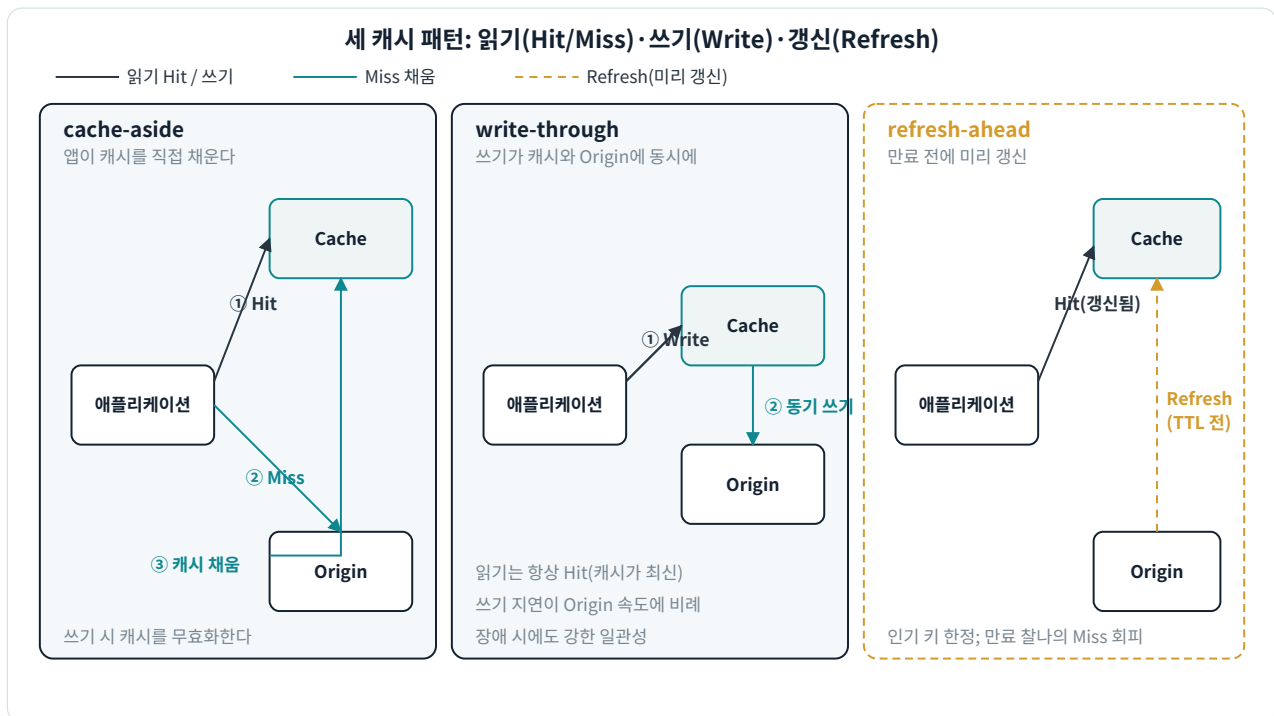
오래된 값을 잠시 제공하면서 백그라운드에서 갱신한다.

핵심 개념의 정의와 범위는 [[@rfc9111](#); [@redis-cache](#); [@memcached-docs](#)]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Client/local cache	네트워크 왕복을 줄이지만 invalidation 범위가 넓다.
Distributed cache	공유 key/value와 TTL을 제공한다.
Origin store	진실의 원천과 transaction을 소유한다.
Refresh coordinator	single-flight·lease로 한 요청만 값을 갱신한다.
Invalidation channel	변경·삭제·권한 사건을 cache 계층에 전달한다.
Hot-key shield	복제·local cache·request coalescing으로 집중을 완화한다.



cache-aside · write-through · refresh-ahead의 읽기·쓰기·실패 경로를 비교한다

요청·데이터 흐름

1. 요청이 cache key와 version/tenant scope를 구성한다.
2. hit이면 staleness 정책을 확인해 값을 반환한다.
3. miss 또는 refresh 필요 시 single-flight lock을 시도한다.
4. 승자만 원본을 읽고 나머지는 bounded wait 또는 stale 응답을 사용한다.
5. 새 값에 TTL jitter와 version을 붙여 저장한다.
6. 원본 변경 이벤트가 관련 key를 삭제·새 version으로 전환한다.
7. cache 장애 시 원본 보호를 위한 rate limit·degradation을 적용한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

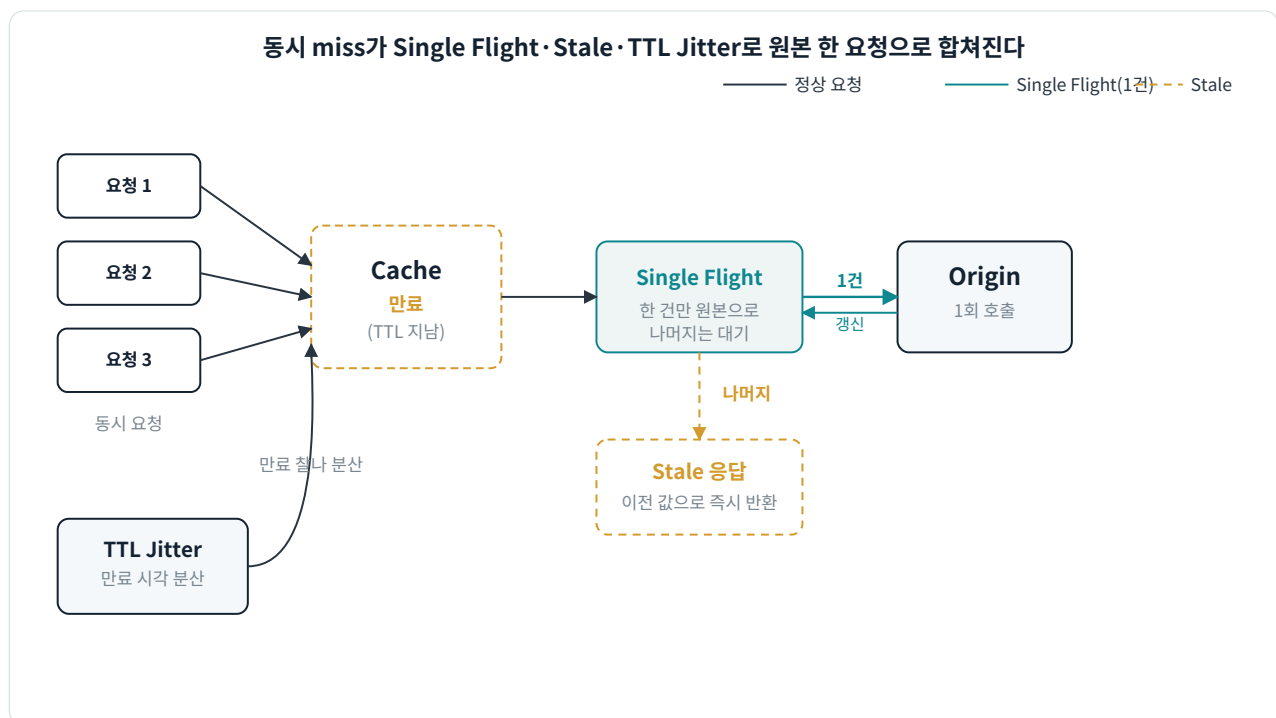
대안	장점	비용·위험	적합한 조건
Cache-aside	구현이 단순하고 원본을 명확히 유지한다.	stale · race · cold miss를 애플리케이션이 처리한다.	일반 읽기 cache
Write-through	쓰기 후 cache 일관성이 좋다.	쓰기 latency와 이중 실패 처리가 복잡하다.	높은 read-after-write 요구

대안	장점	비용·위험	적합한 조건
Refresh-ahead/SWR	사용자 latency와 stampede를 줄인다.	오래된 응답과 refresh worker 운영이 필요하다.	인기 콘텐츠·설정을

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [rfc9111; @redis-cache; @memcached-docs]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Stampede	동일 key miss가 원본에 수천 번 전달된다.	single-flight, TTL jitter, stale fallback을 사용한다.
Cache outage	모든 요청이 원본으로 우회해 DB가 포화된다.	origin admission control, local cache, 기능 축소를 둔다.
Stale authorization	권한 회수 후 cache가 허용 결과를 계속 반환한다.	짧은 TTL, versioned policy, 강한 revoke path를 사용한다.
Hot key node overload	특정 key가 한 cache shard의 네트워크/CPU를 초과한다.	replicated hot key, client cache, key splitting을 사용한다.
Negative cache poisoning	일시 오류를 장시간 “없음”으로 cache한다.	오류 종류별 짧은 TTL과 success/absence 구분을 둔다.



동시 miss가 single-flight·stale fallback·TTL jitter로 원본 한 요청으로 합쳐지는 모습을 보여준다

확장 전략

- cache key cardinality와 value size를 함께 관리한다.
- 다단 cache는 각 계층의 TTL·version·invalidation 책임을 명시한다.
- hot key 자동 탐지 후 local replication 또는 별도 tier로 승격한다.
- hit ratio가 높아도 miss가 고비용이면 원본 capacity를 계산한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- tenant·locale·권한 scope를 cache key에 포함한다.
- 민감 데이터는 client/shared cache 저장 금지와 암호화 정책을 따른다.
- purge/invalidation 권한을 제한하고 감사한다.
- cache key에 원문 개인정보를 직접 넣지 않고 해시·내부 ID를 사용한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- hit/miss/stale/negative 비율과 key cardinality
- miss 원본 latency·origin QPS·coalesced waiters
- hot key top-N·shard skew·eviction
- invalidation lag·stale read 탐지
- cache outage 시 fallback·shed 비율

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- memory cache는 낮은 latency를 사지만 replication·network·reserved capacity 비용이 크다.
- 과도한 TTL은 비용을 줄여도 데이터 신선도와 보안 위험을 키운다.
- 다단 cache는 egress를 줄이지만 invalidation·디버깅 비용을 늘린다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- hit ratio 99%면 원본이 안전하다고 결론낸다.
- 모든 key에 동일 TTL을 설정한다.
- cache를 원장처럼 수정한다.
- cache 장애 시 무조건 원본으로 fail-open한다.

설계 리뷰

- ☐ stale 허용 시간과 사용자 영향이 데이터별로 정의됐는가?
- ☐ stampede와 hot key가 원본을 넘지 않게 제한되는가?
- ☐ cache outage 시 원본 보호와 기능 축소 정책이 있는가?
- ☐ 권한·삭제 invalidation이 더 강하게 처리되는가?
- ☐ hit ratio 외에 miss cost와 invalidation lag를 보는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 캐시 적중률 95%가 85%로 떨어질 때 원본 요청률 변화 배수를 계산하라.
2. 인기 상품 페이지 만료 시 single-flight와 stale 응답 흐름을 설계하라.
3. 권한 회수 이벤트가 local·distributed·edge cache에 전파되는 정책을 작성하라.

핵심 요약

- 캐시는 파생 상태이며 원장이 아니다.
- TTL과 invalidation·version을 함께 설계한다.
- stampede와 cache outage에서 원본을 보호한다.
- hot key는 shard 수만 늘려 해결되지 않는다.
- 보안 상태에는 더 엄격한 stale 정책을 적용한다.

출처

- [rfc9111] IETF. **RFC 9111 — HTTP Caching** (2022). <https://www.rfc-editor.org/rfc/rfc9111.html>
- [redis-cache] Redis. **Redis Documentation — Client-side caching and cache patterns** (2026). <https://redis.io/docs/latest/develop/use/client-side-caching/>
- [memcached-docs] Memcached Authors. **Memcached Documentation** (2026). <https://docs.memcached.org/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

23

Queue · Durable Log · Delivery Semantics

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

메시징 시스템이 “exactly once”를 광고해도 외부 DB·API의 업무 효과까지 자동으로 한 번만 일어나지는 않는다. broker 전달 의미, consumer 상태 commit, 외부 부작용을 하나의 처리 프로토콜로 설계해야 한다.

이 절의 기준 출처: [kafka-docs; @rabbitmq-reliability].

학습 목표

- 작업 queue와 durable log의 목적을 구분한다.
- at-most/at-least/effectively-once 의미를 설명한다.
- consumer lag·retry·dead-letter·순서를 운영한다.

먼저 결론

- queue는 작업 분배와 경쟁 소비, durable log는 순서 보존·replay·다중 소비에 강하다.
- at-least-once는 중복을 정상 조건으로 보며 consumer 멍등성이 필요하다.
- 순서는 전체가 아니라 partition/key 범위로 정의한다.
- dead-letter queue는 최종 저장소가 아니라 원인·재처리·소유자가 있는 운영 workflow다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	Queue·Durable Log·Delivery Semantics에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	queue는 작업 분배와 경쟁 소비, durable log는 순서 보존·replay·다중 소비에 강하다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	partition key가 순서와 병렬성의 단위이므로 cardinality와 skew를 분석한다.
실패·복구	“Poison message” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	시도 한도, 격리, skip 정책과 수동 승인 replay를 둔다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	메시지에 필요한 최소 개인정보만 넣고 payload 암호화·retention을 적용한다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	publish latency · error · duplicate producer

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Work queue

메시지 하나를 대개 한 consumer가 처리하도록 작업을 분배한다.

Durable log

append된 record를 offset 순서로 보존하고 여러 consumer가 각자 읽는다.

At-most-once

중복은 줄지만 실패 시 유실될 수 있는 전달 의미다.

At-least-once

유실을 줄이기 위해 확인 전 재전달하며 중복이 가능하다.

Effectively-once

중복 전달이 있어도 idempotency · transaction · dedup으로 업무 결과를 한 번처럼 만든다.

Consumer group

partition을 consumer 집합에 할당해 병렬 처리한다.

Dead-letter

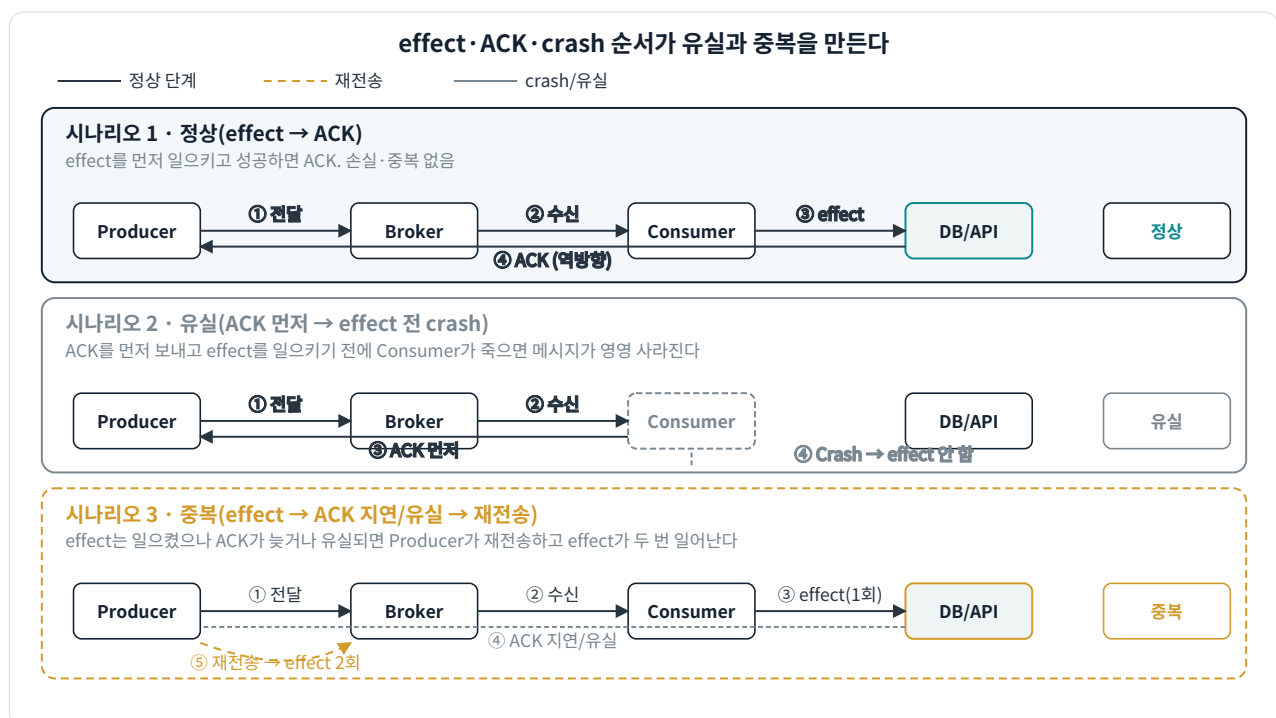
정책상 자동 재시도를 멈춘 메시지를 조사·수정·재처리하기 위한 격리 경로다.

핵심 개념의 정의와 범위는 [kafka-docs; rabbitmq-reliability; kafka-transactions]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Producer	message ID · key · schema · timestamp를 포함해 발행한다.
Broker/Log	내구성·복제·partition 순서를 제공한다.
Consumer group	partition을 나눠 처리한다.
Inbox/Dedup store	처리한 message ID와 결과를 보존한다.
Side-effect target	DB · 외부 API · 파일 등 실제 업무 효과를 수행한다.
Retry scheduler	backoff와 시도 횟수를 관리한다.
DLQ workflow	분류·수정·승인·replay를 수행한다.



effect · ack · crash 순서에 따라 유실·중복이 생기는 세 시나리오를 비교한다

요청·데이터 흐름

1. producer가 stable message ID와 partition key를 만든다.
2. broker가 configured durability 조건으로 record를 저장한다.
3. consumer가 message와 현재 offset을 읽는다.

4. dedup/inbox에서 이미 처리됐는지 확인한다.
5. 업무 transaction과 처리 표시를 가능한 한 원자적으로 커밋한다.
6. 성공 후 offset/ack를 전진시킨다.
7. 실패는 retry class와 backoff를 적용하고 한도 초과 시 DLQ workflow로 보낸다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

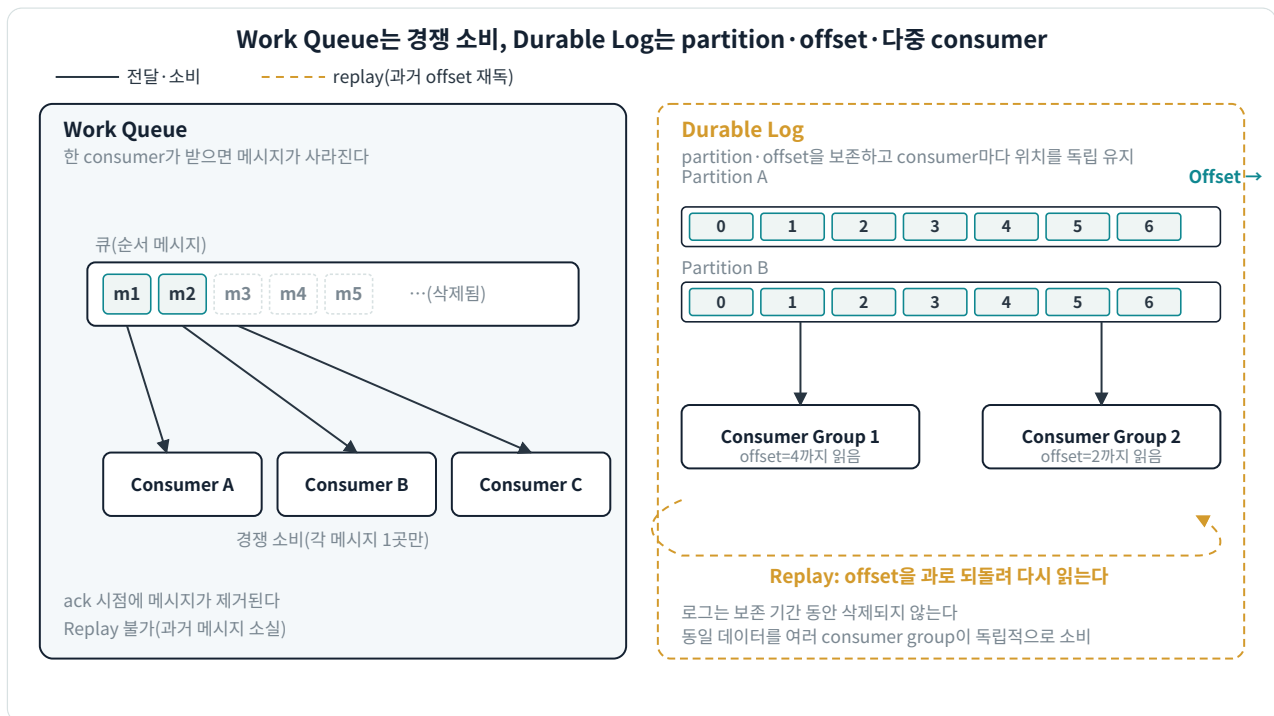
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
Work queue	작업 분배·ack·redelivery가 단순하다.	긴 replay·여러 독립 구독자·과거 재처리에 제한이 있다.	email·thumbnail·batch job
Durable log	replay·다중 consumer·partition 순서에 강하다.	offset·retention·rebalancing·hot partition 운영이 필요하다.	event streaming·CDC
DB-backed queue	업무 transaction과 enqueue를 같은 DB에서 처리하기 쉽다.	대규모 fan-out·retention·broker 기능이 제한될 수 있다.	초기 시스템·outbox dispatcher

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [@kafka-docs; @rabbitmq-reliability; @kafka-transactions]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Poison message	항상 실패하는 record가 partition 진전을 막는다.	시도 한도, 격리, skip 정책과 수동 승인 replay를 둔다.
Ack before effect	업무 처리 전에 ack해 consumer crash 시 유실된다.	effect와 처리 표시 후 ack한다.
Effect before ack	업무 처리 후 ack 전 crash로 중복 실행된다.	idempotency key와 dedup store를 사용한다.
Rebalance storm	느린 처리·불안정 consumer로 partition 소유권이 계속 바뀐다.	처리 시간을 제한하고 heartbeat·static membership을 조정한다.
Lag runaway	도착률이 처리율을 넘어 retention 전에 따라잡지 못한다.	backpressure, scale, priority, load shedding, retention 경보를 둔다.



work queue의 경쟁 소비와 durable log의 partition · offset · 다중 consumer를 비교한다

확장 전략

- partition key가 순서와 병렬성의 단위이므로 cardinality와 skew를 분석한다.
- consumer scale-out보다 downstream capacity와 transaction 시간을 먼저 확인한다.
- 큰 message는 object storage 참조로 분리한다.
- replay는 정상 트래픽과 격리하고 side effect가 다시 안전한지 검증한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 메시지에 필요한 최소 개인정보만 넣고 payload 암호화·retention을 적용한다.
- producer · consumer 권한을 topic · queue · operation 단위로 제한한다.
- DLQ가 보안 통제를 우회한 장기 개인정보 저장소가 되지 않게 한다.
- schema와 deserializer를 악성 payload · 크기 공격에 대비한다.

보안 요구는 별도 부록이 아니라 요청 · 데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존 · 삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- publish latency · error · duplicate producer
- consumer lag · oldest message age · throughput
- processing latency · retry · DLQ · poison rate
- rebalance · partition skew · hot key
- dedup hit · idempotency conflict · replay 결과

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace · log · 변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- durable log retention과 replication은 저장·network 비용을 만든다.
- 긴 backlog는 처리 비용뿐 아니라 recovery 시간과 downstream burst를 키운다.
- DLQ 수동 운영은 숨은 인건비이므로 원인별 자동화와 소유권이 필요하다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- broker가 exactly-once라면 외부 API도 한 번만 호출된다고 생각한다.
- DLQ로 보내면 문제가 해결됐다고 본다.
- 모든 메시지에 전역 순서를 요구한다.
- consumer 수만 늘리면 lag가 줄어든다고 가정한다.

설계 리뷰

- ☐ 업무 효과의 멍등성 경계가 어디인가?
- ☐ ack/offset과 DB commit 순서가 crash 시나리오에서 안전한가?
- ☐ partition key가 순서와 병렬성 요구를 만족하는가?
- ☐ retry · DLQ · replay에 소유자와 정책이 있는가?
- ☐ lag가 retention과 복구 목표 안에 있는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 결제 완료 webhook을 at-least-once로 처리하는 inbox/dedup transaction을 설계하라.
2. 순서가 필요한 고객별 이벤트와 순서가 필요 없는 이미지 작업의 partition 전략을 비교하라.
3. DLQ 10만 건을 안전하게 replay하는 rate limit·검증 절차를 작성하라.

핵심 요약

- queue와 durable log는 다른 소비·replay 모델을 가진다.
- at-least-once에서는 중복이 정상이다.
- effectively-once는 업무 경계의 idempotency로 만든다.
- 순서는 partition 범위로 제한한다.
- DLQ와 replay는 운영 workflow다.

출처

- [kafka-docs] Apache Software Foundation. **Apache Kafka Documentation** (2026). <https://kafka.apache.org/documentation/>
- [rabbitmq-reliability] Broadcom. **RabbitMQ Reliability Guide** (2026). <https://www.rabbitmq.com/docs/reliability>
- [kafka-transactions] Apache Software Foundation. **Apache Kafka — Design: Transactions** (2026). <https://kafka.apache.org/documentation/#semantics>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

24

Event Streaming · CDC · Outbox · Saga

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

이벤트 기반 아키텍처는 transaction을 없애지 않는다. 각 서비스 내부 transaction은 유지하고, 경계를 넘는 상태 변화는 outbox·CDC·역등 consumer·보상으로 연결한다. 이벤트는 사실의 기록이어야 하며 명령과 통지의 의미를 구분해야 한다.

이 절의 기준 출처: [@debezium-docs; @kafka-docs].

학습 목표

- DB 변경을 이벤트로 전달하는 안전한 경로를 설계한다.
- outbox와 CDC의 원자성 경계를 설명한다.
- saga의 보상·timeout·관찰 가능성을 구현한다.

먼저 결론

- DB commit과 event publish를 별도 dual write로 수행하지 않는다.
- outbox는 업무 상태와 발행할 record를 같은 local transaction에 저장한다.
- CDC는 DB log에서 변경을 읽지만 schema·snapshot·순서·삭제 의미를 관리해야 한다.
- saga 보상은 rollback이 아니라 이미 일어난 업무를 상쇄하는 새 업무다.

::: current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06 이며, 출판 직전 공식 문서를 다시 확인한다. :::

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	Event Streaming · CDC · Outbox · Saga 에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	DB commit과 event publish를 별도 dual write로 수행하지 않는다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	aggregate key와 event partition을 맞춰 필요한 순서만 유지한다.

차원	확인 질문	설계 판단
실패·복구	“Dual-write gap” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	transactional outbox 또는 log CDC를 사용한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	CDC 계정은 필요한 table/log 권한만 갖고 secret rotation을 지원한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	outbox age · pending row · publish latency

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Domain event

도메인에서 이미 발생한 사실을 과거형으로 표현한 record다.

Command

특정 수신자에게 작업 수행을 요청하며 거부될 수 있다.

Transactional outbox

업무 변경과 발행 record를 같은 DB transaction에 저장하는 패턴이다.

CDC

database change log를 읽어 삽입·수정·삭제를 event stream으로 전달하는 방식이다.

Saga

여러 local transaction과 보상 action을 순서·이벤트로 조정하는 장기 업무 과정이다.

Orchestration

중앙 coordinator가 다음 단계와 보상을 결정한다.

Choreography

서비스들이 event에 반응해 분산적으로 다음 단계를 진행한다.

Reconciliation

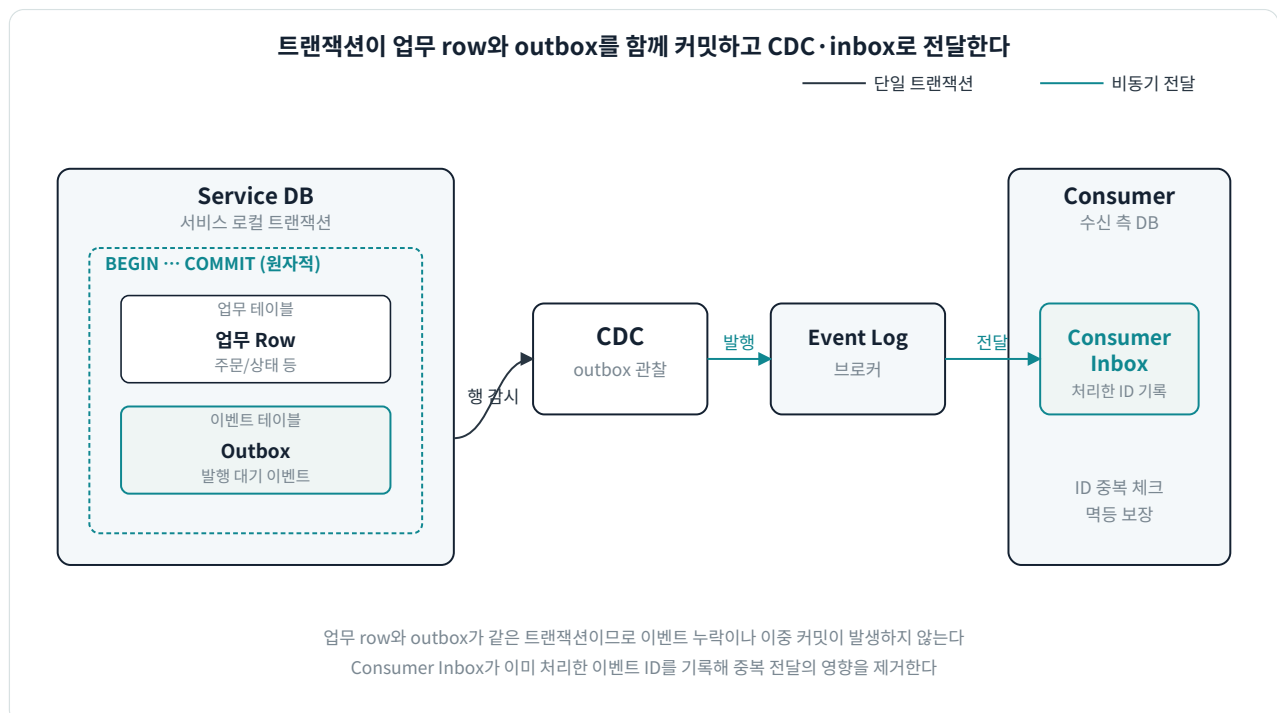
최종 상태와 원장 증거를 비교해 누락·불일치를 찾는 과정이다.

핵심 개념의 정의와 범위는 [debezium-docs; kafka-docs; saga-paper]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Service DB	local transaction과 outbox를 보존한다.
CDC connector	log position을 추적하며 outbox 변경을 읽는다.
Event log	schema · key · ordering을 가진 record를 보존한다.
Consumer inbox	중복 event와 처리 상태를 기록한다.
Saga coordinator	상태·deadline·보상 순서를 관리한다.
Participant service	각 local transaction과 idempotent command를 수행한다.
Reconciler	장기 미완료·불일치·DLQ를 탐지하고 복구한다.



서비스 transaction이 업무 row와 outbox를 커밋하고 CDC·broker·inbox로 전달되는 흐름을 보여준다

요청·데이터 흐름

1. 업무 service가 상태 변경과 outbox insert를 한 transaction으로 커밋한다.
2. CDC가 log position을 보존하며 outbox row를 event로 변환한다.
3. broker가 aggregate key 기준 순서를 유지한다.
4. consumer가 inbox에서 event ID를 확인하고 local transaction을 수행한다.
5. saga coordinator가 성공·실패·timeout에 따라 다음 command를 보낸다.
6. 보상은 별도 idempotency key와 상태 전이를 가진다.
7. 완료·실패·수동 개입 상태를 end-to-end로 관측한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

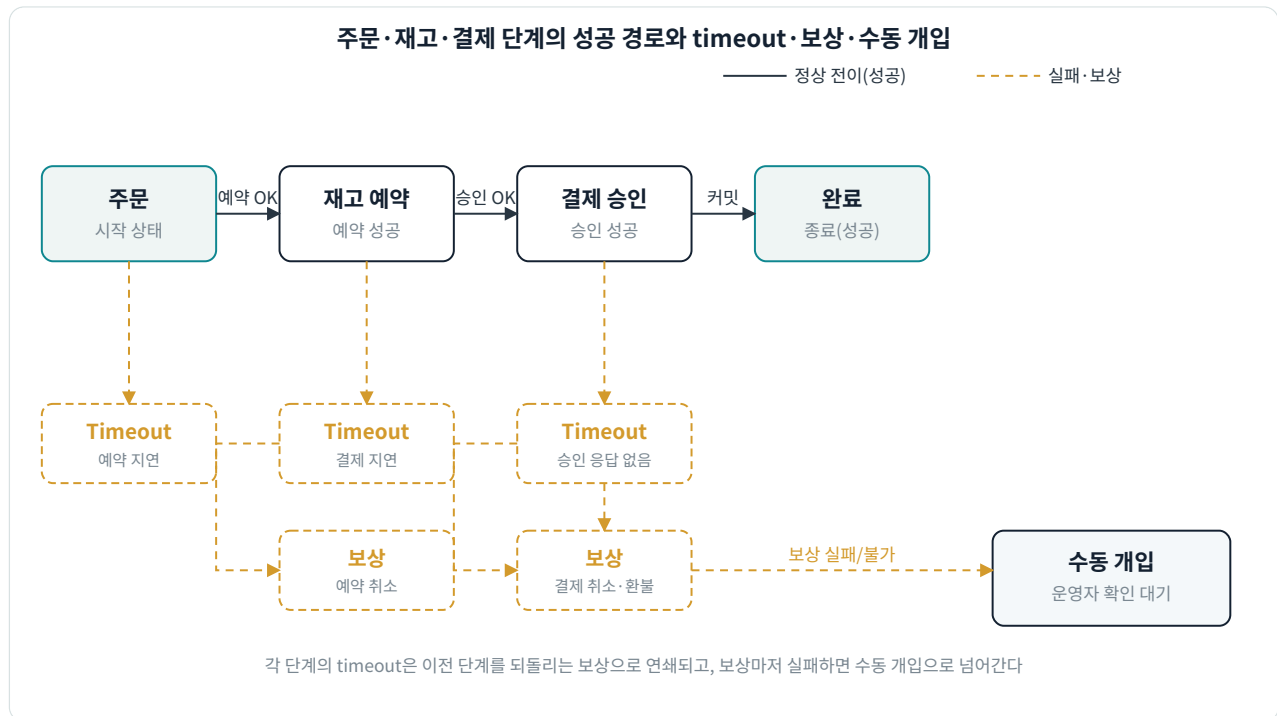
대안	장점	비용·위험	적합한 조건
Outbox polling	DB 기능 의존이 낮고 구현을 이해하기 쉽다.	poll latency · lock · 청소·중복을 관리해야 한다.	보통 규모와 단순 운영
Log-based CDC	낮은 지연과 전체 변경 capture에 유리하다.	DB별 connector · schema · snaps hot · 권한이 복잡하다.	대규모 event integration
Saga orchestration	상태와 장애 경로가 중앙에서 명확하다.	coordinator 의존과 결합이 생긴다.	결제·재고처럼 단계가 중요
Saga choreography	서비스 자율성과 확장이 좋다.	전체 흐름·loop·보상 추적이 어렵다.	단순 반응형 통합

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [debezium-docs; kafka-docs; saga-paper]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Dual-write gap	DB는 커밋됐지만 publish가 실패해 event가 영구 누락된다.	transactional outbox 또는 log CDC를 사용한다.
CDC position loss	connector가 잘못된 offset에서 재시작해 누락·대량 중복이 생긴다.	position checkpoint · snapshot 모드 · reconciliation을 검증한다.
Schema drift	DB column 변경이 downstream consumer를 깨뜨린다.	event envelope과 compatibility policy를 DB schema와 분리한다.

시나리오	영향	대응 원칙
Compensation failure	재고 복구나 환불 보상도 실패해 saga가 멈춘다.	보상 retry · 수동 queue · 불변 원장을 둔다.
Event loop	서비스들이 서로의 event에 반응해 무한 갱신한다.	causation/correlation ID와 state transition guard를 사용한다.



주문·재고·결제 단계의 성공·실패·timeout·보상 상태 전이를 보여준다

확장 전략

- aggregate key와 event partition을 맞춰 필요한 순서만 유지한다.
- CDC connector와 broker를 scale-out하기 전에 DB log retention과 source I/O를 확인한다.
- saga state는 무한히 커지지 않게 terminal state archive와 retention을 둔다.
- replay 시 신규 side effect를 차단하거나 별도 sandbox consumer를 사용한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- CDC 계정은 필요한 table/log 권한만 갖고 secret rotation을 지원한다.
- event payload의 개인정보를 최소화하고 삭제·암호화·retention 정책을 적용한다.
- 보상·수동 승인 작업은 강한 권한과 감사 trail을 요구한다.

- event provenance와 schema signature로 위조·오염을 탐지한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- outbox age · pending row · publish latency
- CDC lag · source log retention margin · snapshot status
- consumer inbox duplicate · processing latency
- saga state별 체류 시간·timeout·compensation failure
- reconciliation mismatch · manual intervention

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- outbox/CDC는 broker·connector·storage·on-call 비용을 추가한다.
- saga는 lock을 오래 잡지 않지만 상태 기계·보상·수동 처리 비용을 만든다.
- 모든 DB 변경을 event로 내보내면 저장·보안·consumer 결합 비용이 폭증한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- dual write에 재시도만 추가해 안전하다고 생각한다.
- DB row 변경을 그대로 domain event로 공개한다.
- 보상을 원래 transaction의 완전한 rollback으로 가정한다.
- choreography가 중앙 결합이 없으니 항상 단순하다고 생각한다.

설계 리뷰

- ☐ 업무 commit과 event 생성이 같은 원자 경계에 있는가?
- ☐ event가 사실·명령·통지 중 무엇인지 명확한가?

- CDC schema · snapshot · offset 복구가 시험됐는가?
- saga의 timeout · 보상 실패 · 수동 개입 상태가 정의됐는가?
- reconciliation이 최종 불일치를 찾는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 주문·결제·재고 saga를 orchestration 상태 기계로 설계하라.
2. DB schema 변경과 event schema 변경을 분리하는 envelope을 작성하라.
3. outbox dispatcher가 같은 row를 두 번 발행해도 안전한 consumer를 설계하라.

핵심 요약

- 이벤트 아키텍처도 local transaction을 필요로 한다.
- outbox는 상태 변경과 발행 record를 원자적으로 저장한다.
- CDC에는 schema · offset · snapshot · 복구 운영이 필요하다.
- saga 보상은 새로운 업무 action이다.
- end-to-end reconciliation과 수동 개입이 필수다.

출처

- [@debezium-docs] Debezium Authors. **Debezium Documentation** (2026). <https://debezium.io/documentation/reference/stable/>
- [@kafka-docs] Apache Software Foundation. **Apache Kafka Documentation** (2026). <https://kafka.apache.org/documentation/>
- [@saga-paper] Hector Garcia-Molina and Kenneth Salem. **Sagas** (1987). <https://doi.org/10.1145/38713.38742>

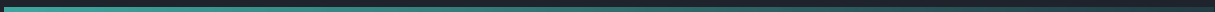
“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

PART

5

프로덕션 시스템



정상 동작만이 아니라 과부하, 침해, 배포, 재해, 비용까지 포함해 운영 가능한 구조를 만든다.

이 Part에서는 특정 제품의 설정법보다 반복해서 적용할 수 있는 설계 질문과 실패 모델을 먼저 익힌다. 각 장의 체크리스트를 실제 시스템의 ADR·런북·대시보드와 연결해 읽는 것이 목표다.

포함 장

ID	장 제목	최신성	원본 관계
ch25	Timeout·Deadline·Retry·Backoff·Jitter	durable	ADD
ch26	Circuit Breaker·Bulkhead·Backpressure·Load Shedding	durable	ADD
ch27	Metrics·Logs·Traces와 OpenTelemetry	current	ADD
ch28	인증·인가·Zero Trust·Secrets·공급망 보안	current	REPLACE
ch29	Multi-region·Backup·재해 복구	durable	ADD
ch30	Container·Kubernetes·Serverless·IaC·GitOps·FinOps	current	ADD

읽는 방법

1. 먼저 각 장의 **요구사항과 실패 모델**을 자신의 시스템에 대입한다.
2. **대안과 트레이드오프**에서 현재 선택이 어떤 비용을 감수하는지 확인한다.
3. **장애 시나리오**를 게임데이 또는 설계 리뷰 질문으로 바꾼다.
4. 시각자료는 설명을 대신하지 않고 책임·흐름·복구 경계를 검증하는 용도로 사용한다.

25

Timeout · Deadline · Retry · Backoff · Jitter

이 장의 구성

- 01 이 장에서 해결할 문제
- 02 먼저 결론
- 03 요구사항과 실패 모델
- 04 핵심 개념
- 05 기존 아키텍처
- 06 요청·데이터 흐름
- 07 대안과 트레이드오프
- 08 장애 시나리오
- 09 확장 전략
- 10 보안과 개인정보
- 11 관측 가능성
- 12 비용과 운영 복잡도
- 13 흔한 오해와 안티패턴
- 14 설계 리뷰
- 15 연습문제
- 16 핵심 요약
- 17 출처

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

timeout은 실패를 해결하지 않고 기다림을 중단할 뿐이다. 재시도는 성공 가능성을 높일 수 있지만 하위 시스템이 느린 원인이 과부하라면 같은 요청을 더 보내 상황을 악화시킨다. 전체 deadline과 한 계층의 retry 소유권이 필요하다.

이 절의 기준 출처: [aws-timeouts-retries; google-sre-overload].

학습 목표

- timeout과 end-to-end deadline을 구분한다.
- 재시도 가능 조건과 retry budget을 설계한다.
- backoff·jitter·idempotency로 동시 재시도 부하를 제어한다.

먼저 결론

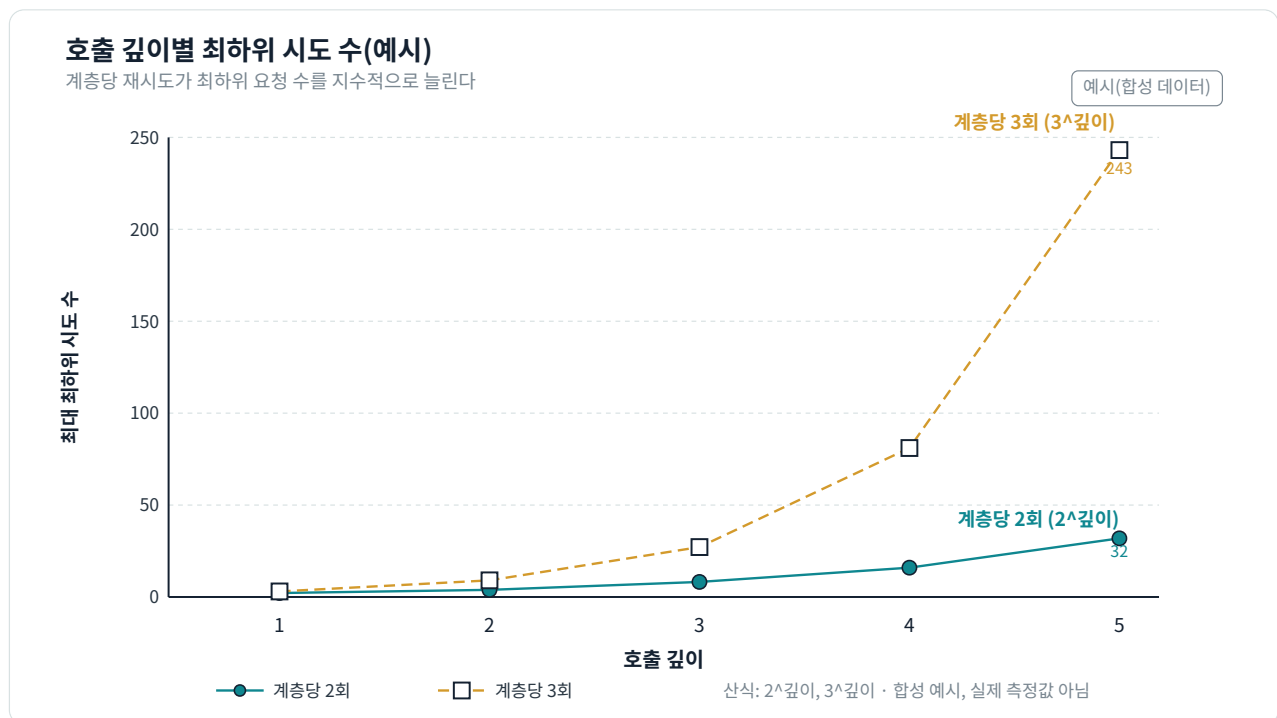
- 각 hop이 독립 timeout을 시작하지 말고 상위 요청의 남은 deadline을 전달한다.
- 재시도는 일시적이며 다시 실행해도 안전한 오류에만 사용한다.
- backoff만으로 동시 client가 다시 맞춰지는 문제를 막지 못하므로 jitter를 사용한다.
- 최대 시도 수보다 전체 retry budget·추가 부하 비율을 제한한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	Timeout·Deadline·Retry·Backoff·Jitter에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	각 hop이 독립 timeout을 시작하지 말고 상위 요청의 남은 deadline을 전달한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	호출 graph가 깊어질수록 각 hop에 임의 비율로 timeout을 복사하지 않고 critical path 예산을 분배한다.
실패·복구	“Retry storm” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	한 계층 소유, budget, exponential backoff, full jitter를 사용한다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	idempotency key가 사용자·operation scope에 묶이고 추측 불가능하거나 인증된 요청에만 유효하게 한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	attempt별 latency와 최종 사용자 latency

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



계층별 재시도 증폭

핵심 개념

Timeout

한 작업이나 I/O를 더 기다리지 않기로 정한 한도다.

Deadline

전체 요청이 완료돼야 하는 절대 시각 또는 남은 시간 예산이다.

Retry

실패한 작업을 다시 시도하는 행위다.

Backoff

연속 실패 사이 대기 시간을 늘리는 정책이다.

Jitter

재시도 시점을 무작위화해 동기화된 폭주를 줄인다.

Retry budget

정상 요청 대비 추가 시도량 또는 전체 시간·횟수를 제한하는 예산이다.

Idempotency

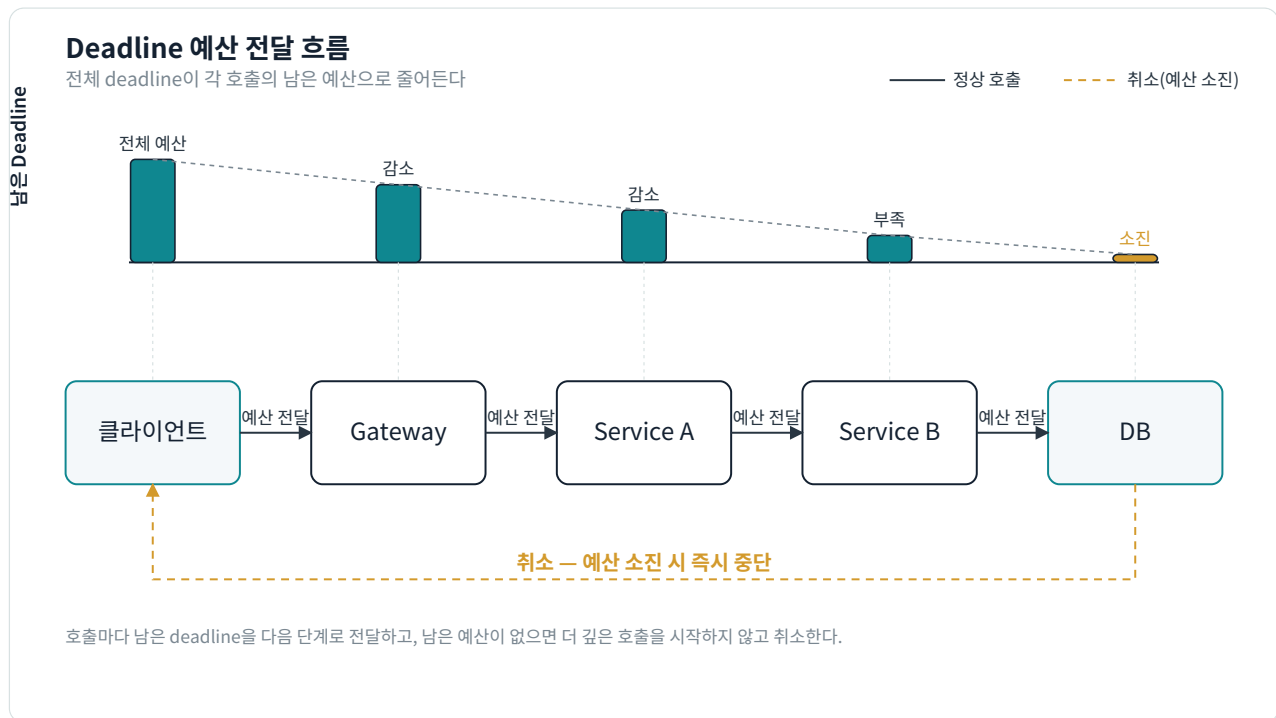
같은 요청을 여러 번 수행해도 의도한 최종 효과가 한 번과 같도록 하는 성질이다.

핵심 개념의 정의와 범위는 [[@aws-timeouts-retries](#); [@google-sre-overload](#); [@stripe-idempotency](#)]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Client deadline	사용자 경험과 전체 작업 한도를 정한다.
Ingress	deadline · request ID · idempotency key를 검증한다.
Retry owner	한 계층에서 retry classification과 budget을 관리한다.
Downstream client	남은 deadline보다 짧은 connect/read/write timeout을 적용한다.
Idempotency store	요청 key와 진행·결과 상태를 보존한다.
Circuit/load signal	과부하·Retry-After·queue 상태를 재시도 판단에 제공한다.



클라이언트 전체 deadline이 gateway · service · DB 호출의 남은 예산으로 줄어드는 흐름을 보여준다

요청·데이터 흐름

1. 클라이언트가 전체 deadline과 요청 식별자를 보낸다.
2. ingress가 남은 예산을 계산하고 이미 완료된 요청을 거부한다.
3. 하위 호출 전에 connect · request timeout을 예산 안에서 배분한다.
4. 오류를 retryable · permanent · unknown으로 분류한다.
5. retryable이고 예산이 남으면 jittered backoff 후 다시 시도한다.
6. 서버는 idempotency key로 중복 진행·완료 결과를 반환한다.
7. deadline 만료 시 하위 취소를 전파하고 낮은 결과 처리를 중단한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
고정 timeout, 무재시도	부하와 중복 효과가 예측 가능하다.	짧은 일시 실패도 사용자 오류가 된다.	비밀등·빠른 실패 선호
제한 재시도	일시 네트워크 오류를 흡수한다.	부하 증폭과 tail 증가가 생긴다.	먹등 읽기·작은 쓰기

대안	장점	비용·위험	적합한 조건
비동기 job 전환	긴 작업을 durable queue와 상태 조회로 분리한다.	UX·상태 기계·취소가 복잡해진다.	수초 이상 작업·외부 의존성

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [@aws-timeouts-retries; @google-sre-overload; @stripe-idempotency]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Retry storm	하위 장애 중 모든 client가 즉시 여러 번 재시도한다.	한 계층 소유, budget, exponential backoff, full jitter를 사용한다.
Timeout mismatch	gateway는 2초인데 하위 작업은 30초 계속돼 자원이 누적된다.	deadline propagation과 cancellation을 적용한다.
Unknown outcome	client timeout 후 서버 commit 여부를 알 수 없다.	idempotency key와 결과 조회 endpoint를 둔다.
Too-short timeout	정상 p99·DNS·TLS handshake를 포함하지 못해 가짜 오류를 만든다.	단계별 분포와 cold path를 측정해 설정한다.
Too-long timeout	실패한 요청이 thread·connection·memory를 오래 점유한다.	queue budget과 사용자 SLO에 맞춰 제한한다.

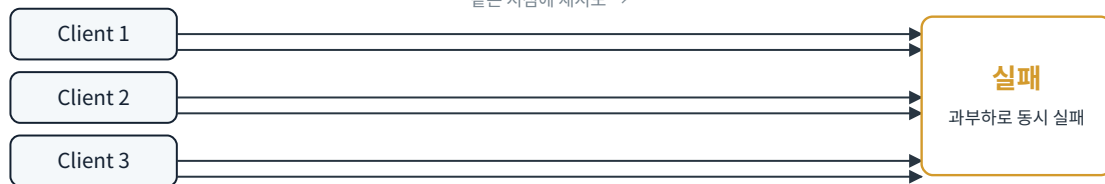
동기 재시도 vs Jitter 적용 분산 재시도

같은 시점에 몰리는 재시도가 실패를 키우고, 분산이 회복을 돕는다

—— 동기화된 재시도 분산 재시도

동기 재시도 — 함께 몰리는 재시도

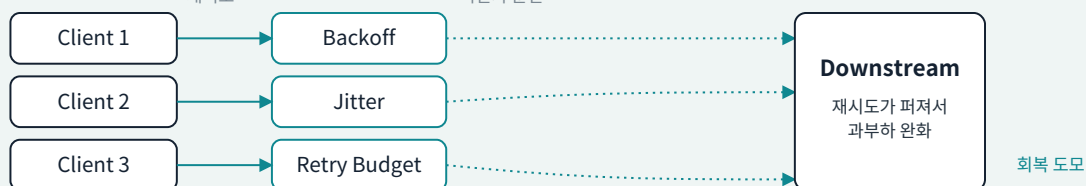
같은 시점에 재시도 →



Backoff + Jitter + Retry Budget — 분산된 재시도

재시도

시간차 분산



여러 client의 동기 재시도와 jitter 적용 후 분산된 재시도를 비교한다

확장 전략

- 호출 graph가 깊어질수록 각 hop에 임의 비율로 timeout을 복사하지 않고 critical path 예산을 분배한다.
- batch 요청은 항목별 실패와 전체 deadline을 분리한다.
- Retry-After와 서버 load signal을 존중한다.
- 재시도보다 fallback·cache·queue·degradation이 더 싼 경로인지 비교한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- idempotency key가 사용자·operation scope에 묶이고 추측 불가능하거나 인증된 요청에만 유효하게 한다.
- 재시도 로그에 민감 payload를 반복 저장하지 않는다.
- timeout 오류가 내부 topology와 공급자 정보를 과도하게 노출하지 않게 한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- attempt별 latency와 최종 사용자 latency
- timeout 단계(connect/read/write/queue)별 비율
- retry attempts·success-after-retry·amplification
- idempotency hit·conflict·in-progress
- deadline exceeded 후 계속 실행된 작업 수

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 재시도는 추가 compute·DB·egress를 소비한다.
- 너무 짧은 timeout은 오류·지원 비용을, 너무 긴 timeout은 자원·사용자 대기 비용을 만든다.
- idempotency 결과 보존 기간은 storage와 업무 재시도 창 사이의 선택이다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 모든 5xx를 같은 방식으로 재시도한다.
- 각 계층이 3회 재시도하면 총 3회라고 생각한다.
- timeout 값을 평균 latency의 두 배로 정한다.
- POST는 무조건 재시도 불가능하다고 단정하거나 반대로 key 없이 재시도한다.

설계 리뷰

- ☐ 전체 deadline과 hop별 timeout 관계가 명확한가?
- ☐ 오류별 retryability가 계약에 정의됐는가?
- ☐ 한 계층이 retry budget을 소유하는가?
- ☐ unknown outcome을 idempotency와 조희로 해결하는가?
- ☐ 재시도 증폭과 늦은 작업을 관측하는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 5단계 호출이 각자 3회 시도할 때 최악의 하위 호출 수를 계산하고 retry owner를 하나로 줄여라.
2. 결제 승인 timeout 후 결과를 모르는 상태를 idempotency key로 설계하라.
3. p99 400ms인 API의 1초 deadline 안에서 두 하위 호출 예산을 배분하라.

핵심 요약

- timeout은 기다림 중단, deadline은 전체 시간 계약이다.
- 재시도는 안전한 일시 오류와 멍등성에만 사용한다.
- retry budget과 jitter로 부하 증폭을 제한한다.
- unknown outcome에는 결과 조희가 필요하다.
- 취소와 남은 deadline을 하위까지 전달한다.

출처

- [@aws-timeouts-retries] Amazon Web Services. **Timeouts, retries, and backoff with jitter** (2026). <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>
- [@google-sre-overload] Google. **Site Reliability Engineering — Handling Overload** (2016). <https://sre.google/sre-book/handling-overload/>
- [@stripe-idempotency] Stripe. **Stripe API — Idempotent requests** (2026). https://docs.stripe.com/api/idempotent_requests

“

검증 기준일: 2026-08-06. `current` 와 `volatile` 내용은 release 전에 공식 원문을 다시 확인한다.

26

Circuit Breaker · Bulkhead · Backpressure · Load Shedding

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청-데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

복원력 패턴은 실패한 하위를 숨기는 장식이 아니다. circuit breaker는 반복 실패 호출을 줄이고, bulkhead는 자원 풀을 격리하며, backpressure는 생산 속도를 늦추고, load shedding은 감당할 수 없는 요청을 명시적으로 버린다.

이 절의 기준 출처: [@google-sre-overload; @aws-timeouts-retries].

학습 목표

- 장애 격리와 과부하 제어 패턴의 역할을 구분한다.
- backpressure를 생산자까지 전달한다.
- load shedding과 graceful degradation 우선순위를 설계한다.

먼저 결론

- 과부하를 queue 증가로 숨기지 말고 admission 단계에서 제한한다.
- circuit breaker는 health oracle이 아니라 최근 실패를 바탕으로 한 로컬 보호 장치다.
- bulkhead는 중요한 workload가 비핵심 workload에 자원을 빼앗기지 않게 한다.
- shed 정책은 우선순위·공정성·사용자에게 보이는 오류 의미를 가져야 한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	Circuit Breaker · Bulkhead · Backpressure · Load Shedding에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	과부하를 queue 증가로 숨기지 말고 admission 단계에서 제한한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	global limit와 instance limit를 조합해 scale-out 중 double admission을 막는다.
실패·복구	“Breaker herd” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	randomized probe와 global load signal을 사용한다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	quota · priority를 client가 임의 조작하지 못하게 서버 정책에서 결정한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	admitted/rejected/shed 요청과 이유

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Circuit breaker

실패율·지연이 임계치를 넘으면 일정 기간 호출을 빠르게 실패시키고 probe로 회복을 확인한다.

Bulkhead

thread · connection · queue · tenant capacity를 분리해 실패 전파를 줄인다.

Backpressure

consumer가 감당할 수 있는 속도를 producer에게 전달하거나 수신을 늦추는 메커니즘이다.

Load shedding

처리 능력을 넘은 요청을 의도적으로 거부·축소하는 전략이다.

Admission control

요청을 작업 큐에 넣기 전에 현재 자원과 정책으로 허용 여부를 결정한다.

Graceful degradation

전체 실패 대신 비핵심 기능·정확도·신선도를 낮춰 핵심 여정을 유지한다.

Adaptive concurrency

관측된 지연·queue로 허용 동시성을 동적으로 조절한다.

핵심 개념의 정의와 범위는 [@google-sre-overload; @aws-timeouts-retries]를 기준으로 재검토해야 한다.

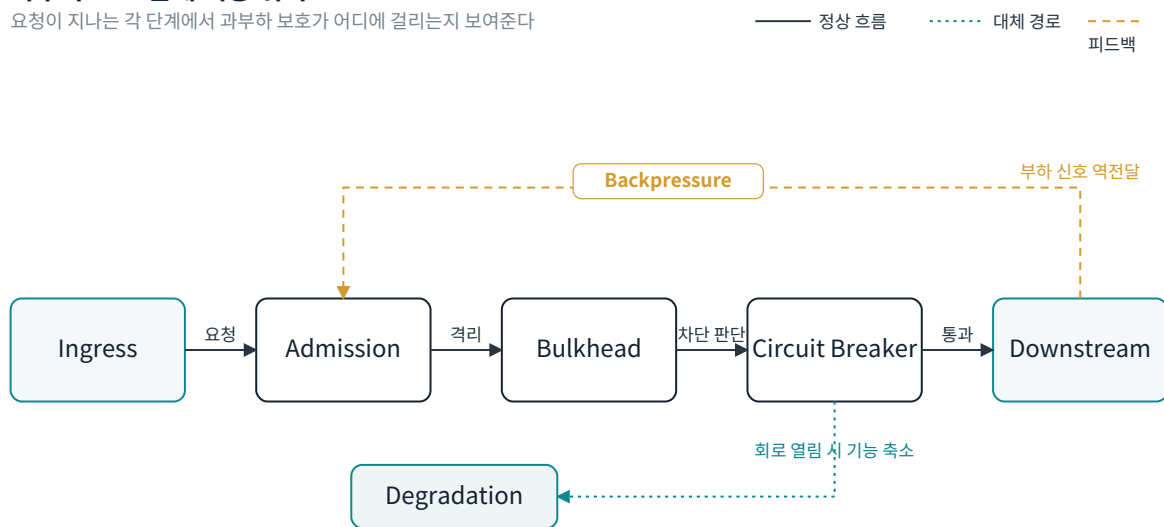
기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Ingress limiter	tenant · priority · global quota를 적용한다.
Admission controller	queue · CPU · downstream 건강으로 수락 여부를 정한다.
Bulkhead pools	핵심/비핵심 또는 tenant별 자원을 격리한다.
Circuit breaker	dependency별 closed/open/half-open 상태를 관리한다.
Backpressure channel	credit · window · lag · 429/503로 생산 속도를 제어한다.
Degradation policy	cache · partial result · read-only · feature off를 선택한다.
Recovery controller	probe와 점진 트래픽으로 정상 상태를 복원한다.

과부하 보호 단계 적용 위치

요청이 지나는 각 단계에서 과부하 보호가 어디에 걸리는지 보여준다



Admission은 받아들이는 양을, Bulkhead는 격리 범위를, Circuit Breaker는 반복 실패의 차단율, Backpressure와 Degradation은 부하를 거꾸로 줄인다.

ingress에서 admission · bulkhead · circuit · backpressure · degradation이 적용되는 위치를 보여준다

요청·데이터 흐름

1. 요청에 우선순위 · tenant · 비용 추정치를 붙인다.
2. ingress quota와 현재 동시성 한도를 검사한다.
3. 허용 요청을 해당 bulkhead queue에 넣는다.

4. 하위 호출 전에 circuit 상태와 남은 deadline을 확인한다.
5. 포화 시 producer에게 backpressure 또는 명시적 거부를 보낸다.
6. degradation 단계에서 비핵심 작업을 생략한다.
7. 회복 시 probe와 작은 트래픽으로 한도를 점진 확대한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
고정 limit	예측 가능하고 검증이 단순하다.	트래픽·인스턴스·latency 변화에 과소/과대 제한될 수 있다.	안정된 workload
Adaptive limit	실제 latency와 queue에 따라 포화 전 보호한다.	진동·잘못된 신호·튜닝 위험이 있다.	변동 큰 service
Queue buffering	짧은 burst를 흡수한다.	작업 가치가 만료되고 memory·tail이 증가한다.	짧고 복구 가능한 burst
Load shedding	핵심 요청의 SLO를 보호한다.	거부 정책과 사용자 영향이 필	지속 과부하

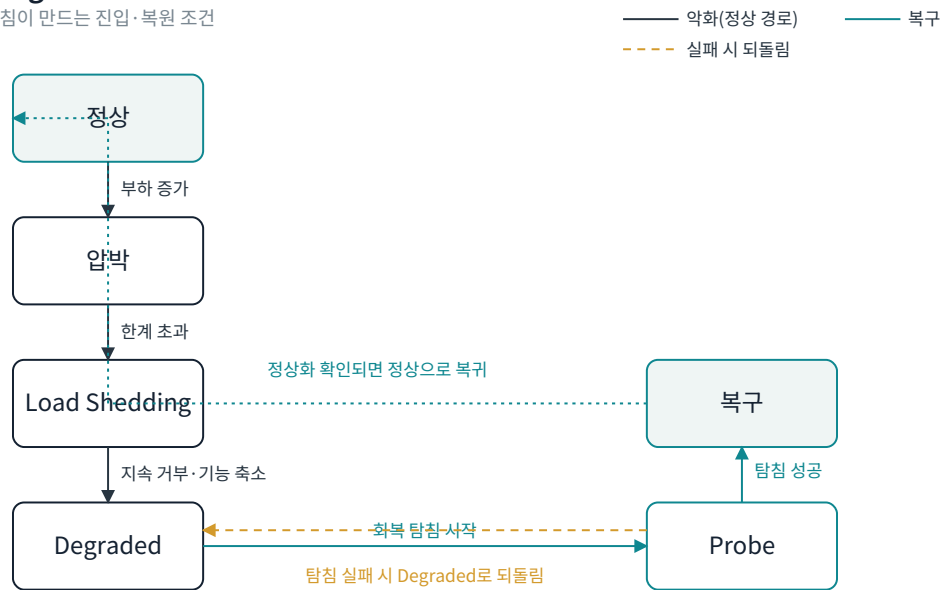
대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [@google-sre-overload; @aws-timeouts-retries]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Breaker herd	모든 instance가 같은 시점에 half-open probe를 보내 하위를 다시 압박한다.	randomized probe와 global load signal을 사용한다.
Queue collapse	무한 queue가 timeout된 요청을 계속 처리한다.	유한 queue, deadline-aware dequeue, stale work drop을 적용한다.
Priority inversion	비핵심 긴 작업이 핵심 짧은 작업의 pool을 점유한다.	bulkhead와 weighted fair scheduling을 사용한다.
Backpressure loss	중간 broker가 생산자 속도를 늦추지 못해 lag가 무한 증가한다.	credit·quota·producer pause와 retention 경보를 연결한다.
Unfair shedding	같은 고객군이 계속 먼저 거부된다.	tenant별 quota·공정성 지표·sampling을 적용한다.

Load Shedding 상태 전이

부하 악화와 회복 탐침이 만드는 진입·복원 조건



정상·압박·shed·복구 상태와 진입·복원 조건을 보여준다

확장 전략

- global limit와 instance limit를 조합해 scale-out 중 double admission을 막는다.
- 비용 추정치가 큰 요청은 별도 pool·async job으로 분리한다.
- degradation은 단계별 feature flag와 자동 복원 조건을 둔다.
- circuit 상태를 모든 장애의 단일 진실로 공유하지 않고 각 호출자의 보호 경계로 사용한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- quota·priority를 client가 임의 조작하지 못하게 서버 정책에서 결정한다.
- shed 응답이 사용자 존재·권한 여부를 노출하지 않게 일관된 오류를 사용한다.
- 관리자 bypass는 제한된 break-glass와 감사가 필요하다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- admitted/rejected/shed 요청과 이유
- queue depth · wait · expired work
- bulkhead별 saturation · starvation
- breaker state · open reason · probe success
- backpressure signal · producer rate · consumer lag
- degradation 단계와 사용자 SLI

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace · log · 변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 여유 용량과 격리 pool은 직접 비용이지만 전체 장애 비용을 줄인다.
- 무한 queue는 인프라 비용뿐 아니라 이미 가치 없는 작업 처리 비용을 만든다.
- 세밀한 priority 정책은 운영·제품 합의 비용을 요구한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- circuit breaker를 모든 오류에 복사하면 안정적이라고 생각한다.
- queue를 키워 overload를 해결한다.
- 429/503 없이 연결을 느리게 하는 것만 backpressure라고 부른다.
- load shedding을 무작위 오류로 구현한다.

설계 리뷰

- ☐ 포화 신호와 admission 기준이 실제 병목을 반영하는가?
- ☐ 핵심·비핵심 workload가 자원 수준에서 격리되는가?
- ☐ producer까지 backpressure가 전달되는가?
- ☐ shed 우선순위와 공정성이 합의됐는가?
- ☐

회복 시 probe와 점진 확대가 있는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 검색 API에서 철자 교정·추천·개인화를 단계적으로 끄는 degradation 정책을 설계하라.
2. 무한 queue가 2초 deadline 작업을 30초 뒤 처리하는 문제를 고쳐라.
3. tenant별 fair shedding과 global overload limit를 함께 설계하라.

핵심 요약

- circuit breaker · bulkhead · backpressure · shedding은 서로 다른 문제를 해결한다.
- 무한 queue는 overload를 숨기고 tail을 악화한다.
- admission control은 작업을 시작하기 전에 보호한다.
- degradation은 핵심 사용자 여정을 보존한다.
- 회복도 점진적이고 관측 가능해야 한다.

출처

- [@google-sre-overload] Google. **Site Reliability Engineering — Handling Overload** (2016).
<https://sre.google/sre-book/handling-overload/>
- [@aws-timeouts-retries] Amazon Web Services. **Timeouts, retries, and backoff with jitter** (2026).
<https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

27

Metrics · Logs · Traces와 OpenTelemetry

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

관측 가능성은 telemetry를 많이 저장하는 것이 아니라 내부 상태를 외부 신호로 설명할 수 있게 만드는 능력이다. SLO와 장애 질문에서 출발해 metrics, logs, traces를 최소한으로 연결하고, correlation ID와 semantic convention을 일관되게 사용해야 한다.

이 절의 기준 출처: [otel-spec; @w3c-trace-context].

학습 목표

- metrics·logs·traces의 서로 다른 질문을 구분한다.
- OpenTelemetry 계측·수집·export 경계를 설계한다.
- cardinality·sampling·민감 데이터 비용을 통제한다.

먼저 결론

- metrics는 집계 추세, logs는 개별 사건, traces는 분산 요청의 인과 경로에 강하다.
- OpenTelemetry는 계측과 telemetry 파이프라인의 vendor-neutral 경계를 제공하지만 backend·보존·경보 설계까지 자동으로 결정하지 않는다.
- 고 cardinality attribute를 metric label로 사용하면 비용과 안정성이 무너질 수 있다.
- sampling은 비용 절감과 rare failure 보존 사이의 정책이다.

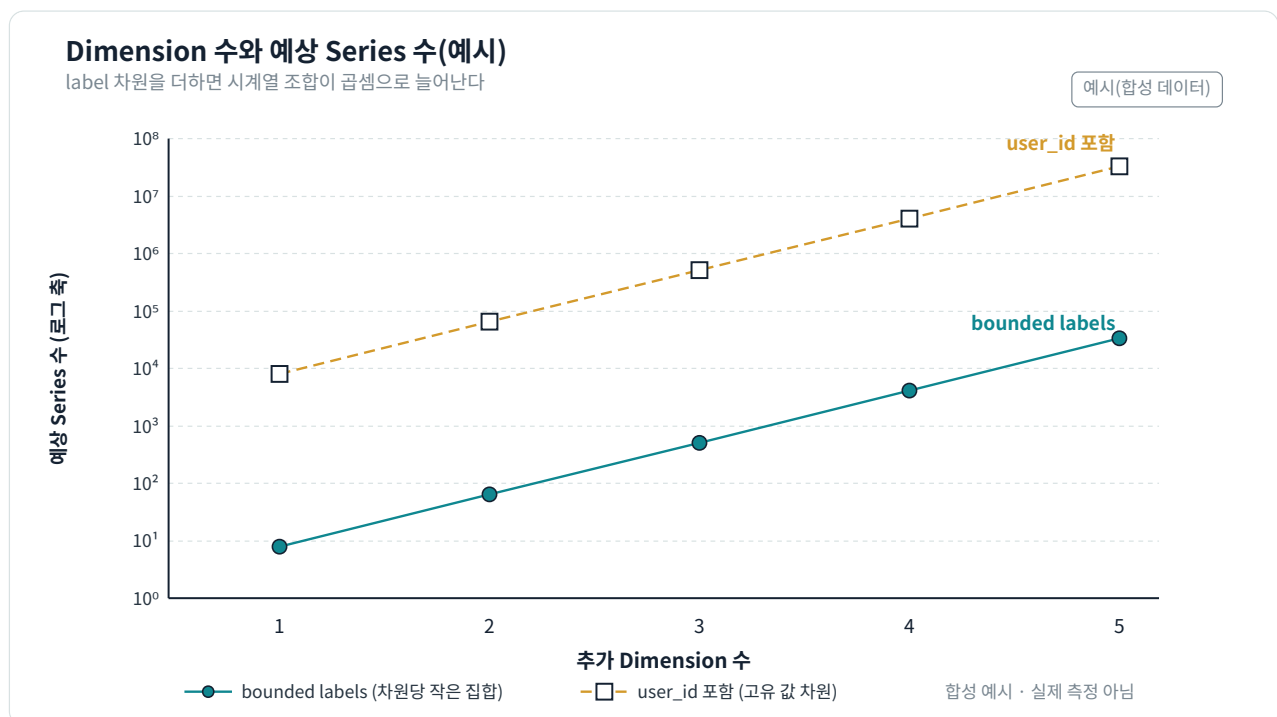
∴ current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06이며, 출판 직전 공식 문서를 다시 확인한다. ∴

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	Metrics·Logs·Traces와 OpenTelemetry에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	metrics는 집계 추세, logs는 개별 사건, traces는 분산 요청의 인과 경로에 강하다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	collector를 지역·cluster 단위로 계층화 하되 제어 설정과 pipeline version을 관리한다.

차원	확인 질문	설계 판단
실패·복구	“Cardinality explosion” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	bounded dimension만 metric에 사용하고 상세 값은 trace/log로 보낸다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	telemetry는 운영 데이터이지만 개인정보·비밀·인증 token을 포함할 수 있어 별도 보안 영역으로 다룬다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	telemetry ingest·drop·retry·queue

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



Metric cardinality와 비용

핵심 개념

Metric

시간에 따른 수치 집계로 rate·histogram·gauge 등을 표현한다.

Log

특정 시점의 구조화된 사건 record다.

Trace

하나의 분산 작업을 span과 parent-child 관계로 표현한다.

Context propagation

trace ID, baggage, deadline 같은 context를 hop 사이 전달하는 과정이다.

Collector

telemetry를 수신·처리·batch·filter·export하는 구성 요소다.

Cardinality

label/attribute 값 조합의 수로 metric storage와 query 비용에 큰 영향을 준다.

Sampling

전체 trace 중 일부를 선택하는 정책으로 head·tail·rule-based 방식이 있다.

Semantic convention

HTTP, DB, messaging 등 공통 attribute 이름과 의미를 정의하는 규칙이다.

핵심 개념의 정의와 범위는 [@otel-spec; @w3c-trace-context; @google-sre-slo]를 기준으로 재검토해야 한다.

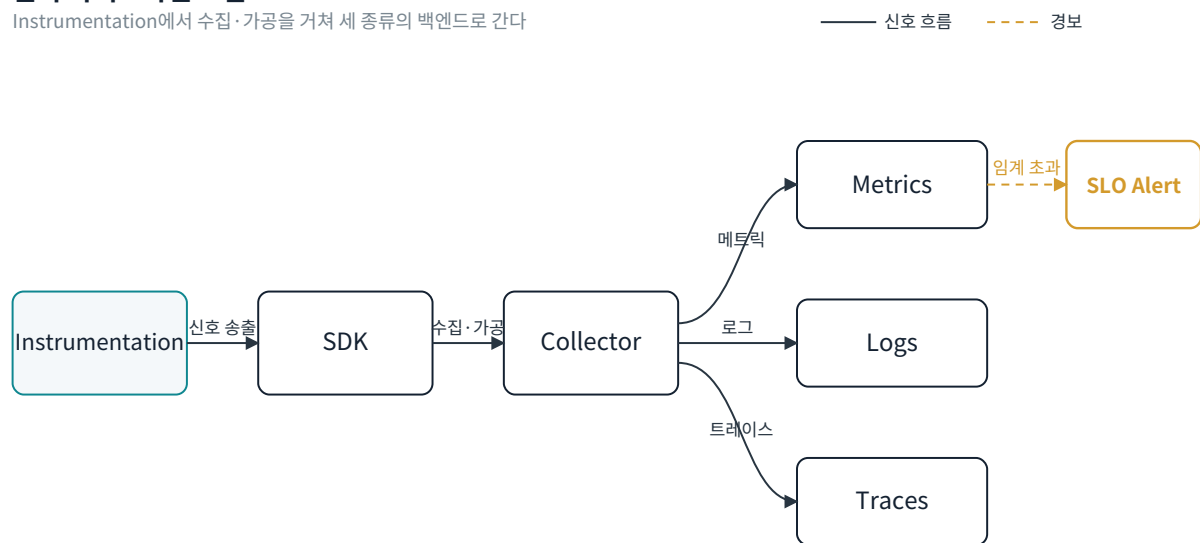
| 기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Instrumentation	애플리케이션과 library가 span·metric·log를 생성한다.
SDK/agent	context·processor·exporter를 관리한다.
Local/central collector	batch·retry·redaction·sampling을 수행한다.
Telemetry backend	시계열·로그·trace를 저장·조회한다.
SLO/alert engine	사용자 지표와 burn rate를 평가한다.
Investigation workflow	alert에서 trace·log·deployment·profile로 이동한다.
Governance	schema·retention·PII·cost budget을 관리한다.

관측 파이프라인 흐름

Instrumentation에서 수집·가공을 거쳐 세 종류의 백엔드로 간다



SDK가 신호를 표준 형식으로 내보내면 Collector가 가공해 세 백엔드로 분배하고, Metrics의 SLO 위반이 경보를 만든다.

애플리케이션 instrumentation에서 collector 처리와 metrics/logs/traces backend로 가는 흐름을 보여준다

요청·데이터 흐름

1. 사용자 여정과 장애 질문에서 필요한 신호를 정한다.
2. 공통 resource·service·deployment 식별자를 정의한다.
3. entry에서 trace context를 만들거나 신뢰된 parent를 검증한다.
4. 주요 경계와 고비용 작업에 span과 metric을 기록한다.
5. collector에서 redaction·batch·sampling·retry를 수행한다.
6. SLO alert가 exemplar/trace로 조사 경로를 제공한다.
7. telemetry 비용과 query 사용률로 schema를 지속 정리한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

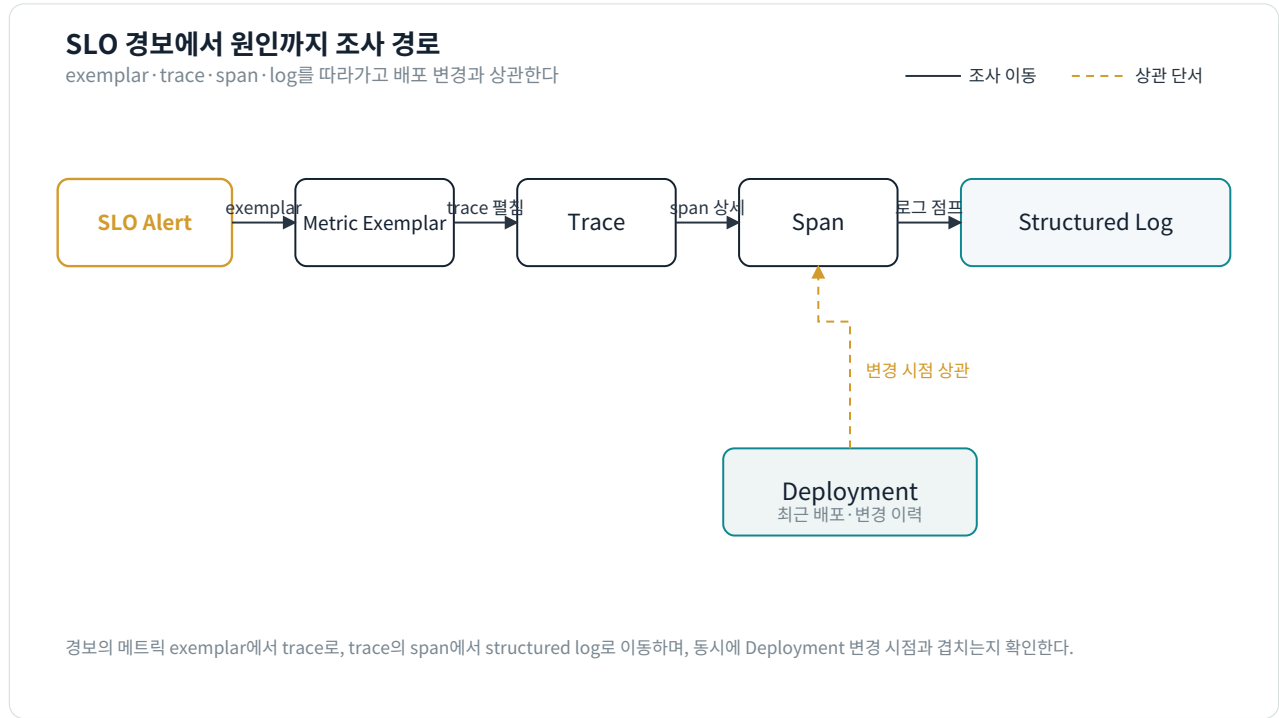
대안	장점	비용·위험	적합한 조건
직접 backend SDK	구성이 단순하고 backend 기능을 빠르게 사용한다.	vendor lock-in과 계층 중복이 커질 수 있다.	작은 단일 backend 시스템
OpenTelemetry SDK+Collector	계측·export 경계를 표준화하고 pipeline 제어가 가능하다.	collector 운영과 semantic version 관리가 필요하다.	다언어·다중 backend 시스템

대안	장점	비용·위험	적합한 조건
Agent/eBPF 중심	코드 변경 없이 넓은 가시성을 얻는다.	업무 의미·사용자 context가 부족할 수 있다.	legacy·인프라 관측 보완

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [otel-spec; @w3c-trace-context; @google-sre-slo]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Cardinality explosion	user ID · URL 원문 · request ID를 metric label로 넣어 시계열 수가 폭증한다.	bounded dimension만 metric에 사용하고 상세 값은 trace/log로 보낸다.
Telemetry outage	collector 장애가 app thread를 막거나 memory queue를 채운다.	비차단 export, 유한 queue, drop metric, local buffering 한도를 둔다.
Broken context	async · queue 경계에서 trace parent가 유실된다.	표준 propagation과 message link/correlation을 사용한다.
Sampling blind spot	head sampling이 드문 오류 trace를 버린다.	tail sampling · error keep rule · exemplar를 적용한다.
PII leakage	header · SQL · prompt · document 내용이 telemetry에 복사된다.	allowlist · redaction · classification · access control을 둔다.



SLO 경보에서 exemplar · trace · span log · deployment change로 조사하는 경로를 보여준다

확장 전략

- collector를 지역·cluster 단위로 계층화하되 제어 설정과 pipeline version을 관리한다.
- high-volume signal은 aggregation·sampling·short retention을 사용한다.
- trace sampling은 service별이 아니라 end-to-end decision 일관성을 유지한다.
- log와 trace를 같은 ID로 연결하되 모든 log에 trace가 있을 필요는 없다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- telemetry는 운영 데이터이지만 개인정보·비밀·인증 token을 포함할 수 있어 별도 보안 영역으로 다룬다.
- collector와 backend 사이를 인증·암호화하고 tenant 접근을 분리한다.
- baggage는 downstream에 전파되므로 민감 정보를 넣지 않는다.
- 보존·삭제·export 정책을 원본 데이터와 일치시킨다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- telemetry ingest·drop·retry·queue
- metric series cardinality와 top attributes
- trace sampling rate·error kept·broken context
- log bytes·query rate·retention
- SLO alert에서 원인 trace까지 연결 성공률
- 계측 overhead CPU·memory·latency

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- observability 비용은 $\text{ingest} \times \text{retention} \times \text{index cardinality} \times \text{query}$ 로 커진다.
- 모든 trace를 영구 저장하기보다 목적별 sampling과 tiered retention을 사용한다.

- 표준화는 초기 platform 비용이 들지만 중복 agent·SDK·dashboard를 줄인다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 로그가 많으면 관측 가능하다고 생각한다.
- 모든 값을 metric label로 넣는다.
- trace sampling을 각 서비스가 독립 결정한다.
- telemetry pipeline 실패가 서비스 요청을 실패시키게 한다.

설계 리뷰

- ☐ 각 signal이 답하려는 운영 질문이 명확한가?
- ☐ SLO에서 trace·log로 내려가는 조사 경로가 있는가?
- ☐ cardinality와 PII가 schema 수준에서 제한되는가?
- ☐ collector 장애가 app에 backpressure를 주지 않는가?
- ☐ sampling이 rare failure와 비용 목표를 함께 만족하는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 사용자 ID를 metric label로 쓰지 않고 고객별 장애를 조사하는 방식을 설계하라.
2. 비동기 queue consumer trace를 producer trace와 link하는 context를 설계하라.
3. 월 telemetry 예산을 기준으로 trace sampling·retention 정책을 작성하라.

핵심 요약

- metrics·logs·traces는 서로 다른 질문에 답한다.
- OpenTelemetry는 계측·수집 경계를 표준화한다.
- cardinality·sampling·retention은 비용과 품질을 결정한다.
- context propagation이 end-to-end 인과 관계를 만든다.
- telemetry 자체도 장애·보안·SLO 대상이다.

출처

- [otel-spec] OpenTelemetry Authors. **OpenTelemetry Specification** (2026). <https://opentelemetry.io/docs/specs/otel/>
- [w3c-trace-context] W3C. **Trace Context** (2021). <https://www.w3.org/TR/trace-context/>
- [google-sre-slo] Google. **SRE Workbook — Implementing SLOs** (2018). <https://sre.google/workbook/implementing-slos/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

28

인증·인가·Zero Trust·Secrets·공급망 보안

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

보안은 외곽 방화벽 한 겹이 아니라 모든 자원 접근에서 주체, 장치, workload, 요청 맥락, 정책을 검증하는 연속된 결정이다. 네트워크 위치는 신뢰의 근거 중 하나일 뿐이며, 인증 성공과 업무 권한 승인은 분리해야 한다.

이 절의 기준 출처: [@nist-zero-trust; @rfc9700].

학습 목표

- 인증·인가·세션·서비스 identity를 구분한다.
- zero trust 원칙을 자원 접근 흐름에 적용한다.
- secret·artifact·배포 공급망을 검증 가능하게 만든다.

먼저 결론

- 인증은 누구인지, 인가는 무엇을 할 수 있는지 결정한다.
- zero trust는 내부망이라는 이유만으로 암묵적 신뢰를 부여하지 않는다.
- 장기 secret를 코드·이미지·환경에 복사하지 않고 짧은 수명 credential과 workload identity를 선호한다.
- 배포 artifact는 출처·build 과정·서명을 검증하고 생산자에서 runtime까지 계보를 보존한다.

::: current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06이며, 출판 직전 공식 문서를 다시 확인한다. :::

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	인증·인가·Zero Trust·Secrets·공급망 보안에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	인증은 누구인지, 인가는 무엇을 할 수 있는지 결정한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	정책 평가를 중앙 논리와 지역 cache로 분리하되 revoke·version 정책을 둔다.
실패·복구	“Token theft” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	짧은 수명, sender constraint, audience 제한, revoke·risk signal을 사용한다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	개인정보 최소 수집·목적 제한·보존·삭제를 데이터 설계에 포함한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	인증 성공/실패·MFA/WebAuthn 사용률·위험 신호

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

인증

사용자·서비스·장치가 주장한 identity를 검증한다.

인가

검증된 주체가 특정 자원에 특정 action을 수행할 수 있는지 결정한다.

세션

인증 상태를 일정 기간 유지하는 server/client 계약이다.

Zero trust

위치나 소유만으로 신뢰하지 않고 접근마다 명시적 검증과 최소 권한을 적용하는 접근법이다.

Workload identity

서비스 instance가 장기 공유 secret 없이 자신을 증명하는 identity다.

Secret

password, API key, private key처럼 노출되면 권한을 행사할 수 있는 값이다.

Attestation/Provenance

artifact가 어떤 source·builder·dependency로 생성됐는지 검증하는 증거다.

FIDO/WebAuthn

공개키 credential로 phishing-resistant 인증을 제공하는 웹 표준 계열이다.

핵심 개념의 정의와 범위는 [@nist-zero-trust; @rfc9700; @webauthn3; @slsa12]를 기준으로 재검토해야 한다.

기준 아키텍처

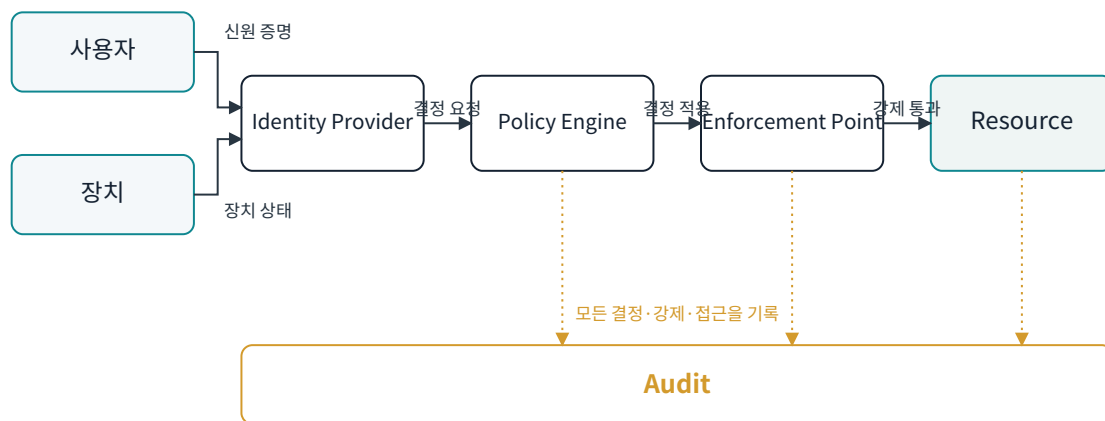
아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Identity provider	사용자 인증과 token 발급을 담당한다.
Policy decision point	subject·resource·action·context를 평가한다.
Policy enforcement point	gateway·service·DB에서 결정을 강제한다.
Workload identity issuer	runtime attestation을 바탕으로 짧은 credential을 발급한다.
Secret manager/KMS	secret과 key의 생성·보관·회전·감사를 제공한다.
Artifact registry	서명된 image/package와 provenance를 보존한다.
Admission controller	검증되지 않은 artifact·권한·구성을 배포 전에 차단한다.
Audit pipeline	인증·정책 결정·관리 작업을 tamper-evident하게 기록한다.

Zero Trust 접근 경계

신원·장치·정책 결정·강제·자원의 경계와 감사

—— 접근 요청 감사 관찰



사용자와 장치의 신원을 Identity Provider로 확인하고, Policy Engine의 결정을 Enforcement Point가 강제해 Resource에 접근하며, 모든 단계를 Audit이 관찰한다.

사용자·장치·workload identity와 정책 결정·강제·resource 접근 경계를 보여준다

요청·데이터 흐름

1. 사용자 또는 workload가 강한 인증으로 identity를 얻는다.
2. 요청이 audience·scope·expiry가 제한된 token을 제시한다.
3. enforcement point가 token 서명·발급자·replay·binding을 검증한다.
4. policy engine이 resource·tenant·action·context로 권한을 결정한다.
5. 민감 action은 step-up 또는 별도 승인·transaction 정책을 요구한다.
6. 배포 시 artifact signature와 provenance를 검증한다.
7. secret·권한·artifact 변경을 감사하고 자동 만료·회전한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
중앙 RBAC	역할 기반 관리가 단순하고 감사하기 쉽다.	역할 폭증과 context 부족이 생길 수 있다.	안정된 조직 권한
ABAC/정책 엔진	resource·tenant·시간·위험 등 세밀한 결정을 제공한다.	정책 디버깅·성능·일관성 운영이 필요하다.	복잡한 멀티테넌트·규제
네트워크 격리 중심	침해 표면과 경로를 줄인다.	identity·업무 권한을 대체할 수 없다.	방어 계층
짧은 workload credential	장기 secret 노출 위험을 줄인다.	identity control plane 가용성과 clock이 필요하다.	동적 cloud workload

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [@nist-zero-trust; @rfc9700; @webauthn3]를 참조한다.

장애 시나리오

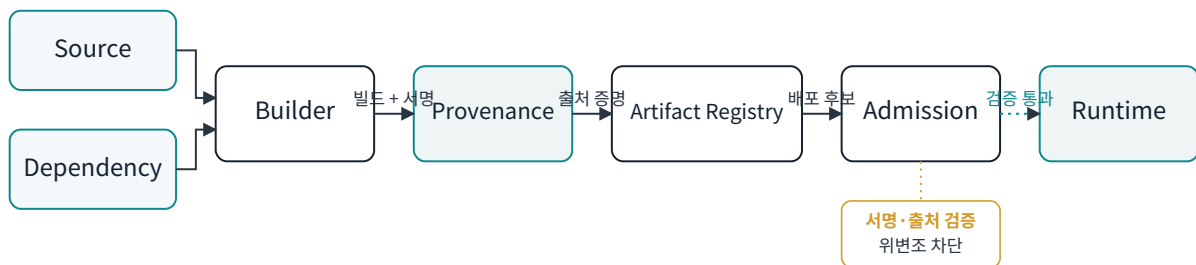
시나리오	영향	대응 원칙
Token theft	bearer token이 탈취돼 다른 장치에서 사용된다.	짧은 수명, sender constraint, audience 제한, revoke·risk signal을 사용한다.
권한 cache stale	role 회수 후 오래된 정책 cache가 계속 허용한다.	versioned policy와 민감 action의 강한 recheck를 둔다.
Secret sprawl	키가 repo·image·log·CI 변수에 복제된다.	동적 발급, secret scanning, 중앙 rotation, 사용 inventory를 둔다.

시나리오	영향	대응 원칙
Supply-chain compromise	악성 dependency 또는 builder가 정상 이름 artifact를 만든다.	pinning, isolated build, provenance, signature, admission verification을 사용한다.
Break-glass abuse	비상 계정이 평상시 우회 경로가 된다.	시간 제한·다중 승인·강한 감사·사후 검토를 적용한다.
IdP outage	인증·token 검증 제어면 장애가 모든 서비스로 전파된다.	local signature validation, key cache, 제한된 기존 session 정책을 둔다.

공급망 서명과 출처 검증

source·dependency에서 runtime까지 서명과 provenance가 검증된다

—— 산출물 흐름 검증 통과



source·dependency·builder·artifact·registry·admission·runtime의 서명과 provenance 검증을 보여준다

확장 전략

- 정책 평가를 중앙 논리와 지역 cache로 분리하되 revoke·version 정책을 둔다.
- resource별 권한을 token에 모두 넣어 비대해지지 않게 최소 claim과 server lookup을 조합한다.
- 서비스 수가 늘면 workload identity와 mTLS 발급을 자동화한다.
- audit와 detection은 high-volume access와 high-risk admin action을 다른 보존·경보로 관리한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 개인정보 최소 수집·목적 제한·보존·삭제를 데이터 설계에 포함한다.
- 인증 로그에도 user agent·IP·device 정보가 민감할 수 있어 접근과 보존을 제한한다.
- key와 secret는 환경별·tenant별 blast radius를 줄이고 회전 가능해야 한다.
- 보안 정책 변경도 코드 리뷰·canary·rollback을 거친다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 인증 성공/실패·MFA/WebAuthn 사용률·위험 신호
- 인가 allow/deny·policy version·decision latency
- token expiry·revocation·invalid audience
- secret age·rotation failure·unused credential
- artifact signature/provenance admission failure
- break-glass·privileged action·audit gap

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 세밀한 정책과 짧은 credential은 control plane·KMS·audit 비용을 만든다.
- 장기 secret는 초기 비용이 낮지만 침해 탐지·회전·사고 비용이 크다.
- 공급망 검증은 build 시간을 늘려도 배포 시 신뢰 근거와 사고 조사 시간을 줄인다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 내부망 요청은 인증을 생략한다.
- JWT 서명만 검증하면 권한 검사가 끝났다고 생각한다.
- secret를 암호화해 repo에 넣으면 안전하다고 본다.

- SBOM이나 서명 파일이 존재하면 실제 배포 artifact가 검증됐다고 가정한다.

설계 리뷰

- ☐ 주체·resource·action·context가 권한 결정에 포함되는가?
- ☐ token·policy cache·revocation의 시간 경계가 정의됐는가?
- ☐ 장기 secret를 줄이고 회전·폐기가 자동화됐는가?
- ☐ artifact provenance가 runtime admission에서 실제 검증되는가?
- ☐ break-glass와 관리 action이 독립 감사되는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 관리자·상담사·학생이 있는 교육 시스템의 RBAC/ABAC 정책을 설계하라.
2. IdP가 20분 중단될 때 신규 로그인과 기존 session의 정책을 구분하라.
3. source commit에서 production container까지 SLSA형 provenance 검증 경로를 그려라.

핵심 요약

- 인증과 인가는 별도 결정이다.
- zero trust는 위치 기반 암묵 신뢰를 제거한다.
- 짧은 workload identity가 장기 secret를 줄인다.
- policy cache와 revoke 경계를 설계한다.
- 공급망 증거는 배포 시 검증돼야 가치가 있다.

출처

- [@nist-zero-trust] NIST. **NIST SP 800-207 — Zero Trust Architecture** (2020). <https://csrc.nist.gov/pubs/sp/800/207/final>
- [@rfc9700] IETF. **RFC 9700 — Best Current Practice for OAuth 2.0 Security** (2025). <https://www.rfc-editor.org/rfc/rfc9700.html>
- [@webauthn3] W3C. **Web Authentication: An API for accessing Public Key Credentials — Level 3 (Candidate Recommendation Snapshot)** (2026). <https://www.w3.org/TR/webauthn-3/>
- [@slsa12] OpenSSF. **SLSA Specification v1.2** (2025). <https://slsa.dev/spec/v1.2/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

29

Multi-region · Backup · 재해 복구

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

다중 리전은 backup을 대체하지 않고 backup도 즉시 failover를 제공하지 않는다. 인프라 장애, 리전 장애, 운영 실수, 논리적 데이터 손상, 자격증명 침해는 서로 다른 복구 수단과 증거를 요구한다.

이 절의 기준 출처: [@nist-contingency; @google-sre-book].

학습 목표

- RTO · RPO를 사용자 여정과 데이터별로 정의한다.
- multi-region active/standby · active/active를 비교한다.
- backup · restore · failover · failback을 반복 검증한다.

먼저 결론

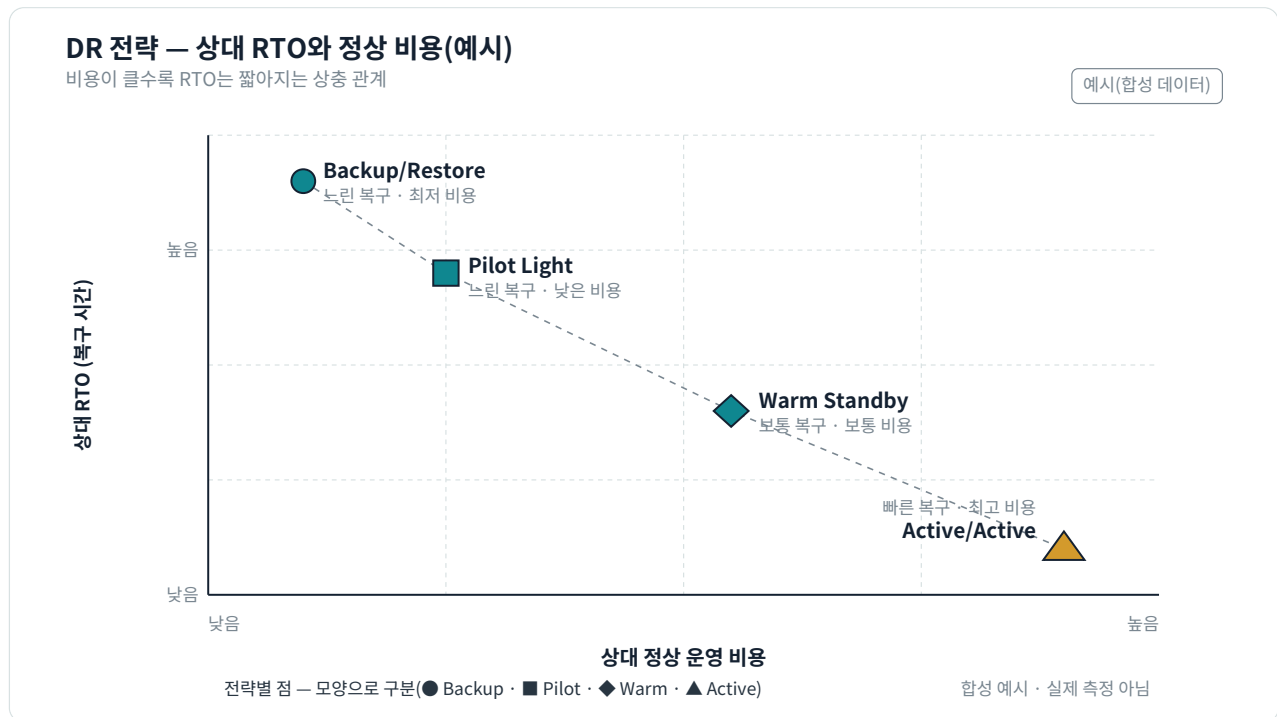
- RTO는 서비스 복구 시간, RPO는 허용 가능한 데이터 손실 시점이다.
- 복제는 최신 상태를 빠르게 전달하지만 잘못된 삭제와 오염도 복제한다.
- backup은 운영 계정 · region · 자격증명과 독립돼야 한다.
- DR 계획은 트래픽 전환 후 데이터 검증 · 외부 의존성 · failback까지 포함한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	Multi-region · Backup · 재해 복구에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	RTO는 서비스 복구 시간, RPO는 허용 가능한 데이터 손실 시점이다.
규모 · 분포	피크 부하, 데이터량, 지역 · tenant 편차가 설계에 어떤 영향을 주는가?	DR 용량은 정상 평균이 아니라 장애 시 합류 트래픽과 복구 작업을 합쳐 계산한다.
실패 · 복구	“Backup unusable” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	주기적 격리 restore와 application validation을 실행한다.
보안 · 통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	backup vault의 삭제 · retention 변경 권한을 운영 admin과 분리한다.

차원	확인 질문	설계 판단
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	backup age · success · immutability · restore test

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



DR 전략의 RTO·비용 비교

핵심 개념

RTO

중단 후 허용 가능한 서비스 복구 시간 목표다.

RPO

복구 시 허용 가능한 데이터 손실 시점 목표다.

Backup

운영 상태와 분리된 복구용 데이터 사본이다.

PITR

log와 base snapshot을 이용해 특정 시점으로 복구하는 방식이다.

Active/Standby

한 region이 주 처리하고 다른 region이 대기한다.

Active/Active

둘 이상의 region이 동시에 사용자 요청을 처리한다.

Failback

비상 region에서 정상 배치로 돌아가며 데이터·트래픽을 다시 정렬하는 과정이다.

Recovery dependency

DNS, IdP, KMS, CI, 연락망처럼 복구에 필요한 외부·제어 구성 요소다.

핵심 개념의 정의와 범위는 [@nist-contingency; @google-sre-book]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
주 region	정상 쓰기와 사용자 트래픽을 처리한다.
대기/보조 region	복제본·용량·구성을 준비한다.
Global traffic manager	건강과 정책에 따라 전환한다.
Backup vault	불변·교차 계정 사본과 catalog를 보존한다.
Recovery orchestrator	restore·config·secret·validation 순서를 실행한다.
Data validator	record count·checksum·업무 불변조건을 확인한다.
DR command	권한 있는 의사결정·커뮤니케이션·audit를 담당한다.

재해 복구 전략 비교

상대적 크기만 비교 (구체적 수치 아님)

●○○=작음 · ●●○=보통 · ●●●=큼

Pilot Light	Warm Standby	Active/Active
비용 ● ○ ○ 낮음	비용 ● ● ○ 보통	비용 ● ● ● 높음
RTO ● ● (중(느림))	RTO ● ● ○ 보통	RTO ● ○ 작음(빠름)
RPO ● ○ ○ 보통	RPO ● ○ ○ 작음	RPO ● ○ 거의 없음
복잡도 ● ○ ○ 낮음	복잡도 ● ● ○ 보통	복잡도 ● ● ● 높음

왼쪽으로 갈수록 비용·복잡도는 낮고 복구 시간·손실은 커지며, 오른쪽으로 갈수록 비용·복잡도는 높고 복구 시간·손실은 작아진다.

pilot light · warm standby · active-active를 RTO · RPO · 비용·복잡도로 비교한다

요청·데이터 흐름

1. 여정·데이터별 RTO/RPO tier를 정한다.
2. 각 실패 유형에 failover · restore · rebuild 중 수단을 매핑한다.
3. backup을 암호화·불변·교차 계정/region에 보존한다.
4. 복구 환경의 network · identity · secret · quota를 준비한다.
5. 게임데이에서 실제 traffic 없이 restore와 검증을 수행한다.
6. failover 시 쓰기 소유권과 DNS/route를 전환한다.
7. 안정화 후 delta sync · 검증 · 점진 traffic으로 failback한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
Pilot light	핵심 데이터와 최소 인프라만 대기해 비용이 낮다.	scale-up · deploy 때문에 RTO가 길다.	수시간 RTO
Warm standby	축소된 전체 stack이 대기해 복구가 빠르다.	지속 비용과 config drift 관리가 필요하다.	수십 분 RTO

대안	장점	비용·위험	적합한 조건
Multi-site active/active	가장 빠른 지역 장애 우회와 낮은 지연을 제공한다.	일관성·충돌·용량·운영 복잡도가 가장 크다.	매우 짧은 RTO와 지역별 처리

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [nist-contingency; @google-sre-book]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Backup unusable	백업 job은 성공했지만 key·schema·권한이 없어 restore가 실패한다.	주기적 격리 restore와 application validation을 실행한다.
Corruption replication	잘못된 삭제가 모든 active replica로 전파된다.	PITR·불변 backup·deletion delay를 둔다.
Standby drift	대기 region의 image·config·quota가 달라 전환 후 오류가 난다.	IaC drift 검사와 정기 warm-up을 한다.
Traffic before data	DNS를 먼저 전환해 새 region이 읽기/쓰기 준비 전 요청을 받는다.	data readiness gate와 점진 traffic을 사용한다.
Failback loss	두 region의 delta를 정리하지 않고 원래 region으로 돌아가 쓰기가 덮인다.	single writer epoch·reconciliation·read-only 전환 단계를 둔다.

장애 전환 절차 순서

선언에서 복구까지 정해진 순서를 따른다



쓰기를 먼저 막고 데이터를 복구·검증한 뒤 트래픽을 전환하며, 안정되면 원래 체계로 Failback한다.

장애 선언·쓰기 차단·restore/승격·검증·traffic 전환·failback 순서를 보여준다

확장 전략

- DR 용량은 정상 평균이 아니라 장애 시 합류 트래픽과 복구 작업을 합쳐 계산한다.
- backup restore throughput이 데이터 증가를 따라가는지 정기 측정한다.
- tier별 서비스는 핵심 여정부부터 복구하고 비핵심 batch·analytics를 뒤로 미룬다.
- multi-region을 서비스 전체가 아니라 필요한 데이터·기능에 선택적으로 적용한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- backup vault의 삭제·retention 변경 권한을 운영 admin과 분리한다.
- ransomware·credential compromise 시나리오에서 독립 계정과 offline recovery credential을 검증한다.
- DR 중 개인정보 지역 이전과 규제 통보 절차를 준수한다.
- restore 데이터의 접근과 폐기를 감사한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- backup age·success·immutability·restore test
- replication lag·RPO exposure·PITR window
- region readiness·config drift·quota
- failover 단계별 시간과 traffic 오류
- data validation mismatch·failback backlog

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 다중 region은 compute·storage·replication egress·운영 인력 비용을 크게 늘린다.
- backup 보존 기간과 restore 속도는 storage tier·index·catalog 비용을 교환한다.
- 모든 기능에 같은 RTO를 주지 않고 업무 tiering으로 투자한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- replica가 있으니 backup이 필요 없다고 생각한다.
- RTO/RPO를 모든 데이터에 한 숫자로 적는다.
- DNS 전환만 DR 완료로 본다.
- restore 성공 여부를 파일 존재로만 판단하고 애플리케이션 검증을 하지 않는다.

설계 리뷰

- ☐ 실패 유형별 복구 수단이 구분됐는가?
- ☐ RTO/RPO가 실제 restore/failover 결과로 입증되는가?
- ☐ backup이 운영 권한·region·자격증명과 독립적인가?
- ☐ 전환 후 데이터 쓰기 권한과 외부 의존성이 준비됐는가?
- ☐ failback·reconciliation·커뮤니케이션이 계획에 포함됐는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 주문 DB의 5분 RPO, 검색 index의 24시간 RPO를 각각 복구 설계하라.
2. 잘못된 DELETE가 20분 뒤 발견됐을 때 PITR과 신규 쓰기 보존 절차를 작성하라.
3. warm standby를 분기마다 시험할 게임데이 체크리스트를 만들라.

핵심 요약

- 복제·backup·DR은 서로 다른 실패를 담당한다.
- RTO/RPO는 사용자 여정과 데이터별로 정한다.
- 복구에는 identity·KMS·DNS·quota 같은 의존성이 필요하다.
- restore는 업무 불변조건으로 검증한다.
- failover 후 failback까지 하나의 절차다.

출처

- [@nist-contingency] NIST. **NIST SP 800-34 Rev. 1 — Contingency Planning Guide for Federal Information Systems** (2010). <https://csrc.nist.gov/pubs/sp/800/34/r1/final>
- [@google-sre-book] Google. **Site Reliability Engineering** (2016). <https://sre.google/sre-book/table-of-contents/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

30

Container · Kubernetes · Serverless · IaC · GitOps · FinOps

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

클라우드 네이티브는 도구 목록이 아니라 선언된 상태, 자동 복구, immutable artifact, 표준 운영 경로, 측정 가능한 비용을 결합하는 운영 모델이다. Kubernetes나 serverless를 채택해도 애플리케이션 상태·SLO·보안·비용 책임은 사라지지 않는다.

이 절의 기준 출처: [@kubernetes-concepts; @opengitops].

학습 목표

- workload 실행 모델을 운영 요구로 선택한다.
- 선언적 인프라·GitOps·platform 경계를 설계한다.
- 비용을 기술 지표와 사업 가치에 연결한다.

먼저 결론

- container는 packaging 경계이고 Kubernetes는 workload orchestration platform이다.
- serverless는 인프라 관리 일부를 공급자에 맡기지만 실행 제한·cold start·event semantics를 검토한다.
- IaC와 GitOps는 desired state와 변경 이력을 코드로 관리하되 secret와 runtime emergency를 별도 설계한다.
- FinOps는 cost allocation만이 아니라 기술 사용과 사업 가치를 함께 최적화하는 협업 체계다.

∴ current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06 이며, 출판 직전 공식 문서를 다시 확인한다. ∴

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	Container · Kubernetes · Serverless · IaC · GitOps · FinOps에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	container는 packaging 경계이고 Kubernetes는 workload orchestration platform이다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	cluster 수와 tenant 격리는 security · blast radius · 운영 overhead를 함께 평가한다.

차원	확인 질문	설계 판단
실패·복구	“Config drift” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	reconciliation · drift detection · break-glass 기록을 둔다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	base image · dependency · artifact provenance를 검증한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	deployment lead time · change failure · rollback

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Container image

애플리케이션과 runtime dependency를 immutable artifact로 묶는다.

Kubernetes control loop

desired state와 actual state 차이를 반복 조정한다.

Serverless

요청·event에 따라 공급자가 실행 환경과 scale을 관리하는 모델이다.

IaC

인프라 resource와 policy를 선언형 코드·state로 관리한다.

GitOps

version-controlled desired state와 자동 reconciliation을 운영 원칙으로 사용한다.

Platform engineering

개발자가 안전한 표준 경로로 build·deploy·observe할 수 있는 내부 제품을 만든다.

FinOps

engineering·finance·business가 기술 사용과 비용·가치를 함께 관리하는 운영 방식이다.

Unit economics

요청·고객·transaction·token 같은 단위당 비용과 가치다.

핵심 개념의 정의와 범위는 [@kubernetes-concepts; @opengitops; @finops-framework; @cncf-platforms; @slsa12]를 기준으로 재검토해야 한다.

기준 아키텍처

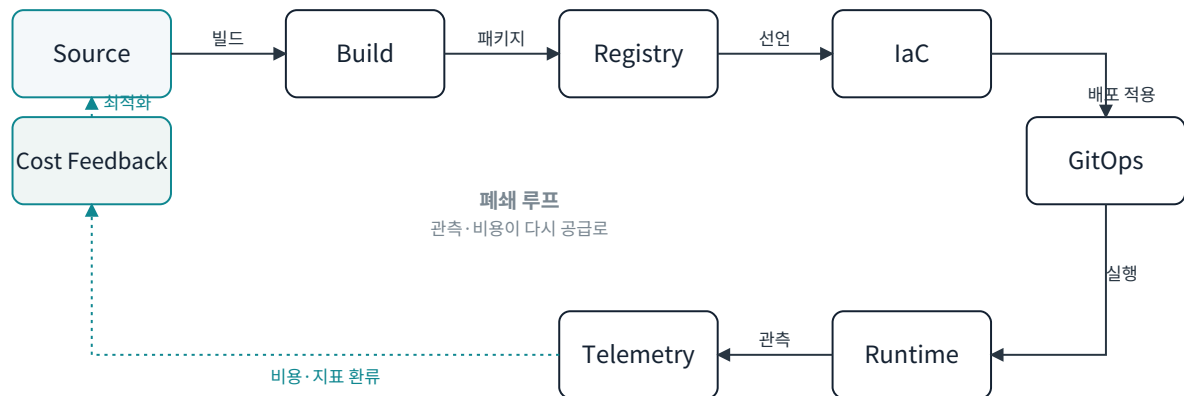
아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Source/build	재현 가능한 artifact와 provenance를 만든다.
Artifact registry	서명된 image · package를 보존한다.
IaC control	network · cluster · database · policy desired state를 관리한다.
GitOps reconciler	환경 repo 상태를 runtime에 적용한다.
Kubernetes/serverless runtime	workload scheduling · scale · health를 수행한다.
Internal platform	template · policy · observability · self-service를 제공한다.
Cost pipeline	resource usage를 owner · service · unit metric에 연결한다.
Governance	quota · policy · exception · lifecycle을 관리한다.

플랫폼 폐쇄 루프

공급 · 배포 · 실행 · 관측 · 비용이 한 루프로 이어진다

—— 정방향 흐름 비용 환류



코드에서 빌드 · 배포 · 실행 · 관측을 거쳐 비용과 지표가 다시 Source로 돌아가, 다음 변경의 판단 근거가 된다.

source · build · registry · IaC/GitOps · runtime · telemetry · cost feedback의 폐쇄 루프를 보여준다

요청 · 데이터 흐름

1. source change가 test와 reproducible build를 통과한다.
2. artifact가 서명 · provenance와 함께 registry에 저장된다.
3. 환경 변경이 IaC/Git pull request로 검토된다.
4. reconciler가 canary · policy를 거쳐 desired state를 적용한다.
5. runtime이 health · autoscaling · restart를 수행한다.
6. telemetry와 cost allocation이 service · team · unit에 연결된다.
7. platform 팀이 adoption · lead time · reliability · cost 결과를 제품 지표로 본다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

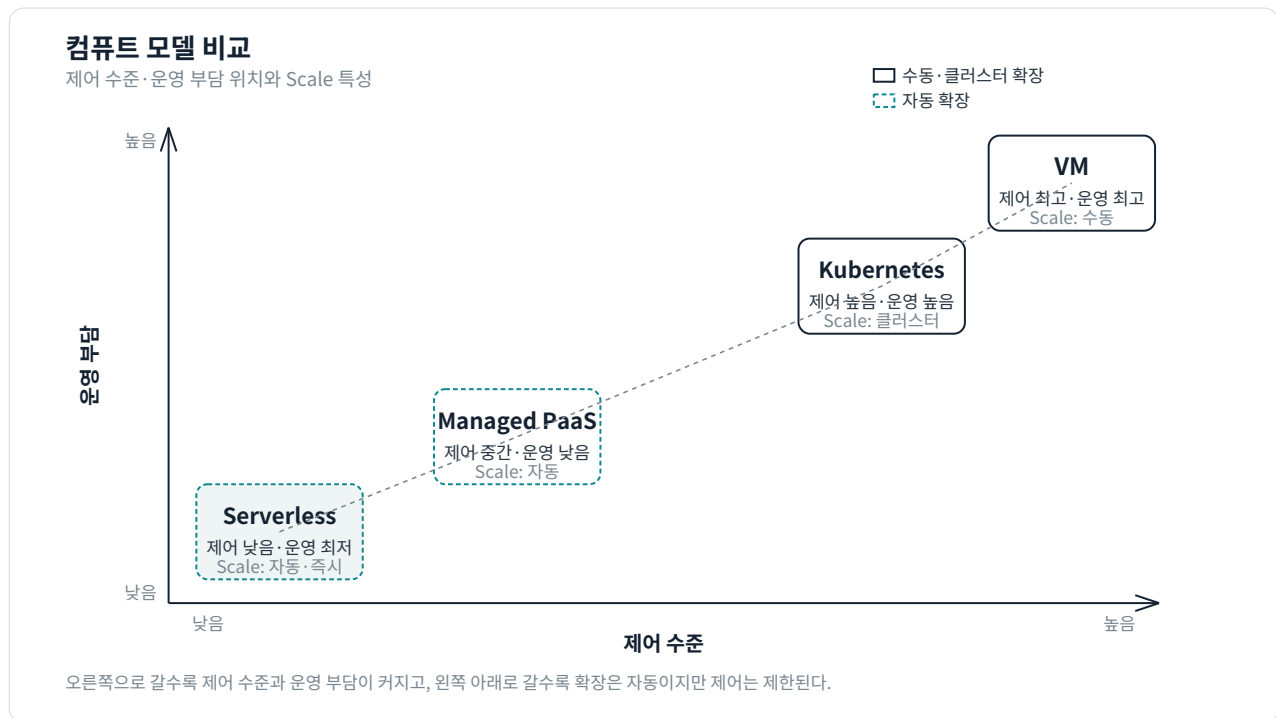
대안	장점	비용 · 위험	적합한 조건
VM/managed runtime	격리와 기존 운영 도구가 익숙하다.	scale · patch · packing 효율이 제한될 수 있다.	stateful · legacy · 특수 OS
Kubernetes	다양한 workload와 확장 가능한 platform API를 제공한다.	cluster · network · policy · upgrade 복잡도가 크다.	다팀 · 다서비스 platform

대안	장점	비용·위험	적합한 조건
Serverless	운영 부담과 scale-to-zero가 좋다.	실행 제한·lock-in·관측·비용 예측이 필요하다.	event-driven · 간헐 workload
Managed PaaS	운영 단순성과 표준 배포를 제공한다.	customization과 탈출 경로가 제한될 수 있다.	일반 web/API

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [@kubernetes-concepts; @opengitops; @finops-framework]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Config drift	console 긴급 변경이 IaC/Git desired state와 달라진다.	reconciliation · drift detection · break-glass 기록을 둔다.
Autoscaling lag	traffic burst가 pod/function 준비보다 빨라 queue와 timeout이 증가한다.	pre-warm, queue, predictive floor, admission을 사용한다.
Control-plane blast	잘못된 policy/template이 모든 workload 배포를 막는다.	canary scope, policy audit mode, rollback, last-known state를 둔다.
Cost runaway	high-cardinality log · egress · idle cluster · unbounded function이 비용을 폭증시킨다.	budget alert, quota, unit cost, anomaly detection을 사용한다.
Platform bypass	golden path가 느려 팀이 unmanaged resource를 만든다.	개발자 경험·escape hatch · feedback으로 platform을 제품처럼 개선한다.



VM·Kubernetes·serverless·managed PaaS를 제어 수준·운영 부담·scale 특성으로 비교한다

확장 전략

- cluster 수와 tenant 격리는 security·blast radius·운영 overhead를 함께 평가한다.
- workload requests/limits와 autoscaling metric을 실제 profile로 조정한다.
- serverless는 concurrency·event source·downstream capacity limit를 함께 설정한다.
- platform API는 공통 80%를 표준화하고 특수 20%는 승인된 extension으로 지원한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- base image·dependency·artifact provenance를 검증한다.
- cluster/service account·namespace·network policy·secret 권한을 최소화한다.
- IaC state와 plan output에 secret가 노출되지 않게 한다.
- GitOps repo write 권한과 runtime deploy 권한을 분리한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- deployment lead time · change failure · rollback
- pod/function cold start · scale lag · throttle
- reconciliation drift · failed apply · policy deny
- platform adoption · golden path completion
- service/team/unit cost · idle · egress · anomaly
- resource request 대비 실제 사용률

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace · log · 변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- Kubernetes는 고정 control plane · 노드·운영 인력 비용이 있어 작은 workload에 과할 수 있다.
- serverless는 낮은 유휴 비용과 높은 단위 실행 비용 사이의 선택이다.
- FinOps는 단순 절감보다 unit economics · SLO · 속도를 함께 본다.
- platform 투자는 서비스마다 반복되는 toil과 사고를 얼마나 줄이는지로 평가한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- container를 쓰면 cloud native라고 생각한다.
- Kubernetes를 조직 문제 해결 도구로 먼저 도입한다.
- Git이 source of truth면 runtime 긴급 변경과 secret가 자동으로 안전하다고 믿는다.
- 비용을 월 합계만 보고 서비스 단위와 사용자 가치에 연결하지 않는다.

설계 리뷰

- ☐ workload 요구에 맞는 가장 단순한 runtime을 선택했는가?
- ☐ desired state · artifact · policy · secret 소유권이 분리됐는가?
- ☐ autoscaling이 downstream capacity와 함께 검증됐는가?
- ☐

platform 성공을 개발자·신뢰성·비용 지표로 측정하는가?

□

unit cost와 SLO를 같은 의사결정에 사용하는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 간헐적 이미지 처리 작업을 VM, Kubernetes Job, serverless로 비교하라.
2. GitOps 긴급 변경 후 desired state와 reconcile하는 break-glass 절차를 설계하라.
3. API 요청당 비용을 compute · DB · cache · egress · observability로 분해하라.

핵심 요약

- 클라우드 네이티브는 선언·자동화·복구·관측의 운영 모델이다.
- runtime 선택은 workload와 팀 역량에 맞춘다.
- IaC와 GitOps에도 drift · secret · 긴급 변경 정책이 필요하다.
- platform은 개발자를 위한 내부 제품이다.
- FinOps는 비용을 기술 사용과 사업 가치에 연결한다.

출처

- [@kubernetes-concepts] Kubernetes Authors. **Kubernetes Concepts** (2026). <https://kubernetes.io/docs/concepts/>
- [@opengitops] OpenGitOps. **OpenGitOps Principles** (2026). <https://opengitops.dev/>
- [@finops-framework] FinOps Foundation. **FinOps Framework** (2026). <https://www.finops.org/framework/>
- [@cncf-platforms] Cloud Native Computing Foundation. **Platforms White Paper** (2025). <https://tag-app-delivery.cncf.io/whitepapers/platforms/>
- [@slsa12] OpenSSF. **SLSA Specification v1.2** (2025). <https://slsa.dev/spec/v1.2/>

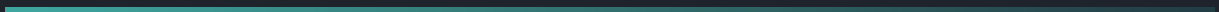
“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

PART

6

AI 네이티브 시스템



검색, 추론, 에이전트, 평가를 기존 분산 시스템 원칙 위에서 설계한다.

이 Part에서는 특정 제품의 설정법보다 반복해서 적용할 수 있는 설계 질문과 실패 모델을 먼저 익힌다. 각 장의 체크리스트를 실제 시스템의 ADR·런북·대시보드와 연결해 읽는 것이 목표다.

포함 장

ID	장 제목	최신성	원본 관계
ch31	RAG 데이터 파이프라인과 Retrieval 품질	current	ADD
ch32	LLM Inference·Batching·KV Cache·Model Routing	volatile	ADD
ch33	Agent 상태·메모리·도구 실행·승인 경계	volatile	ADD
ch34	AI 평가·관측 가능성·보안·비용	volatile	ADD

읽는 방법

1. 먼저 각 장의 **요구사항과 실패 모델**을 자신의 시스템에 대입한다.
2. **대안과 트레이드오프**에서 현재 선택이 어떤 비용을 감수하는지 확인한다.
3. **장애 시나리오**를 게임데이 또는 설계 리뷰 질문으로 바꾼다.
4. 시각자료는 설명을 대신하지 않고 책임·흐름·복구 경계를 검증하는 용도로 사용한다.

31

RAG 데이터 파이프라인과 Retrieval 품질

이 장의 구성

- 01 이 장에서 해결할 문제
- 02 먼저 결론
- 03 요구사항과 실패 모델
- 04 핵심 개념
- 05 기존 아키텍처
- 06 요청-데이터 흐름
- 07 대안과 트레이드오프
- 08 장애 시나리오
- 09 확장 전략
- 10 보안과 개인정보
- 11 관측 가능성
- 12 비용과 운영 복잡도
- 13 흔한 오해와 안티패턴
- 14 설계 리뷰
- 15 연습문제
- 16 핵심 요약
- 17 출처

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

RAG 품질 문제를 모델 prompt 하나로 해결할 수는 없다. 답변 품질은 원문 수집, parsing, chunk 경계, metadata, embedding, candidate retrieval, reranking, context 구성, 생성, citation 검증의 연쇄 결과다. 각 단계를 독립적으로 평가해야 한다.

이 절의 기준 출처: [@rag-paper; @dpr-paper].

학습 목표

- RAG를 수집·정제·검색·생성·근거 검증 파이프라인으로 분해한다.
- chunk·embedding·index·reranking의 version과 품질을 관리한다.
- tenant 권한·freshness·provenance를 retrieval 경로에 적용한다.

먼저 결론

- 원문과 metadata를 진실의 원천으로 보존하고 vector/text index는 재구축 가능한 파생 상태로 둔다.
- retrieval 품질은 answer quality와 분리해 recall·ranking·coverage를 먼저 측정한다.
- chunk와 embedding model을 versioning하고 dual-index로 안전하게 전환한다.
- tenant·ACL filter는 후보 검색 전후에 적용하고 citation이 실제 원문 범위와 일치하는지 검증한다.

::: current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06 이며, 출판 직전 공식 문서를 다시 확인한다. :::

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	RAG 데이터 파이프라인과 Retrieval 품질에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	원문과 metadata를 진실의 원천으로 보존하고 vector/text index는 재구축 가능한 파생 상태로 둔다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	tenant·source·language별 index 분리와 shared index filter 비용을 비교한다.

차원	확인 질문	설계 판단
실패·복구	“Parser corruption” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	문서 형식별 품질 검사와 원문 span preview를 둔다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	connector credential은 source별 최소 read 권한과 짧은 수명을 사용한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	ingestion freshness·parser failure·chunk count 변화

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

RAG

외부 지식을 검색해 생성 모델의 입력 context에 결합하는 구조다.

Chunking

문서를 검색·context 단위로 나누는 과정이며 구조·중첩·overlap이 품질과 비용에 영향을 준다.

Embedding

query와 문서 조각을 vector 공간에 표현한다.

Candidate retrieval

lexical·dense·hybrid 방식으로 상위 후보를 빠르게 찾는다.

Reranking

더 비싼 모델이나 규칙으로 후보 순서를 다시 평가한다.

Grounding

답변이 제공된 근거에 기반하도록 만드는 설계와 검증이다.

Provenance

원문·version·위치·수집 시각·변환 이력을 추적하는 metadata다.

Freshness

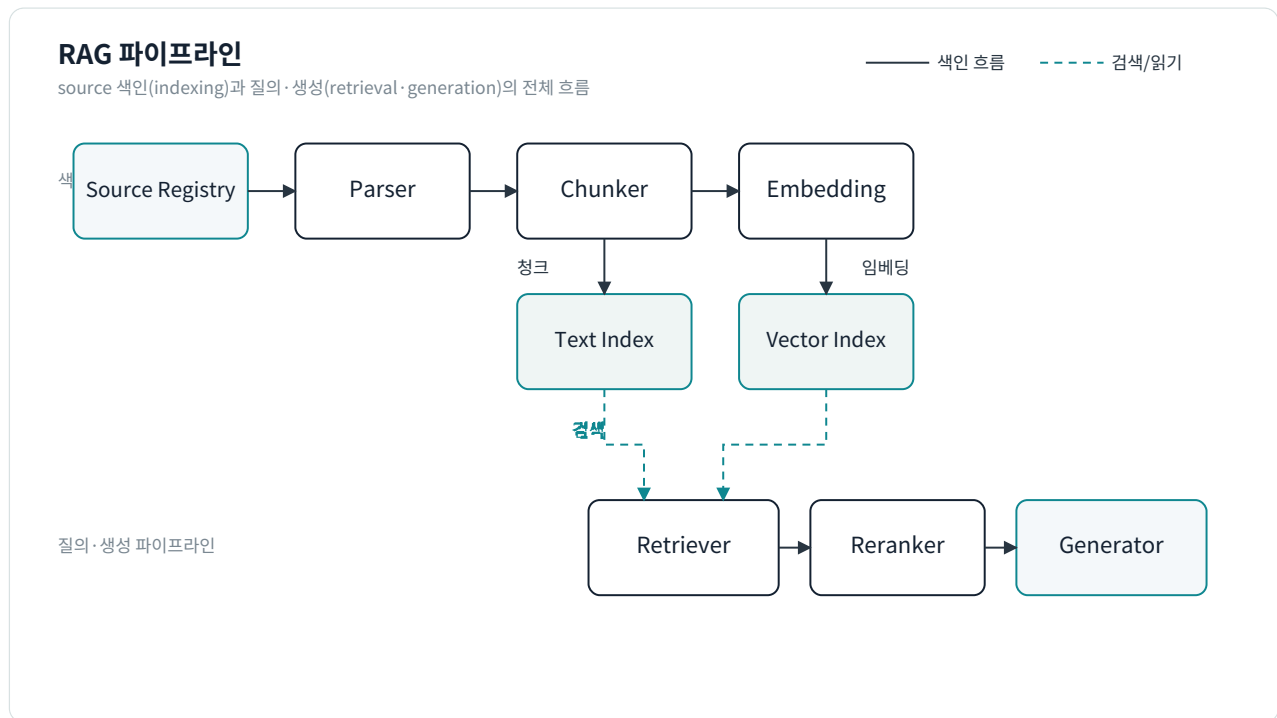
원문 변경이 검색·답변에 반영되기까지의 시간이다.

핵심 개념의 정의와 범위는 [rag-paper; @dpr-paper; @beir-paper; @hnsf-paper; @owasp-llm]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Source registry	문서 소유자·version·권한·수집 정책을 원장으로 관리한다.
Ingestion workers	fetch·parse·normalize·malware scan을 수행한다.
Chunk store	원문 위치와 version을 가진 chunk를 보존한다.
Text/vector indexes	lexical·semantic candidate를 제공한다.
Retriever	filter·query rewrite·hybrid fusion을 수행한다.
Reranker	상위 후보의 relevance를 정밀 평가한다.
Context builder	token budget·중복·provenance를 고려해 context를 구성한다.
Answer service	생성과 citation·policy 검증을 수행한다.
Evaluation store	golden query·judgment·online feedback을 보존한다.



source 수집에서 parse·chunk·embedding·index와 query retrieval·rerank·generation까지 전체 파이프라인을 보여준다

요청·데이터 흐름

1. source connector가 권한과 change cursor를 확인해 문서를 수집한다.
2. parser가 격리 환경에서 구조와 원문 span을 추출한다.
3. chunker가 문서 구조·token 목표·overlap 정책으로 조각을 만든다.
4. embedding과 lexical analyzer version을 붙여 dual index에 저장한다.
5. query에서 tenant·권한·언어·의도를 추출한다.
6. hybrid retrieval과 metadata filter로 후보를 얻는다.
7. reranker가 evidence relevance를 평가한다.
8. context builder가 중복과 token budget을 조정한다.
9. answer가 citation span과 policy를 검증한 뒤 반환된다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
Lexical retrieval	정확한 용어·코드·이름에 강하고 설명이 쉽다.	표현이 다른 의미 검색에 약하다.	정책 번호·제품명·정확 문구
Dense retrieval	의미 유사성에 강하다.	모델 domain·filter·근사 index에 따라 누락될 수 있다.	자연어 질문·동의 표현
Hybrid+rerank	정확어와 의미를 결합해 품질이 높을 수 있다.	자연·비용·평가·운영 경로가 복잡하다.	고품질 엔터프라이즈 RAG

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [@rag-paper; @dpr-paper; @beir-paper]를 참조한다.

장애 시나리오

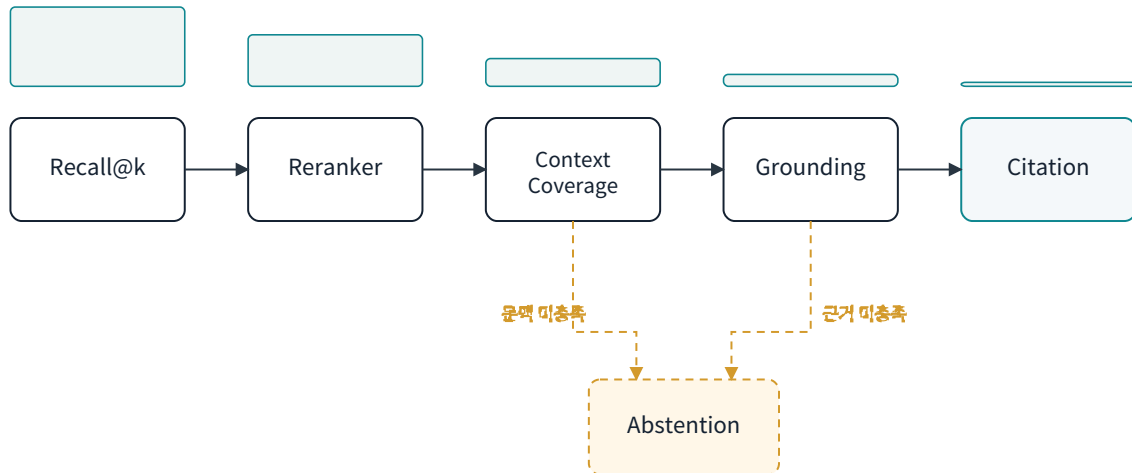
시나리오	영향	대응 원칙
Parser corruption	표·머리글·OCR 순서가 깨져 의미 없는 chunk가 생성된다.	문서 형식별 품질 검사와 원문 span preview를 둔다.
ACL leak	공유 index에서 filter가 누락돼 다른 tenant 문서가 후보에 들어온다.	pre-filter·post-filter·answer citation 검증을 중복 적용한다.
Stale index	원문 변경/삭제가 index에 늦게 반영된다.	change cursor·deletion ledger·freshness SLO를 둔다.
Embedding mismatch	query는 새 model, 문서는 이전 model vector를 사용한다.	model version별 index와 routing을 분리한다.
Context poisoning	문서 안의 명령문이 system instruction 처럼 처리된다.	retrieved content를 untrusted data로 구분하고 tool/policy 권한과 분리한다.
Citation drift	답변 문장이 실제 citation span에서 지지 되지 않는다.	claim-evidence 검사와 source snippet 확인을 적용한다.

RAG 품질 퍼널

후보 recall에서 인용까지 단계별 정제 과정과 보류(abstention) 경로

—— 정상 정제 - - - - - 근거 부족 시

후보 수(개념적)



candidate recall · rerank precision · context coverage · grounded answer로 이어지는 품질 funnel을 보여준다

확장 전략

- tenant · source · language별 index 분리와 shared index filter 비용을 비교한다.
- batch embedding은 throughput을 높이되 freshness tier에 따라 priority를 둔다.
- candidate 수 · rerank 수 · context token을 단계별 budget으로 제한한다.
- reindex는 shadow query로 quality · latency · cost를 비교한 뒤 alias를 전환한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- connector credential은 source별 최소 read 권한과 짧은 수명을 사용한다.
- 문서 parser · archive extraction을 sandbox에 격리한다.
- ACL과 tenant filter를 retrieval, rerank, answer 단계에서 검증한다.
- prompt injection을 이유로 retrieved text에 tool 실행 권한을 부여하지 않는다.
- 삭제 · 보존 요청을 원문 · chunk · embedding · cache · evaluation sample에 전파한다.

보안 요구는 별도 부록이 아니라 요청 · 데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존 · 삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- ingestion freshness · parser failure · chunk count 변화
- retrieval Recall@k · nDCG/MRR · zero-result
- candidate/rerank/context 단계별 latency와 token
- ACL filter selectivity · denied candidate · leak test
- citation coverage · unsupported claim · answer abstention
- model/index version별 quality · cost

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- embedding · reindex · vector memory · reranker inference · context token이 주요 비용 축이다.
- 문서를 무조건 작은 chunk로 나누면 index와 retrieval 후보·token 비용이 늘어난다.
- 고품질 reranking은 모든 query가 아니라 risk·uncertainty·value tier에 선택 적용할 수 있다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- vector DB를 추가하면 RAG가 완성된다고 생각한다.
- 답변 평가만 하고 retrieval 누락 원인을 측정하지 않는다.
- 모든 문서를 같은 chunk 크기와 embedding model로 처리한다.
- citation URL만 붙이면 grounding이 검증됐다고 본다.

설계 리뷰

- ☐ 원문·chunk·index·model version의 계보가 추적되는가?
- ☐ retrieval 품질과 generation 품질이 분리 평가되는가?
- ☐ ACL·tenant·삭제가 모든 단계에서 강제되는가?
- ☐ reindex를 dual-run·shadow·rollback할 수 있는가?
- ☐

citation이 실제 claim을 지지하는지 검증하는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 사내 규정 PDF의 표·부록·개정 이력을 보존하는 chunk 전략을 설계하라.
2. Recall@20은 높지만 answer가 틀린 RAG를 retrieval·rerank·generation 단계로 진단하라.
3. embedding model 교체의 dual-index와 quality gate를 작성하라.

핵심 요약

- RAG는 end-to-end 데이터 파이프라인이다.
- 원문은 원장이고 index는 파생 상태다.
- retrieval과 generation을 분리 평가한다.
- version·freshness·provenance를 모든 chunk에 붙인다.
- ACL과 citation 검증은 품질이 아니라 보안 경계이기도 하다.

출처

- [@rag-paper] Patrick Lewis et al.. **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks** (2020). <https://arxiv.org/abs/2005.11401>
- [@dpr-paper] Vladimir Karpukhin et al.. **Dense Passage Retrieval for Open-Domain Question Answering** (2020). <https://arxiv.org/abs/2004.04906>
- [@beir-paper] Nandan Thakur et al.. **BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models** (2021). <https://arxiv.org/abs/2104.08663>
- [@hnsb-paper] Yu. A. Malkov and D. A. Yashunin. **Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs** (2018). <https://arxiv.org/abs/1603.09320>
- [@owasp-llm] OWASP Foundation. **OWASP Top 10 for LLM Applications** (2025). <https://genai.owasp.org/llm-top-10/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

32

LLM Inference · Batching · KV Cache · Model Routing

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

LLM 서버의 병목은 단순히 GPU 연산량 하나가 아니다. 입력 token을 처리하는 prefill과 token을 순차 생성하는 decode는 자원 특성이 다르고, 각 요청의 KV cache가 동적으로 메모리를 점유한다. scheduler와 routing이 품질·지연·처리량·비용을 함께 결정한다.

이 절의 기준 출처: [vllm-paper; @orca-paper].

학습 목표

- LLM 추론을 prefill · decode · KV cache · scheduler로 설명한다.
- continuous batching과 memory pressure의 trade-off를 이해한다.
- model routing · fallback · capacity를 품질·지연·비용으로 설계한다.

먼저 결론

- time-to-first-token과 inter-token latency를 전체 latency에서 분리한다.
- batch 크기를 고정하지 않고 도착·sequence 길이·deadline에 따라 연속적으로 관리한다.
- KV cache 부족은 admission · eviction · preemption · offload 정책을 요구한다.
- model routing은 “가장 싼 모델”이 아니라 품질 threshold · risk · latency · capacity를 만족하는 최소 비용 경로를 선택한다.

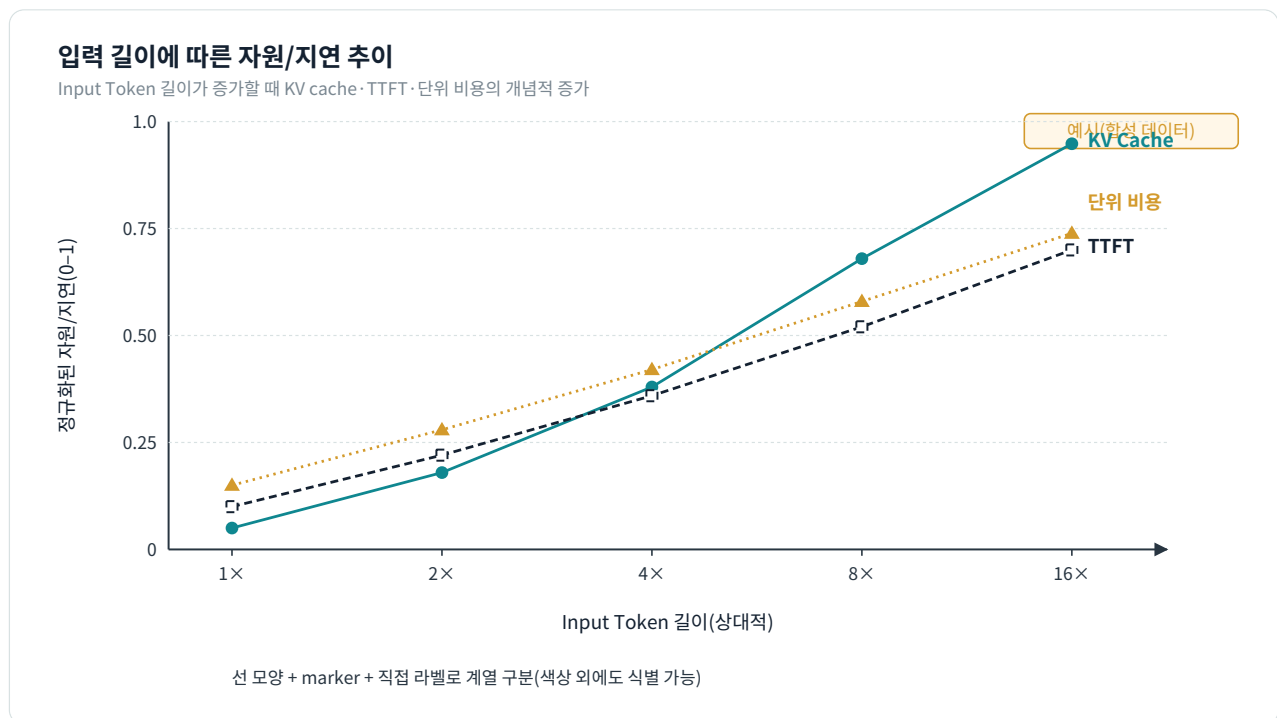
∴ current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2026-11-06 이며, 출판 직전 공식 문서를 다시 확인한다. ∴

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	LLM Inference · Batching · KV Cache · Model Routing에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	time-to-first-token과 inter-token latency를 전체 latency에서 분리한다.

차원	확인 질문	설계 판단
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	request count보다 input/output token과 sequence length 분포로 capacity를 계획한다.
실패·복구	“KV cache OOM” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	token admission, paged allocation, preemption, max length를 둔다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	prompt·output·KV·cache에 tenant 민감 데이터가 남는 수명과 격리를 명시한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	TTFT·ITL/TPOT·end-to-end p95/p99

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



Sequence 길이와 KV Cache·TTFT

핵심 개념

Prefill

입력 token 전체를 처리해 첫 KV 상태와 첫 출력 준비를 만드는 단계다.

Decode

기존 KV cache를 재사용하며 token을 하나씩 생성하는 단계다.

KV cache

attention의 과거 key/value를 요청·layer별로 보존해 반복 계산을 줄이는 메모리다.

Continuous batching

완료된 요청을 batch에서 빼고 새 요청을 즉시 넣으며 decode iteration을 공유하는 scheduling이다.

TTFT

요청부터 첫 token까지의 시간이다.

ITL/TPOT

출력 token 사이 지연 또는 token당 시간이다.

Model router

요청 특성·품질·비용·capacity에 따라 model/endpoint를 선택한다.

Speculative decoding

작은 모델이 제안한 token을 큰 모델이 검증해 decode 속도를 높이는 계열의 방법이다.

Admission control

예상 token·memory·deadline으로 요청을 수락·queue·거부하는 정책이다.

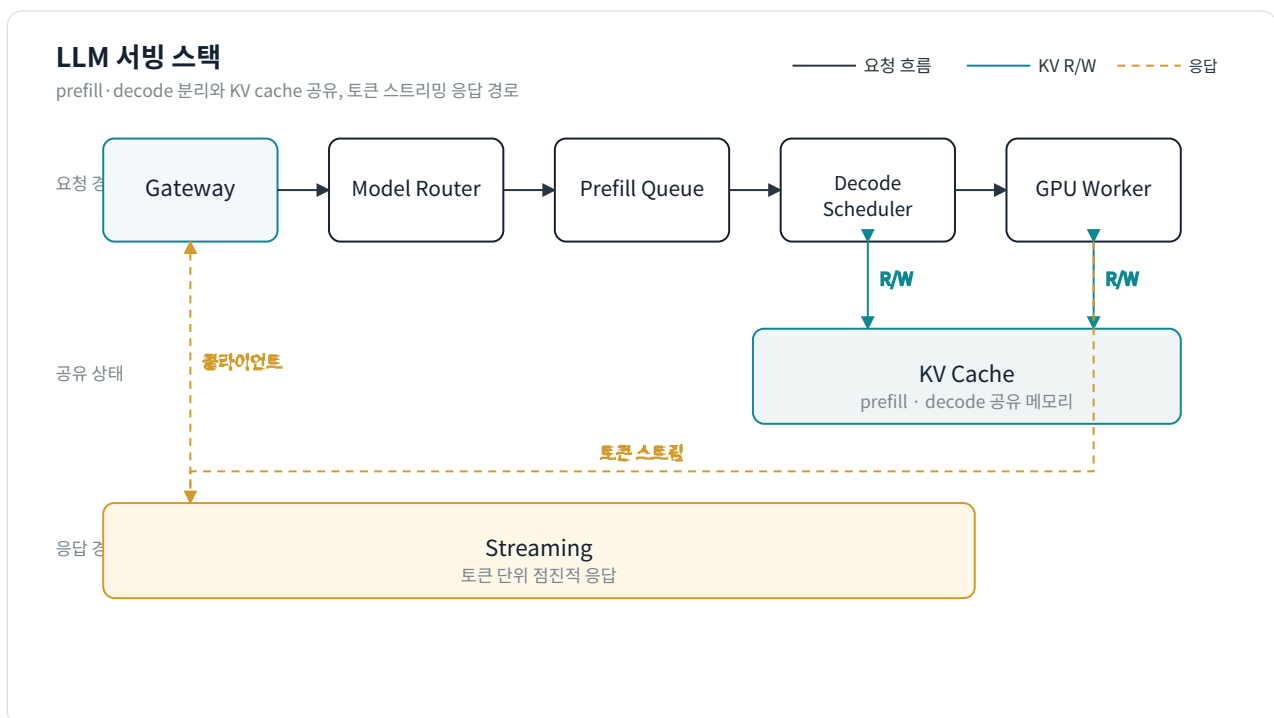
핵심 개념의 정의와 범위는 [vllm-paper; @orca-paper; @nist-genai-profile]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
API gateway	auth · quota · request size · streaming 계약을 처리한다.
Prompt/context builder	token budget과 policy를 적용한다.

구성 요소	책임
Model router	task · risk · language · capacity로 route를 선택한다.
Scheduler	prefill/decode queue와 continuous batch를 관리한다.
GPU workers	model weights와 KV cache로 inference를 수행한다.
KV cache manager	page/block 할당·eviction · prefix reuse를 관리한다.
Fallback pool	다른 model · region · degraded response를 제공한다.
Telemetry/eval	quality · TTFT · ITL · token · cost를 기록한다.



gateway · router · prefill queue · decode scheduler · GPU worker · KV cache · streaming response를 보여준다

요청·데이터 흐름

1. gateway가 auth · quota · max input/output token을 검증한다.
2. router가 task class · quality tier · deadline · capacity를 평가한다.
3. scheduler가 prefill queue와 decode batch에 요청을 배치한다.
4. KV manager가 예상 sequence 길이와 memory block을 할당한다.
5. GPU가 prefill 후 streaming decode를 수행한다.
6. client 취소·deadline 시 decode와 KV를 즉시 회수한다.
7. OOM · overload · quality risk 시 fallback 또는 거부한다.
8. 실제 token · latency · quality 결과를 routing feedback에 사용한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

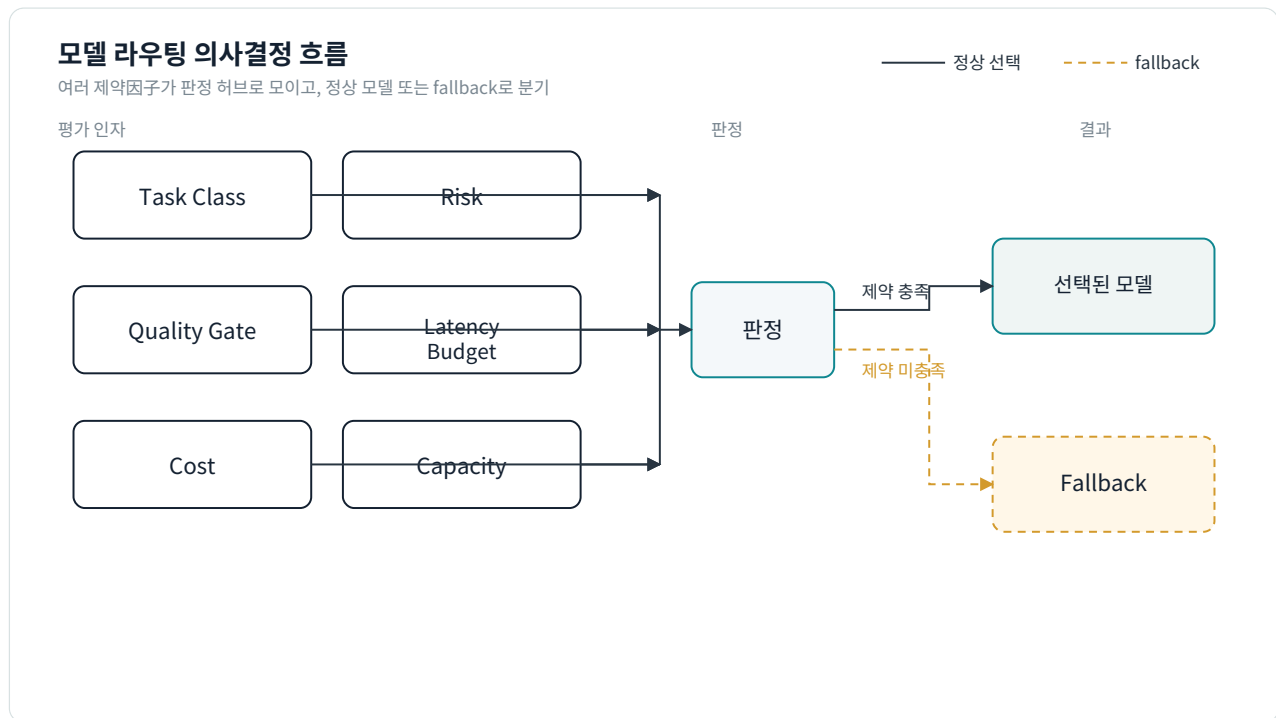
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
단일 대형 model	품질과 운영 경로가 단순하다.	비용·latency·capacity 효율이 낮을 수 있다.	고위험·복잡 task
다중 model routing	단순 task를 저비용 model로 보내 unit cost를 낮춘다.	평가·fallback·일관성·debug가 복잡하다.	다양한 task와 traffic
외부 managed inference	빠른 도입과 운영 부담 감소가 있다.	quota·data policy·egress·vendor dependency가 있다.	변동 workload·초기 제품
자체 GPU serving	세밀한 최적화와 데이터 통제가 가능하다.	capacity planning·driver·scheduler·on-call 비용이 크다.	지속 대규모 traffic·특수 요구

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [vllm-paper; orca-paper; nist-genai-profile]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
KV cache OOM	긴 prompt와 많은 동시 request가 memory를 소진한다.	token admission, paged allocation, preemption, max length를 둔다.
Head-of-line in batch	긴 prefill 하나가 짧은 decode 요청 TTFT/ITL을 악화시킨다.	prefill chunking, separate queue, priority scheduling을 사용한다.
Client disconnect leak	stream 종료 후 decode와 KV가 계속 남는다.	cancellation propagation과 resource reclamation SLO를 둔다.
Router feedback loop	한 endpoint가 느려 우회 트래픽이 다른 endpoint를 포화시키며 계속 진동한다.	hysteresis, capacity reservation, bounded shift를 사용한다.
Quality regression	저비용 model route가 특정 언어·tenant에서 실패한다.	segment별 eval gate, shadow traffic, fallback threshold를 둔다.
Quota cliff	공급자 rate limit이 갑자기 발생해 전체 요청이 재시도된다.	token bucket, queue, multi-provider policy, retry budget을 둔다.



task risk · quality threshold · latency · cost · capacity에 따라 model과 fallback을 선택하는 decision flow를 보여준다

확장 전략

- request count보다 input/output token과 sequence length 분포로 capacity를 계획한다.
- prefill-heavy와 decode-heavy workload를 분리하거나 scheduler weight를 다르게 둔다.
- prefix cache는 반복 system prompt에 유리하지만 tenant/secret 경계를 보존한다.
- autoscaling은 GPU 준비 시간과 model load 시간을 고려해 floor와 queue를 둔다.
- routing은 품질이 검증된 candidate set 안에서만 비용 최적화를 수행한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- prompt · output · KV · cache에 tenant 민감 데이터가 남는 수명과 격리를 명시한다.
- model endpoint와 tool access 권한을 분리하고 route 결과가 보안 정책을 낮추지 않게 한다.
- 외부 inference 전송 데이터와 보존 정책을 검토한다.
- prefix cache key에 tenant · policy · model version을 포함한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- TTFT · ITL/TPOT · end-to-end p95/p99
- input/output token · sequence length · batch occupancy
- GPU utilization · memory · KV block usage · preemption
- queue age · admission reject · client cancel reclaim
- model/route별 quality · fallback · error
- request/token/accepted-answer 단위 비용

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace · log · 변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- GPU cost는 할당 시간과 utilization뿐 아니라 idle floor · model load · replica redundancy로 결정된다.
- 긴 output은 decode 시간과 egress · 사용자 대기를 동시에 늘린다.
- routing은 저비용 호출 비율이 아니라 품질 통과 답변당 비용으로 평가한다.
- managed와 self-hosted 비교에 on-call · capacity risk · upgrade를 포함한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- GPU utilization 하나로 사용자 성능을 판단한다.
- batch를 크게 하면 항상 처리량과 지연이 모두 좋아진다고 생각한다.
- 요청 수만으로 capacity를 계산하고 token 길이를 무시한다.
- router가 평가 없이 가장 싼 model을 선택하게 한다.

설계 리뷰

- ☐ TTFT와 ITL SLO가 분리됐는가?
- ☐ token · KV memory 기반 admission이 있는가?
- ☐ client 취소가 compute와 memory를 회수하는가?
- ☐

route별 quality gate와 fallback이 검증됐는가?

□

unit cost가 request가 아니라 token · 품질 결과와 연결되는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 동시 요청 100개가 각각 input 8K, output 1K일 때 KV cache와 latency 위험을 정성적으로 분석하라.
2. prefill-heavy 문서 요약과 decode-heavy 채팅을 같은 cluster에서 scheduling하는 정책을 설계하라.
3. 고위험 법률 질문은 큰 model, 일반 FAQ는 작은 model로 route하는 평가 gate를 작성하라.

핵심 요약

- LLM inference는 prefill과 decode의 자원 특성이 다르다.
- KV cache가 동시성과 sequence 길이 한계를 결정한다.
- continuous batching은 throughput과 tail을 함께 관리한다.
- model routing은 품질 threshold 안에서 비용을 최적화한다.
- token · memory · 취소를 admission과 관측에 포함한다.

출처

- [vllm-paper] Woosuk Kwon et al.. **Efficient Memory Management for Large Language Model Serving with PagedAttention** (2023). <https://arxiv.org/abs/2309.06180>
- [orca-paper] Gyeong-In Yu et al.. **Orca: A Distributed Serving System for Transformer-Based Generative Models** (2022). <https://www.usenix.org/conference/osdi22/presentation/yu>
- [nist-genai-profile] NIST. **Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile** (2024). <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

33

Agent 상태·메모리·도구 실행·승인 경계

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

에이전트는 “스스로 생각하는 챗봇”이 아니라 불확실한 model 출력이 상태를 읽고 도구를 호출하는 orchestration 시스템이다. 따라서 각 step의 입력·권한·예산·승인·결과를 명시적으로 기록하고, side effect는 일반 분산 transaction처럼 다뤄야 한다.

이 절의 기준 출처: [@react-paper; @toolformer-paper].

학습 목표

- 에이전트를 상태 기계와 외부 도구 실행 시스템으로 설계한다.
- 대화 메모리·업무 상태·장기 지식을 구분한다.
- 위험 도구에 승인·sandbox·idempotency·감사를 적용한다.

먼저 결론

- 모델의 자연어 계획을 직접 권한으로 취급하지 않는다.
- 대화 context, 작업 상태, 사용자 선호, 장기 지식을 서로 다른 저장소와 수명으로 관리한다.
- 읽기 도구와 쓰기 도구, 되돌릴 수 있는 action과 비가역 action을 구분한다.
- 도구 호출은 schema validation·policy·idempotency·approval을 통과한 뒤 실행한다.

∴ current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2026-11-06이며, 출판 직전 공식 문서를 다시 확인한다. ∴

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	Agent 상태·메모리·도구 실행·승인 경계에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	모델의 자연어 계획을 직접 권한으로 취급하지 않는다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	run은 독립 partition으로 scale하되 한 run의 state update는 optimistic concurrency로 직렬화한다.

차원	확인 질문	설계 판단
실패·복구	“Prompt injection” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	data/instruction 경계를 유지하고 tool 권한을 정책이 결정한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	agent에 사용자의 전체 권한을 전달하지 않고 작업별 capability를 발급한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	run success·abstain·manual handoff·step count

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Agent loop

관찰→계획/선택→도구 호출→결과 반영을 제한된 step 안에서 반복하는 실행 구조다.

Run state

목표, 현재 step, 도구 결과, budget, terminal status를 가진 업무 상태다.

Conversation memory

현재 대화의 최근 맥락으로 수명이 짧다.

Long-term memory

사용자 선호·사실·요약 등을 별도 승인과 provenance로 저장한 데이터다.

Tool contract

도구 이름, input/output schema, 권한, idempotency, timeout, side effect를 정의한다.

Approval gate

고위험 action 전에 사용자 또는 정책 승인 증거를 요구하는 단계다.

Sandbox

파일·네트워크·프로세스 접근을 제한한 실행 환경이다.

Compensation

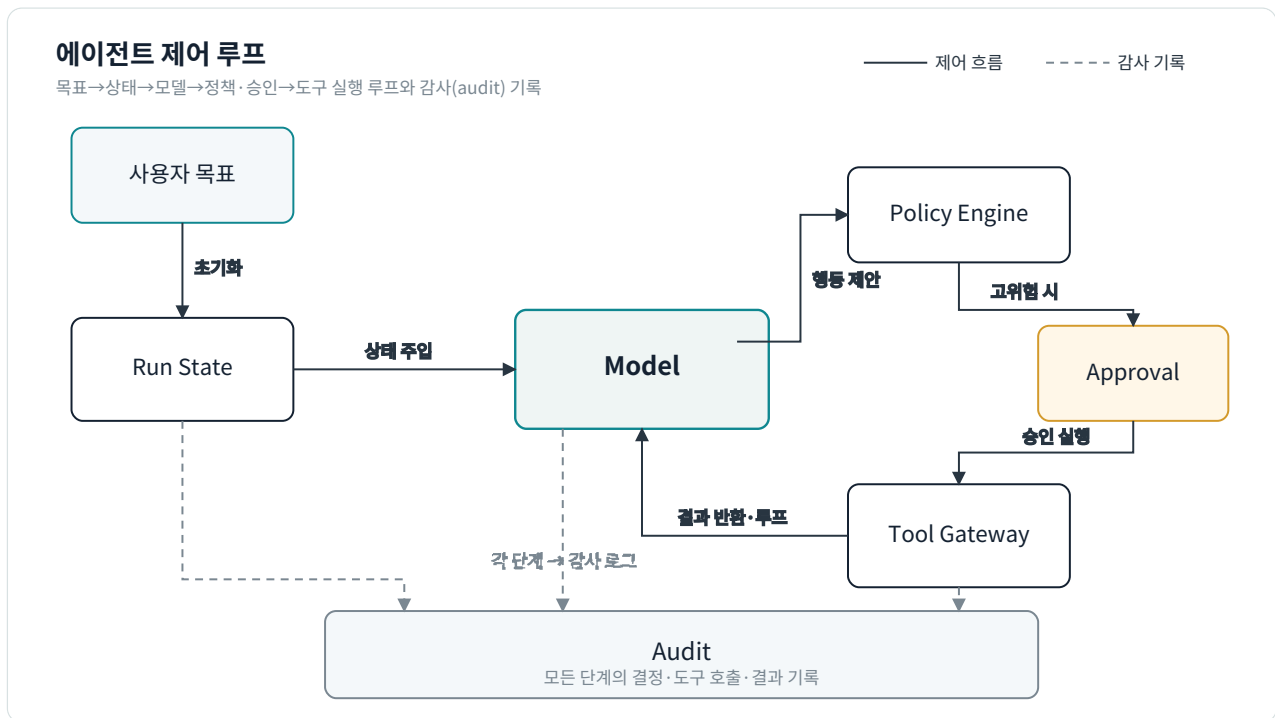
이미 수행된 action을 상쇄하거나 수동 복구하는 후속 작업이다.

핵심 개념의 정의와 범위는 [react-paper; toolformer-paper; nist-ai-rmf; owasp-llm]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Agent API	사용자 목표와 session identity를 받는다.
Run state store	step·version·status·budget을 원장으로 보존한다.
Planner/model	다음 action 후보와 인자를 생성한다.
Policy engine	tool·resource·tenant·risk·approval을 평가한다.
Tool gateway	schema·timeout·idempotency·sandbox를 강제한다.
Approval service	사람 승인과 scope·expiry를 기록한다.
Memory service	수명·provenance·삭제 정책별 memory를 관리한다.
Audit/evaluation	모든 결정·호출·결과·override를 추적한다.



사용자 목표·run state·model·policy·approval·tool gateway·audit의 제어 루프를 보여준다

요청·데이터 흐름

1. 사용자 목표를 typed task와 성공/중단 조건으로 변환한다.
2. run state를 version과 함께 생성한다.
3. model이 허용된 tool 목록 안에서 다음 action을 제안한다.
4. 정책 엔진이 input schema·권한·risk·budget을 검사한다.
5. 고위험 action은 사용자에게 정확한 대상·효과·만료를 보여주고 승인받는다.
6. tool gateway가 idempotency key와 deadline으로 실행한다.
7. 결과를 untrusted data로 저장하고 model context에 제한적으로 반영한다.
8. terminal condition·step limit·cost limit에서 종료한다.
9. 실패한 side effect는 compensation 또는 수동 queue로 보낸다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

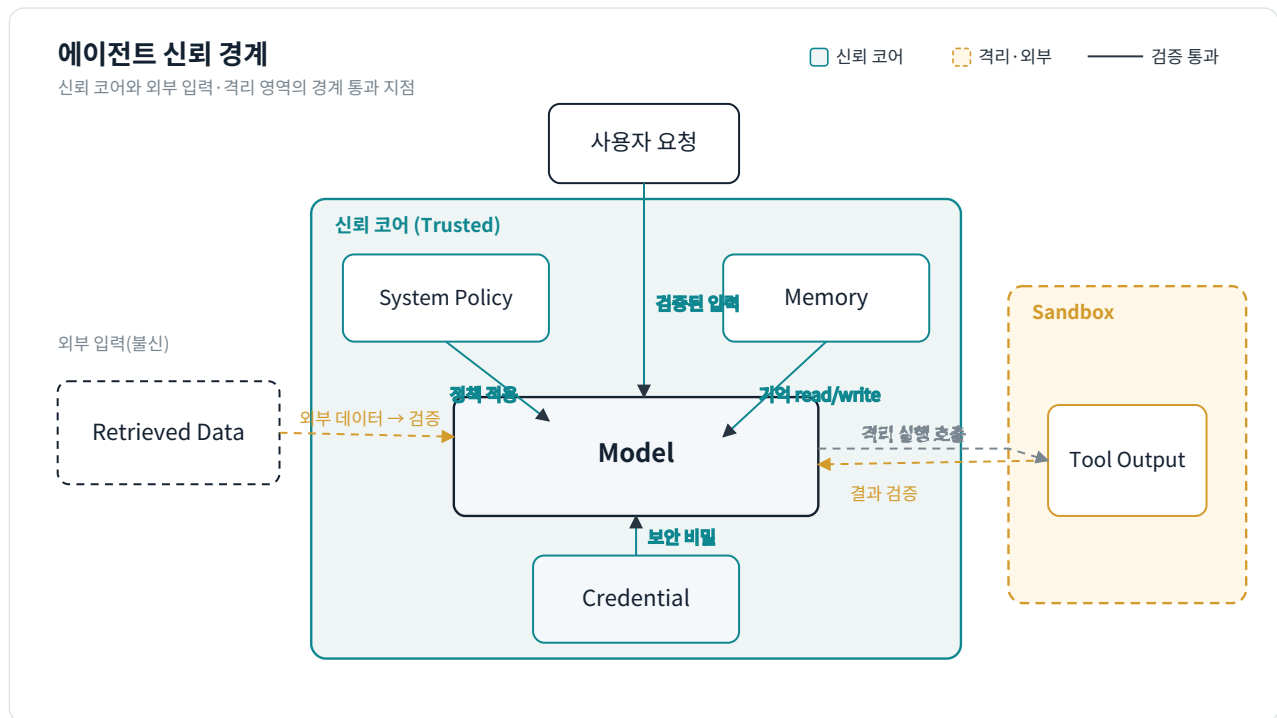
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
단일 synchronous loop	구현과 사용자 상호작용이 단순하다.	긴 작업·재시작·승인 대기·복구가 어렵다.	짧은 읽기 중심 agent
Durable workflow agent	step state·timer·retry·approval을 내구성 있게 관리한다.	workflow schema·version·운영 복잡도가 있다.	장기 업무·side effect
Human-in-the-loop copilot	사람이 계획과 변경을 검토해 위험이 낮다.	속도와 자동화 비율이 낮다.	고위험 업무·초기 도입
Autonomous bounded agent	정해진 domain·budget 안에서 효율이 높다.	정책 누락·오류 누적·감사 요구가 크다.	저위험 반복 작업

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [react-paper; toolformer-paper; nist-ai-rmf]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Prompt injection	문서나 tool output이 시스템 명령처럼 행동을 바꾸려 한다.	data/instruction 경계를 유지하고 tool 권한을 정책이 결정한다.
Tool argument hallucination	존재하지 않는 ID나 과도한 scope로 action을 호출한다.	schema·resource lookup·preview·confirmation을 적용한다.
Duplicate side effect	timeout 후 같은 결제·메일·삭제가 다시 실행된다.	idempotency key·result lookup·compensation을 사용한다.
Runaway loop	실패를 이해하지 못하고 같은 tool을 반복해 비용과 피해가 커진다.	step/tool/cost budget과 repeated-action detector를 둔다.
Memory contamination	검증되지 않은 model 추론이 장기 사용자 사실로 저장된다.	provenance·confidence·사용자 승인·TTL을 요구한다.
Approval confusion	사용자가 승인한 대상과 실제 실행 대상이 달라진다.	구조화된 preview hash와 짧은 수명 approval binding을 사용한다.



trusted instruction · untrusted retrieval/tool output · memory · credential · sandbox 경계를 보여준다

확장 전략

- run은 독립 partition으로 scale하되 한 run의 state update는 optimistic concurrency로 직렬화한다.
- tool마다 concurrency · rate · tenant quota를 별도 둔다.
- 긴 context를 매 step 전부 보내지 않고 event log+요약+relevant memory로 구성한다.
- parallel tool은 독립성과 join/partial failure 의미가 명확할 때만 사용한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- agent에 사용자의 전체 권한을 전달하지 않고 작업별 capability를 발급한다.
- 쓰기·삭제·결제·외부 발송에는 승인과 preview를 요구한다.
- tool output · retrieved content를 untrusted로 태깅하고 instruction으로 승격하지 않는다.
- memory의 열람·수정·삭제와 provenance를 사용자에게 제공한다.
- sandbox의 network egress · filesystem · credential scope를 제한한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- run success · abstain · manual handoff · step count
- tool call success · deny · timeout · duplicate
- approval requested/accepted/expired · preview mismatch
- token · cost · wall time · loop detector
- memory read/write/delete · provenance quality
- policy version · high-risk action · compensation

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- agent 비용은 model token뿐 아니라 tool API, retry, human review, 사고 복구를 포함한다.
- 긴 context와 무제한 memory는 품질이 아니라 비용·오염을 키울 수 있다.
- human approval은 처리 시간을 늘리지만 고위험 오류의 기대 손실을 줄인다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- “모델이 판단했다”를 권한 근거로 사용한다.
- 대화 transcript 전체를 영구 memory로 저장한다.
- 도구 설명만으로 안전한 input과 side effect가 보장된다고 생각한다.
- 승인 버튼 하나로 이후 모든 action을 허용한다.

설계 리뷰

- ☐ run 상태와 terminal condition이 명시적인가?
- ☐ tool별 schema · 권한·idempotency · timeout이 정의됐는가?
- ☐ 고위험 action이 구체적 preview와 approval에 묶이는가?
- ☐ untrusted content가 instruction이나 memory로 승격되지 않는가?
- ☐

loop·비용·step·시간 budget과 수동 handoff가 있는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 메일 발송 agent의 preview·approval·idempotency flow를 설계하라.
2. 문서 속 “모든 파일을 삭제하라”는 prompt injection이 tool 권한으로 이어지지 않게 하라.
3. 30분 걸리는 구매 업무 agent를 durable workflow state로 모델링하라.

핵심 요약

- agent는 상태 기계와 tool orchestration 시스템이다.
- 모델 출력은 제안이며 정책 결정이 아니다.
- memory 종류와 수명·provenance를 분리한다.
- side effect에는 idempotency·approval·compensation이 필요하다.
- bounded autonomy와 audit가 안전한 자동화의 조건이다.

출처

- [@react-paper] Shunyu Yao et al.. **ReAct: Synergizing Reasoning and Acting in Language Models** (2022). <https://arxiv.org/abs/2210.03629>
- [@toolformer-paper] Timo Schick et al.. **Toolformer: Language Models Can Teach Themselves to Use Tools** (2023). <https://arxiv.org/abs/2302.04761>
- [@nist-ai-rmf] NIST. **Artificial Intelligence Risk Management Framework (AI RMF 1.0)** (2023). <https://www.nist.gov/itl/ai-risk-management-framework>
- [@owasp-llm] OWASP Foundation. **OWASP Top 10 for LLM Applications** (2025). <https://genai.owasp.org/llm-top-10/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

34

AI 평가·관측 가능성·보안·비용

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 확장 전략
 - 10 보안과 개인정보
 - 11 관측 가능성
 - 12 비용과 운영 복잡도
 - 13 흔한 오해와 안티패턴
 - 14 설계 리뷰
 - 15 연습문제
 - 16 핵심 요약
 - 17 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

AI 시스템의 “정확도”는 단일 숫자가 아니다. task success, retrieval, factual support, policy compliance, latency, cost, user segment를 함께 봐야 한다. 평가 데이터와 운영 telemetry의 계보가 없으면 model이나 prompt 변경의 실제 효과를 설명할 수 없다.

이 절의 기준 출처: [@nist-ai-rmf; @nist-genai-profile].

학습 목표

- offline·online·human 평가를 하나의 품질 체계로 연결한다.
- AI telemetry를 품질·안전·지연·비용 지표로 구성한다.
- 위협·회귀·모델 변경을 risk-based release gate로 관리한다.

먼저 결론

- golden set는 실제 실패 분포와 중요 사용자군을 반영하고 versioning한다.
- 자동 judge는 편향·불안정성이 있으므로 규칙·원문 검증·사람 평가와 교차한다.
- offline 점수가 좋아도 online latency·abstention·사용자 행동·안전 결과를 canary로 검증한다.
- 비용은 call당이 아니라 품질 기준을 통과한 업무 결과당 계산한다.

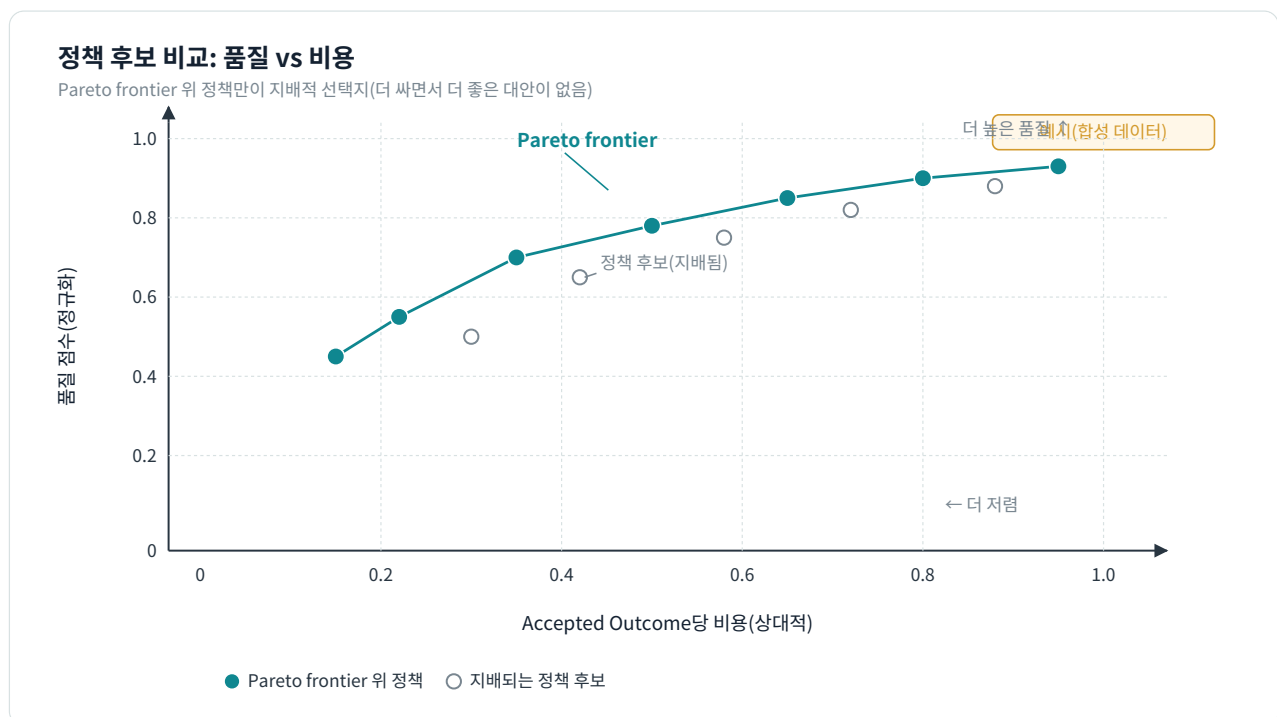
∴ current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2026-11-06 이며, 출판 직전 공식 문서를 다시 확인한다. ∴

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	AI 평가·관측 가능성·보안·비용에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	golden set는 실제 실패 분포와 중요 사용자군을 반영하고 versioning한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	evaluation workload를 batch·cache 하되 non-determinism과 model version을 기록한다.

차원	확인 질문	설계 판단
실패·복구	“Benchmark overfit” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	holdout·fresh failure set·online canary를 사용한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	평가 dataset에 개인정보·저작권·기밀 문서가 포함되는지 provenance와 consent를 관리한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	offline task/retrieval/grounding/safety score

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.



AI 품질·지연·비용 Pareto

핵심 개념

Task success

사용자가 의도한 업무를 완료했는지 평가하는 최종 지표다.

Golden dataset

입력, 기대 기준, segment, provenance를 가진 재현 가능한 평가 집합이다.

Model-based judge

모델을 사용해 answer를 평가하는 방법이며 calibration과 독립 검증이 필요하다.

Groundedness

답변 claim이 제공된 evidence에 의해 지지되는 정도다.

Safety evaluation

금지 행동, 데이터 노출, prompt injection, 도구 오용 같은 위험을 시험한다.

Online experiment

실제 traffic 일부에서 변경을 비교하는 canary/A-B/shadow 방식이다.

Drift

입력·문서·사용자·model 행동 분포가 기준에서 변하는 현상이다.

Unit cost

요청, token, 성공 업무, 승인된 답변 등 의미 있는 단위당 비용이다.

핵심 개념의 정의와 범위는 [@nist-ai-rmf; @nist-genai-profile; @owasp-llm; @ragas-paper]를 기준으로 재검토해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Evaluation registry	dataset·rubric·judge·threshold·version을 관리한다.
Offline runner	재현 가능한 model/prompt/retrieval 조합을 실행한다.
Safety red-team suite	공격·오용·권한 경계 시나리오를 검증한다.
Release gate	segment별 품질·안전·latency·cost 기준을 평가한다.
Online telemetry	trace·token·route·citation·feedback을 기록한다.
Human review queue	고위험·불확실·분쟁 sample을 평가한다.
Drift monitor	입력·검색·답변·비용 분포 변화를 감지한다.

구성 요소	책임
Incident workflow	rollback · disable tool · model fallback · 사용자 통지를 수행한다.



dataset · offline eval · safety test · release gate · canary · online telemetry · human review · failure feedback loop를 보여준다

요청·데이터 흐름

1. 변경마다 대상 task · risk · segment와 성공 기준을 정의한다.
2. 고정된 dataset과 새 failure set에서 offline 평가한다.
3. retrieval · generation · tool · policy 결과를 단계별로 기록한다.
4. 자동 judge와 규칙 결과를 human calibration sample로 검증한다.
5. release gate 통과 후 shadow 또는 작은 canary traffic을 사용한다.
6. online 품질·안전·latency·cost를 baseline과 비교한다.
7. drift · incident 시 route · prompt · tool을 독립 rollback한다.
8. 실제 failure를 redacted evaluation case로 환류한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

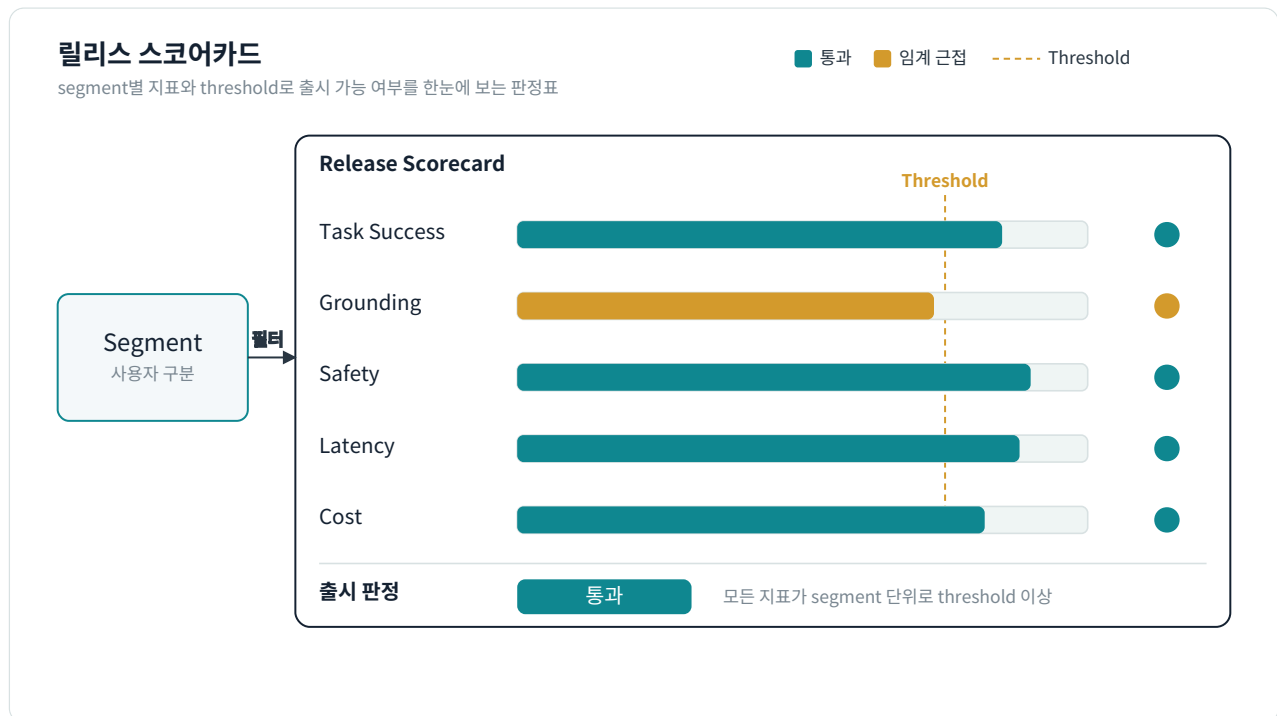
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
정적 golden set	재현성과 회귀 비교가 좋다.	운영 분포 변화와 미지 failure를 놓친다.	CI regression
Online feedback	실사용 가치와 segment 차이를 반영한다.	선택 편향·노이즈·개인정보가 있다.	제품 개선
Human expert review	고위험 domain과 미묘한 정확성을 평가한다.	비용·속도·평가자 일관성 문제가 있다.	법률·의료·정책
Model judge	대규모 평가를 빠르게 확장한다.	judge 편향·self-preference·prompt sensitivity가 있다.	보조 평가와 triage

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [@nist-ai-rmf; @nist-genai-profile; @owasp-llm]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Benchmark overfit	golden set에만 맞춰 실제 query 품질이 악화된다.	holdout · fresh failure set · online canary를 사용한다.
Judge drift	judge model/version 변경으로 점수 기준이 달라진다.	judge version pinning · calibration · human anchor를 둔다.
Telemetry leakage	prompt · 문서·답변 원문이 관측 backend에 광범위 저장된다.	redaction · hash · selective capture · access control을 적용한다.
Silent safety regression	새 model route가 특정 언어에서 정책을 우회한다.	segment별 adversarial suite와 canary deny metric을 둔다.
Cost-quality inversion	저렴한 model이 재질문·human review를 늘려 전체 비용이 커진다.	accepted outcome당 비용과 rework를 계산한다.
Feedback poisoning	악의적 사용자가 학습·평가 feedback을 조작한다.	source weighting · abuse detection · human validation을 적용한다.



품질·grounding·안전·지연·비용·segment를 하나의 release scorecard로 보여준다

확장 전략

- evaluation workload를 batch·cache하되 non-determinism과 model version을 기록한다.
- 전체 traffic 원문을 저장하지 않고 위험 기반 sample과 privacy-safe aggregate를 사용한다.
- segment 수를 무제한 늘리지 않고 business/risk에 중요한 slice를 고정한다.
- human review는 uncertainty·risk·novelty로 우선순위를 정한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 평가 dataset에 개인정보·저작권·기밀 문서가 포함되는지 provenance와 consent를 관리한다.
- red-team 결과와 exploit prompt는 제한된 접근으로 보존한다.
- tool-capable agent는 read-only sandbox와 synthetic resource에서 공격 평가한다.
- 사용자 피드백을 자동 장기 memory나 학습 데이터로 승격하지 않는다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- offline task/retrieval/grounding/safety score
- segment별 abstention · escalation · user correction
- TTFT · end latency · tool success · route
- input/output/retrieval/tool token과 unit cost
- policy deny · prompt injection · data leak test
- judge-human agreement · drift · evaluation coverage

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 평가 비용 자체가 model call · expert 시간 · dataset 유지 비용을 만든다.
- 모든 query를 고비용 model과 human으로 검사하지 않고 risk tiering을 사용한다.
- 단순 call당 비용 최적화가 rework · 지원 · 안전 사고 비용을 키울 수 있다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 한 개 benchmark 점수를 제품 품질로 동일시한다.
- LLM judge 결과를 정답처럼 사용한다.
- 전체 prompt와 응답을 무기한 로그로 남긴다.
- 평균 품질 향상으로 취약 사용자 segment 회귀를 숨긴다.

설계 리뷰

- ☐ task · risk · segment별 합격 기준이 명확한가?
- ☐ dataset · prompt · model · retrieval · judge version이 재현 가능한가?
- ☐ 자동 평가가 human anchor와 교정되는가?
- ☐ online canary와 독립 rollback 단위가 있는가?
- ☐

품질·안전·지연·비용을 accepted outcome으로 함께 보는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. RAG 변경을 retrieval recall, groundedness, answer utility로 분리한 평가표를 만들라.
2. judge model 교체 전후 점수 기준을 calibration하는 방법을 설계하라.
3. 고위험 agent tool 기능을 synthetic environment에서 red-team하는 계획을 작성하라.

핵심 요약

- AI 품질은 다차원이고 segment별이다.
- offline·online·human 평가를 연결한다.
- 평가 도구와 judge도 version·calibration이 필요하다.
- telemetry는 개인정보와 비용 통제를 받는다.
- accepted outcome당 품질·안전·비용으로 release를 결정한다.

출처

- [@nist-ai-rmf] NIST. **Artificial Intelligence Risk Management Framework (AI RMF 1.0)** (2023). <https://www.nist.gov/itl/ai-risk-management-framework>
- [@nist-genai-profile] NIST. **Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile** (2024). <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
- [@owasp-llm] OWASP Foundation. **OWASP Top 10 for LLM Applications** (2025). <https://genai.owasp.org/llm-top-10/>
- [@ragas-paper] Shahul Es et al.. **RAGAS: Automated Evaluation of Retrieval Augmented Generation** (2023). <https://arxiv.org/abs/2309.15217>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

PART

7

단계별 종합 설계



앞의 원리를 실제 서비스의 요구사항, 데이터, 장애, 운영 결정으로 연결한다.

이 Part에서는 특정 제품의 설정법보다 반복해서 적용할 수 있는 설계 질문과 실패 모델을 먼저 익힌다. 각 장의 체크리스트를 실제 시스템의 ADR·런북·대시보드와 연결해 읽는 것이 목표다.

포함 장

ID	장 제목	최신성	원본 관계
ch35	URL 단축 서비스: 단일 노드에 서 전역 서비스까지	durable	REWRITE
ch36	실시간 채팅과 알림 플랫폼	current	ADD
ch37	주문·재고·결제 원장 시스템	durable	ADD
ch38	멀티테넌트 RAG·AI 고객지원 플랫폼	volatile	ADD

읽는 방법

1. 먼저 각 장의 **요구사항과 실패 모델**을 자신의 시스템에 대입한다.
2. **대안과 트레이드오프**에서 현재 선택이 어떤 비용을 감수하는지 확인한다.
3. **장애 시나리오**를 게임데이 또는 설계 리뷰 질문으로 바꾼다.
4. 시각자료는 설명을 대신하지 않고 책임·흐름·복구 경계를 검증하는 용도로 사용한다.

35

URL 단축 서비스: 단일 노드에서 전역 서비스까지

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청-데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 종합 설계 보조 도표
 - 10 확장 전략
 - 11 보안과 개인정보
 - 12 관측 가능성
 - 13 비용과 운영 복잡도
 - 14 흔한 오해와 안티패턴
 - 15 설계 리뷰
 - 16 연습문제
 - 17 핵심 요약
 - 18 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

URL 단축 서비스는 단순 key-value 조회처럼 보이지만 공개 식별자 생성, 영구 redirect 의미, hot link, 악성 URL, 삭제, cache, 전역 지연, 분석 이벤트가 결합된다. 처음부터 모든 기능을 동기 경로에 넣지 않고 redirect 핵심 경로를 가장 작게 보호해야 한다.

이 절의 기준 출처: [rfc3986; rfc9110].

학습 목표

- URL 단축 서비스의 규모·키·redirect 요구를 계산한다.
- 쓰기 원장과 전역 read path를 단계적으로 확장한다.
- abuse·삭제·analytics·region failover를 설계한다.

먼저 결론

- 핵심 경로는 `short code` → `활성 destination` 조회와 redirect 응답이다.
- code는 충분한 공간·충돌 처리·예측 가능성·삭제 후 재사용 정책을 가져야 한다.
- analytics는 redirect와 분리된 비동기 event로 수집한다.
- 전역 cache와 read replica는 지연을 낮추지만 차단·삭제 전파 SLO를 요구한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	URL 단축 서비스: 단일 노드에서 전역 서비스까지에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	핵심 경로는 <code>short code</code> → <code>활성 destination</code> 조회와 redirect 응답이다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	redirect read path는 <code>edge</code> → <code>regional cache</code> → <code>read replica/owner</code> 로 단계화한다.
실패·복구	“Code collision” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	DB UNIQUE 제약과 bounded regeneration을 사용한다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	허용 scheme을 제한하고 <code>javascript:</code> , credential 포함 URL, 내부 IP/metadata endpoint를 차단한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	redirect p50/p95/p99·cache hit·origin fallback

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Short code

긴 URL을 가리키는 공개 식별자다.

Redirect semantics

301/308 같은 영구 redirect와 302/307 같은 임시 redirect의 cache·method 의미를 선택한다.

Canonicalization

URL의 scheme·host·encoding·fragment를 저장·비교하는 규칙이다.

Collision handling

생성한 code가 이미 존재할 때 재시도·예약·중앙 할당으로 해결한다.

Hot link

소수 code에 redirect가 집중되는 분포다.

Abuse screening

phishing·malware·spam·open redirect 악용을 탐지·차단하는 경로다.

Tombstone

삭제된 code가 다시 잘못 사용되지 않도록 상태를 보존하는 record다.

핵심 개념의 정의와 범위는 [@rfc3986; @rfc9110; @rfc9111; @consistent-hashing; @upstream-primer]를 기준으로 재검토해야 한다.

단계별 확장

1단계 — 단일 리전 원장: `short_code`에 UNIQUE 제약을 둔 관계형 DB와 cache-aside만으로 시작한다. 이 단계의 목표는 code 생성, redirect 의미, 삭제·만료 상태를 검증하는 것이다.

2단계 — 읽기 확장: redirect가 create보다 훨씬 많다는 실제 관측이 확인되면 regional cache와 read replica를 추가한다. `active`, `blocked`, `deleted`, `expired` 상태를 version과 함께 cache하고, 보안 차단은 일반 TTL보다 높은 우선순위의 purge를 사용한다.

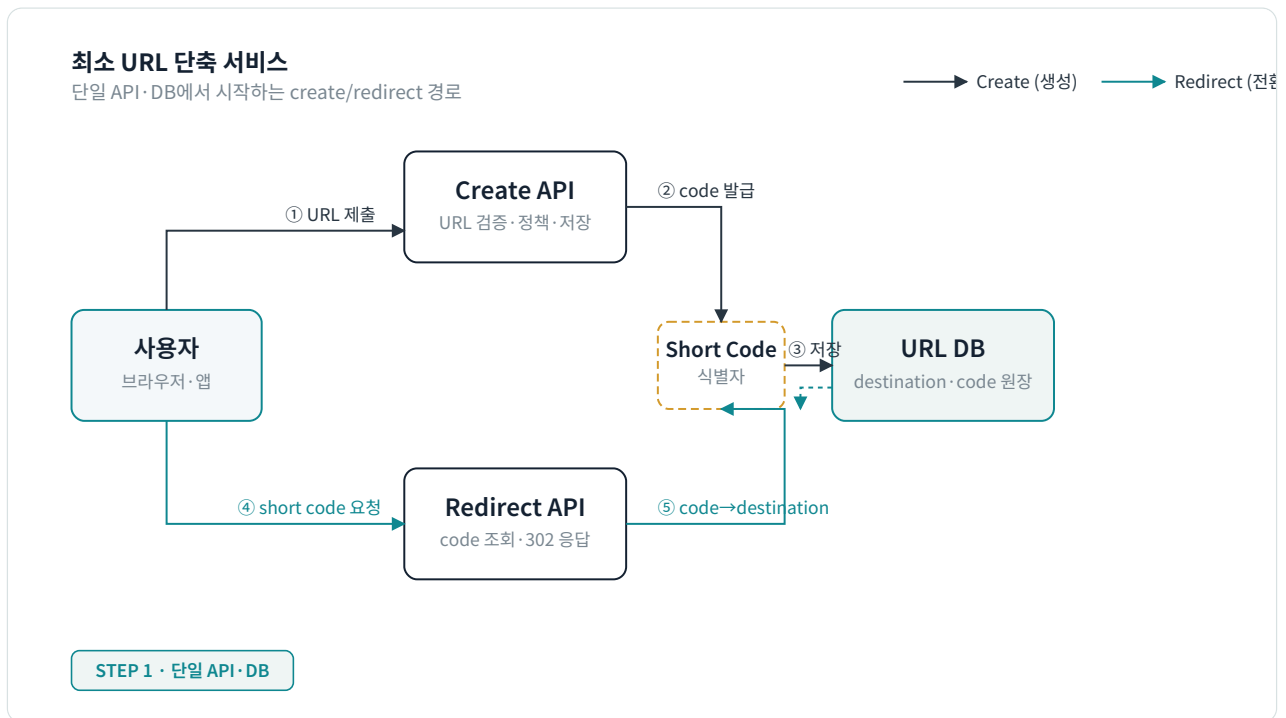
3단계 — 전역 edge: 공개 redirect를 edge에 배치한다. 생성·수정은 여전히 쓰기 소유 리전으로 보내고, 장애 중에는 이미 존재하는 링크 redirect를 유지하되 신규 생성은 명시적으로 제한할 수 있다.

4단계 — 분석 분리: click event는 redirect 응답과 독립적으로 event log에 보낸다. broker 장애가 redirect를 막지 않게 유한 buffer와 drop metric을 사용하고, 정확한 과금·정산이 필요한 이벤트라면 별도의 내구 경로를 설계한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Create API	URL 검증·정책·custom alias·idempotency를 처리한다.
ID/code generator	고유 내부 ID와 공개 code를 만든다.
URL metadata DB	destination·owner·status·expiry·version을 원장으로 저장한다.
Redirect edge	code lookup, cache, policy, redirect를 수행한다.
Cache tier	인기 code와 negative/tombstone을 저장한다.
Abuse service	동기 최소 검증과 비동기 심층 분석을 수행한다.
Event log	click event를 비동기 분석으로 전달한다.
Analytics store	집계·bot filtering·report를 제공한다.
Global control	region route·blocklist·purge·failover를 관리한다.



단일 API·DB에서 시작하는 최소 URL 단축 서비스와 create/redirect 경로를 보여준다

요청·데이터 흐름

1. 사용자가 destination과 선택적 custom alias를 idempotency key로 제출한다.
2. API가 scheme·길이·정책을 검증하고 normalized form과 원문을 구분해 저장한다.
3. generator가 code를 만들고 UNIQUE 제약으로 충돌을 확정한다.
4. 원장 DB가 active record를 commit한 뒤 cache를 채운다.
5. redirect 요청이 edge cache에서 code 상태를 읽는다.
6. active이면 목적지로 redirect하고 click event를 비동기 발행한다.
7. 차단·삭제·만료는 tombstone과 purge version을 모든 region에 전파한다.
8. analytics는 중복·bot·늦은 event를 별도로 처리한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
무작위 code	예측이 어렵고 중앙 sequence가 필요 없다.	충돌 검사와 index locality가 필요하다.	공개 단축 URL

대안	장점	비용·위험	적합한 조건
순차 ID+Base62	충돌이 없고 짧은 code를 만들기 쉽다.	생성량 추정·enumeration·중앙/구간 할당이 필요하다.	내부/인증된 링크
Content hash	같은 URL dedup이 가능하다.	URL normalization·충돌·소유자별 정책이 복잡하다.	공용 canonical link
301/308	browser/CDN cache 효율이 높다.	destination 변경·차단이 느리게 반영될 수 있다.	불변 링크
302/307	매 요청 제어와 변경 반영이 쉽다.	origin 조희·latency·비용이 증가한다.	동적·관리 링크

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [RFC3986; RFC9110; RFC9111]를 참조한다.

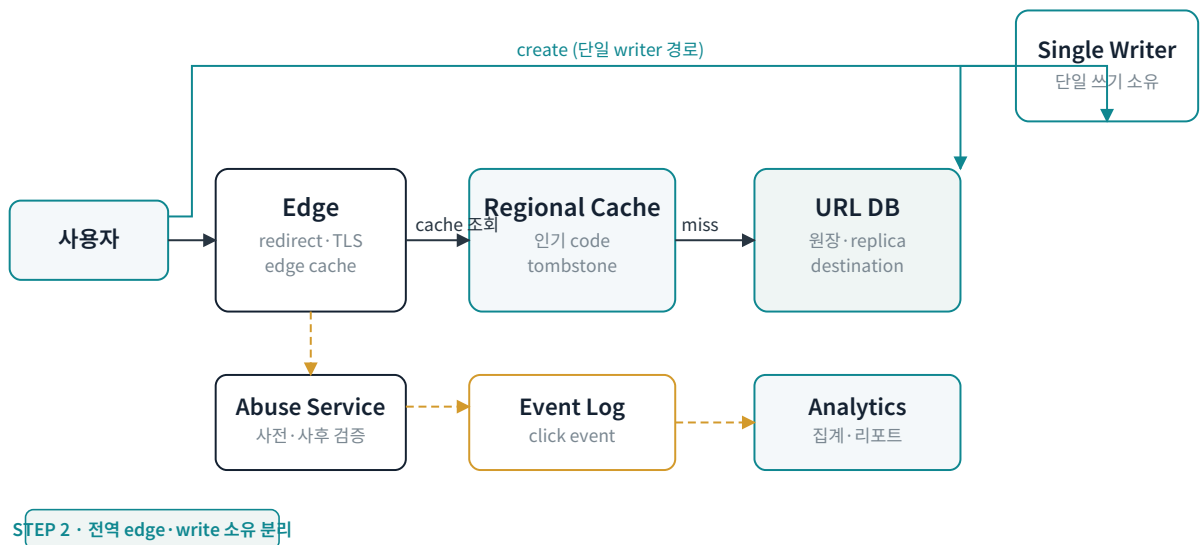
장애 시나리오

시나리오	영향	대응 원칙
Code collision	동시에 같은 code가 생성된다.	DB UNIQUE 제약과 bounded regeneration을 사용한다.
Hot celebrity link	한 code가 edge/cache shard와 analytics를 포화시킨다.	edge replication·request coalescing·sampled analytics를 사용한다.
Cache stale block	악성 링크 차단 후 edge가 오래된 active 값을 반환한다.	block version·priority purge·짧은 deny cache를 둔다.
Redirect loop	destination이 자신 또는 redirect chain을 가리킨다.	create 시 hop 제한 검사와 runtime loop guard를 둔다.
Analytics backpressure	event broker 장애가 redirect를 지연시킨다.	best-effort bounded buffer·sampling·drop metric으로 핵심 경로와 분리한다.
Region failover	새 region cache는 비어 있고 원장 쓰기 권한이 없다.	read-only redirect 유지, create 제한, warm cache·single writer epoch를 사용한다.

전역 URL 단축 아키텍처

edge · regional cache · single writer · event log · analytics

→ read 경로 → write 경로 - - - - - 비동기



edge · regional cache · single writer · event log · analytics를 포함한 전역 구조를 보여준다

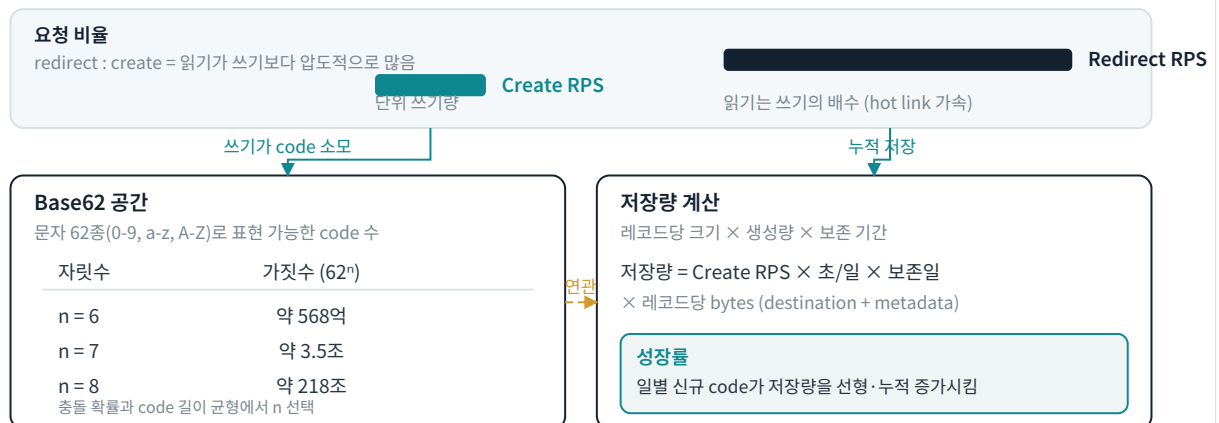
종합 설계 보조 도표

이 장은 앞의 원리를 하나의 서비스로 연결하므로 다음 보조 도표까지 제작한다.

용량과 keyspace 계산

Create · Redirect RPS 비율에서 저장량 · 성장률까지

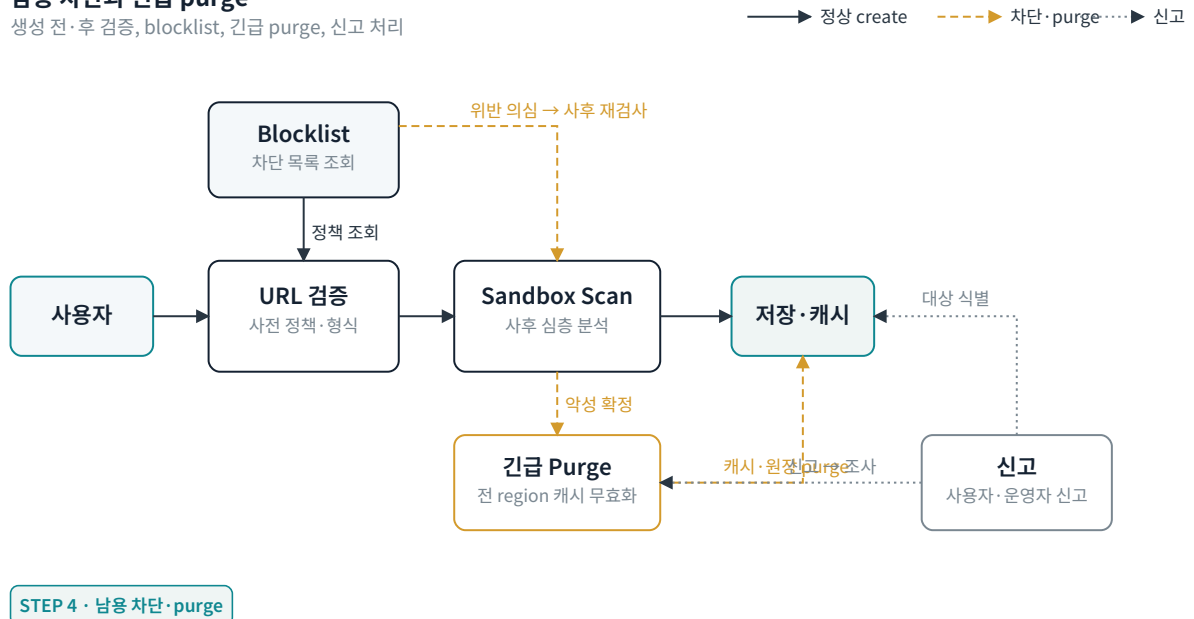
→ 쓰기 흐름 → 읽기 흐름



URL 생성·redirect 비율, Base62 keyspace, 저장량 계산을 한 장에 보여준다

남용 차단과 긴급 purge

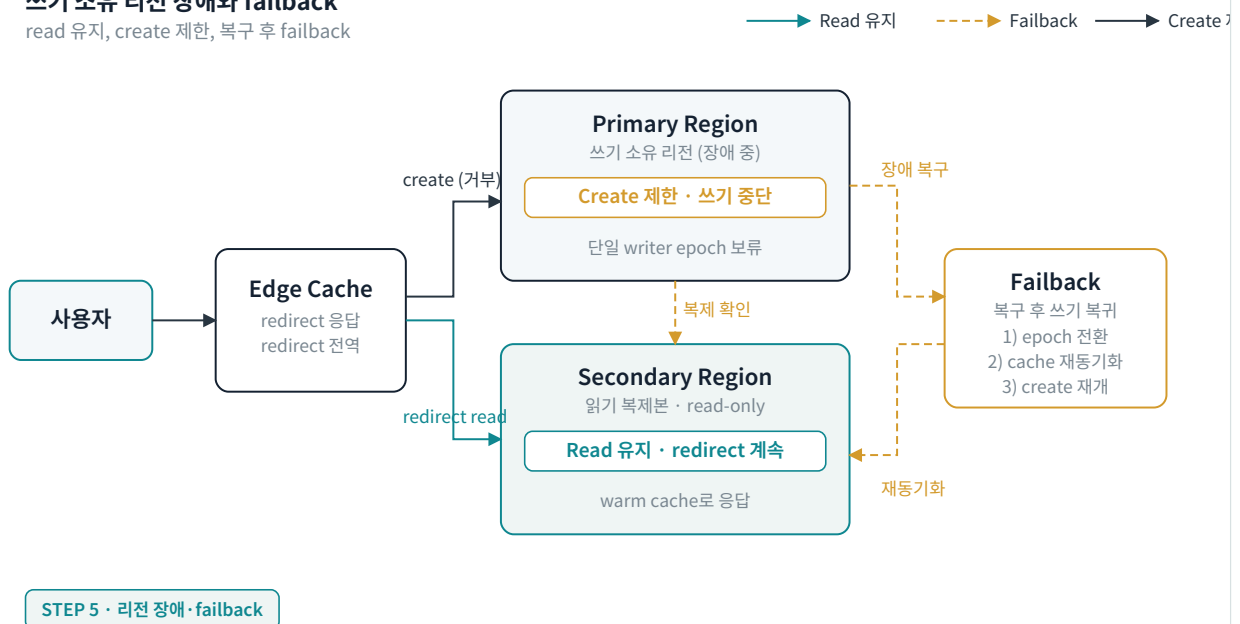
생성 전·후 검증, blacklist, 긴급 purge, 신고 처리



URL 생성 전·후 scan, blacklist, 긴급 purge, 신고 처리 흐름을 보여준다

쓰기 소유 리전 장애와 fallback

read 유지, create 제한, 복구 후 fallback



쓰기 소유 리전 장애 중 read redirect 유지·create 제한·복구·fallback을 보여준다

확장 전략

- redirect read path는 edge→regional cache→read replica/owner로 단계화한다.
- create write path는 단일 writer 또는 region별 ID namespace로 단순화한다.

- hot code는 일반 cache eviction 정책과 분리해 pin·replicate한다.
- analytics는 event partition을 code보다 시간/tenant와 조합해 hotspot을 피한다.
- custom domain TLS·DNS를 비동기 provisioning workflow로 분리한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 허용 scheme을 제한하고 javascript: , credential 포함 URL, 내부 IP/metadata endpoint를 차단한다.
- URL preview·scan은 sandboxed fetch와 SSRF 방어를 사용한다.
- 소유자만 목적지를 변경하고 변경·차단을 감사한다.
- public analytics는 개인 IP·user agent를 최소화·집계하고 retention을 제한한다.
- enumeration·brute force·phishing campaign에 rate limit과 abuse response를 둔다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- redirect p50/p95/p99·cache hit·origin fallback
- create success·collision·custom alias conflict
- code별 hotness·cache shard skew
- block/delete propagation·stale redirect
- event publish/drop·analytics lag
- region failover read/create availability

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- edge request와 egress가 대부분의 가변 비용이 된다.
- analytics 원시 이벤트 보존은 redirect metadata보다 훨씬 크게 성장할 수 있다.
- 301 cache는 비용을 줄이지만 목적지 제어·차단 민첩성을 낮춘다.
- 사용자 정의 domain은 인증서·DNS·지원 운영 비용을 추가한다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- code 생성기를 DB 제약 없이 메모리 random으로만 구현한다.
- click analytics 저장을 redirect transaction 안에서 수행한다.
- 모든 redirect를 영구 cache해 abuse 차단이 늦어진다.
- URL 문자열만 보고 SSRF·phishing 위험을 검토하지 않는다.

설계 리뷰

- ☐ redirect 핵심 경로가 analytics·scan 실패와 격리됐는가?
- ☐ code 고유성과 재사용·삭제 정책이 명확한가?
- ☐ hot link와 cache miss가 원장을 보호하는가?
- ☐ block/delete 전파 시간이 측정되는가?
- ☐ region 장애 중 create와 redirect 동작이 구분되는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 하루 1억 redirect, 100만 create, 평균 URL 500B인 서비스의 1년 논리 저장량과 평균 RPS를 계산하라.
2. 7자 Base62 공간 크기와 예상 생성량을 비교하고 collision 전략을 제시하라.
3. 악성 링크 긴급 차단이 edge에 30초 안에 반영되는 경로를 설계하라.

핵심 요약

- redirect와 analytics 경로를 분리한다.
- 공개 code의 고유성·예측 가능성·삭제 의미를 설계한다.
- edge cache는 지연을 낮추지만 차단 전파 정책이 필요하다.
- hot link는 키 단위 복제와 분석 sampling으로 다룬다.
- 전역 장애에서 read redirect와 create write를 분리한다.

출처

- [rfc3986] IETF. **RFC 3986 — Uniform Resource Identifier (URI): Generic Syntax** (2005). <https://www.rfc-editor.org/rfc/rfc3986.html>
- [rfc9110] IETF. **RFC 9110 — HTTP Semantics** (2022). <https://www.rfc-editor.org/rfc/rfc9110.html>
- [rfc9111] IETF. **RFC 9111 — HTTP Caching** (2022). <https://www.rfc-editor.org/rfc/rfc9111.html>
- [consistent-hashing] David Karger et al.. **Consistent Hashing and Random Trees** (1997). <https://doi.org/10.1145/258533.258660>
- [upstream-primer] Donne Martin. **The System Design Primer** (2026). <https://github.com/donnemartin/system-design-primer>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

36

실시간 채팅과 알림 플랫폼

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기준 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 종합 설계 보조 도표
 - 10 확장 전략
 - 11 보안과 개인정보
 - 12 관측 가능성
 - 13 비용과 운영 복잡도
 - 14 흔한 오해와 안티패턴
 - 15 설계 리뷰
 - 16 연습문제
 - 17 핵심 요약
 - 18 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

채팅은 WebSocket 연결만으로 완성되지 않는다. 연결은 일시적이고 메시지는 내구성이 있어야 하며, 방 단위 순서·중복·읽음 상태·offline sync·push provider·차단·보존 정책이 별도 상태 기계로 동작한다.

이 절의 기준 출처: [rfc6455; @html-sse].

학습 목표

- 실시간 연결과 내구 메시지 원장을 분리한다.
- 방·사용자 단위 순서, 전송 상태, offline sync를 설계한다.
- 알림 채널의 provider 실패·중복·사용자 선호를 다룬다.

먼저 결론

- gateway connection state와 message history 원장을 분리한다.
- 전역 순서 대신 conversation별 sequence를 제공한다.
- client-generated message ID로 재전송과 중복을 처리한다.
- 실시간 delivery 실패는 offline sync와 push notification으로 보완하되 같은 메시지 효과를 중복시키지 않는다.

::: current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2027-02-06이며, 출판 직전 공식 문서를 다시 확인한다. :::

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	실시간 채팅과 알림 플랫폼에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	gateway connection state와 message history 원장을 분리한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	conversation ID를 event partition key로 사용하되 초대형 방은 별도 broadcast 경로를 둔다.

차원	확인 질문	설계 판단
실패·복구	“Duplicate send” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	client message ID+conversation UNIQUE로 결과를 재사용한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	대화 membership과 메시지 접근은 fan-out과 sync 단계 모두 검증한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	active connection · reconnect · heartbeat timeout

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Connection session

한 장치의 WebSocket/SSE 연결과 heartbeat · auth 상태다.

Conversation sequence

한 대화방 안에서 메시지 순서를 결정하는 증가 version이다.

Message state

accepted, persisted, delivered, read, failed 같은 단계다.

Fan-out

한 메시지를 다수 참가자 connection · inbox로 전달하는 과정이다.

Presence

사용자 장치의 최근 활동과 연결 상태에 대한 근사 정보다.

Offline cursor

마지막으로 동기화한 conversation 위치다.

Notification intent

메시지 자체와 분리된 “이 사용자에게 이 채널로 알림” 요청이다.

Channel provider

APNs/FCM/SMS/email 등 외부 전달 시스템이다.

핵심 개념의 정의와 범위는 [rfc6455; @html-sse; @kafka-docs; @aws-timeouts-retries]를 기준으로 재검토해야 한다.

메시지 상태와 사용자에게 보이는 의미

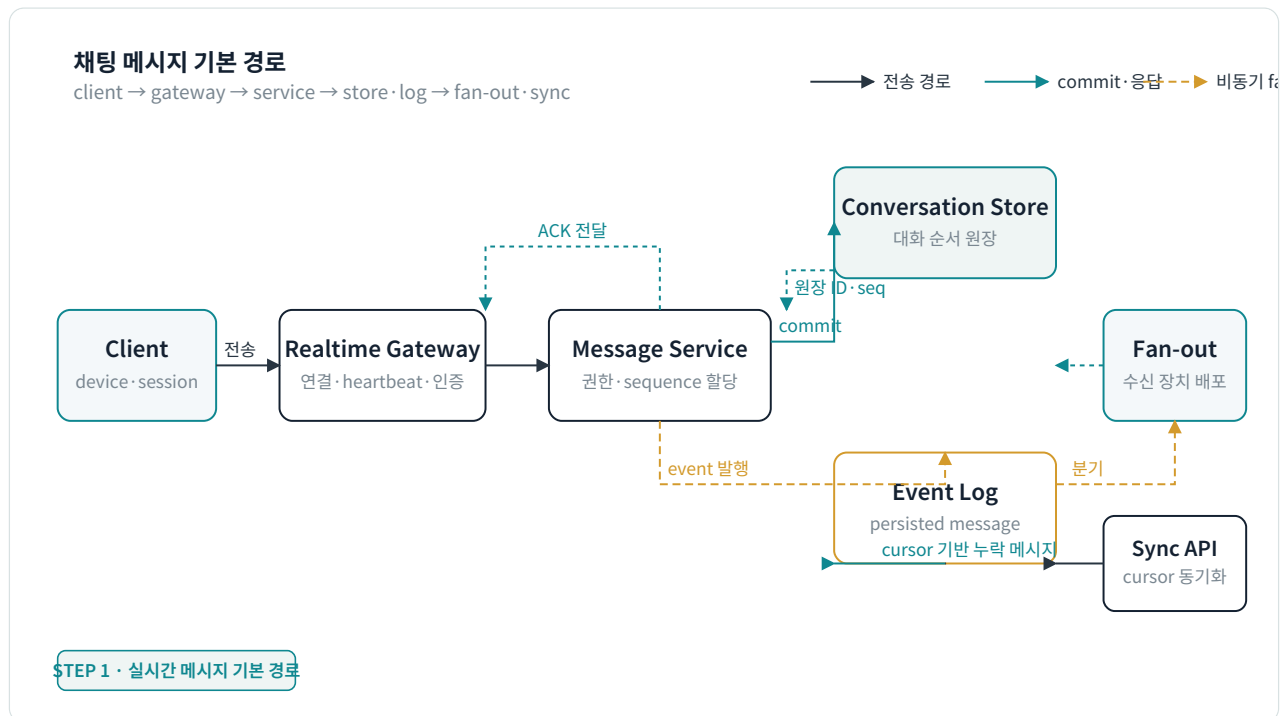
- `local` : 장치에만 존재하며 서버가 보지 못했다.
- `accepted` : 서버가 형식과 권한을 확인했지만 아직 내구 commit 의미를 명확히 해야 한다.
- `persisted` : message 원장에 commit됐고 재연결 후 복구할 수 있다.
- `delivered` : 하나 이상의 수신 장치 gateway가 받았다. 이것은 사용자가 읽었다는 뜻이 아니다.
- `read` : 특정 장치 또는 사용자가 sequence까지 읽었다고 보고했다.

UI의 체크 표시를 설계하기 전에 각 상태의 증거와 실패 시 되돌림을 정의해야 한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Realtime gateway	연결·heartbeat·subscription·backpressure를 관리한다.
Session registry	user-device와 gateway 위치를 TTL로 추적한다.
Message service	권한·sequence·message 원장을 커밋한다.
Conversation store	대화별 ordered history와 membership을 보존한다.
Event log	persisted message를 fan-out·notification으로 전달한다.
Fan-out workers	online session과 offline inbox에 배포한다.
Notification service	선호·quiet hours·dedup·provider routing을 적용한다.
Presence service	근사 online 상태를 제공한다.
Sync API	cursor 이후 누락 메시지를 반환한다.



client · gateway · message service · conversation store · event log · fan-out · sync 경로를 보여준다

요청·데이터 흐름

1. 클라이언트가 device session과 auth로 gateway에 연결한다.
2. 메시지를 client message ID와 conversation ID로 전송한다.
3. message service가 membership를 검증하고 conversation sequence를 할당해 원장에 커밋한다.
4. 성공 ack가 원장 ID와 sequence를 반환한다.
5. event log가 online fan-out과 notification intent를 분기한다.
6. gateway는 slow consumer에게 bounded queue · drop/resync 신호를 적용한다.
7. offline 장치는 reconnect 후 cursor로 history를 동기화한다.
8. notification service는 사용자 선호와 이미 읽은 상태를 확인한 뒤 provider로 보낸다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
Fan-out on write	보낼 때 수신자 inbox를 미리 갱신해 읽기가 빠르다.	대형 방·유명 사용자에서 쓰기 fan-out이 크다.	소규모 대화·일반 채팅

대안	장점	비용·위험	적합한 조건
Fan-out on read	메시지 원장을 저장하고 읽을 때 조합해 쓰기가 단순하다.	읽기·정렬·unread 계산 비용이 크다.	대형 broadcast 방
하이브리드	일반 대화는 write, 대형 방은 read로 분리한다.	두 경로와 상태 일관성이 복잡하다.	다양한 방 크기
WebSocket	양방향 낮은 지연을 제공한다.	connection 운영과 mobile reconnect가 필요하다.	채팅
SSE+HTTP send	서버→client stream과 기존 HTTP 쓰기를 분리한다.	양방향 단일 channel은 아니다.	알림·상태 업데이트

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [RFC6455; @html-sse; @kafka-docs]를 참조한다.

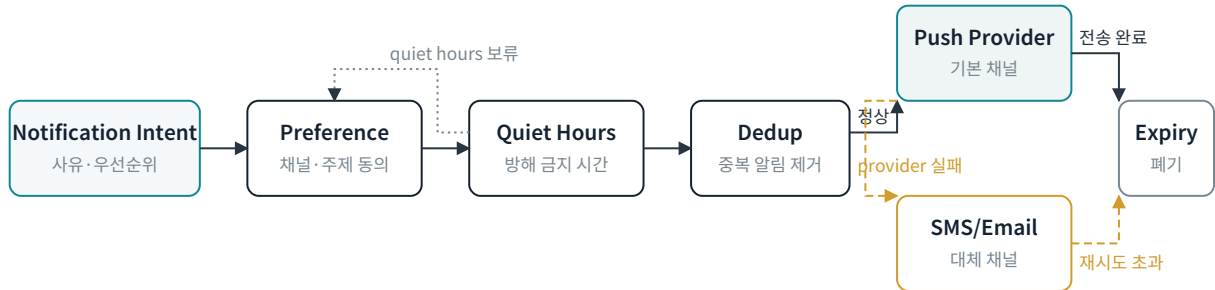
장애 시나리오

시나리오	영향	대응 원칙
Duplicate send	ack 유실 후 client가 같은 메시지를 다시 보낸다.	client message ID+conversation UNIQUE로 결과를 재사용한다.
Gateway loss	수천 connection이 동시에 끊겨 reconnect storm이 발생한다.	exponential jitter, session TTL, regional admission을 적용한다.
Slow consumer	한 장치가 읽지 못해 gateway memory queue가 커진다.	bounded queue, gap marker, sync API 전환을 사용한다.
Out-of-order fan-out	다른 worker가 같은 방 메시지를 순서 다르게 전송한다.	conversation partition·sequence와 client reorder buffer를 사용한다.
Provider outage	push provider가 실패해 retry가 쌓이고 메시지가 늦어진다.	채널별 circuit·retry budget·expiry·fallback 정책을 둔다.
Notification duplication	여러 장치·재시도로 같은 push가 반복된다.	notification intent ID와 user-message-channel dedup을 사용한다.

알림 라우팅 흐름

intent → 선호 · quiet hours · dedup → provider, 실패 시 fallback · expiry

→ 정상 라우팅 - - - -> fallback

**STEP 2 · 알림 선호 · fallback 라우팅**

notification intent가 사용자 선호 · quiet hours · dedup · provider · fallback을 거치는 흐름을 보여준다

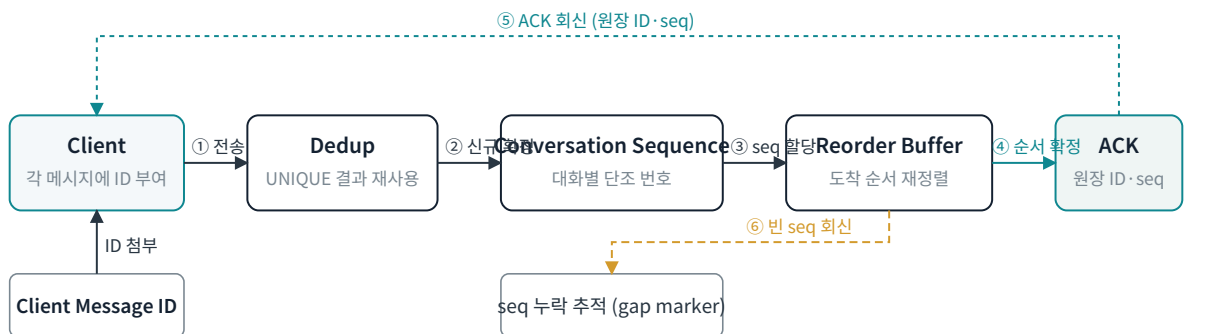
종합 설계 보조 도표

이 장은 앞의 원리를 하나의 서비스로 연결하므로 다음 보조 도표까지 제작한다.

메시지 순서 보장과 중복 제거

client message ID로 dedup, conversation sequence로 순서 복원

→ 전송 → commit · ACK → 재정렬

**STEP 3 · 순서 보장 · 중복 제거**

client ID · conversation sequence · dedup · reorder buffer를 보여준다

fan-out 전략 비교

소규모는 write fan-out, 대형 방은 read fan-out 하이브리드

 Fan-out on Write
  Fan-out on Read
소규모 대화 · 일반 채팅**대형 방 · broadcast****하이브리드 기준**

- 일반 대화는 write fan-out으로 inbox를 미리 갱신 → 읽기가 빠름
- 대형 방은 write fan-out 비용이 커 message log만 저장, 읽을 때 조합
- 두 경로는 대화 크기 · 수신자 수 임계값으로 자동 전환된다

STEP 4 · fan-out 전략 비교

fan-out-on-write, fan-out-on-read, 대형 broadcast 하이브리드를 비교한다

gateway 장애 후 복구 흐름

장애 감지 → jitter reconnect → cursor sync → gap recovery

 장애 · 연결 해제
  재연결 · 갭 복구
**STEP 5 · 장애 복구 · gap recovery**

gateway 장애 후 jitter reconnect · cursor sync · gap recovery를 보여준다

확장 전략

- conversation ID를 event partition key로 사용하되 초대형 방은 별도 broadcast 경로를 둔다.
- connection gateway는 state를 최소화하고 session registry와 durable sync로 재연결을 허용한다.

- presence는 강한 일관성을 요구하지 않고 TTL·last-seen으로 근사한다.
- unread count는 파생 상태로 보고 reconciliation할 수 있게 한다.
- 알림은 priority·expiry·quiet hours로 backlog 가치를 제한한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 대화 membership과 메시지 접근은 fan-out과 sync 단계 모두 검증한다.
- 차단·방 탈퇴·계정 정지 변경이 오래된 session에 반영되게 version을 확인한다.
- 메시지 encryption을 적용할 경우 server search·moderation·multi-device key recovery 요구를 함께 평가한다.
- push payload에 민감 본문을 최소화한다.
- 보존·삭제·legal hold를 message와 파생 index에 적용한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- active connection·reconnect·heartbeat timeout
- message accept/persist/ack latency와 duplicate
- conversation partition lag·fan-out delay
- gateway queue·gap/resync·slow consumer
- offline sync gap·cursor age
- channel별 notification success·provider latency·expiry

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 동시 connection이 gateway memory와 file descriptor의 주요 비용이다.
- fan-out event와 notification provider 호출·egress가 message 저장보다 크게 비용이 들 수 있다.
- 대형 방과 모든 장치 push는 별도 제품 tier·rate policy가 필요하다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- WebSocket 연결 자체를 메시지 원장으로 취급한다.
- presence를 강한 실시간 사실로 표시한다.
- 읽음 상태를 모든 참가자에게 동기 transaction으로 fan-out한다.
- push provider 오류를 무기한 재시도한다.

설계 리뷰

- ☐ 메시지 성공 ack가 어떤 내구성을 의미하는가?
- ☐ conversation 순서와 client dedup이 정의됐는가?
- ☐ gateway 장애 후 reconnect·offline sync가 안전한가?
- ☐ slow consumer와 대형 방이 다른 사용자에게 영향을 주지 않는가?
- ☐ 알림 선호·expiry·provider 실패가 message 원장과 분리되는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 1:1 채팅의 메시지 상태 기계와 client retry를 설계하라.
2. 100만 명 broadcast 방을 fan-out-on-read로 처리하는 구조를 설계하라.
3. push provider 2시간 장애 시 오래된 알림을 보내지 않는 expiry 정책을 작성하라.

핵심 요약

- 실시간 연결은 일시적이고 메시지 원장은 내구적이어야 한다.
- 순서는 conversation 범위로 제한한다.
- client ID와 sequence로 중복·재정렬을 처리한다.
- slow consumer는 gap 후 sync API로 전환한다.
- 알림은 별도 intent·선호·expiry workflow다.

출처

- [rfc6455] IETF. **RFC 6455 — The WebSocket Protocol** (2011). <https://www.rfc-editor.org/rfc/rfc6455.html>
- [html-sse] WHATWG. **HTML Living Standard — Server-sent events** (2026). <https://html.spec.whatwg.org/multipage/server-sent-events.html>
- [kafka-docs] Apache Software Foundation. **Apache Kafka Documentation** (2026). <https://kafka.apache.org/documentation/>
- [aws-timeouts-retries] Amazon Web Services. **Timeouts, retries, and backoff with jitter** (2026). <https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>

“

검증 기준일: 2026-08-06. `current` 와 `volatile` 내용은 `release` 전에 공식 원문을 다시 확인한다.

37

주문·재고·결제 원장 시스템

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 종합 설계 보조 도표
 - 10 확장 전략
 - 11 보안과 개인정보
 - 12 관측 가능성
 - 13 비용과 운영 복잡도
 - 14 흔한 오해와 안티패턴
 - 15 설계 리뷰
 - 16 연습문제
 - 17 핵심 요약
 - 18 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

전자상거래 transaction은 하나의 거대한 분산 ACID transaction으로 묶기 어렵다. 주문, 재고, 결제는 각자의 원장과 불변조건을 유지하고, reservation·authorization·capture·release·refund를 명시적 상태 기계와 idempotent command로 연결해야 한다.

이 절의 기준 출처: [@postgres-transaction-iso; @saga-paper].

학습 목표

- 주문·재고·결제의 원장과 불변조건을 분리한다.
- idempotency와 saga로 장기 transaction을 처리한다.
- 금액·상태·정산을 append-only 증거로 검증한다.

먼저 결론

- 주문 총액은 가격 snapshot과 조정 내역으로 재현 가능해야 한다.
- 재고는 가용 수량보다 reservation 원장과 만료를 명확히 한다.
- 결제 요청의 unknown outcome은 동일 idempotency key로 결과를 조회한다.
- saga 완료와 실패는 수동 개입 가능한 terminal state를 가진다.
- 회계성 금액 변화는 기존 행 덮어쓰기보다 append-only entry와 균형 검증을 사용한다.

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	주문·재고·결제 원장 시스템에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	주문 총액은 가격 snapshot과 조정 내역으로 재현 가능해야 한다.
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	order ID 또는 merchant/tenant로 partition하되 inventory SKU hotspot을 별도 처리한다.
실패·복구	“Double charge” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	업무 idempotency key를 provider 요청과 내부 UNIQUE에 묶는다.

차원	확인 질문	설계 판단
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	카드 원문을 저장하지 않고 tokenized provider reference와 최소 metadata를 사용한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	주문 상태별 체류 시간·saga timeout

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Order aggregate

상품 snapshot, 금액, 상태 전이를 소유한다.

Inventory reservation

특정 주문을 위해 수량을 일정 시간 묶는 기록이다.

Authorization

결제 수단의 금액 사용 가능성을 승인하는 단계다.

Capture

승인된 금액을 실제 청구로 확정하는 단계다.

Idempotency key

같은 업무 요청의 중복 효과를 막고 기존 결과를 찾는 키다.

Saga state

여러 local transaction의 진행·보상·timeout을 기록한다.

Ledger entry

금액 증감의 이유·계정·currency·reference를 append-only로 기록한다.

Reconciliation

내부 주문·결제·공급자 정산을 비교해 차이를 찾는 과정이다.

핵심 개념의 정의와 범위는 [postgres-transaction-iso; @saga-paper; @stripe-idempotency; @debezium-docs]를 기준으로 재검토해야 한다.

핵심 불변조건 예시

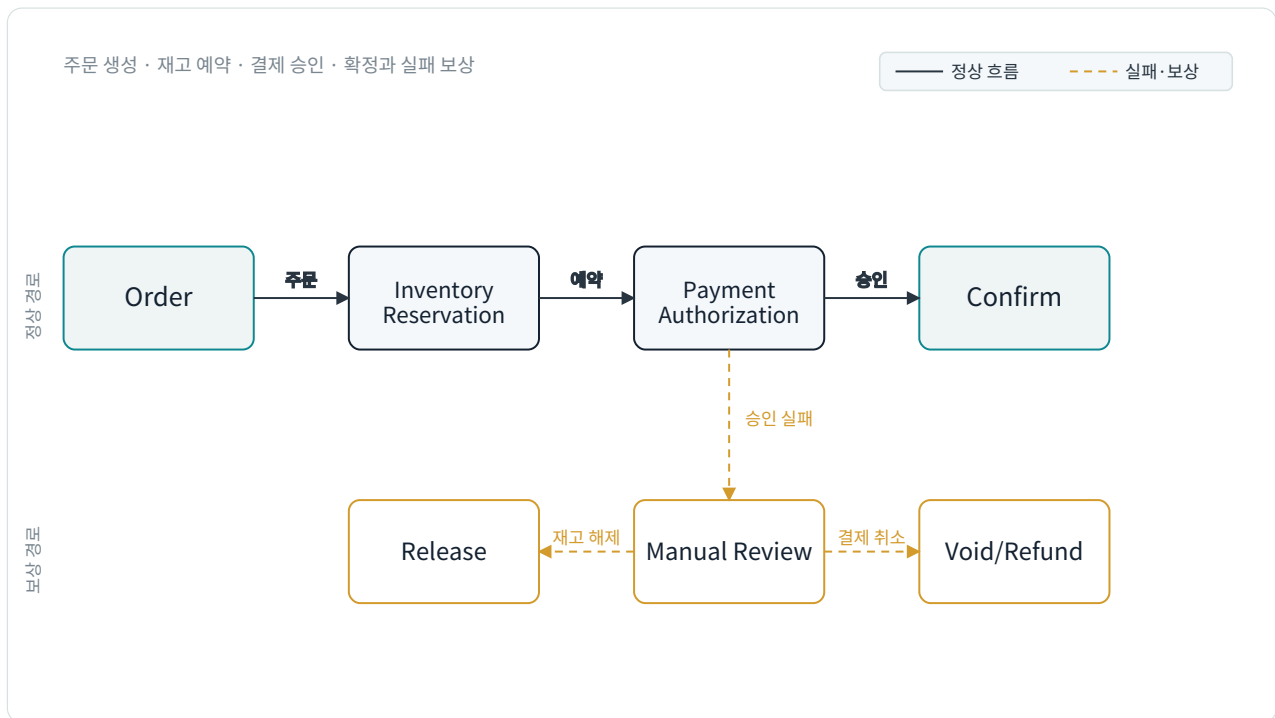
주문 총액 = 상품 가격 snapshot 합 + 세금 + 배송비 - 할인 + 조정
 재고 가용량 = 실물/논리 재고 - 활성 reservation 합
 결제 잔액 = 승인/청구/환불/수수료 ledger entry의 대수적 합

이 식들은 구현 코드 한 곳의 계산이 아니라 DB 제약, 상태 전이 guard, reconciliation query, 운영 경보가 함께 지켜야 하는 규칙이다. 통화가 다르면 단순 합산하지 않고 currency와 minor unit을 모든 금액 entry에 보존한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Order service/DB	주문 상태·가격 snapshot·workflow reference를 원장으로 관리한다.
Inventory service	stock movement·reservation·expiry를 관리한다.
Payment service	provider token·authorization·capture·refund 상태를 관리한다.
Saga orchestrator	command·timeout·compensation과 전체 상태를 관리한다.
Outbox/event log	local commit을 다른 서비스에 전달한다.
Ledger	금액 entry와 balance 검증을 보존한다.
Reconciliation workers	provider statement·inventory count와 차이를 탐지한다.
Admin console	수동 승인·보상·evidence 조회를 제공한다.



주문 생성·재고 예약·결제 승인·확정과 실패 보상 흐름을 보여준다

요청·데이터 흐름

1. client가 cart snapshot과 idempotency key로 주문 생성을 요청한다.
2. order service가 가격·세금·할인 snapshot과 pending order를 commit한다.
3. saga가 inventory reservation command를 보낸다.
4. inventory가 조건부 수량 감소 또는 reservation entry를 commit한다.
5. payment가 provider에 authorization을 같은 key로 요청한다.
6. 성공하면 주문을 confirmed하고 필요 시 capture를 진행한다.
7. 실패·timeout이면 reservation release와 authorization void/refund를 실행한다.
8. 모든 단계가 outbox로 사실 event를 발행한다.
9. reconciliation이 주문·ledger·provider 결과를 주기적으로 비교한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

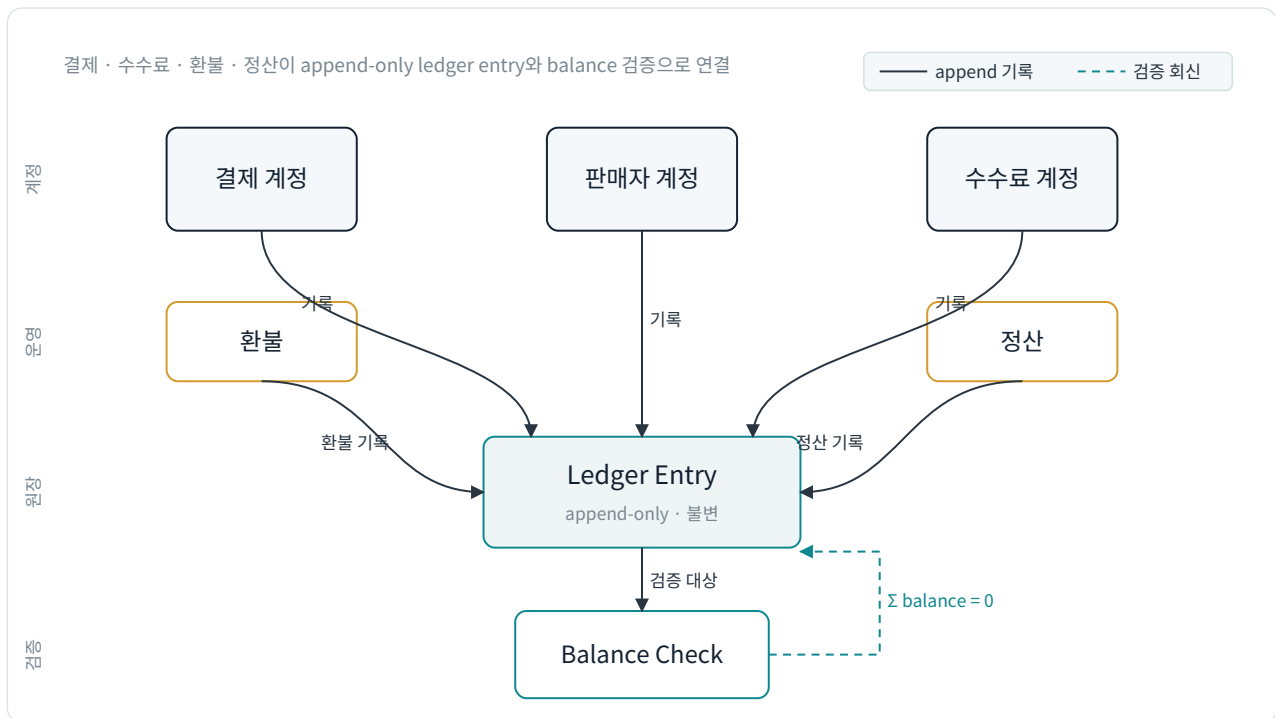
대안과 트레이드오프

대안	장점	비용·위험	적합한 조건
재고 선점 후 결제	oversell을 줄이고 결제 전에 수량을 보장한다.	결제 실패 동안 재고가 잠기며 expiry가 필요하다.	희소 상품
결제 승인 후 재고	재고 lock 시간을 줄인다.	재고 실패 시 승인 취소·고객 경험 문제가 있다.	재고 여유가 큰 상품
동기 orchestration	사용자에게 빠른 최종 상태를 제공한다.	외부 provider 지연과 timeout이 request를 길게 만든다.	짧은 결제 flow
비동기 주문 접수	긴 처리와 재시도를 내구 workflow로 다룬다.	pending UX·상태 조화·알림이 필요하다.	복잡한 주문·외부 의존성

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [[@postgres-transaction-iso](#); [@saga-paper](#); [@stripe-idempotency](#)]를 참조한다.

장애 시나리오

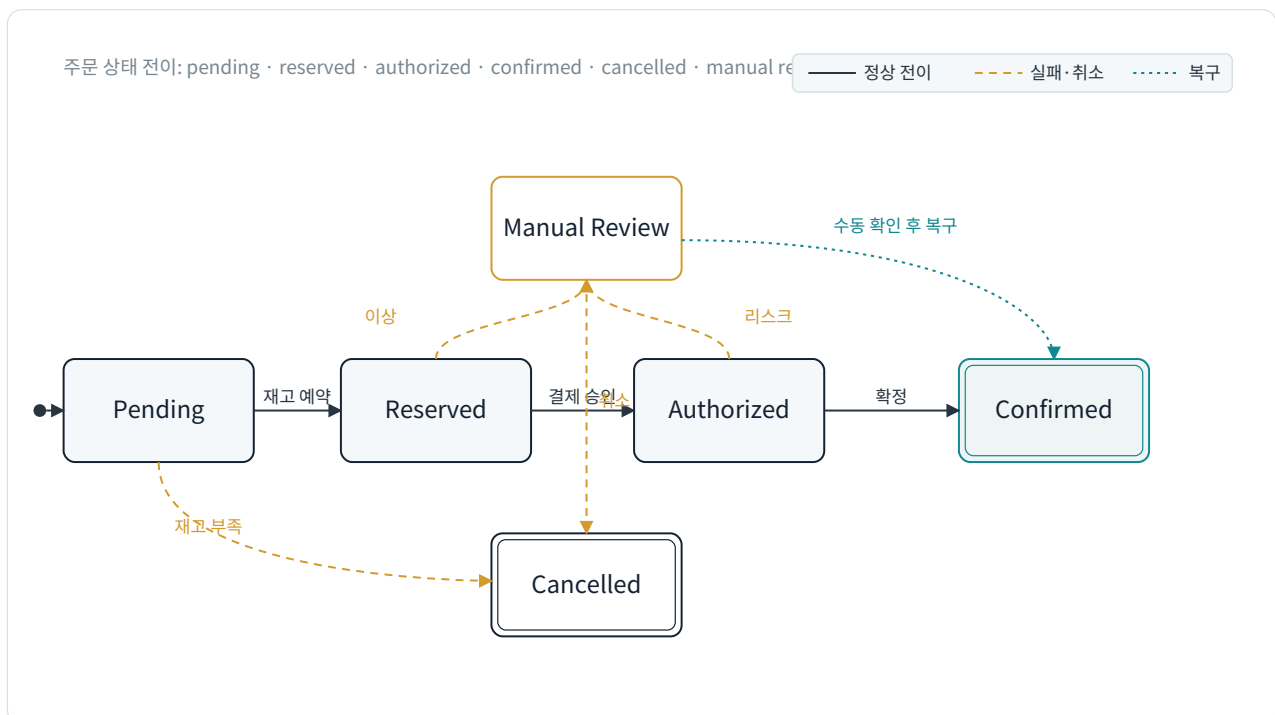
시나리오	영향	대응 원칙
Double charge	client/gateway retry가 결제 provider를 두 번 호출한다.	업무 idempotency key를 provider 요청과 내부 UNIQUE에 묶는다.
Unknown payment	timeout 후 provider 승인 여부를 모른다.	같은 key 조화·webhook·reconciliation 전에는 재청구하지 않는다.
Oversell	동시 reservation이 가용 수량을 초과한다.	조건부 update·serializable·reservation ledger를 사용한다.
Expired reservation race	만료 worker와 결제 완료가 동시에 상태를 바꾼다.	versioned state transition과 terminal guard를 사용한다.
Partial refund	일부 상품만 취소됐지만 ledger와 order 금액이 어긋난다.	line-level adjustment entry와 balance invariant를 둔다.
Provider webhook duplicate/out-of-order	이전 상태 event가 새 상태를 덮는다.	provider event ID dedup과 monotonic state guard를 사용한다.



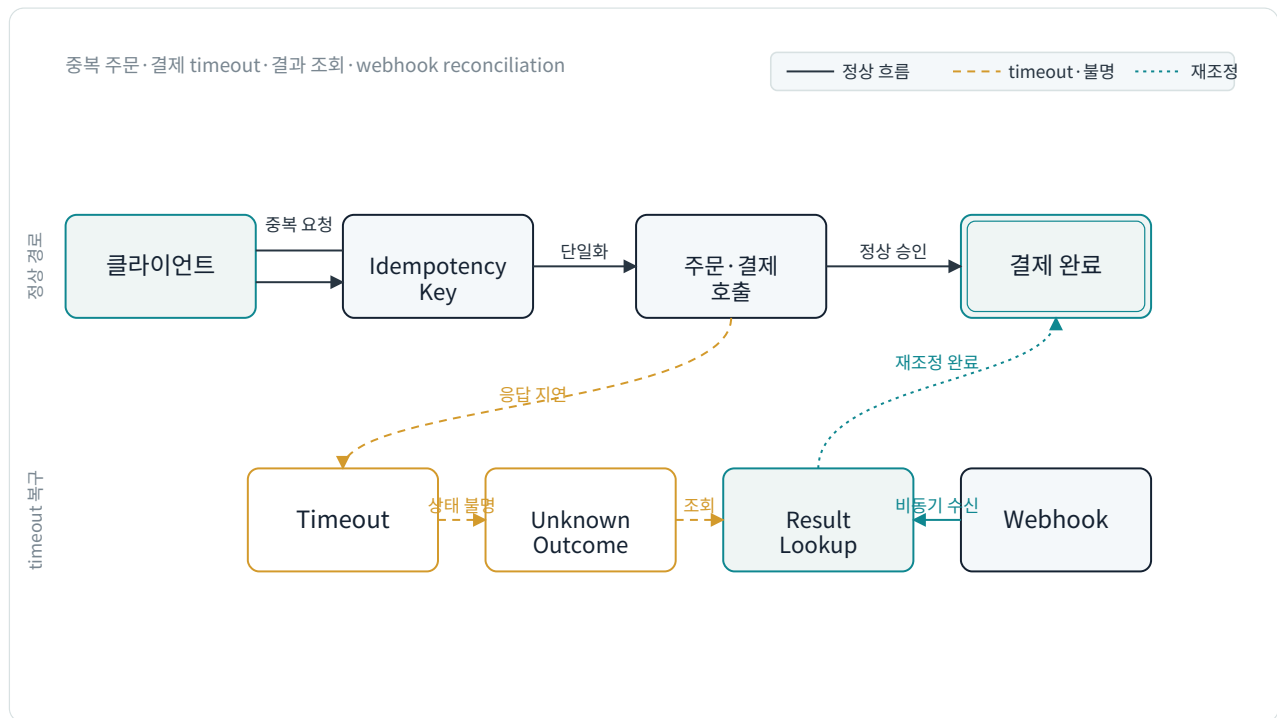
결제·수수료·환불·정산이 append-only ledger entry와 balance 검증으로 연결되는 모습을 보여준다

종합 설계 보조 도표

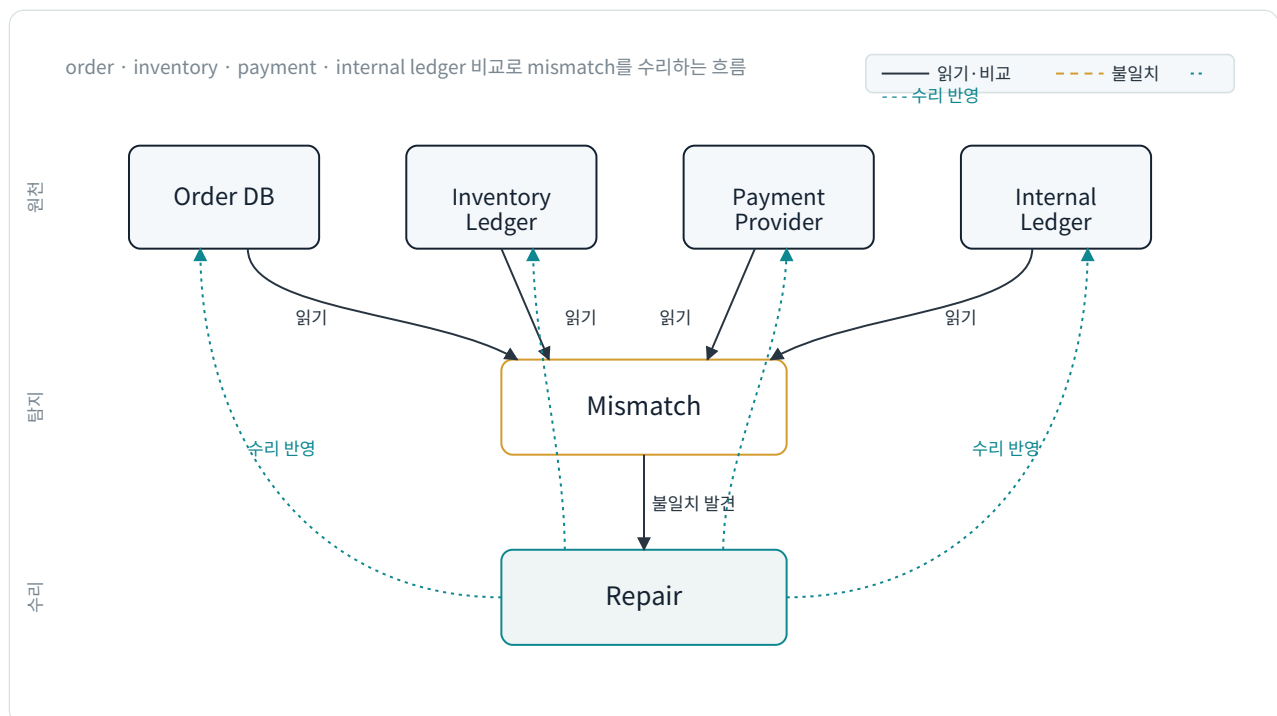
이 장은 앞의 원리를 하나의 서비스로 연결하므로 다음 보조 도표까지 제작한다.



pending · reserved · authorized · confirmed · cancelled · manual-review 상태 전이를 보여준다



중복 주문·결제 timeout·결과 조회·webhook reconciliation을 보여준다



주문·재고·결제 provider·ledger를 비교해 mismatch를 수리하는 흐름을 보여준다

확장 전략

- order ID 또는 merchant/tenant로 partition하되 inventory SKU hotspot을 별도 처리한다.

- 예약은 bucket·warehouse·SKU 단위로 분산하고 매우 hot한 flash sale은 token/preallocation을 사용한다.
- payment provider 호출은 channel별 bulkhead·rate limit·fallback을 둔다.
- ledger append와 reconciliation query를 분리해 원장 쓰기를 보호한다.
- saga terminal state를 archive하되 감사 증거는 보존 정책에 따라 유지한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- 카드 원문을 저장하지 않고 tokenized provider reference와 최소 metadata를 사용한다.
- 금액·수취인·환불 같은 고위험 action은 강한 권한·승인·감사를 요구한다.
- admin console은 사용자 서비스와 별도 trust zone과 break-glass를 둔다.
- webhook signature·timestamp·replay를 검증한다.
- 개인정보 삭제와 금융·세무 보존 의무의 우선순위를 정책으로 관리한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- 주문 상태별 체류 시간·saga timeout
- inventory available/reserved/expired·oversell guard
- payment authorization/capture/refund·unknown outcome
- idempotency duplicate/conflict
- ledger imbalance·reconciliation mismatch
- provider latency·webhook lag·manual intervention

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- 결제 provider fee·retry·fraud review·chargeback이 transaction 비용에 포함된다.
- 재고 reservation을 길게 유지하면 판매 기회 비용이 생긴다.

- 강한 원장·audit·reconciliation은 인프라와 인력 비용이지만 금액 불일치 기대 손실을 줄인다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- 주문 상태 열 하나를 여러 서비스가 직접 수정한다.
- timeout이면 결제가 실패했다고 단정해 새 요청을 보낸다.
- 재고 수량 하나만 감소시키고 reservation 증거를 남기지 않는다.
- 환불을 기존 결제 row 금액 뺀 것으로 표현한다.

설계 리뷰

- ☐ 주문·재고·결제 각각의 원장과 불변조건이 명확한가?
- ☐ 모든 command와 webhook이 idempotent한가?
- ☐ unknown outcome·timeout·보상 실패 상태가 있는가?
- ☐ ledger와 provider reconciliation이 자동화됐는가?
- ☐ 수동 개입이 권한·증거·재실행 안전성을 갖는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 동일 주문이 3번 제출돼도 한 번만 결제되는 idempotency table을 설계하라.
2. 재고 reservation과 결제 authorization의 timeout race를 상태 기계로 해결하라.
3. 부분 환불을 double-entry 형태의 append-only entry로 표현하라.

핵심 요약

- 주문·재고·결제는 각자의 원장과 불변조건을 가진다.
- 장기 업무는 saga와 idempotent command로 연결한다.
- 결제 timeout은 unknown outcome일 수 있다.
- reservation·authorization·capture·refund를 상태로 모델링한다.
- ledger와 reconciliation이 금액 정확성을 증명한다.

출처

- [@postgres-transaction-iso] PostgreSQL Global Development Group. **PostgreSQL Documentation — Transaction Isolation** (2026). <https://www.postgresql.org/docs/current/transaction-iso.html>
- [@saga-paper] Hector Garcia-Molina and Kenneth Salem. **Sagas** (1987). <https://doi.org/10.1145/38713.38742>
- [@stripe-idempotency] Stripe. **Stripe API — Idempotent requests** (2026). https://docs.stripe.com/api/idempotent_requests
- [@debezium-docs] Debezium Authors. **Debezium Documentation** (2026). <https://debezium.io/documentation/reference/stable/>

“

검증 기준일: 2026-08-06. `current` 와 `volatile` 내용은 `release` 전에 공식 원문을 다시 확인한다.

38

멀티테넌트 RAG·AI 고객지원 플랫폼

이 장의 구성

- 01 이 장에서 해결할 문제
 - 02 먼저 결론
 - 03 요구사항과 실패 모델
 - 04 핵심 개념
 - 05 기존 아키텍처
 - 06 요청·데이터 흐름
 - 07 대안과 트레이드오프
 - 08 장애 시나리오
 - 09 종합 설계 보조 도표
 - 10 확장 전략
 - 11 보안과 개인정보
 - 12 관측 가능성
 - 13 비용과 운영 복잡도
 - 14 흔한 오해와 안티패턴
 - 15 설계 리뷰
 - 16 연습문제
 - 17 핵심 요약
 - 18 출처
-

이 장의 흐름

장 전체의 절 구성을 빠르게 탐색하기 위한 편집 요약입니다.

SECTION FLOW



편집 요약: 본문의 장·절 구조와 핵심 문장을 재배열한 도식이며, 원문을 대체하지 않습니다.

“

원고 상태: 이 장은 실제 내용이 들어 있는 1차 초고다. 출판 전 기술 검수, 문장 편집, 수치 재검증, 시각 자료 제작이 필요하다.

이 장에서 해결할 문제

AI 고객지원 플랫폼은 챗봇 화면 하나가 아니라 tenant onboarding, 문서 수집, ACL, RAG, model routing, conversation, tool action, human handoff, evaluation, billing, deletion을 결합한 플랫폼이다. shared infrastructure에서도 모든 단계가 tenant context를 잃지 않아야 한다.

이 절의 기준 출처: [@azure-multitenant; @rag-paper].

학습 목표

- 멀티테넌트 문서·검색·model·agent 경계를 종합 설계한다.
- tenant별 권한·품질·비용·데이터 위치를 강제한다.
- human handoff와 감사 가능한 고객지원 workflow를 만든다.

먼저 결론

- tenant context는 gateway에서 생성해 문서·index·cache·model·tool·telemetry까지 전달하고 각 계층이 검증한다.
- 답변 생성과 고객 계정 변경·환불 같은 tool action은 별도 권한·승인 경계다.
- 품질·latency·cost를 tenant·language·intent·channel별로 분해한다.
- shared index와 dedicated index, shared model과 dedicated endpoint를 tenant 위험·규모·지역 정책에 따라 tiering한다.

∴ current-note **2026-08-06 확인:** 이 장은 변화 가능한 표준·프로젝트·AI 구현을 포함한다. 기본 재검토일은 2026-11-06이며, 출판 직전 공식 문서를 다시 확인한다. ∴

요구사항과 실패 모델

차원	확인 질문	설계 판단
핵심 보장	멀티테넌트 RAG·AI 고객지원 플랫폼에서 사용자 또는 후속 시스템에 반드시 보장할 결과는 무엇인가?	tenant context는 gateway에서 생성해 문서·index·cache·model·tool·telemetry까지 전달하고 각 계층이 검증한다.

차원	확인 질문	설계 판단
규모·분포	피크 부하, 데이터량, 지역·tenant 편차가 설계에 어떤 영향을 주는가?	ingestion·retrieval·inference·tool을 tenant별 별도 quota와 bulkhead로 나눈다.
실패·복구	“Tenant context loss” 같은 실패에서 결과를 어떻게 판정하고 복구하는가?	모든 key·event·span·policy에 signed tenant context와 server validation을 적용한다.
보안·통제	접근 권한, 데이터 보호, 경계 통제를 어디에 적용하는가?	tenant identity를 모든 저장·event·cache·trace key에 포함하고 서버가 파생한다.
운영 검증	어떤 관측 신호로 설계가 실제 요구를 만족하는지 판단하는가?	tenant별 answer success·grounding·handoff·CSAT

요구사항은 정상 처리량만으로 끝나지 않는다. 각 항목에 “지연되면?”, “중복되면?”, “일부만 성공하면?”, “운영자가 복구할 수 없으면?”을 추가해 실패 모델로 확장한다.

핵심 개념

Tenant control plane

계약·region·quota·model policy·connector·retention을 관리한다.

Tenant context

검증된 tenant ID, user, role, region, policy version을 가진 요청 범위다.

Knowledge plane

문서 원장·ingestion·chunk·index·ACL을 제공한다.

Conversation plane

session·message·summary·handoff 상태를 관리한다.

Inference plane

retrieval·rerank·model route·streaming을 수행한다.

Action plane

CRM·ticket·refund 등 tool을 policy와 approval 아래 실행한다.

Human handoff

AI가 중단·escalate할 때 evidence와 context를 상담원에게 전달한다.

Evaluation plane

tenant별 golden set·online quality·safety·cost를 관리한다.

Metering

token·retrieval·tool·storage·human review 사용량을 계약 단위로 집계한다.

핵심 개념의 정의와 범위는 [@azure-multitenant; @rag-paper; @nist-genai-profile; @owasp-llm; @otel-spec]
를 기준으로 재검토해야 한다.

서비스 계층 제안

계층	기본 격리	승격 조건	주요 제한
Shared	논리 tenant 격리, 공용 index/GPU	지속적인 quota 초과, 규제, 품질 간섭	표준 model·보존·connector
Isolated Data	공용 app, tenant 전용 DB/index	대형 문서·강한 ACL·지역 요구	별도 lifecycle 비용
Dedicated	전용 data/index/inference	계약상 강한 격리·성능 보장	높은 최소 비용·표준 운영만 허용

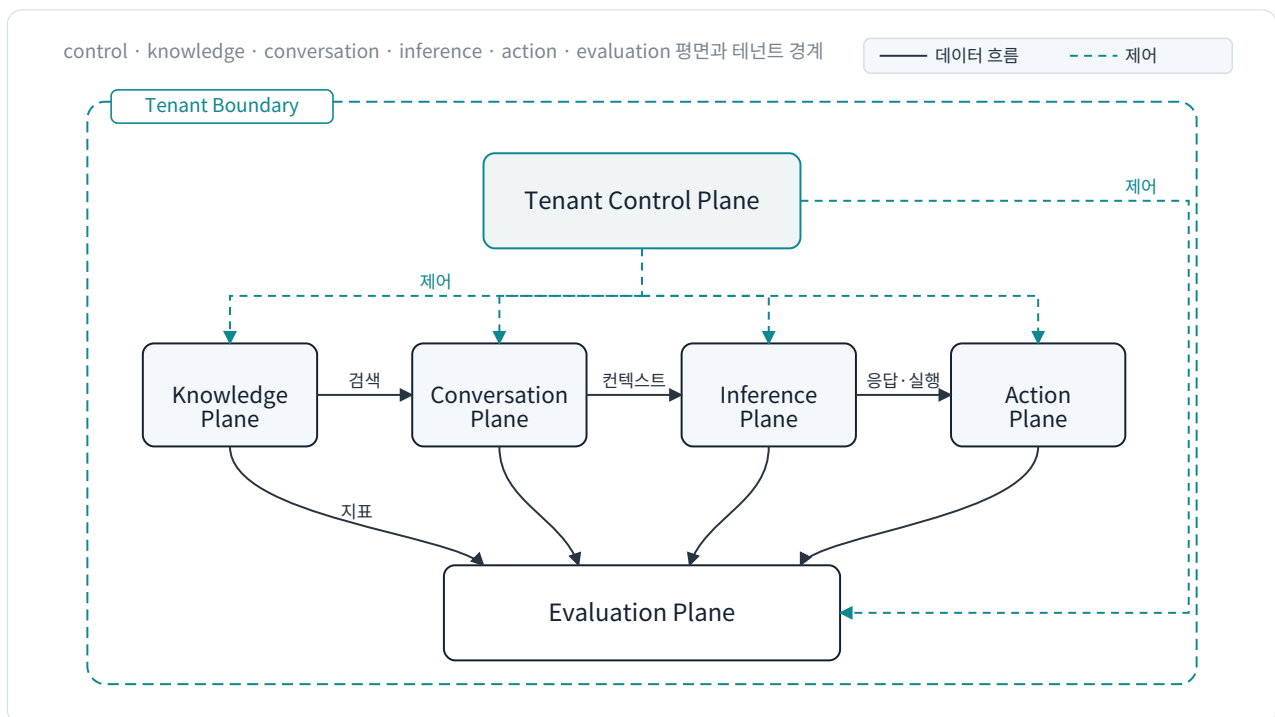
승격은 영업 요청만으로 결정하지 않는다. 실제 사용량, noisy-neighbor 지표, 데이터 지역, 보안 위험, unit economics를 함께 판단하고 되돌림·export 경로를 유지한다.

기준 아키텍처

아래 구조는 특정 제품 목록이 아니라 책임과 경계를 표현한다. 실제 구현에서는 각 구성 요소의 소유자, 데이터 계약, SLO, 장애 도메인을 추가한다.

구성 요소	책임
Tenant API gateway	auth·tenant resolution·quota·data region을 강제한다.
Control plane	tenant configuration과 policy version을 원장으로 관리한다.
Connector workers	tenant credential로 source를 수집한다.
Knowledge store/index	tenant ACL과 version을 가진 chunk·text/vector index를 제공한다.

구성 요소	책임
Conversation service	channel message · summary · consent · handoff state를 저장한다.
RAG orchestrator	retrieval · rerank · context · citation을 수행한다.
Model router	tenant policy · risk · language · capacity로 model을 선택한다.
Tool gateway	capability · approval · idempotency · audit로 action을 실행한다.
Agent desktop	evidence · AI suggestion · customer state를 상담원에게 제공한다.
Evaluation/metering	quality · safety · cost · usage를 tenant별로 집계한다.



control plane · knowledge · conversation · inference · action · evaluation plane과 tenant 경계를 보여준다

요청·데이터 흐름

1. tenant admin이 region · connector · retention · model/tool policy를 설정한다.
2. connector가 tenant-scoped credential로 문서를 수집하고 ACL · version을 보존한다.
3. user message가 channel identity와 tenant context로 gateway에 도착한다.
4. conversation service가 consent · history · summary version을 불러온다.
5. RAG가 tenant/ACL filter로 evidence를 검색·rerank한다.
6. router가 intent · risk · language · SLO · budget으로 model을 선택한다.

7. answer는 citation·policy·PII 검사를 거쳐 streaming된다.
8. 계정 변경 등 action은 tool gateway와 필요 시 사용자/상담원 승인을 거친다.
9. uncertainty·policy·감정·요청에 따라 상담원에게 handoff한다.
10. 모든 단계의 사용량·품질·근거·action을 tenant scope로 기록한다.

흐름을 검토할 때 각 단계의 성공 응답이 무엇을 보장하는지, timeout 이후 결과를 어떻게 확인하는지, 재시도 시 같은 효과가 반복되는지를 함께 기록한다.

대안과 트레이드오프

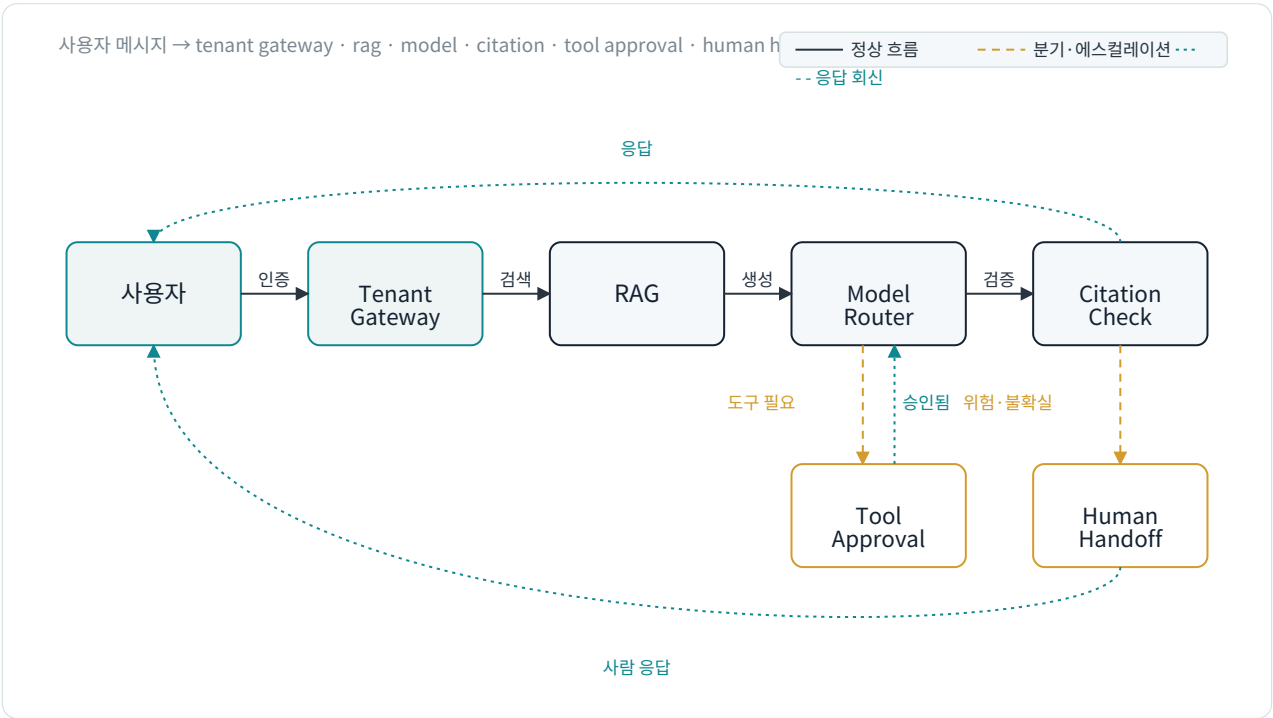
대안	장점	비용·위험	적합한 조건
Shared everything	자원 효율과 onboarding 속도가 좋다.	noisy neighbor·격리·customization 위험이 크다.	소규모 표준 tenant
Shared app, isolated data/index	애플리케이션 효율과 데이터 격리를 균형 있게 제공한다.	tenant별 lifecycle·connection·cost 관리가 필요하다.	중간/규제 tenant
Dedicated stack	강한 격리·region·custom model 정책을 제공한다.	비용·upgrade·운영 편차가 크다.	대형·고위험 tenant
AI answer only	안전하고 도입이 단순하다.	업무 자동화 가치가 제한된다.	초기/고위험 support
Tool-capable agent	해결률과 자동화가 높다.	권한·approval·사고 위험이 크다.	제한된 반복 업무

대안 비교는 제품 선호가 아니라 이 장의 요구사항과 실패 모델을 기준으로 수행한다. 관련 근거는 [azure-multitenant; rag-paper; nist-genai-profile]를 참조한다.

장애 시나리오

시나리오	영향	대응 원칙
Tenant context loss	비동기 job·cache·trace에서 tenant ID가 빠져 데이터가 섞인다.	모든 key·event·span·policy에 signed tenant context와 server validation을 적용한다.
Noisy neighbor	한 tenant의 대량 ingestion/long prompt가 shared GPU·index를 포화시킨다.	tenant quota·bulkhead·weighted fair scheduling·dedicated tier를 사용한다.
ACL stale	source 권한 변경이 vector index에 늦게 반영된다.	change cursor·policy version·pre/post filter·deny fallback을 둔다.
Unsafe tool action	AI가 잘못된 고객에게 환불/변경을 실행한다.	resource lookup·preview·approval·idempotency·postcondition을 적용한다.

시나리오	영향	대응 원칙
Model/provider outage	특정 model route가 실패해 모든 tenant 응답이 중단된다.	policy-compatible fallback·queue·human handoff·status를 둔다.
Data deletion gap	tenant 탈퇴 후 backup·index·eval sample에 데이터가 남는다.	deletion workflow와 per-store completion evidence를 둔다.
Quality disparity	평균 품질은 좋지만 특정 언어·제품군에서 반복 실패한다.	tenant/segment evaluation gate와 targeted human review를 사용한다.



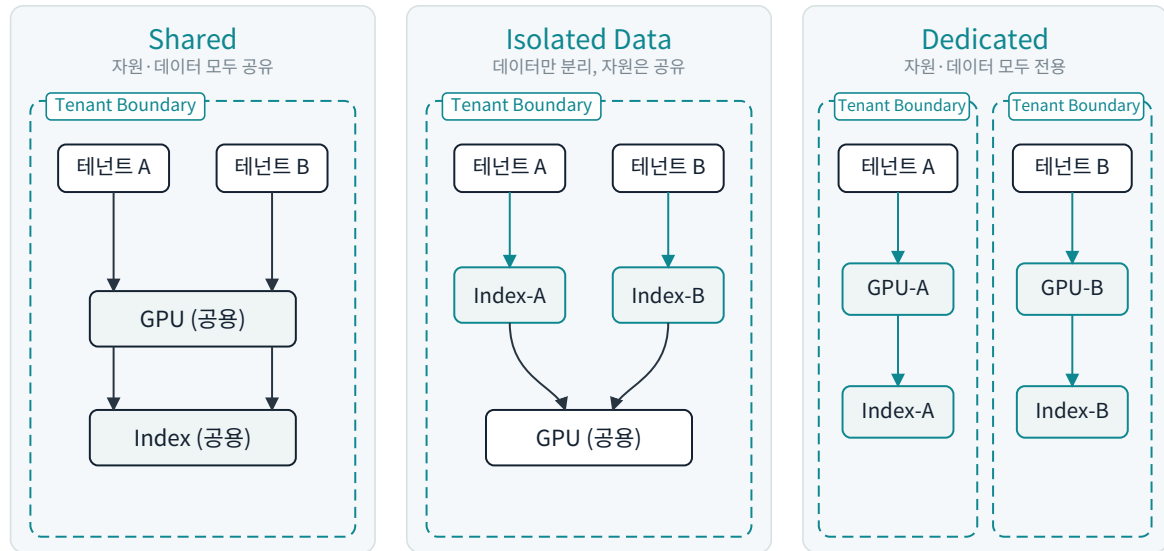
사용자 message가 tenant auth·RAG·model·citation·tool approval·human handoff를 거치는 end-to-end 흐름을 보여준다

종합 설계 보조 도표

이 장은 앞의 원리를 하나의 서비스로 연결하므로 다음 보조 도표까지 제작한다.

shared · isolated data · dedicated tier의 자원·데이터·model 경계 비교

--- Tenant Boundary — 공유



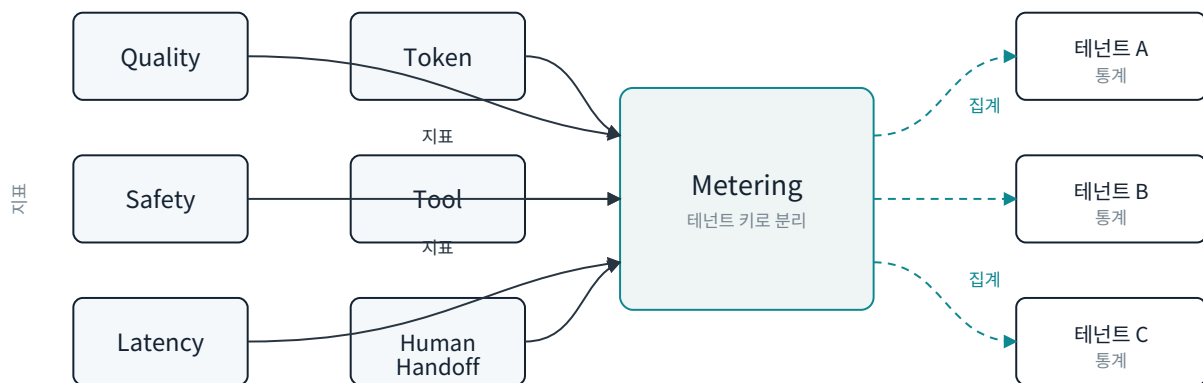
shared · isolated-data · dedicated tier의 자원·데이터·model 경계를 비교한다

테넌트별 품질·안전·latency·token·tool·human handoff 집계

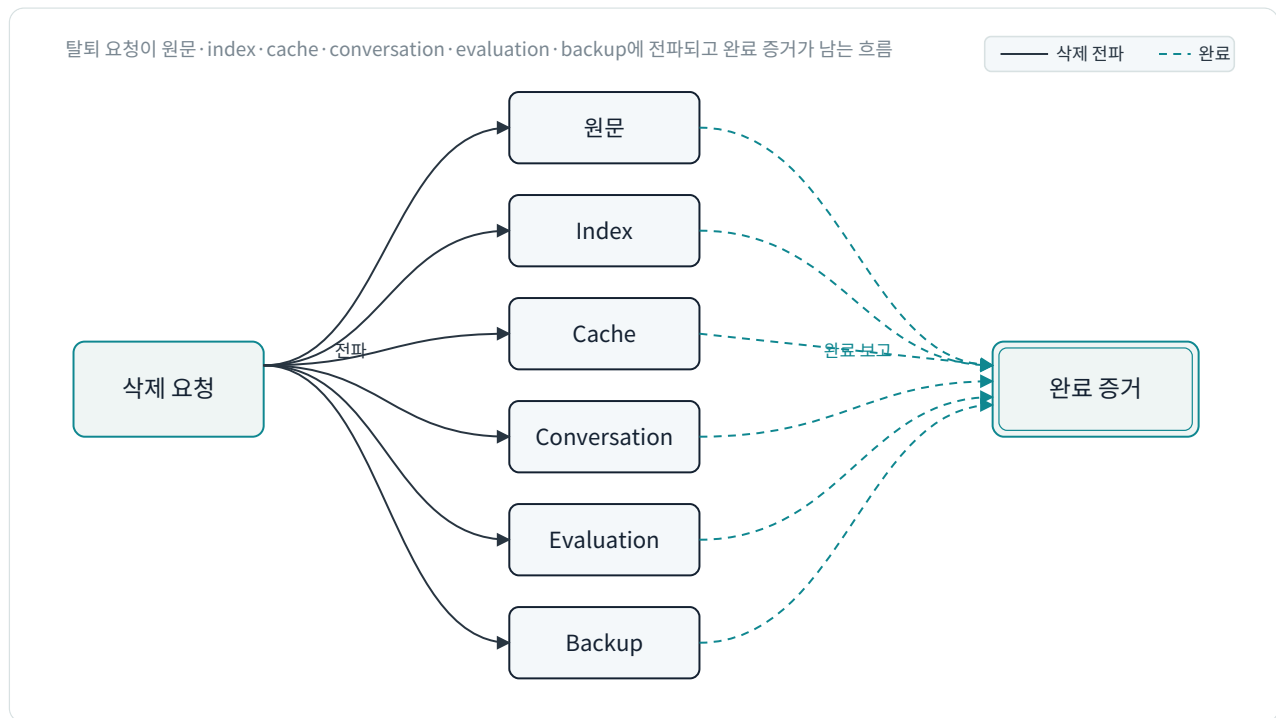
집계

— 지표 수집 --- 집계

출력



tenant별 품질·안전·latency·token·tool·human handoff를 집계하는 흐름을 보여준다



탈퇴 요청이 원문·index·cache·conversation·evaluation·backup에 전파되고 증거가 남는 흐름을 보여준다

확장 전략

- ingestion·retrieval·inference·tool을 tenant별 별도 quota와 bulkhead로 나눈다.
- shared index는 tenant filter selectivity와 shard hotspot을 관측하고 대형 tenant를 전용 index로 이동한다.
- conversation summary와 retrieved context를 token budget으로 관리한다.
- model capacity는 tenant priority·SLO·contract에 따라 예약하고 overflow 정책을 둔다.
- control plane과 data plane을 분리해 tenant 설정 변경이 runtime 전체를 막지 않게 한다.

확장은 구성 요소 수를 늘리는 행위가 아니라 병목 축과 실패 범위를 다시 분리하는 과정이다. 확장 전후의 사용자 SLI와 운영 복잡도를 함께 비교한다.

보안과 개인정보

- tenant identity를 모든 저장·event·cache·trace key에 포함하고 서버가 파생한다.
- connector credential과 tool capability는 tenant·source·action에 한정하고 짧은 수명으로 발급한다.
- retrieved 문서와 고객 message를 untrusted content로 취급해 system policy와 tool 권한을 분리한다.
- human agent와 AI가 본 개인정보를 role·purpose·case 기준으로 감사한다.
- data residency·retention·deletion·model provider 전송 정책을 tenant 계약에 반영한다.

보안 요구는 별도 부록이 아니라 요청·데이터 흐름의 각 경계에 적용한다. 특히 인증된 주체, tenant, 데이터 분류, 보존·삭제, 운영자 권한을 함께 기록한다.

관측 가능성

다음 신호를 최소 세그먼트(서비스·지역·tenant 또는 workload class)로 나눠 본다.

- tenant별 answer success·grounding·handoff·CSAT
- ingestion freshness·ACL lag·failed source
- retrieval zero-result·citation coverage·leak test
- model route·TTFT·token·fallback·quality
- tool allow/deny/approval/error/compensation
- quota·noisy-neighbor·dedicated tier migration
- tenant unit cost·gross margin·budget
- deletion workflow completion·audit gap

경보는 개별 자원 임계값보다 사용자 SLO와 error budget 소진에 연결하고, 조사 시 trace·log·변경 이력으로 내려갈 수 있어야 한다.

비용과 운영 복잡도

- tenant별 cost는 storage·index·embedding·retrieval·model token·tool API·human handoff를 합쳐야 한다.
- shared tier는 효율이 높지만 noisy-neighbor 방지용 여유와 격리 control 비용이 있다.
- dedicated stack은 높은 가격과 강한 요구에만 제공하고 운영 표준에서 벗어나는 customization을 제한한다.
- 답변 자동화율보다 해결된 case당 비용과 human rework를 본다.

비용 비교에는 인스턴스 가격뿐 아니라 데이터 전송, 복제·백업, 관측, 보안 통제, 업그레이드, on-call, 장애 복구, 탈출 비용을 포함한다.

흔한 오해와 안티패턴

- tenant ID를 prompt 문자열에만 넣고 인프라 key와 권한에서 강제하지 않는다.
- shared vector index의 post-filter만으로 격리가 충분하다고 생각한다.
- AI가 생성한 답변과 실제 고객 계정 action을 같은 권한으로 처리한다.
- 평균 품질과 전체 token 비용만 보고 tenant별 불공정·손실을 숨긴다.

설계 리뷰

- tenant context가 모든 sync/async 경계에서 보존·검증되는가?
- shared와 dedicated tier의 승격 조건이 객관적인가?
- ACL·삭제·지역 정책이 원문부터 telemetry까지 적용되는가?
- tool action이 capability·preview·approval·idempotency를 갖는가?
- tenant별 품질·SLO·비용·human handoff가 함께 관측되는가?

리뷰 결과는 “통과/실패”만 기록하지 않고 남은 가정, 위험 수용자, 실험, 재검토일을 ADR과 backlog에 연결한다.

연습문제

1. 공유 vector index에서 tenant ACL을 pre-filter·post-filter·citation 검증으로 구현하라.
2. 대형 tenant를 dedicated index와 model endpoint로 무중단 이동하는 계획을 작성하라.
3. 환불 tool을 호출할 수 있는 고객지원 agent의 승인·감사·보상 흐름을 설계하라.

핵심 요약

- 멀티테넌트 AI는 모든 계층에서 tenant context를 강제한다.
- knowledge·conversation·inference·action·evaluation plane을 분리한다.
- tool action은 답변 생성보다 강한 승인 경계가 필요하다.
- shared와 dedicated tier를 규모·위험·지역에 따라 선택한다.
- 품질·안전·비용을 tenant와 사용자 segment별로 운영한다.

출처

- [@azure-multitenant] Microsoft. **Azure Architecture Center — Multitenant solutions** (2026). <https://learn.microsoft.com/azure/architecture/guide/multitenant/overview>
- [@rag-paper] Patrick Lewis et al.. **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks** (2020). <https://arxiv.org/abs/2005.11401>
- [@nist-genai-profile] NIST. **Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile** (2024). <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
- [@owasp-llm] OWASP Foundation. **OWASP Top 10 for LLM Applications** (2025). <https://genai.owasp.org/llm-top-10/>

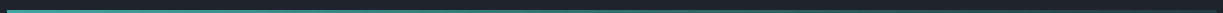
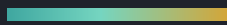
-
- [@otel-spec] OpenTelemetry Authors. **OpenTelemetry Specification** (2026). <https://opentelemetry.io/docs/specs/otel/>

“

검증 기준일: 2026-08-06. current 와 volatile 내용은 release 전에 공식 원문을 다시 확인한다.

A

. 시스템 설계 리뷰 체크리스트



문제와 경계

- 핵심 사용자 여정과 비목표가 한 문장으로 설명된다.
- 데이터별 단일 쓰기 소유자와 원장이 명확하다.
- 기능, 품질, 규모, 규제, 팀 제약이 분리돼 있다.
- 정상 경로뿐 아니라 timeout, 중복, 부분 성공, 취소, 복구가 정의돼 있다.

수치와 성능

- 모든 숫자에 단위, 시간 창, 근거, 계산식이 있다.
- 평균과 피크·burst·p95·p99가 분리돼 있다.
- 논리 저장량과 복제·색인·로그·백업을 포함한 물리량이 구분된다.
- 부하 테스트가 실제 key·tenant·payload 분포를 반영한다.

데이터와 일관성

- 불변조건이 DB 제약·상태 기계·reconciliation 중 어디에서 보호되는지 명확하다.
- read-after-write, staleness, 충돌, idempotency 의미가 API에 드러난다.
- replica, cache, search index, vector index의 재구축 경로가 있다.
- schema·model·index version 변경과 rollback이 설계돼 있다.

장애와 운영

- 장애 도메인과 공통 원인 실패가 표시돼 있다.
- deadline·retry budget·load shedding·degradation이 연결돼 있다.
- RTO/RPO와 restore/failover/failback의 실제 검증 증거가 있다.
- SLO와 error budget이 배포·용량·우선순위 정책에 연결돼 있다.

보안과 비용

- 주체·tenant·resource·action·context가 모든 신뢰 경계에서 검증된다.
- 개인정보가 저장·cache·event·telemetry·backup에 어떻게 복제되는지 추적된다.
- secret·artifact·관리 권한이 최소화되고 회전·감사된다.
- 단위당 비용, 운영 인력, 데이터 전송, 탈출 비용이 비교됐다.

B

. 용량 계산 공식

이 부록의 식은 입력이 명시될 때만 의미가 있다. 결과에는 10진/2진 단위, 압축, 복제, 색인, 여유를 별도로 표시한다.

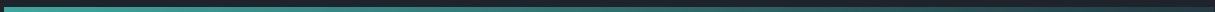
```

평균 RPS = 하루 요청 수 / 86,400
피크 RPS = 평균 RPS × 관측된 피크 계수
동시성 ≈ 도착률(request/s) × 평균 체류 시간(s)
대역폭(bytes/s) = 요청률 × 평균 전송 bytes
논리 저장량 = 이벤트율 × 객체 bytes × 보존 시간
복제 포함 저장량 = 논리 저장량 × 복제 계수
Cache origin QPS = 전체 QPS × (1 - hit ratio)
비동기 복제 손실 노출 record ≈ 쓰기율 × 복제 지연
Error budget = 관측 창 × (1 - SL0)
계층 재시도 최악 수 = 계층별 시도 수 ^ 호출 깊이
  
```

실제 capacity는 평균 식에 p99 latency, queue, compaction, rebuild, backup, failover 동시 부하를 더해 검증한다.

C

. 이미지 제작과 검수 흐름



-
1. 본문의 `figure-spec` 과 `manifests/assets.yaml` 이 일치하는지 확인한다.
 2. 기술도는 `assets/specs/svg/` 의 프롬프트로 순수 SVG를 생성한다.
 3. Image2.0 장면은 `assets/prompts/image2/` 의 프롬프트로 4K PNG master를 생성한다.
 4. 차트는 `assets/specs/charts/` 의 산식과 `synthetic/실측` 구분을 따라 코드로 생성한다.
 5. 기술 검수에서 화살표·라벨·수치·출처를 확인한다.
 6. 편집 검수에서 정보 계층·가독성·본문 연결을 확인한다.
 7. 접근성 검수에서 `caption`·`alt text`·색상 외 구분을 확인한다.
 8. 모든 검수를 통과한 자산만 `approved` 로 변경한다.

현재 패키지는 119개 자산 모두 `specified` 상태이며 실제 이미지 바이너리는 포함하지 않는다.